

PART 1: CORE COMMUNICATION PATTERNS

Sync vs Async Communication

Core Idea

In microservices, services communicate either:

- Synchronous (Sync): Immediate request-response
- Asynchronous (Async): Fire-and-forget via events/messages

📊 Comparison Table

Factor Sync Async

Latency Low (immediate) Higher (event delay)

Coupling Tight Loose

Scalability Limited High

Consistency Strong Eventual

Complexity Simple Complex

Use when Immediate response needed Processing can wait

📌 Decision Tree

text

Step 1: Immediate response required?

YES → Sync

NO → Step 2

Step 2: User-facing critical path?

YES → Sync

NO → Step 3

Step 3: Can processing be delayed?

YES → Async

NO → Sync

Step 4: High load/scalability needs?

YES → Async preferred

NO → Sync acceptable

Step 5: Strong consistency required?

YES → Sync

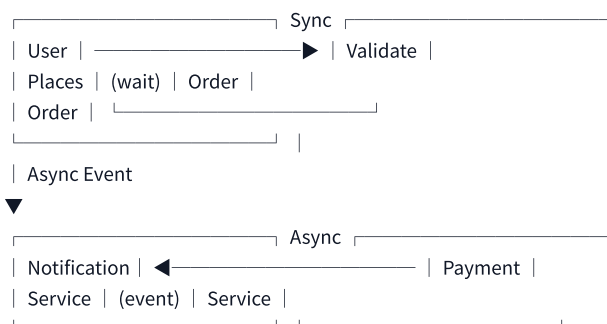
NO → Async

🧠 Mental Model

- Sync = Phone call (wait for reply)
- Async = Message/postbox (no waiting)

🎯 Hybrid Example (Order Flow)

text



PART 2: DATA MANAGEMENT

Data Decomposition

Core Principle

Database per service — each microservice owns its own database.

Why No Shared Database?

- Creates tight coupling
- Limits scalability
- Prevents independent deployment

Data Ownership Examples

Service Data Owned

Order order_id, items, order_status

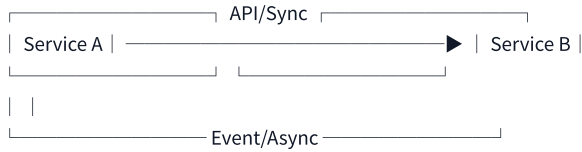
Customer customer_id, name, address

Payment payment_id, order_id, payment_status

Shipping shipment_id, tracking_status

🔄 Communication Without Shared DB

text



Key Challenge: Eventual Consistency

Solution Patterns:

- Saga Pattern for distributed transactions
- CQRS for separating read/write models
- Event Sourcing for maintaining consistency

🧠 Mental Model

- Monolith = one shared brain
- Microservices = multiple independent brains

PART 3: RESILIENCE PATTERNS

The Resilience Stack (Order of Protection)

text

Level 1: TIMEOUT —————▶ "Don't wait forever"

|



Level 2: RETRY —————▶ "Try again if temporary"

|



Level 3: CIRCUIT BREAKER —▶ "Stop calling failing service"

|



Level 4: FALLBACK —————▶ "Provide alternative response"

1. Timeout

Aspect Detail

What Limits max wait time for a call

Rule EVERY remote call needs timeout

Why Prevents thread blocking, resource exhaustion

2. Retry Pattern

Aspect Detail

When Network glitches, temporary overload, 5xx errors

When NOT 4xx errors, non-idempotent operations

Best Practices Exponential backoff + jitter + limited attempts

Retry Backoff Diagram

text

Attempt 1: Wait 100ms —▶ Fail

Attempt 2: Wait 200ms —▶ Fail

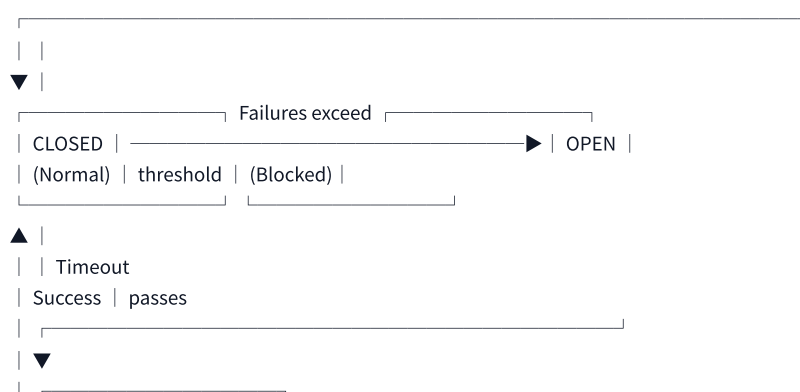
Attempt 3: Wait 400ms —▶ Success ✓

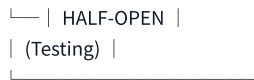
(Exponential backoff with jitter)

3. Circuit Breaker

States Flow

text





4. Fallback Pattern

Type Example

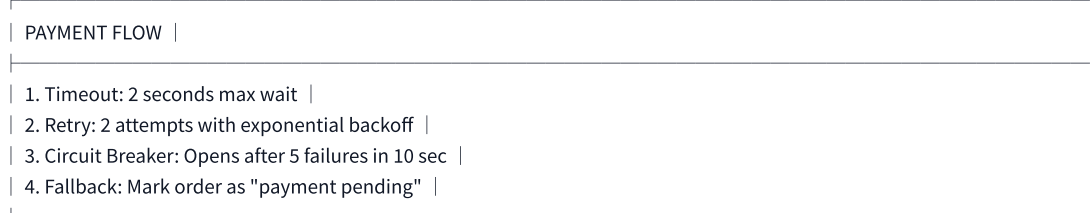
Cached data Serve stale cache when live API fails

Default response "Service temporarily unavailable"

Secondary provider Switch to backup service

🎯 Real-World Example (Payment Flow)

text



PART 4: RATE LIMITING

What It Does

Controls number of requests a client/user/service can make within a time window.

Algorithms Comparison

Algorithm Burst Allowed? Complexity Best For

Token Bucket ✓ Yes Medium Most production systems

Fixed Window ✗ No Simple Basic throttling

Sliding Window Limited High Accurate rate limiting

Leaky Bucket ✗ No Medium Smooth output traffic

Token Bucket Explained

text

Tokens added at steady rate (10/sec)

↓



↓

Each request consumes 1 token

↓

Burst: 50 requests can come instantly!

HTTP Status Code

429 Too Many Requests → Rate limit exceeded

🧠 Mental Model

Rate limiter = Traffic police (decides who goes, who waits, who stops)

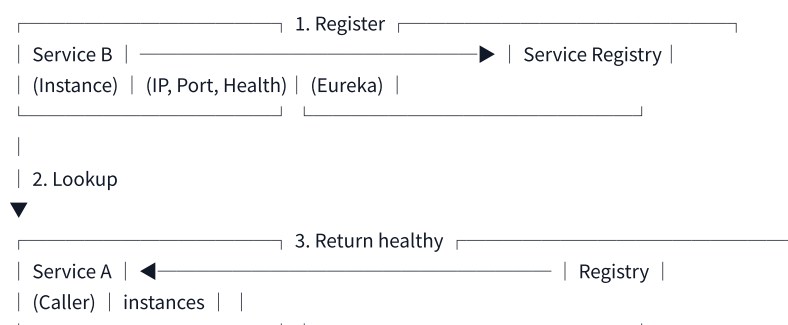
PART 5: SERVICE DISCOVERY

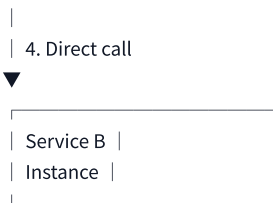
Definition

Mechanism that allows microservices to find and communicate with each other dynamically.

How It Works

text





Types

Type Who Decides Routing Example Tools

Client-Side Client Eureka + Ribbon

Server-Side Load Balancer Kubernetes, AWS ALB

🧠 Mental Model

- Service Discovery = "Google Maps for microservices"
- Registry = Directory of all services
- Load Balancer = Traffic controller

PART 6: SAGA PATTERN

Definition

Manages distributed transactions across multiple microservices without using a single ACID transaction.

Why Needed

- No shared database
- No global ACID transaction support
- Failures are common

Types of Saga

1. Choreography (Event-Driven)

text

Order Created —event—► Payment Service —event—► Inventory —event—► Shipping

▲ | | |

No central controller

2. Orchestration (Central Controller)

text



| | |

▼▼▼



Saga vs ACID

Feature ACID Transaction Saga Pattern

Scope Single DB Multiple services

Consistency Strong Eventual

Locking Yes No

Scalability Limited High

Failure handling Rollback Compensating actions

Compensating Transactions Example

text

Forward Actions: Compensating Actions:

3. Create Order → Cancel Order

4. Deduct Payment → Refund Payment

5. Reserve Stock → Release Stock

6. Create Shipment → Cancel Shipment

🧠 Mental Model

- ACID = Single chef in one kitchen
 - Saga = Multiple chefs in different kitchens coordinating via messages
-

PART 7: CQRS & DATA DUPLICATION

Definition

CQRS = Command Query Responsibility Segregation

Core Architecture

text



Intentional Data Duplication

Write Model Read Model

Normalized data Denormalized data

Transactional focus Query-optimized focus

Single source of truth Multiple projections possible

Example: Order Data

Write Model (Normalized):

sql

orders: order_id, customer_id, status

order_items: order_id, product_id, quantity

Read Model (Denormalized):

sql

order_view: order_id, customer_name, product_names, order_status, total_amount

🧠 Mental Model

- Write Model = Official record ledger
- Read Model = Optimized display copy
- Users never query the ledger directly

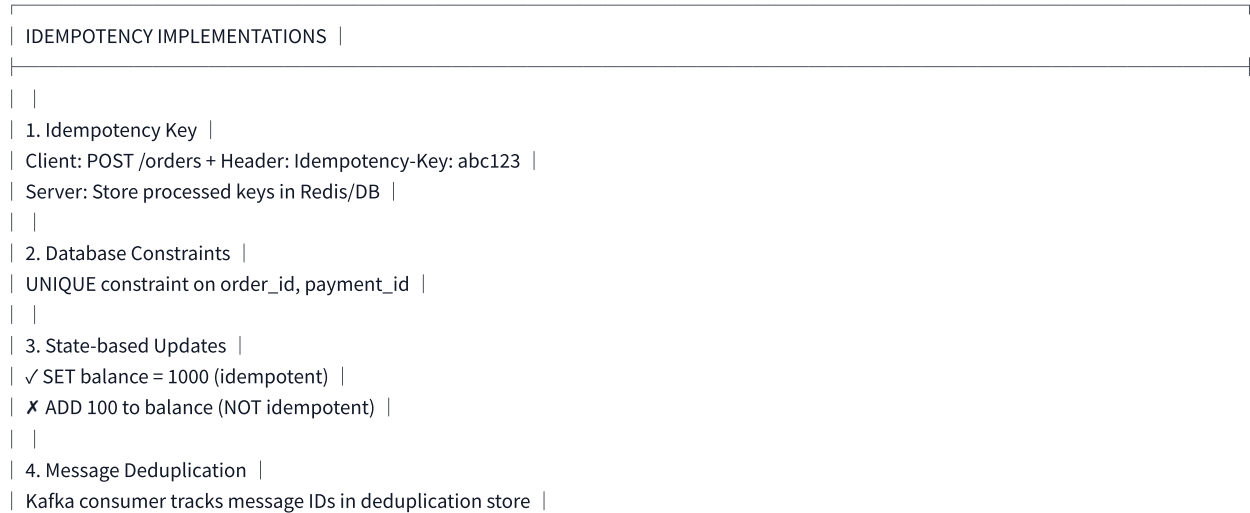
PART 8: IDEMPOTENCY

Definition

Executing the same operation multiple times produces the same result as executing it once.

Implementation Strategies

text



Where Critical

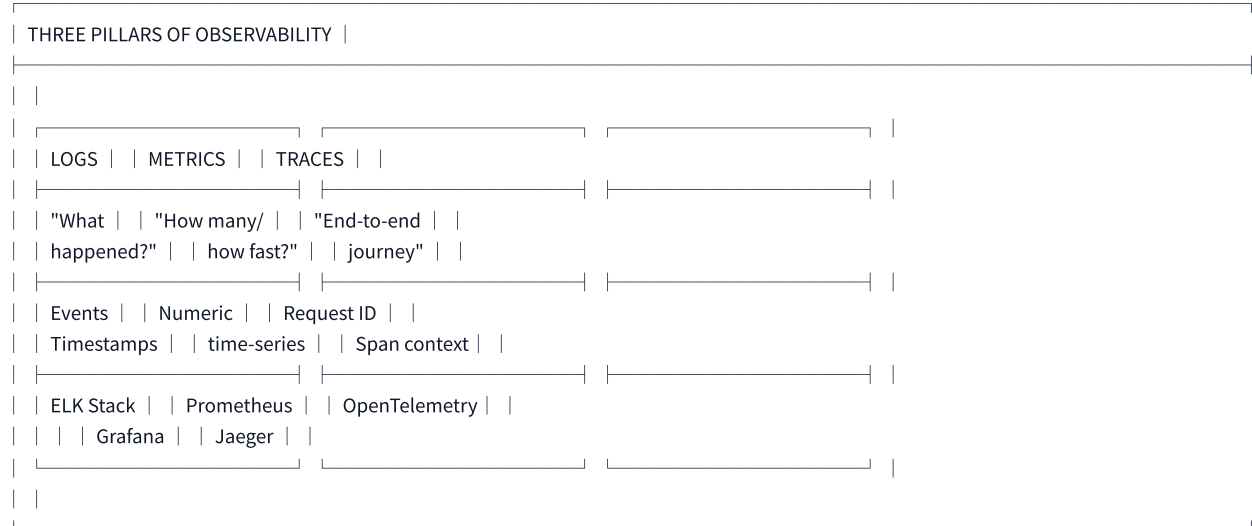
- Payment systems
- Order creation
- Inventory updates
- Event processing systems

🧠 Mental Model

- Idempotent = Light switch (ON → ON → ON = same state)
- Non-idempotent = Adding coins (each action changes state)

PART 9: OBSERVABILITY (3 Pillars)

text



How They Work Together

text

Trace ID: abc-123



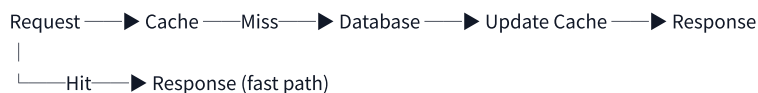
🧠 Mental Model

- Logs = Diary entries
- Metrics = Heartbeat monitor
- Traces = GPS route map

PART 10: CACHING STRATEGIES

Cache Aside (Lazy Loading) — MOST COMMON

text



Write-Through

text



Write-Back (Write-Behind)

text



|



Cache Invalidation Strategies

Strategy How It Works Best For
TTL Data expires after time Product catalogs
Event-based Update cache on data change User profiles
Manual Explicit invalidation API Admin-triggered updates
Cache Stampede Problem & Solutions
text
Problem: Cache expires → 1000 requests hit DB simultaneously
Solutions:

- 1. Locking (only 1 request fetches)
 - 2. Request coalescing (deduplicate in-flight requests)
 - 3. Random TTL jitter (stagger expiration times)
- 🧠 Mental Model
- Cache = Fast memory desk
 - Database = Slow library
 - You keep notes on your desk instead of going to the library every time

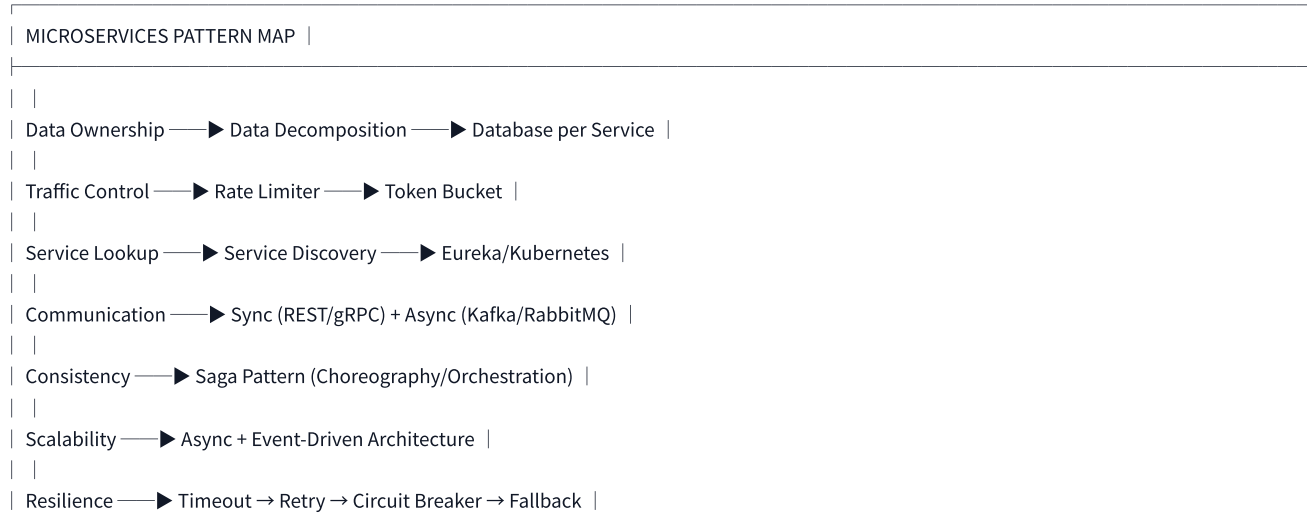
PART 11: DEPLOYMENT MODELS

Quick Comparison Matrix
Model Downtime Risk Cost Complexity Use Case
Monolithic High High Low Low Legacy systems
Rolling None Medium Medium Medium Standard deployments
Blue-Green None Low High Medium Critical apps
Canary None Very Low Medium High Large-scale systems
Active-Passive Minimal Low Medium Low DR systems
Active-Active None Low High Very High Global high-scale apps
Visual Summary
text
Rolling: [████████] → [███████] → [███████] → [███████] → [███████]
(Gradual instance replacement)
Blue-Green: [████████] BLUE [██████] GREEN
\\
└── Switch ───┘
(Instant traffic cutover)
Canary: [████████████████████] [██████] 5% traffic
Main version New version
(Gradual traffic increase)
Active-Active: 🌐 US ↵
└── All active, load balanced
🌐 EU ↵

📄 ULTRA CHEAT SHEET (1-Page Revision)

CORE PATTERNS MAP

text



	Performance	→ CQRS + Caching (Cache Aside)
	Safety	→ Idempotency (Idempotency Key + Unique Constraints)
	Visibility	→ Observability (Logs + Metrics + Traces)
	Deployment	→ Blue-Green / Canary / Active-Active

QUICK REFERENCE CARD

Pattern One-Line Summary

Sync Immediate response, tight coupling, user-facing ops

Async Event-driven, loose coupling, scalable processing

Saga Distributed transactions via compensating actions

CQRS Separate read/write models with intentional duplication

Circuit Breaker Stop calling failing services, prevent cascading failures

Retry Handle transient failures with exponential backoff

Timeout Never wait indefinitely for any remote call

Idempotency Same operation multiple times = same result

Rate Limiter Control request flow, return 429 when exceeded

MENTAL MODEL CHEAT SHEET

text

API Gateway = Front door of the system

Rate Limiter = Security guard at entrance

Service Registry = Phone directory of services

Kafka = Postal system (async messages)

Circuit Breaker = Emergency electrical switch

Saga = Workflow conductor/orchestrator

Cache = Sticky notes on your desk

Trace = GPS tracker of a request

Idempotency Key = Unique receipt ID for each operation

INTERVIEW POWER PHRASES

"Microservices are independently deployable services, each owning its own data model aligned to business domains."

"We use synchronous communication for real-time user-driven flows, and asynchronous for scalability and decoupling."

"The resilience stack is timeout first, then retry, then circuit breaker, finally fallback."

"Saga replaces ACID with eventual consistency using compensating transactions."

"CQRS intentionally duplicates data — that's the trade-off for scalability and performance."

"Idempotency is essential for safe retries in distributed systems."

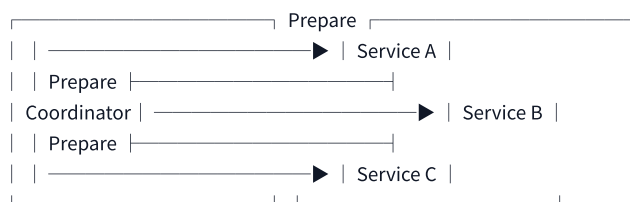
This document now serves as a complete interview preparation guide with:

- ✓ Clean formatting with emojis and visual hierarchy
- ✓ Text-based diagrams for easy recall
- ✓ Comparison tables for quick reference
- ✓ Mental models for intuitive understanding
- ✓ One-line summaries for interview delivery

TOPIC 1: DISTRIBUTED TRANSACTIONS — BEYOND SAGA

Two-Phase Commit (2PC) — Why Microservices Avoid It

text



| If ALL ready → Commit

| If ANY failure → Rollback

Aspect 2PC Saga

Locking Yes (blocks resources) No

Consistency Strong Eventual

Availability Lower Higher

Scalability Poor Excellent

Microservices fit  No  Yes

Interview Insight: "2PC doesn't work in microservices because locks can't span services and network partitions kill availability."

XA Transactions vs Saga vs TCC (Try-Confirm-Cancel)

Pattern Locking Compensating Best For

XA / 2PC Long locks Rollback Monoliths, not microservices

Saga No locks Compensating actions Long-running workflows

TCC Short "try" locks Explicit cancel High-performance scenarios

TCC Pattern Explained

text

Phase 1: TRY

- |—— Reserve resources (soft lock)
- |—— No actual business execution
- |—— Return confirmation token

Phase 2: CONFIRM (if all TRY succeed)

- |—— Execute actual business
- |—— Release soft locks

Phase 2 (alt): CANCEL (if any TRY fails)

- |—— Rollback reservations
- |—— Release soft locks

When TCC > Saga: Financial transactions where double-spending must be prevented during the reservation phase.

TOPIC 2: EVENT DRIVEN ARCHITECTURE — DEEP DIVE

Event Sourcing vs Change Data Capture vs Event Streaming

text

EVENT PATTERNS COMPARED	
Event Sourcing: State = Replay(All Events)	
["added", "changed", "deleted"]	
Source of truth = Event Store	
Change Data Capture: Monitor DB Log → Publish Events	
Debezium → Kafka	
Non-invasive, existing DB stays source	
Event Streaming: Kafka/Kinesis/Pulsar	
Infinite replay window	
Real-time + batch in one	

When to Use Which

Pattern When to Choose

Event Sourcing Need full audit trail, time travel, debugging past states

CDC Existing monolith, can't change application code

Event Streaming Real-time analytics, data pipelines, Kafka as backbone

Idempotent Event Processing — Deep Dive

text

IDEMPOTENT EVENT PROCESSOR PATTERN	
Consumer:	
1. Receive event { id: "evt-123", type: "order.created" }	
2. Check Redis SETNX "processed:evt-123"	
3. If redis says "already seen" → SKIP	
4. If new → process + store result in DB	
5. Atomic commit: DB update + Redis mark	
Exactly-Once Semantics = Idempotent Consumer + Transactional	
Outbox + Kafka idempotent producer	

TOPIC 3: API GATEWAY — ADVANCED PATTERNS

Gateway Responsibilities — Complete Map

text



Gateway Implementation Comparison

Gateway Best For Notable Features

Kong Enterprise Plugin ecosystem, DB-backed

NGINX High performance Low latency, small memory

Envoy Service mesh integration xDS protocol, gRPC native

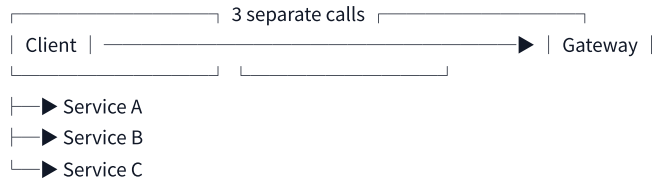
Spring Cloud Gateway Java ecosystem Reactive, Spring integrated

AWS API Gateway Serverless Lambda integration, managed

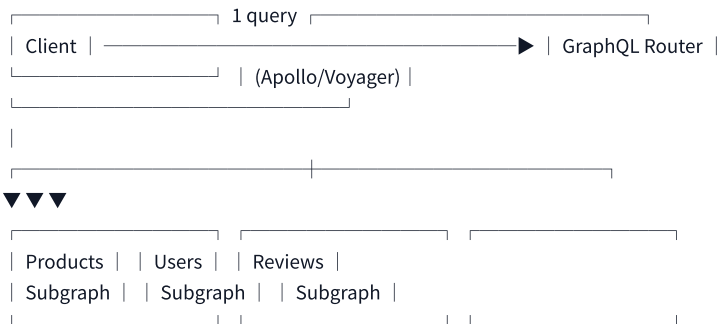
GraphQL Federation vs API Gateway Aggregation

text

API Gateway Aggregation:



GraphQL Federation:



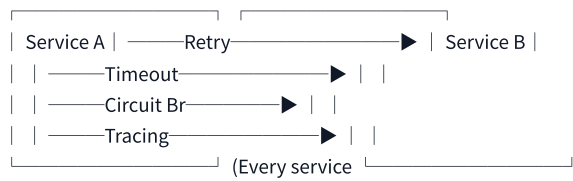
Interview Insight: "API Gateway aggregates at request level; GraphQL federation aggregates at field level with type safety."

TOPIC 4: SERVICE MESH — ENVOY, ISTIO, LINKERD

What Problem Service Mesh Solves

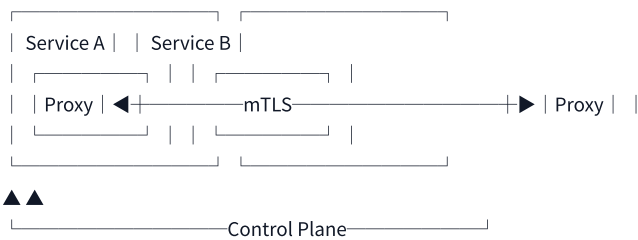
text

Without Service Mesh:



implements its own!)

With Service Mesh:



(Istio/Pilot, Linkerd control)

Control Plane vs Data Plane

Plane Responsibility Components

Data Plane Handles actual traffic Envoy proxies (sidecars)

Control Plane Manages configuration Pilot, Mixer, Citadel (Istio)

Service Mesh Features Matrix

Feature Envoy Istio (on Envoy) Linkerd

mTLS ✓✓✓

Retry/Timeout ✓✓✓

Circuit Breaking ✓✓✓

Traffic splitting ✓✓✓

Fault injection Limited ✓ Limited

Complexity Medium High Low

When to Adopt Service Mesh

text

✓ GOOD FIT:

- Many services (50+)
- Multiple languages (Java, Go, Python, Node)
- Security/compliance requiring mTLS
- Dedicated platform team exists

✗ NOT NEEDED:

- < 10 services
- Single language (library approach works)
- No security requirements
- Small team, no platform expertise

Interview Insight: "Service mesh is NOT for everyone. For 10-20 services, client libraries (Resilience4j, Hystrix) are simpler. At 50+ services, the standardization of service mesh pays off."

TOPIC 5: STRANGER PATTERN — MONOLITH TO MICROSERVICES

The Six Stranger Pattern Strategies

text



- | 2. STRANGLER FIG (MARTIN FOWLER) |
- | Phase 1: Intercept and redirect specific routes |
- | Phase 2: Gradually move functionality |
- | Phase 3: Retire old system |
- | |
- | 3. BUBBLE CONTEXT (DDD) |
- | Create anti-corruption layer between bounded contexts |
- | |
- | 4. SAGA OF SPLITTING |
- | Move one table → Migrate one use case → Iterate |
- | |
- | 5. EVENT INTERCEPTION |
- | Monolith publishes events → Services consume |
- | |
- | 6. DATA SYNCHRONIZATION |
- | Dual-write during transition → Switch when ready |
- | |

Strangler Fig — Detailed Timeline

text

- Week 1-2: |
- | Add proxy in front of monolith |

- Week 3-6: |
- | Route /customers/* to new service |
 - | Everything else → monolith |

- Week 7-10: |
- | Route /orders/* to new service |
 - | Keep /reports/* on monolith |

- Week 11-14: |
- | Route /reports/* → new service |
 - | Monolith = empty shell |

- Week 15: |
- | Decommission monolith |

Interview Insight: "Never do Big Bang rewrites. The Strangler Pattern is the only proven safe path to microservices."

TOPIC 6: DORA METRICS & MICROSERVICES MATURITY

The Four Key DORA Metrics

Metric Definition Target (Elite)

Deployment Frequency How often deploy to production Multiple times/day

Lead Time for Changes Code commit → deployment < 1 hour

Time to Restore Incident → resolution < 1 hour

Change Failure Rate % of deployments causing incidents 0-15%

Microservices Maturity Model

text

Level 1: MONOLITH

- |— Single deployment unit
- |— Shared database
- |— Deploy frequency: Monthly

Level 2: MICROSERVICES NAIVE

- |— Split by technical layers (not domains)
- |— Shared database hidden
- |— Synchronous calls everywhere
- |— Deploy frequency: Weekly

Level 3: DOMAIN-DRIVEN

- |— Split by business domains
- |— Database per service
- |— Hybrid sync/async

- └─ Saga for transactions
- └─ Deploy frequency: Daily

Level 4: OPERATIONALLY EXCELLENT

- └─ Fully independent deployments
- └─ Blue-green/canary
- └─ Service mesh
- └─ Platform engineering team
- └─ DORA Elite metrics
- └─ Deploy frequency: Multiple times/day

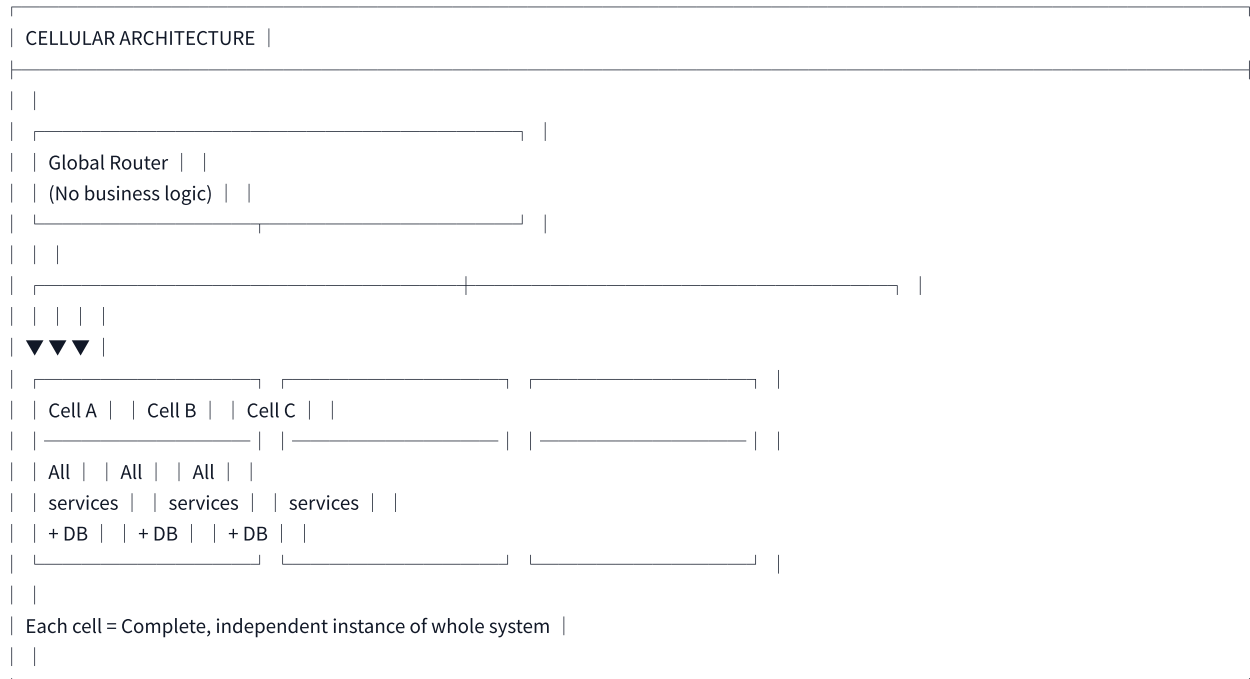
Level 5: CELLULAR ARCHITECTURE

- └─ Self-contained cells
- └─ Regional isolation
- └─ Chaos engineering in production
- └─ Deploy frequency: Hourly

TOPIC 7: CELLULAR ARCHITECTURE — THE NEXT STEP

What is Cellular Architecture

text



Why Cellular > Traditional Microservices

Aspect Traditional Microservices Cellular Architecture

Blast radius Full system Single cell

Deployment risk Global Per cell (canary cells)

Scaling Per service Whole cells

Regional failover Complex Built-in (cells per region)

Tenancy Hard Easy (tenant → cell mapping)

Complexity High per service High per cell but isolated

Companies Using Cellular Architecture

Company Scale Why Cells

Uber Thousands of services Blast radius containment

Netflix Global streaming Regional isolation

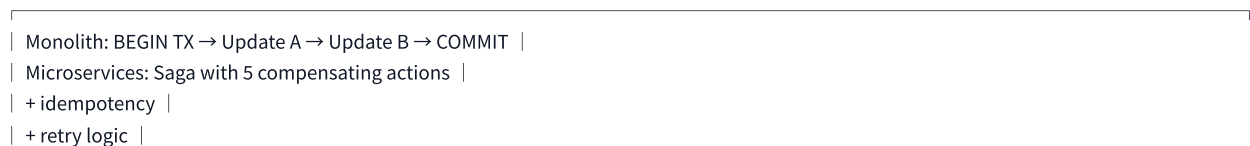
DoorDash 10K+ microservices Developer independence

TOPIC 8: DATABASE PER SERVICE — DEEP TRADE-OFFS

The Hidden Costs

text

Cost 1: Distributed Transactions



| + dead letter queues |

Cost 2: Reporting

| Solutions: |
	—— CQRS (separate read models)
	—— Data warehouse (ETL from all services)
	—— Materialized views across services
	—— Kafka + Flink for real-time analytics

Cost 3: Shared Reference Data

| Problem: Product catalog shared across 10 services |
| |
| Solutions: |
	—— Product Service as source + cache elsewhere
	—— Replicate product data to each service (eventually)
	—— Library for product validation (breaking independence!)

When Database Per Service Fails

text

✗ Anti-patterns to recognize:

1. Too many join operations across services
→ Suggests wrong service boundaries
2. Report queries constantly aggregating
→ Needs CQRS or data warehouse
3. Shared reference data changing rapidly
→ Cache invalidation becomes nightmare
4. Transactional boundaries span 5+ services
→ Maybe the monolith was better

TOPIC 9: CHAOS ENGINEERING

Principles of Chaos Engineering

text

| CHAOS ENGINEERING MATURITY MODEL |

| Level 0: No testing |

| Level 1: Test in staging only |

| Level 2: Test in production with small blast radius |

| Example: Kill one pod out of 1000 |

| Level 3: Production experiments with automated rollback |

| Example: 2% latency injection, rollback if error rate >0.1% |

| Level 4: Game days (simulate major outages proactively) |

| Example: Simulate entire region failure |

Common Chaos Experiments

Experiment Target Success Criteria

Pod kill Kubernetes deployment Self-healing, no user impact

Latency injection Service dependency Timeout + circuit breaker works

Database failover Primary → replica Zero downtime failover

Network partition Service A ↔ Service B Graceful degradation

Resource exhaustion CPU/Memory limits Throttling, not crash

Certificate expiry mTLS certs Auto-rotation works

Tools

Tool Use Case Company

Chaos Monkey Kill instances Netflix
Gremlin Commercial platform Gremlin Inc
Litmus Kubernetes native ChaosNative
PowerfulSeal Pod/VMi chaos Bloomberg
Interview Insight: "Chaos engineering isn't about breaking things randomly. It's about building confidence in system resilience by running controlled experiments."

TOPIC 10: PLATFORM ENGINEERING FOR MICROSERVICES

Internal Developer Platform (IDP) — Golden Paths

text

PLATFORM AS A PRODUCT — GOLDEN PATHS		
Developer Experience		
▼		
SELF-SERVICE PLATFORM		
"I want a new service with:		
• REST API		
• Kafka consumer		
• PostgreSQL DB		
• Canary deployment"		
Platform provisions:		
✓ Repo + CI pipeline		
✓ Service registry entry		
✓ Database (isolated)		
✓ Monitoring dashboards		
✓ Tracing configured		
▼		
Infrastructure		

Platform Team Responsibilities

Responsibility Without Platform With Platform

Kubernetes clusters Each team learns k8s Abstracted away

Service discovery Complex setup Built-in

Observability Self-configure Auto-instrumented

Secrets management DIY Vault integration

CI/CD Each team builds Standardized pipelines

When to Build Platform

text

Small team (<20 devs): NO platform → Too much overhead

Growing team (20-100 devs): ONE platform engineer → Start tooling

Mature org (100+ devs): Dedicated platform team → 1:10 ratio

Large org (500+ devs): Platform as internal product → 1:20 ratio

📄 ADVANCED TOPICS QUICK REFERENCE

One-Line Summaries for Interview

Topic One-Line Summary

2PC Distributed ACID fails in microservices due to locks and network partitions

TCC Try-Confirm-Cancel gives short-term reservation before execution

Event Sourcing Store events, compute state; audit trail is the source of truth

CDC Listen to DB transaction logs to publish events without code changes

Service Mesh Offload resilience, observability, security to infrastructure layer

Strangler Fig Incrementally replace monolith by intercepting and rerouting requests

DORA Metrics Measure DevOps maturity: frequency, lead time, restore time, failure rate

Cellular Architecture Deploy complete independent cells for blast radius isolation

Chaos Engineering Run experiments to build confidence in system resilience

text

1. Cohesion over consistency
 - └── Eventual consistency is acceptable; strong coupling is not
2. Autonomy over orchestration
 - └── Services should decide their own fate, not be directed centrally
3. Observability over guessing
 - └── If you can't measure it, you can't improve it
4. Resilience over correctness
 - └── Better to be partially available than fully unavailable
5. Simplicity over completeness
 - └── The best architecture is the simplest one that meets requirements
6. Discovery over configuration
 - └── Services find each other; don't hardcode anything
7. Idempotency over retry
 - └── Without idempotency, retry is dangerous
8. Exponential backoff over immediate retry
 - └── Give failing systems time to recover
9. Circuit breaking over endless retry
 - └── Know when to stop calling
10. Platform over point solutions
 - └── Standardize the infrastructure, not the applications

INTERVIEW POWER PHRASES (Advanced Edition)

"Two-phase commit doesn't work in microservices because network partitions are unavoidable. We use Saga with compensating transactions for eventual consistency."

"Service mesh shifts complexity from application to infrastructure — you pay with operational overhead, but gain standardization across polyglot services."

"The Strangler Pattern is the only safe path to microservices. Anyone proposing a big bang rewrite hasn't experienced one."

"Database per service isn't dogma — it's a trade-off. You pay with distributed transactions and reporting complexity in exchange for independent scalability."

"Chaos engineering is about controlled experiments, not random breaking. We start with 1% blast radius and expand confidence over time."

"Cellular architecture is microservices' answer to blast radius: each cell fails independently, the system as a whole doesn't."

"Platform engineering is a product. Our customers are internal developers. We build golden paths, not golden cages."

TOP 3 MICROSERVICES INTERVIEW QUESTIONS — Complete Analysis

For 16-Year Experience Professional

QUESTION 1: HOW TO CONVERT MONOLITH TO MICROSERVICES

Quick Analysis (30-Second Answer)

"Convert incrementally using the Strangler Fig Pattern. Never big bang. Start with identifying bounded contexts, extract one service at a time, use anti-corruption layer, and gradually redirect traffic. The goal is zero downtime and continuous delivery throughout migration."

Detailed Answer (5-Minute Interview Response)

Phase 1: Assessment & Planning (Weeks 1-4)

MIGRATION ASSESSMENT FRAMEWORK

Step 1: Identify Bounded Contexts (DDD)

Look for:

- Naturally isolated business capabilities
- Teams already organized around functions
- Database tables with clear ownership
- Low cross-domain transaction frequency

Step 2: Score Candidates for Extraction

Criteria:

- Business value (high = extract early)
- Technical complexity (low = extract early)
- Independence from monolith (high = extract early)
- Team ownership (clear = extract early)

Step 3: Understand Dependencies

Tools:

- Code analysis (JDepend, SonarQube)
- Runtime tracing (distributed tracing)
- Database foreign key analysis
- Team interviews

Phase 2: Strangler Fig Pattern Implementation

STRANGLER FIG – STEP BY STEP

STEP 1: Add Proxy Layer

Client → Proxy/API Gateway → Monolith
(routes all traffic)

STEP 2: Extract First Service + Redirect

Client → Proxy └─→ Payment Service (new)
 └─→ Monolith (others)

- /payments/* → Payment Service
- /* → Monolith

STEP 3: Extract More Services

Client → Proxy └─→ Payment Service
 └─→ Order Service
 └─→ Inventory Service
 └─→ Monolith (shrinking)

STEP 4: Decommission Monolith

Client → Proxy └─→ Payment Service
 └─→ Order Service
 └─→ Inventory Service
 └─→ Shipping Service

Monolith: DECOMMISSIONED ✓

Concrete Example: E-Commerce Migration

Before — Monolith Structure

```
// MONOLITH: Single application with all features
@SpringBootApplication
public class ECommerceMonolith {
    // Contains: Order, Payment, Inventory, Shipping, User, Review
}

// Single database with all tables
// orders, payments, inventory, shipments, users, reviews

// Tightly coupled code
@Service
public class OrderService {
    @Autowired
    private PaymentService paymentService; // Direct dependency

    @Autowired
    private InventoryService inventoryService; // Direct dependency

    public Order createOrder(OrderRequest request) {
        // All in one transaction
        inventoryService.reserveStock(request.getItems());
        paymentService.processPayment(request.getPayment());
        Order order = orderRepository.save(createOrder(request));
        shippingService.createShipment(order);
        return order;
    }
}
```

Step 1: Extract Payment Service


```

// NEW PAYMENT SERVICE (Independent Microservice)
@SpringBootApplication
@EnableDiscoveryClient
public class PaymentServiceApplication {
    // Own database: payment_db
}

@RestController
@RequestMapping("/api/payments")
public class PaymentController {

    @PostMapping("/process")
    public PaymentResponse process(@RequestBody PaymentRequest request) {
        // Payment logic only
        return paymentService.process(request);
    }

    @PostMapping("/refund")
    public RefundResponse refund(@RequestBody RefundRequest request) {
        return paymentService.refund(request);
    }
}

// MODIFIED MONOLITH – Call Payment Service via HTTP
@Service
public class OrderService {

    // Remove direct dependency, add HTTP client
    private final PaymentClient paymentClient; // Feign/RestClient

    public Order createOrder(OrderRequest request) {
        // Call Payment Service via API
        PaymentResponse payment = paymentClient.process(request.getPayment());

        // Still in monolith
        inventoryService.reserveStock(request.getItems());
        Order order = orderRepository.save(createOrder(request));
        shippingService.createShipment(order);
        return order;
    }
}

```

Step 2: Anti-Corruption Layer (ACL)

```
// ANTI-CORRUPTION LAYER – Protects monolith from service changes
@Component
public class PaymentAntiCorruptionLayer {

    private final PaymentClient paymentClient;
    private final PaymentTransformer transformer;

    public PaymentResponse processPayment(MonolithPaymentRequest request) {
        // Transform from monolith model to service model
        PaymentServiceRequest serviceRequest =
transformer.toServiceModel(request);

        // Call service
        PaymentServiceResponse serviceResponse =
paymentClient.process(serviceRequest);

        // Transform back to monolith model
        return transformer.toMonolithModel(serviceResponse);
    }

    // Fallback when service is down
    public PaymentResponse fallbackPayment(MonolithPaymentRequest request) {
        return new PaymentResponse("PENDING", "Payment queued for processing");
    }
}
```

Step 3: Data Migration Strategy

```

// DATA MIGRATION – Dual Write Pattern
@Component
public class DataMigrationService {

    // Phase 1: Dual Write (Write to both)
    @Transactional
    public Order createOrder(OrderRequest request) {
        // Write to monolith database (legacy)
        Order order = monolithOrderRepository.save(createOrder(request));

        // Also write to new service via API
        CompletableFuture.runAsync(() -> {
            orderServiceClient.createOrder(order);
        });

        return order;
    }

    // Phase 2: Backfill Historical Data
    @Scheduled(cron = "0 0 2 * * *") // Daily at 2 AM
    public void backfillHistoricalData() {
        List<Order> oldOrders = monolithOrderRepository.findNotMigrated();
        for (Order order : oldOrders) {
            try {
                orderServiceClient.createOrder(order);
                order.setMigrated(true);
                monolithOrderRepository.save(order);
            } catch (Exception e) {
                log.error("Failed to migrate order {}", order.getId());
            }
        }
    }

    // Phase 3: Switch to New Service as Source of Truth
    @Transactional
    public Order createOrder(OrderRequest request) {
        // Write only to new service
        OrderResponse response = orderServiceClient.createOrder(request);

        // Optional: Write to monolith for rollback capability
        if (rollbackMode) {
            monolithOrderRepository.save(convert(response));
        }
    }
}

```

```
    }

    return convert(response);
  }
}
```

Common Pitfalls & Solutions

Pitfall	Symptom	Solution
Distributed transactions	Data inconsistency	Saga pattern + compensating transactions
Shared database	Tight coupling continues	Database per service + event-driven sync
Chatty APIs	Too many network calls	API composition + GraphQL/BFF
Fallback failures	Cascade failures	Circuit breaker + timeout + retry
Migration never ends	Monolith still running	Set hard deadline, incentivize completion

Interview Power Phrases

- "The Strangler Fig Pattern isn't just about technical migration—it's about business continuity. Each extracted service must provide value independently, so stakeholders see progress."
- "The Anti-Corruption Layer is your most important safety net. It isolates the monolith from changes in extracted services, allowing independent evolution."
- "Database migration is harder than code migration. Use the Dual Write pattern: write to both, verify consistency, then switch readers, finally stop writing to legacy."

QUESTION 2: PATTERNS IN MICROSOFT SERVICES

Quick Analysis (30-Second Answer)

"Microservices patterns fall into five categories: Decomposition (Database per Service, Bounded Context), Integration (API Gateway, Service Discovery, Circuit Breaker), Data Management (Saga, CQRS, Event Sourcing), Observability (Distributed Tracing, Log Aggregation), and Cross-cutting (Sidecar, Ambassador, Externalized Configuration)."

Detailed Answer — Complete Pattern Catalog

Pattern 1: Decomposition Patterns

DECOMPOSITION PATTERNS

1. DECOMPOSE BY BUSINESS CAPABILITY

Split based on business functions:

E-Commerce → Order, Payment, Inventory, Shipping

Banking → Account, Transaction, Loan, Customer

✓ Aligns with business structure

✓ Clear ownership

✗ May cause chatty APIs if domains are coupled

2. DECOMPOSE BY SUBDOMAIN (DDD)

- Identify Bounded Contexts

- Define Ubiquitous Language

- Map Context Maps

Insurance → Policy, Claims, Underwriting, Billing

✓ Most aligned with business logic

✓ Natural service boundaries

✗ Requires deep domain expertise

3. DECOMPOSE BY VERB/USE CASE

- Split by actions: Create, Update, Delete, Query

- CQRS naturally fits here

Example: OrderWrite Service, OrderRead Service

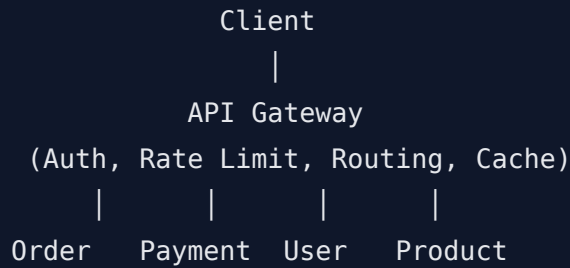
✓ Optimized scaling

✗ Complex consistency

Pattern 2: Integration Patterns

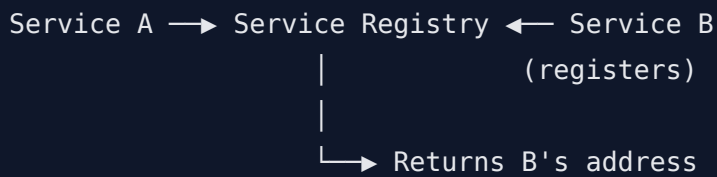
INTEGRATION PATTERNS

1. API GATEWAY



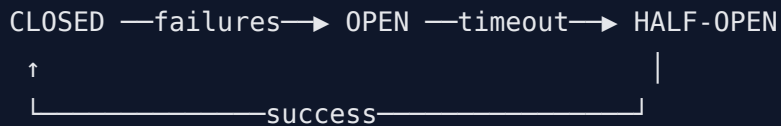
Use: Single entry point, cross-cutting concerns

2. SERVICE DISCOVERY



Use: Dynamic environments (K8s, cloud)

3. CIRCUIT BREAKER



Use: Prevent cascading failures

4. RETRY WITH BACKOFF

Attempt 1: — 100ms → Fail
Attempt 2: — 200ms → Fail


```
| | Attempt 3: — 400ms → Success ✓ | |
```

```
| | Use: Transient failures (network, timeout) | |
```

Pattern 3: Data Management Patterns

DATA MANAGEMENT PATTERNS

1. DATABASE PER SERVICE

Order Service → order_db (PostgreSQL)
Payment Service → payment_db (PostgreSQL)
User Service → user_db (MongoDB)

- ✓ Independent scaling, technology choice
- ✗ Distributed transactions

2. SAGA PATTERN (Choreography)

Order Created —event→ Payment Service
|
Payment Success
|
▼
Inventory Service
|
Stock Reserved
|
▼
Shipping Service

- ✓ Decentralized, scalable
- ✗ Hard to debug, no central view

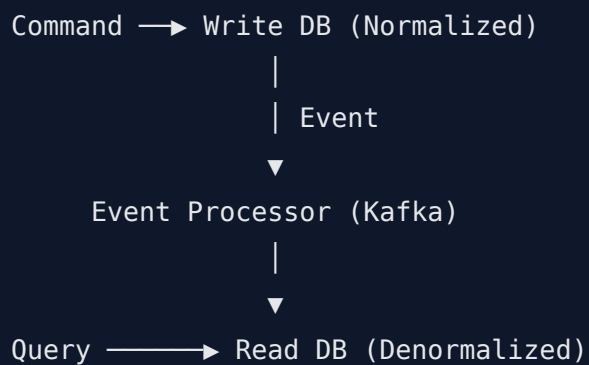
3. SAGA PATTERN (Orchestration)

Saga Orchestrator
|
├──
| ▼ ▼ ▼
Order Svc Payment Svc Inventory Svc

Compensating Actions
(refund, release, cancel)

- ✓ Central view, easier to manage
- ✗ Single point of coordination

4. CQRS (Command Query Responsibility Segregation)



- ✓ Optimized reads/writes independently
- ✗ Eventual consistency, duplication

Pattern 4: Observability Patterns

OBSERVABILITY PATTERNS

1. DISTRIBUTED TRACING

Trace ID: abc-123

Gateway (5ms) → Order (50ms) → Payment (200ms)

Span 1

Span 2

Span 3

Tools: Jaeger, Zipkin, OpenTelemetry

2. HEALTH CHECK API

GET /health

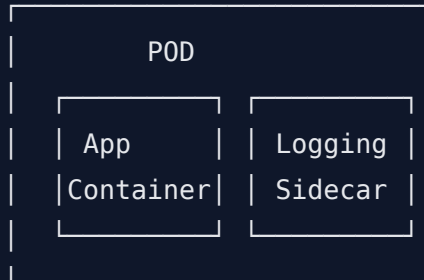
```
{
  "status": "UP",
  "components": {
    "database": { "status": "UP" },
    "redis": { "status": "UP" },
    "kafka": { "status": "DOWN" }
  }
}
```

Use: Kubernetes liveness/readiness probes

Pattern 5: Cross-Cutting Patterns

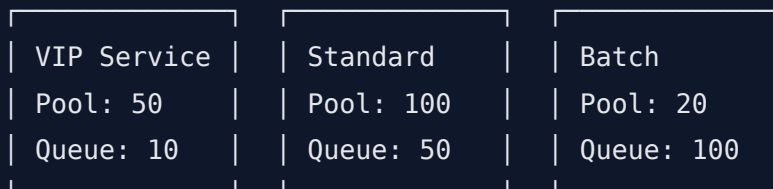
CROSS-CUTTING PATTERNS

1. SIDECAR PATTERN



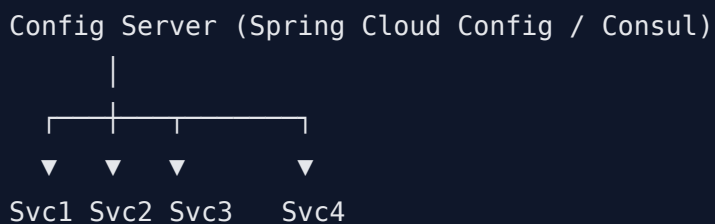
- ✓ Separation of concerns, reusable
- ✗ Resource overhead

2. BULKHEAD PATTERN



- ✓ Isolates failures, guarantees resources
- ✗ Complex configuration

3. EXTERNALIZED CONFIGURATION



- ✓ No hardcoded config, environment-specific

x Extra infrastructure

Complete Pattern Selection Guide

PATTERN SELECTION DECISION TREE

- Q1: How to split the monolith?
- Decompose by Business Capability
 - Decompose by Subdomain (if DDD expert available)
- Q2: How do services communicate?
- Need immediate response? Use Sync (API Gateway)
 - Can wait? Use Async (Event-Driven)
- Q3: How to handle service failures?
- Use Retry + Circuit Breaker + Timeout + Fallback
- Q4: How to manage distributed transactions?
- Simple workflow? Saga Choreography
 - Complex workflow? Saga Orchestration
- Q5: Read vs Write performance issues?
- Use CQRS + Event Sourcing
- Q6: Need to share cross-cutting concerns?
- Use Sidecar pattern
- Q7: How to find services dynamically?
- Service Discovery (Client-side or Server-side)

QUESTION 3: ADVANTAGES, DISADVANTAGES & ANTI-PATTERNS

Quick Analysis (30-Second Answer)

"Advantages: independent deployment, technology diversity, team autonomy, fault isolation, elastic scaling. Disadvantages: distributed system complexity, operational overhead, data consistency challenges, network latency, debugging difficulty. Anti-patterns: shared database, distributed monolith, too fine-grained, sync by default, ignoring fallbacks."

Detailed Answer — Complete Analysis

Advantages (Pros)

ADVANTAGES DETAILED

1. INDEPENDENT DEPLOYMENT

- ✓ Deploy Payment Service without touching Order
- ✓ Rollback single service
- ✓ Reduced deployment risk
- ✓ Faster time-to-market

Real Example: Amazon deploys every 11.7 seconds

2. TECHNOLOGY DIVERSITY

- ✓ Use Java for Order Service
- ✓ Use Go for Gateway (high concurrency)
- ✓ Use Python for ML Service
- ✓ Use Node.js for Real-time Service
- ✓ Choose right tool for each job

3. TEAM AUTONOMY & SCALING

- ✓ Each team owns service end-to-end
- ✓ No coordination between teams for features
- ✓ Scale teams independently (Amazon's two-pizza teams)
- ✓ Hire specialists per service

4. FAULT ISOLATION

Payment Service fails → Order Service degrades
Product Service → Still works!
User Service → Still works!

- ✓ Partial failures don't bring down whole system

5. ELASTIC SCALING

Black Friday: Scale Order Service to 200 instances

Normal day: Scale down to 10 instances

Payment Service: Always 50 instances

Reporting Service: 2 instances

✓ Scale only what needs scaling

✓ Cost optimization

6. SMALL CODEBASE

✓ Easier to understand (10K lines vs 500K)

✓ Faster builds (30 seconds vs 30 minutes)

✓ Faster onboarding

✓ Easier refactoring

Disadvantages (Cons)

DISADVANTAGES DETAILED

1. DISTRIBUTED SYSTEM COMPLEXITY

- x Network latency (10-100x slower than in-memory)
- x Partial failures
- x Network partitions (CAP theorem)
- x Distributed transactions

Example: Monolith: 5ms response → Microservices: 50ms

2. OPERATIONAL OVERHEAD

- x Need service discovery
- x Need API gateway
- x Need distributed tracing
- x Need log aggregation
- x Need circuit breakers
- x Need container orchestration

Team size needed:

Monolith: 5 developers

Microservices: 5 devs + 2 platform engineers

3. DATA CONSISTENCY CHALLENGES

- x No ACID transactions across services
- x Eventual consistency (stale reads)
- x Complex Saga patterns
- x Data duplication required (CQRS)

Example: Order created but inventory not reserved

Solution: Saga with 5 compensating actions

4. DEBUGGING COMPLEXITY

- x Stack traces across services

- x Hard to reproduce bugs
- x Multiple log files to search
- x Distributed tracing required

Monolith: `grep error.log`

Microservices: Search 50 services × 10 instances

5. TESTING COMPLEXITY

- x Integration tests across services
- x Contract testing needed
- x End-to-end tests are slow and flaky
- x Need test environments with all services

Time to run tests:

Monolith: 10 minutes

Microservices: 60+ minutes

6. VERSIONING & DEPENDENCY MANAGEMENT

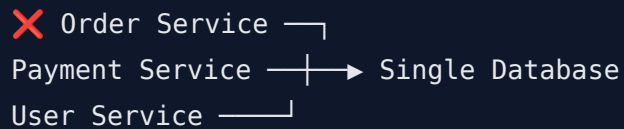
- x Service A uses API v1 of Service B
- x Service C uses API v2 of Service B
- x Need backward compatibility
- x Breaking changes require coordinated rollout

Solution: API versioning + consumer-driven contracts

Anti-Patterns (What NOT to Do)

ANTI-PATTERNS – AVOID THESE

ANTI-PATTERN 1: SHARED DATABASE



Problem: Tight coupling, single point of failure, no independent scaling, schema changes affect all services

Solution: Database per service + event-driven sync

ANTI-PATTERN 2: DISTRIBUTED MONOLITH

- ✗ Services split technically, not by business
- ✗ Deployed separately but must upgrade together
- ✗ Synchronous calls chain (A→B→C→D→E)

Symptoms:

- Changing one service requires changing 5 others
- Can't deploy independently
- High network latency

Solution: Proper bounded contexts, async where possible

ANTI-PATTERN 3: NANO-SERVICES (Too Fine-Grained)

- ✗ 200 services for a simple e-commerce site
- ✗ Each service has 2-3 endpoints
- ✗ Single request calls 15 services

Problem:

- High network overhead

- Hard to manage deployment
- Complex monitoring
- Team coordination overhead

Solution: Start with macro-services (10-50 lines)
Split only when necessary

ANTI-PATTERN 4: SYNCHRONOUS BY DEFAULT

❌ Order → Payment → Inventory → Shipping → Notification
(ALL SYNCHRONOUS)

Problem:

- User waits for entire chain
- Single failure kills whole request
- Low scalability

Solution: Async for non-critical path
(Notifications, emails, reporting)

ANTI-PATTERN 5: NO CIRCUIT BREAKER / RETRY

❌ Direct calls with no protection

Problem:

- Cascading failures
- Thread pool exhaustion
- Whole system goes down

Solution: Timeout + Retry + Circuit Breaker + Fallback

ANTI-PATTERN 6: VENDOR LOCK-IN

- ❌ All services use proprietary message bus
- ❌ All services use vendor-specific APIs

Problem: Can't migrate, expensive, vendor dictates

Solution: Use standards (OpenTelemetry, Prometheus,

Kafka vs proprietary)

ANTI-PATTERN 7: IGNORING FALLBACKS

❌ Payment fails → Order fails → User sees error

Better Approach:

- Payment fails → Mark order as "Payment Pending"
- Notify user, retry later
- Show partial data from cache

Solution: Always provide degraded but functional UI

Quick Reference Card

Aspect	Pro	Con	Anti-Pattern
Deployment	Independent	Complex orchestration	Distributed monolith
Data	Technology choice	Consistency	Shared database
Scale	Elastic per service	Network overhead	Nano-services
Resilience	Fault isolation	Partial failures	No circuit breaker
Performance	Parallel processing	Network latency	Sync by default
Debugging	Small codebase	Distributed traces	No observability

Interview Power Phrases

- "Microservices solve organizational problems more than technical ones. Conway's Law says system architecture mirrors communication structure—if your teams can't communicate, microservices won't fix that."
- "The biggest anti-pattern is the distributed monolith. You get the worst of both worlds: network latency AND coordinated deployments."

"Start with a monolith. Seriously. Extract services only when you have proven need—different scaling requirements, different teams, or different tech stacks."

"Database per service is non-negotiable. If services share a database, you don't have microservices—you have a distributed monolith with extra network calls."

"Fallbacks are not optional. Every service dependency must answer: 'What happens when this call fails?' If you can't answer, you're not production-ready."

Final Comparison: Monolith vs Microservices

MONOLITH VS MICROSERVICES – TRADE-OFFS		
DIMENSION	MONOLITH	MICROSERVICES
Deployment	Unified	Independent
Startup time	Slow (10 min)	Fast (30 sec)
Scalability	All or nothing	Per service
Tech stack	Single	Polyglot
Team structure	Centralized	Autonomous
Development speed	Fast initially	Fast at scale
Debugging	Easy (local)	Hard (distributed)
Consistency	Strong	Eventual
Network calls	0	Many
Operational cost	Low	High
Best for	Startups, MVP	Large orgs, scale

Sync vs Async Communion

22 April 2026 18:30

Sync vs Async Communication in Microservices

Core Idea

In microservices, services communicate either:

- **Synchronous (Sync):** Immediate request-response
- **Asynchronous (Async):** Fire-and-forget via events/messages

Choosing correctly impacts scalability, latency, and system resilience.

Synchronous Communication

What it is

A service calls another service and **waits for response**.

Examples

- REST APIs
- gRPC calls

When to use Sync

Use synchronous communication when:

- Immediate response is required
- User-facing request/response flow
- Simple workflows
- Strong consistency is needed

Typical use cases

- Login authentication
- Payment confirmation check
- Fetching user profile
- Real-time validation (e.g., inventory check)

Downsides

- Tight coupling between services
- Higher latency (blocking calls)
- Cascading failures risk
- Harder to scale under heavy load

Asynchronous Communication

What it is

Services communicate via **messages/events without waiting for response**.

Examples

- Kafka

- RabbitMQ
- Event streaming systems

When to use Async

Use asynchronous communication when:

- Processing can be delayed
- High scalability is required
- Loose coupling is needed
- Event-driven workflows exist

Typical use cases

- Order processing pipelines
- Email/SMS notifications
- Payment processing workflows
- Logging and audit events
- Data synchronization between services

Downsides

- Eventual consistency
- Harder debugging
- Requires idempotency handling
- Complex event tracking and retries

How to Decide (Decision Framework)

Step 1: Is immediate response required?

- YES → Sync
- NO → Async

Step 2: Is it user-facing critical path?

- YES → Sync
- NO → Async

Step 3: Can processing be delayed?

- YES → Async
- NO → Sync

Step 4: Do we expect high load/scalability needs?

- YES → Async preferred
- NO → Sync acceptable

Step 5: Is strong consistency required?

- YES → Sync
- NO → Async

Real-world Hybrid Example

Order Flow:

1. User places order → **Sync call** (validate order)
2. Order service creates order → **Async event**

3. Payment service processes → Async
 4. Notification service sends SMS/email → Async
- 🔑 Best systems are usually **hybrid**, not purely sync or async.

Key Trade-offs

Factor	Sync	Async
Latency	Low (immediate)	Higher (event delay)
Coupling	Tight	Loose
Scalability	Limited	High
Consistency	Strong	Eventual
Complexity	Simple	Complex

Interview Key Lines

- "Use synchronous communication for real-time user-driven flows."
- "Use asynchronous communication for scalability and decoupling."
- "Modern microservices systems are hybrid, balancing both patterns."

Mental Model

- Sync = phone call 📞 (wait for reply)
- Async = message/postbox 📧 (no waiting)

One-line Summary

Synchronous communication is used for immediate, user-critical operations, while asynchronous communication is preferred for scalable, decoupled, and event-driven processing where immediate response is not required.

Data decomposition

22 April 2026 18:32

Data Decomposition in Microservices — Quick Revision Flashcards

Basics

Q: What is data decomposition in microservices?

A: Splitting a single shared data model into multiple independent, service-owned data models aligned with business domains.

Q: Core principle of data decomposition?

A: Database per service — each microservice owns its own database.

Q: Why is shared database avoided?

A: It creates tight coupling, limits scalability, and prevents independent deployment.

Design Concepts

Q: On what basis is data decomposed?

A: Based on business domains (DDD - Domain Driven Design).

Q: What is data ownership in microservices?

A: Each service is the single source of truth for its own data.

Q: How do services communicate if they don't share DB?

A: Via APIs (sync) or events (async).

Example Mapping

Q: Example of Order Service data?

A: order_id, items, order_status

Q: Example of Customer Service data?

A: customer_id, name, address

Q: Example of Payment Service data?

A: payment_id, order_id, payment_status

Q: Example of Shipping Service data?

A: shipment_id, tracking_status

Communication

Q: Types of service communication?

A: Synchronous (REST/gRPC) and Asynchronous (Kafka/events)

Q: Example of event-driven flow?

A: OrderPlaced event → triggers Payment Service

Challenges

Q: Key challenge of data decomposition?

A: Eventual consistency instead of strong consistency

Q: Why are distributed transactions hard?

A: No single database manages all services

Q: What issue arises in reporting?

A: Data is distributed, so analytics requires separate pipelines

Solutions

Q: Pattern used for distributed transactions?

A: Saga pattern

Q: What is CQRS?

A: Separates read and write models for scalability

Q: Why use event sourcing/event-driven architecture?

A: To maintain consistency across distributed services

Interview Key Points

Q: One-liner for data decomposition?

A: Each microservice owns its data and schema independently.

Q: Why avoid shared DB in microservices?

A: To reduce coupling and enable independent scaling and deployment.

Q: How is consistency handled?

A: Through events and eventual consistency, not transactions.

Mental Model

Q: Monolith vs Microservices analogy?

A: Monolith = one shared brain  | Microservices = multiple independent brains 

Final Summary

Q: What is data decomposition in one line?

A: It is the practice of splitting data ownership across services to achieve independent scalability, deployment, and evolution using APIs and events instead of shared databases.

Circuit breaker + retry + fallback patterns

22 April 2026 18:34

Circuit Breaker, Retry & Fallback Patterns (Microservices)

Core Idea

In distributed microservices, failures are normal, not exceptions. To build resilient systems, we use:

- **Retry** → Try again when failure is temporary
- **Circuit Breaker** → Stop calling failing services
- **Fallback** → Provide alternative response when service fails

1. Retry Pattern

What it is

Automatically re-attempting a failed request assuming the failure is temporary.

When to use Retry

- Network glitches
- Temporary service overload
- Timeout errors
- Transient database issues

How it works

1. Call service
2. If failure occurs → retry
3. Stop after max attempts

Risks

- Can increase load on already failing system
- May worsen outages if misused
- Requires backoff strategy

Best Practices

- Use **exponential backoff**
- Add **jitter** (random delay)
- Limit retry count
- Retry only idempotent operations

2. Circuit Breaker Pattern

What it is

Prevents repeated calls to a failing service by "opening the circuit" after failures.

States

Closed

- Normal operation
- Requests pass through

Open

- Service is failing
- Requests are blocked immediately

Half-Open

- Testing recovery
- Limited requests allowed

When to use Circuit Breaker

- High dependency services
- External APIs
- Payment gateways
- Downstream microservices prone to failure

Benefits

- Prevents cascading failures
- Reduces load on failing services
- Improves system stability

Risks

- Incorrect thresholds may block healthy services
- Requires tuning

3. Fallback Pattern

What it is

Providing an alternative response when the primary service fails.

When to use Fallback

- Non-critical features
- Read-heavy systems
- Degraded service acceptable scenarios

Examples

- Cached data instead of live API
- Default response message
- Secondary service provider
- "Service temporarily unavailable" response

Benefits

- Better user experience
- System remains partially functional

Risks

- Stale data
- Inconsistent user experience
- Can hide real issues if overused

How They Work Together

Typical flow:

1. Client calls Service A
2. Service A calls Service B
3. If failure:
 - Retry first
 - If still failing → Circuit Breaker may open
 - If circuit open → Fallback response used

Comparison Table

Pattern	Purpose	Trigger	Outcome
Retry	Recover from transient failures	Temporary error	Re-attempt request
Circuit Breaker	Prevent system overload	Repeated failures	Block requests
Fallback	Maintain functionality	Service unavailable	Alternative response

Key Interview Insights

- Retry alone can amplify load during outages
- Circuit breaker prevents failure propagation
- Fallback ensures graceful degradation
- All three are often used together in production systems

Real-world Example (E-commerce)

Payment Flow:

- Retry: Retry payment gateway call on timeout
- Circuit Breaker: Stop calling gateway if failures spike
- Fallback: Mark order as "pending payment" and notify user

Mental Model

- Retry = knock again on door 🚪
- Circuit Breaker = stop knocking, door might be broken ⚡
- Fallback = leave a note and walk away 📝

One-line Summary

Retry handles temporary failures, circuit breaker prevents system overload from persistent failures, and fallback ensures graceful degradation when services are unavailable.

Timeout vs Retry vs Circuit Breaker — Decision Tree

22 April 2026 18:38

Timeout vs Retry vs Circuit Breaker — Decision Tree (Microservices)

Core Idea

In distributed systems, failures are expected. These three patterns work together to prevent cascading failures and improve resilience:

- **Timeout** → Don't wait forever
- **Retry** → Try again if failure is temporary
- **Circuit Breaker** → Stop calling a failing service

1. TIMEOUT — First Line of Defense

What it does

Limits how long a service call can take before it is aborted.

When to use Timeout

Always. Every remote call should have a timeout.

- REST/gRPC calls
- DB queries
- External APIs

Why it matters

- Prevents thread blocking
- Avoids resource exhaustion
- Stops infinite waiting

Key Rule

If you don't set timeout, system stability is already compromised.

2. RETRY — For Temporary Failures

What it does

Re-attempts failed operations assuming failure is transient.

When to use Retry

Use only when failure is likely temporary:

- Network glitches
- Temporary overload
- 5xx server errors
- Timeout from downstream service

When NOT to retry

- 4xx errors (client errors)
- Non-idempotent operations (unless carefully handled)



Best Practices

- Exponential backoff
- Add jitter (random delay)
- Limit retry attempts
- Retry only idempotent operations



3. CIRCUIT BREAKER — System Protection Layer



What it does

Stops calling a failing service to prevent cascading failures.



States



Closed

- Normal operation
- Requests pass through



Open

- Failures exceed threshold
- Requests are blocked immediately



Half-Open

- Limited test requests allowed
- Checks if service recovered



When to use Circuit Breaker

- Downstream services frequently failing
- External dependencies (payment gateways, third-party APIs)
- High traffic systems



DECISION TREE (Interview Gold)

Step 1: Should I wait for response?

- YES → Apply **Timeout**
- NO → Async flow (not covered here)

Step 2: Did the request fail?

- YES → Go to Retry decision
- NO → Done

Step 3: Is failure temporary?

- YES → Apply **Retry**
- NO → Skip retry

Step 4: Are failures frequent/repeated?

- YES → Activate **Circuit Breaker**
- NO → Continue normal retry logic

Step 5: Is service currently unavailable?

- YES → Use **Circuit Breaker + Fallback**
- NO → Proceed normally

HOW THEY WORK TOGETHER

Typical flow:

1. Client calls Service B
2. Timeout ensures call doesn't hang
3. If failure occurs → Retry logic kicks in
4. If failures persist → Circuit Breaker opens
5. When circuit is open → Fallback response is used

QUICK COMPARISON

Pattern	Purpose	Trigger	Effect
Timeout	Limit waiting time	Slow response	Abort request
Retry	Handle transient failures	Temporary error	Re-attempt call
Circuit Breaker	Prevent cascading failures	Repeated failures	Block requests

INTERVIEW INSIGHT

- Timeout is mandatory for every remote call
- Retry improves reliability but can increase load
- Circuit breaker protects system stability
- Fallback ensures graceful degradation

REAL-WORLD EXAMPLE (E-Commerce)

Payment Service Call:

- Timeout: 2 seconds max wait
- Retry: 2 attempts with backoff
- Circuit Breaker: opens after repeated gateway failures
- Fallback: mark order as "payment pending"

MENTAL MODEL

- Timeout = alarm clock  (stop waiting)
- Retry = knock again  (try once more)
- Circuit breaker = safety switch  (stop the system)

ONE-LINE SUMMARY

Timeout prevents indefinite waiting, retry handles temporary failures, and circuit breaker prevents system overload from persistent downstream issues.

Rate Limiter — Quick Explanation (Microservices)

What is a Rate Limiter?

A **rate limiter** controls the number of requests a client/user/service can make within a given time window.

 It protects systems from:

- Overload
- Abuse (DDoS-like spikes)
- Noisy neighbors

Why it is used

- Prevent backend crashes under high traffic
- Ensure fair usage among users/services
- Maintain system stability and SLA
- Protect expensive downstream dependencies

Where it is applied

- API Gateway (most common)
- Load balancer
- Application layer (service-level rate limiting)
- CDN / edge layer

Common Rate Limiting Algorithms

1. Fixed Window

- Counts requests in fixed time buckets (e.g., 100 req/min)

✓ Simple ✗ Can cause burst at window edges

2. Sliding Window

- Smooths request counting over time

✓ More accurate than fixed window ✗ Slightly more complex

3. Token Bucket

- Tokens are added at a fixed rate
- Each request consumes a token

✓ Allows bursts ✓ Widely used in production

4. Leaky Bucket

- Requests are processed at a constant rate
- Excess requests are queued or dropped

✓ Smooth output traffic ✗ Less flexible for bursts

Key Trade-offs

Aspect	Benefit	Drawback
Strict limiting	Protects system	May block valid users
Burst allowance	Better UX	Temporary overload risk
Distributed control	Scales well	Requires coordination

Key Interview Points

- Rate limiter prevents system overload and abuse
- Most systems use **token bucket at API gateway level**
- Can be implemented in-memory or distributed (Redis)
- Works alongside retries, circuit breakers, and timeouts

Real-world Example

API Gateway:

- 100 requests/sec per user
- Token bucket used to allow short bursts
- Excess requests return HTTP 429 (**Too Many Requests**)

Common HTTP Status Code

- **429 Too Many Requests** → Rate limit exceeded

Mental Model

- Rate limiter = traffic police 🚔
- It decides who goes, who waits, and who stops

One-line Summary

A rate limiter controls request flow to ensure system stability, fairness, and protection against overload by enforcing limits per user or service over time.

⚡ What is Burst in Rate Limiting?

🤖 Definition

****Burst**** refers to a short-term spike in requests that exceeds the normal allowed rate, but is temporarily permitted by the system.

It allows sudden traffic surges without immediately rejecting requests.

📄 Simple Meaning

If a system allows:

> 100 requests/sec

A burst might allow:

> 150–200 requests in a very short time window

As long as the long-term average stays within limits.

⚙️ Where Burst Comes From (Token Bucket Model)

Burst handling is most commonly explained using the ****Token Bucket algorithm****:

- Tokens are added at a steady rate (e.g., 10/sec)
- Each request consumes 1 token
- Bucket has a maximum capacity (e.g., 50 tokens)

💡 Key idea:

If tokens accumulate during low traffic, they can be used later all at once.

📊 Example

Configuration:

- Rate: 10 requests/sec
- Bucket capacity: 50 tokens

Scenario:

****1. Idle period****

- No requests → tokens accumulate up to 50

****2. Sudden spike****

- 50 requests arrive instantly
- All 50 are allowed → this is a burst

****3. After burst****

- System returns to steady rate (10/sec)

⚡ Why Burst is Allowed

Real-world traffic is not smooth. Bursts happen due to:

- User rapid clicks
- App retries after network reconnect
- Batch jobs or cron triggers
- Flash sales / sudden traffic spikes

Without burst handling:

- Valid users get throttled unfairly
- System becomes too strict and rigid

⚖️ Key Concept

Concept Meaning
----- -----
Rate limit Long-term request control
Burst Short-term flexibility using stored capacity

🧠 Mental Model

- Rate limit = speed limit on highway 🚗
- Burst = temporarily using buffer lanes during traffic spikes 🚧

🚀 One-line Summary

A burst is a temporary spike in traffic allowed by using stored capacity (like tokens) so that systems can handle real-world unpredictable request patterns smoothly.

Service discovery

22 April 2026 18:46

🎯 Service Discovery in Microservices

🤖 What is Service Discovery?

Service discovery is the mechanism that allows microservices to **find and communicate with each other dynamically**, without hardcoding IP addresses or hostnames.

In simple terms:

> Services don't "remember" where others are — they **ask a registry** where to find them.

🧩 Why Service Discovery is needed

In microservices:

- Instances scale up/down frequently
- IP addresses change dynamically
- Services run in containers/pods (Kubernetes, Docker, cloud)

🔑 Hardcoding service locations breaks quickly.

So we need a system that:

- Tracks available service instances
- Provides real-time location info
- Handles dynamic scaling

⚙️ How Service Discovery works

🕒 Step 1: Service Registration

When a service starts:

- It registers itself with a **Service Registry**
- Example: IP, port, health status

🕒 Step 2: Service Lookup

When Service A wants Service B:

- It queries the registry
- Gets a list of healthy instances

🕒 Step 3: Communication

- Service A calls Service B using returned address

📖 Types of Service Discovery

1. Client-Side Discovery

Client decides which instance to call.

Flow:

1. Service A queries registry
2. Gets list of Service B instances
3. Chooses one (load balancing logic)
4. Calls it directly

Example tools:

- Netflix Eureka
- Ribbon (client-side load balancing)

Example:

Comparison

Feature	Client-Side	Server-Side
-----	-----	-----
Who decides routing	Client	Load Balancer
Complexity	Higher	Lower
Control	More control	Centralized
Examples	Eureka + Ribbon	Kubernetes, AWS LB

Real-world Example

E-commerce system:

Services:

- Order Service
- Payment Service
- Inventory Service

Without service discovery:

- Hardcoded IP: `http://10.0.1.5:8080`
- Breaks when service restarts

With service discovery:

1. Payment Service registers itself
2. Order Service asks registry:
 - “Where is Payment Service?”
3. Registry returns healthy instances
4. Order Service calls dynamically

Key Challenges

- Registry can become a single point of failure (needs clustering)
- Latency in lookup (mitigated with caching)
- Health check accuracy is critical
- Complex in large distributed systems

Mental Model

- Service Discovery = “Google Maps for microservices” 
- Registry = directory of all services 
- Load Balancer = traffic controller 

One-line Summary

Service discovery enables microservices to dynamically find and communicate with each other using a registry or load balancer instead of hardcoded service locations.

Saga Pattern — Key Points Summary (Microservices)

What is Saga Pattern?

The **Saga Pattern** is a design pattern used to manage **distributed transactions across multiple microservices** without using a single ACID transaction.

👉 It ensures data consistency through a sequence of **local transactions + compensating actions**.

Why Saga Pattern is needed

In microservices:

- No shared database
- No global ACID transaction support
- Failures are common in distributed calls

👉 Problem:

How do we maintain consistency across services?

👉 Solution:

Break one big transaction into multiple small transactions + rollback logic

How Saga Pattern works

A saga is a sequence of steps:

Step 1: Local transaction

Each service performs its own transaction.

Step 2: Event or command triggers next step

Moves flow to next service.

Step 3: If failure occurs

Compensating transactions are executed to undo previous steps.

Example (Order Flow)

Step flow:

1. Order Service → Create Order
2. Payment Service → Deduct Money
3. Inventory Service → Reserve Stock
4. Shipping Service → Create Shipment

Failure scenario

If Inventory fails after payment:

- Payment must be rolled back (refund)
- Order marked as failed/cancelled

Types of Saga

1. Choreography (Event-driven)

- No central controller
- Services communicate via events

Flow:

Order Created → Payment Service listens

Payment Success → Inventory Service listens

Inventory Success → Shipping Service listens

✓ Simple ✓ Decentralized ✗ Hard to track flow in complex systems

2. Orchestration (Central Controller)

- Central saga orchestrator controls flow

Flow:

Saga Orchestrator → calls Order Service

→ calls Payment Service

→ calls Inventory Service

✓ Easier to manage ✓ Clear flow ✗ Central dependency

Key Concepts

Compensating Transaction

- Reverse action for each step
- Example: refund payment, release stock

Local Transaction

- Each service commits independently
- No distributed lock

Event-driven coordination

- Kafka / RabbitMQ commonly used

Saga vs ACID Transaction

Feature	ACID Transaction	Saga Pattern
Scope	Single DB	Multiple services
Consistency	Strong	Eventual
Locking	Yes	No
Scalability	Limited	High
Failure handling	Rollback	Compensating actions

When to use Saga Pattern

Use Saga when:

- Multiple microservices involved
- Distributed transactions required
- High scalability needed
- Eventual consistency is acceptable



Challenges

- Complex failure handling
- Debugging multi-step flows
- Designing correct compensating actions
- Handling partial failures



Interview Key Points

- Saga = distributed transaction management pattern
- Replaces ACID with eventual consistency
- Uses compensating transactions instead of rollback
- Two types: choreography and orchestration



Mental Model

- ACID = single chef cooking in one kitchen 🍳
- Saga = multiple chefs in different kitchens coordinating via messages 📧



One-line Summary

The Saga Pattern manages distributed transactions in microservices by breaking them into a sequence of local transactions with compensating actions to ensure eventual consistency.

Saga vs event driven

22 April 2026 19:31

Saga vs Event-Driven Architecture — Key Differences

Core Idea

Both Saga and Event-Driven Architecture (EDA) use **events for communication**, but they solve **different problems**:

- **Saga Pattern** → manages **distributed transactions + consistency**
- **Event-Driven Architecture** → enables **loose coupling + asynchronous communication between services**

What is Saga Pattern?

Saga is a **distributed transaction management pattern**.

🔑 It ensures data consistency across multiple microservices using:

- Local transactions
- Events or commands
- Compensating transactions (rollback-like behavior)

What is Event-Driven Architecture?

EDA is a **communication style where services emit and react to events asynchronously**.

🔑 Focus is on:

- Decoupling services
- Asynchronous processing
- Event propagation

Key Difference in Purpose

Aspect	Saga Pattern	Event-Driven Architecture
Primary goal	Data consistency across services	Loose coupling & async communication
Problem solved	Distributed transaction management	Service interaction model
Focus	Correctness of business workflow	Communication and scalability

Example Scenario (Order Flow)

Saga Pattern

Used when strict workflow consistency is needed:

1. Create Order
2. Deduct Payment
3. Reserve Inventory
4. If failure → compensate (refund, release stock)

🔑 Ensures end-to-end correctness

Event-Driven Architecture

Same flow, but focus is different:

1. OrderCreated event emitted
2. Payment Service reacts
3. Inventory Service reacts
4. Shipping Service reacts

🔗 No strict coordination, just event reactions

Relationship Between Them

🔗 Saga is often implemented using Event-Driven Architecture

But:

- Not all EDA systems are Sagas
- Not all Sagas are pure EDA (orchestration-based saga uses commands)

Types Comparison

Saga Types

- Choreography (event-driven)
- Orchestration (central controller)

Event-Driven Styles

- Pub/Sub model
- Event streaming (Kafka)
- Event notification systems

Side-by-Side Comparison

Feature	Saga	Event-Driven Architecture
Nature	Pattern	Architecture style
Purpose	Transaction management	Communication model
Consistency	Stronger (eventual + compensations)	Eventual consistency
Coupling	Moderate	Very low
Control flow	Defined workflow	No strict workflow

Key Interview Insight

- Saga = business workflow correctness across services
- EDA = system design for async communication
- Saga often *uses* EDA, but EDA is broader

Mental Model

- Saga = choreographed dance 🕺 (steps must complete or be undone)
- EDA = crowd reacting to signals 🗣️ (everyone responds independently)

One-line Summary

Saga is a pattern for managing distributed transactions, while Event-Driven Architecture is a broader communication style where services interact via events asynchronously.

⚡ Microservices Ultra Cheat Sheet (1-Page Revision)

🧠 CORE ARCHITECTURE IDEA

Microservices = independently deployable services with:

- Own database (data decomposition)
- Loose coupling
- Async + sync hybrid communication
- Resilience + scalability patterns

🔗 DATA DECOMPOSITION

- Each service owns its DB (no shared DB)
- Based on business domains (DDD)
- Ensures independent scaling + deployment

⚠ Trade-off:

- Eventual consistency
- Data duplication

📡 COMMUNICATION

⚡ Sync

- REST / gRPC
- Immediate response required
- Use for: login, validation, payments

⚡ Async

- Kafka / RabbitMQ
- Event-driven
- Use for: notifications, workflows, processing

🔗 Most systems = HYBRID

🔍 SERVICE DISCOVERY

- Dynamic service lookup
- Avoid hardcoded URLs

Types:

- Client-side (Eureka)
- Server-side (Kubernetes / Load Balancer)

🔍 RATE LIMITING

- Controls request traffic per user/service
- Prevents overload & abuse

Algorithms:

- Token Bucket (best for bursts)
- Fixed Window
- Sliding Window

RELIABILITY PATTERNS

Timeout

- Always set for remote calls
- Prevents hanging requests

Retry

- Handles transient failures
- Use exponential backoff + jitter
- Only for idempotent operations

Circuit Breaker

- Stops calling failing service States:
- Closed → Open → Half-Open

 Prevents cascading failures

SAGA PATTERN

- Distributed transaction management
- Replaces ACID with eventual consistency

Types:

- Choreography (event-driven)
- Orchestration (central controller)

Key concept:

- Compensating transactions (rollback logic)

EVENT-DRIVEN ARCHITECTURE

- Services communicate via events
- Loose coupling
- Asynchronous processing backbone

NOT same as Saga (But Saga often uses EDA)

SAGA vs EDA

Saga	Event-Driven
Transaction consistency	Communication model
Workflow control	Event propagation
Uses compensations	No built-in rollback

SYNC vs ASYNC

Sync	Async
Immediate response	Event-based
Tight coupling	Loose coupling
Lower scalability	High scalability

RESILIENCE STACK

Order of protection:

1. Timeout (stop waiting)

2. Retry (try again)
3. Circuit Breaker (stop calling)
4. Fallback (graceful response)

SYSTEM DESIGN FLOW (REAL WORLD)

1. API Gateway entry
2. Rate limiting applied
3. Service discovery resolves services
4. Sync call for critical flow
5. Async events for workflows
6. Saga ensures consistency
7. Retry + timeout + circuit breaker ensure resilience

KEY PATTERN MAP

- Data ownership → decomposition
- Traffic control → rate limiter
- Service lookup → discovery
- Communication → sync/async
- Consistency → saga
- Scalability → async + EDA
- Resilience → retry + timeout + circuit breaker

MENTAL MODEL

- API Gateway = front door 
- Rate limiter = security guard 
- Services = independent buildings 
- Kafka = postal system 
- Circuit breaker = emergency switch 
- Saga = workflow conductor 

ONE-LINE SUMMARY

Microservices architecture is a layered system combining decomposition, hybrid communication, event-driven design, and resilience patterns to achieve scalability, reliability, and independence at global scale.

Api gateway

22 April 2026 19:46

📖 Topic 1: API Gateway (Deep Dive)

🧠 What is an API Gateway?

An **API Gateway** is the single entry point into a microservices system that handles all incoming client requests and routes them to appropriate backend services.

👉 Think of it as the **front-door brain of the system**.

⚙️ Why API Gateway exists

Without API Gateway:

- Clients call multiple services directly ✗
- Security logic is duplicated ✗
- Tight coupling between client and services ✗

With API Gateway:

- Single entry point ✓
- Centralized control ✓
- Simplified client logic ✓

⚙️ Core Responsibilities

📋 1. Routing

Routes requests to correct microservices.

Example:

- `/orders` → Order Service`
- `/payments` → Payment Service`

📋 2. Authentication & Authorization

- Validates JWT / OAuth2 tokens
- Enforces roles and permissions

Example:

- Admin APIs vs User APIs access control

📋 3. Rate Limiting

- Controls request volume per user/service

- Prevents abuse and overload

(Protects backend from traffic spikes)

📦 4. Request Aggregation (BFF Pattern)
Combines multiple service responses into one.

Example:

Instead of 3 calls:

- Order Service
- Payment Service
- Shipping Service

🔗 Gateway returns a single combined response

⚡ 5. Load Balancing (Indirect)
Distributes traffic across service instances:

- Round robin
- Least connections
- Weighted routing

🌐 6. Edge Caching
- Caches frequent responses at the gateway
- Reduces backend load and latency

📊 7. Observability Layer
Central place for:

- Logs
- Metrics
- Tracing

🔗 Because ALL traffic flows through it

📄 Request Flow Example

```text

Client

↓

API Gateway

↓

Authentication (JWT validation)

↓

Rate Limiting check

↓

Routing → Order Service

↓

Order Service → Payment Service (internal call)

# Consistency models

22 April 2026 19:49

## # 🧠 Topic 2: Consistency Models in Distributed Systems

---

## # 🌀 What is Consistency?

Consistency defines **\*\*how and when all nodes in a distributed system see the same data after a write operation\*\***.

👉 In simple terms:

> "If I update data, when will everyone else see it?"

---

## # ⚡ Why Consistency Matters

In microservices + distributed systems:

- Data is spread across services
- Network delays are real
- Failures are normal

So we must define:

👉 How "up-to-date" the system should be

---

## # 🗃️ 1. Strong Consistency

### ## 🧠 Definition

After a write, **\*\*all reads immediately see the latest value\*\***.

---

### ## ⚙️ Behavior

- Write happens
- All nodes instantly reflect update
- No stale reads allowed

---

### ## 🔄 Example

Bank balance update:

- You transfer ₹1000
- Immediately every system shows updated balance

---

### ## ⚠️ Trade-offs

- Slower performance

- High coordination cost
- Lower availability in distributed systems

---

## ## 🧠 Mental Model

> “Everyone must agree immediately, even if it slows things down”

---

## # ⌚ 2. Eventual Consistency

### ## 🧠 Definition

System will \*\*become consistent over time\*\*, not immediately.

---

### ## ⚙️ Behavior

- Write happens in one node/service
- Other nodes update later asynchronously
- Temporary inconsistency allowed

---

### ## ✂️ Example

E-commerce order system:

- Order placed
- Payment or inventory updates arrive a few seconds later

---

### ## ⚠️ Trade-offs

- Stale reads possible
- Harder to reason about system state
- Better scalability and availability

---

## ## 🧠 Mental Model

> “Everyone will agree eventually, but not right now”

---

## # 🔗 3. Causal Consistency (Advanced)

### ## 🧠 Definition

Operations that are \*\*causally related are seen in the same order by all nodes\*\*.

---

### ## ⚙️ Behavior

- If A causes B → everyone sees A before B
- Independent operations may appear in different order

---

## ## 📌 Example

Social media:

- You post a comment
- Then reply to it

👁️ Everyone sees post before reply

---

## ## ⚠️ Trade-offs

- More complex than eventual consistency
- Less strict than strong consistency

---

## ## 🧠 Mental Model

> “Cause must always come before effect”

---

## # ⚖️ Comparison Table

| Model    | Speed  | Consistency      | Availability | Use Case                 |
|----------|--------|------------------|--------------|--------------------------|
| Strong   | Slow   | Immediate        | Lower        | Banking, payments        |
| Eventual | Fast   | Delayed          | High         | E-commerce, social feeds |
| Causal   | Medium | Partial ordering | High         | Messaging, collaboration |

---

## # 🏠 Real-world Mapping (Microservices)

### ## 🏠 Strong Consistency

- Payments
- Wallet systems
- Account balance

---

### ## 📦 Eventual Consistency

- Orders
- Inventory updates
- Notifications

---

### ## 💬 Causal Consistency

- Chat systems
- Comments + replies
- Collaboration tools (Google Docs style)

---

## # 🔄 Connection to Microservices

Consistency directly affects:

- Saga pattern (uses eventual consistency)
- Event-driven architecture
- Retry + compensation logic
- System design trade-offs

---

### # ⚠ Interview Trap Insight

If interviewer asks:

> “Why not always use strong consistency?”

👉 Answer:

- It reduces availability
- In distributed systems, network partitions are unavoidable (CAP theorem)

---

### # 🧠 Mental Model

- Strong = synchronized orchestra 🎻 (everyone plays same note together)
- Eventual = crowd gradually syncing claps 🖐
- Causal = conversation flow 💬 (reply must follow message)

---

### # 🚀 One-line Summary

Consistency models define how quickly and strictly distributed systems agree on data, ranging from strong immediate consistency to flexible eventual consistency optimized for scalability.



# CQRS

22 April 2026 19:56

## Markdown

### # 🔄 CQRS + Data Duplication (Clean Interview Notes)

---

### # ⚙️ What is CQRS?

CQRS = **Command Query Responsibility Segregation**

It separates a system into two parts:

- 🎯 **Command Model** → Handles writes (create, update, delete)
- 🎯 **Query Model** → Handles reads (fetch/display)

👉 Both models are independent and optimized for their purpose.

---

### # ⚙️ Core Idea

Instead of one shared model for everything:

- Write side focuses on **correctness + transactions**
- Read side focuses on **speed + query efficiency**

---

### # 📄 CQRS Flow

```text

User Request



Command (Write Operation)



Write Model (Source of Truth DB)



Event Published (Kafka / Queue)



Read Model Updated (Projection DB)



Query Request



Read Model Response

📊 Does Data Duplication happen in CQRS?

☑️ Yes — data duplication is expected

CQRS intentionally maintains two copies of data:

Write Model (authoritative data)

Read Model (optimized copy for queries)

👉 This duplication is by design, not a flaw

⚙️ Why duplication happens

Write model stores normalized transactional data

Read model stores denormalized, query-optimized data

Read models are created as projections of events

⚙️ Example

📊 Write Model (Source of Truth)

Focus: correctness, consistency, transactions

Plain text

Order Table

- order_id

- customer_id

- status

📊 Read Model (Query Optimized View)

Focus: fast reads, UI efficiency

Plain text

Order View

- order_id

- customer_name

- product_names

- order_status

👉 This is a duplicated + enriched version of data

📦 How duplication happens

Command updates Write Model

Write Model emits event (e.g., OrderCreated)

Event is published via Kafka / messaging system

Read Model consumes event

Read DB updates its own projection

Plain text

Write DB → Event → Read DB (Projection / Duplicate Data)

🏛️ Why duplication is acceptable

🚀 Performance

No complex joins required

Faster read responses

🔄 Scalability

Read and write systems scale independently

🌀 Flexibility

Multiple read models can exist for different use cases

⚠️ Trade-offs

✖️ Eventual consistency

Read data may lag behind write updates

✖️ Storage overhead

Same data stored multiple times

✖️ System complexity

Requires event handling + synchronization logic

🧠 Mental Model

📊 Write Model = official record ledger 📖

📊 Read Model = optimized display copy 📺

👉 Users never query the ledger directly — they see a shaped version of it.

🔗 Key Interview Insight

CQRS is NOT about avoiding duplication.

👉 It is about:

Accepting controlled duplication to achieve scalability, performance, and flexibility

🚀 One-line Summary

CQRS separates read and write models, and intentionally duplicates data in read models using event-driven projections to achieve high scalability and optimized read performance.

Obserbality

22 April 2026 20:02

Markdown

📊 Topic 5: Observability in Microservices

🗨️ What is Observability?

Observability is the ability to **understand what is happening inside a distributed system from the outside**, using logs, metrics, and traces.

👉 If something breaks in microservices:

> Observability tells you **what broke, where it broke, and why it broke**

⚙️ Why Observability is critical

In microservices:

- Many services interact
- Failures are distributed
- Debugging is not straightforward

👉 Without observability:

- You see symptoms ✖
- Not root cause ✖

🏛️ The 3 Pillars of Observability

📄 1. Logs

What it is:

- Event-based records of what happened

Example:

```text id="log"```

OrderService: OrderCreated for orderId=123

PaymentService: Payment failed due to timeout

Use:

Debugging individual events

Root cause investigation

📊 2. Metrics

What it is:

Numeric data over time

Example:

Request count/sec

Latency (ms)

Error rate (%)

Use:

Monitoring system health

Alerting

### 3. Traces

What it is:

End-to-end request journey across services

Example:

Plain text

User Request

→ API Gateway

→ Order Service

→ Payment Service

→ Inventory Service

Use:

Identify bottlenecks

Understand service dependencies

 How they work together

Plain text

Trace → shows request path

Metrics → shows system health

Logs → show detailed events inside each step

 Together they form full visibility

 Observability Architecture

Common tools:

Logs → ELK Stack (Elasticsearch, Logstash, Kibana)

Metrics → Prometheus + Grafana

Tracing → OpenTelemetry / Jaeger

 Why observability is hard in microservices

Too many services

Distributed failures

Network latency variability

Async event flows (Kafka, queues)

 Example (E-commerce checkout failure)

User says:

“Order failed”

Without observability:

You don't know why 

With observability:

Trace shows: Payment Service timeout

Logs show: Gateway timeout after 2s

Metrics show: Payment latency spike

 Root cause identified instantly

 Logs vs Metrics vs Traces

Type. Focus. Use Case

Logs. Events. Debugging

Metrics Numbers Monitoring

Traces. Flow End-to-end analysis

 Mental Model

Logs = diary entries 

Metrics = heartbeat monitor 

Traces = GPS route map 

 Connection to other patterns

Observability is critical for:

Circuit breaker tuning

Retry analysis

Rate limiter monitoring

Saga debugging

Event-driven systems troubleshooting

 One-line Summary

Observability is the combination of logs, metrics, and traces that provides full visibility into distributed microservices systems for debugging, monitoring, and performance analysis.

# Caching

22 April 2026 20:07

Markdown

## # 📁 Topic 6: Caching Strategies in Microservices

---

### # 🗨️ What is Caching?

Caching is the technique of **storing frequently accessed data in a fast storage layer** so future requests are served faster.

👉 Goal:

> Reduce latency + reduce load on backend systems

---

### # ⚙️ Why caching is needed

In microservices:

- Databases are slow compared to memory
- High traffic systems need fast responses
- Repeated reads are common (products, profiles, configs)

👉 Without caching:

- High DB load
- Higher latency
- Poor scalability

---

### # 🔗 Where caching is used

- API Gateway (edge caching)
- Application layer (service cache)
- Database query results
- CDN (static content caching)

---

## # 📁 Common Caching Patterns

---

### ## 📁 1. Cache Aside (Lazy Loading)

### 🗨️ How it works:

- Application checks cache first
- If miss → fetch from DB → store in cache

### Flow:

```text id="cache-aside"

Request → Cache → (Miss) → DB → Cache updated → Response

✓ Pros:

Simple

Most commonly used

✗ Cons:

First request is slow (cache miss)

📦 2. Write Through Cache

🧠 How it works:

Write goes to cache AND database simultaneously

Flow:

Plain text

Write → Cache → DB (synchronous)

✓ Pros:

Cache always up to date

Strong consistency

✗ Cons:

Slower writes

📦 3. Write Back (Write Behind)

🧠 How it works:

Write goes only to cache first

DB updated later asynchronously

Flow:

Plain text

Write → Cache → Async DB update

✓ Pros:

Very fast writes

✗ Cons:

Risk of data loss if cache fails

📦 4. Cache Refresh / TTL-Based

🧠 How it works:

Data expires after time (TTL)

Fresh data fetched on expiry

Example:

Product details cached for 5 minutes

🔗 Cache Invalidation Problem (BIG INTERVIEW TOPIC)

👉 Hardest problem in caching

Options:

TTL expiration

Event-based invalidation

Manual invalidation on updates

👉 Problem:

“When data changes, how do you ensure cache is updated correctly?”

🧠 Real-world Example

E-commerce Product Page

Product details → cached (high read traffic)

Price updates → invalidate cache

Inventory → real-time or short TTL cache

⚠️ Challenges in caching

Cache inconsistency

Stale data

Cache stampede (many requests on expiry)

Memory overhead

Invalidation complexity

🧠 Cache Stampede Problem

When cache expires:

Many requests hit DB at same time

🔑 Solution:

Locking

Request coalescing

Random TTL jitter

🔗 Cache in Microservices Context

Caching improves:

API Gateway performance

Service response time

Database scalability

🧠 Mental Model

Cache = fast memory desk 🧠

Database = slow library 📖

You don't go to library every time → you keep notes on desk

🚀 One-line Summary

Caching is a performance optimization technique that stores frequently used data in fast storage layers to reduce latency and backend load, using patterns like cache-aside, write-through, and write-back.

Idempotent

22 April 2026 20:14

IDENTITY: IDEMPOTENCY IN MICROSERVICES

What is Idempotency?

Idempotency means:

Executing the same operation multiple times produces the same result as executing it once.

👉 In simple terms: Repeated requests should NOT create duplicate side effects.

Why Idempotency is Important

In distributed systems:

Network failures happen

Requests get retried

Messages get duplicated (Kafka / queues)

Timeouts cause re-execution

👉 Without idempotency:

Duplicate payments 💰

Duplicate orders 🛒

Data corruption ✖

Example

✖ Non-idempotent operation

Operation: Add ₹100 to account

If executed 3 times:

₹100 → ₹200 → ₹300 ✖ (wrong in retry scenarios)

☑ Idempotent operation

Operation: Set balance = ₹1000

No matter how many times executed:

Result = ₹1000 ✔

Real-world Microservices Example

Order Service Scenario

✖ Without idempotency:

User clicks "Place Order" twice

Two orders are created

✔ With idempotency:

Same request is detected using request ID

Only one order is created

How Idempotency is Implemented

1. Idempotency Key

Client sends unique request ID

Server stores processed request IDs

Flow: Request (ID = 123) → Process → Store result

Request (ID = 123 again) → Return same result

2. Database Constraints

Unique keys (e.g., order_id UNIQUE)

Prevent duplicate inserts

3. State-based Updates

Use absolute updates instead of incremental updates

Example: SET balance = 1000 (not +100)

4. Message Deduplication

Kafka consumer tracks message IDs

Prevents duplicate processing

⚠ Where Idempotency is Critical

Payment systems 💰

Order creation 🛒

Inventory updates 📦

Event processing systems 📡

🚨 Problems Without Idempotency

Double billing

Duplicate orders

Inventory mismatch

Retry storms causing corruption

🔗 Relation with Retry Pattern

Retry + No Idempotency = ❌ Data corruption

Scenario

Result

Retry + Idempotent API

Safe ✔

Retry + Non-idempotent API

Risk of duplication ❌

🧠 Mental Model

Idempotent → Light switch 💡 (ON → ON → ON = same state)

Non-idempotent → Adding coins 🪙 (each action changes state)

🔗 Connection to Microservices

Idempotency is essential for:

Retry mechanisms 🔄

Event-driven systems 📡

Saga pattern 📋

Messaging systems (Kafka / RabbitMQ) 📬

💡 Interview Key Points

Idempotency ensures safe retries in distributed systems

Prevents duplicate side effects

Critical in payments and order systems

Works closely with retry and messaging systems

🚀 One-line Summary

Idempotency ensures that repeating the same operation multiple times does not change the result, making distributed systems safe against retries, failures, and duplicate events.

If you want next, I can: ➡ Convert ALL 7 topics into a single “microservices interview PDF booklet”

➡ Or start Topic 8 (Messaging guarantees: Kafka, at-least-once, exactly-once, DLQ)

Deployment models

22 April 2026 20:22

Markdown

📌 Deployment Models in Microservices (Combined Notes)

🧠 What are Deployment Models?

Deployment models define **how microservices are released, run, and managed in production environments**.

👉 In simple terms:

> "Where services run, how they scale, and how new versions are rolled out."

🧩 1. Monolithic Deployment (Baseline)

🧠 Definition

Entire application is deployed as a single unit.

⚙️ Characteristics

- One build → one deployment
- Single scaling unit
- Tight coupling

❌ Problems

- No independent deployment
- Hard scaling
- High risk during release

🧩 2. Microservices Deployment (Core Model)

🧠 Definition

Each service is deployed independently.

⚙️ Characteristics

- Independent deployment per service
- Independent scaling
- Separate CI/CD pipelines

- ##  Benefits
 - Faster releases
 - Fault isolation
 - Better scalability

3. Cloud-Based Deployment

- ##  Definition
 - Microservices deployed on cloud platforms.

- ##  Examples
 - AWS (EKS, ECS)
 - Azure (AKS)
 - Google Cloud (GKE)

- ##  Features
 - Auto scaling
 - High availability
 - Managed infrastructure

4. Container-Based Deployment

- ##  Definition
 - Services packaged as containers (Docker) and deployed via orchestration tools.

- ##  Stack
 - Docker → packaging
 - Kubernetes → orchestration

- ##  Benefits
 - Environment consistency
 - Portability
 - Easy scaling

5. Rolling Deployment

- ##  Definition
 - Gradual replacement of old version with new version.

⚙️ Flow

- Replace instances one by one
- No downtime

☑️ Pros

- Safe deployment
- Continuous availability

❌ Cons

- Mixed versions during rollout

🟦 6. Blue-Green Deployment

🗨️ Definition

Two identical environments:

- Blue = current version
- Green = new version

⚙️ Flow

- Deploy new version in Green
- Switch traffic from Blue → Green

☑️ Pros

- Instant rollback
- Zero downtime

❌ Cons

- Higher infrastructure cost

🟦 7. Canary Deployment

🗨️ Definition

New version is released to a small subset of users first.

⚙️ Flow

- 5% traffic → new version
- Monitor metrics
- Gradually increase traffic

- ##  Pros
 - Very low risk
 - Real user validation

- ##  Cons
 - Requires strong observability

8. Multi-Region Deployment

- ##  Definition
 - Services deployed across multiple geographic regions.

- ##  Purpose
 - Low latency
 - High availability
 - Disaster recovery

- ##  Example
 - US region
 - Europe region
 - Asia region

9. Hybrid Deployment

- ##  Definition
 - Combination of:
 - On-prem systems
 - Cloud systems

- ##  Use case
 - Legacy + modern systems coexist

10. Active-Passive Deployment

- ##  Definition
 - Only one environment actively serves traffic, others are standby.

- ##  Flow
 - ```text id="active-passive"

Active Region → Handles all traffic

Passive Region → Standby (failover backup)

☒ Pros

Simple design

Easy consistency management

☒ Cons

Underutilized resources

Slower failover

11. Active-Active Deployment

Definition

Multiple environments actively serve traffic at the same time.

Flow

Plain text

Region A → part of traffic

Region B → part of traffic

Region C → part of traffic

☒ Pros

High availability

Better performance

Fast failover

☒ Cons

Data consistency complexity

Conflict resolution challenges

Deployment Model Comparison

Model

Downtime

Risk

Cost

Complexity

Use Case

Monolithic

High

High

Low

Low

Legacy systems

Microservices

None

Medium

Medium

High

Modern systems

Rolling

None

Medium

Medium

Medium

Standard deployments

Blue-Green

None

Low

High

Medium

Critical apps

Canary

None

Very Low
Medium
High
Large-scale systems
Multi-Region
None

Very Low
Very High
Very High
Global systems
Active-Passive

Minimal
Low
Medium
Low

DR systems
Active-Active
None

Low
High
Very High
Global high-scale apps

🧠 Mental Model

Rolling = replacing parts while machine runs 🔧

Blue-Green = switching between two factories 🏭

Canary = testing with small group first 🐦

Active-Passive = backup standby system 🛡️

Active-Active = multiple live systems sharing load 🌐

🔗 Connection to Microservices

Deployment models directly affect:

Service discovery 🔍

Observability 📊

Resilience patterns ⚡

Saga workflows 📋

Scaling strategies 📈

💡 Interview Key Points

“Microservices enable independent deployment per service.”

“Blue-green and canary reduce production risk.”

“Active-active improves availability but increases complexity.”

“Deployment strategy depends on cost, scale, and consistency needs.”

🚀 One-line Summary

Deployment models define how microservices are released and operated in production, ranging from rolling updates to canary, blue-green, and multi-region active-active/passive strategies for scalability and reliability.

PART 1: CORE COMMUNICATION PATTERNS

Sync vs Async Communication

Core Idea

In microservices, services communicate either:

- **Synchronous (Sync):** Immediate request-response
- **Asynchronous (Async):** Fire-and-forget via events/messages

 Comparison Table

| Factor | Sync | Async |
|-------------|---------------------------|----------------------|
| Latency | Low (immediate) | Higher (event delay) |
| Coupling | Tight | Loose |
| Scalability | Limited | High |
| Consistency | Strong | Eventual |
| Complexity | Simple | Complex |
| Use when | Immediate response needed | Processing can wait |

 Decision Tree

text

Step 1: Immediate response required?

YES → Sync

NO → Step 2

Step 2: User-facing critical path?

YES → Sync

NO → Step 3

Step 3: Can processing be delayed?

YES → Async

NO → Sync

Step 4: High load/scalability needs?

YES → Async preferred

NO → Sync acceptable

Step 5: Strong consistency required?

YES → Sync

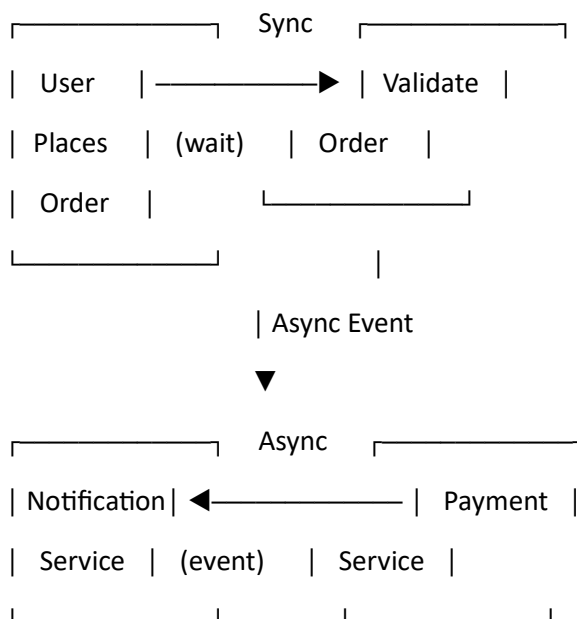
NO → Async

Mental Model

- **Sync** = Phone call (wait for reply)
- **Async** = Message/postbox (no waiting)

Hybrid Example (Order Flow)

text



PART 2: DATA MANAGEMENT

Data Decomposition

Core Principle

Database per service — each microservice owns its own database.

Why No Shared Database?

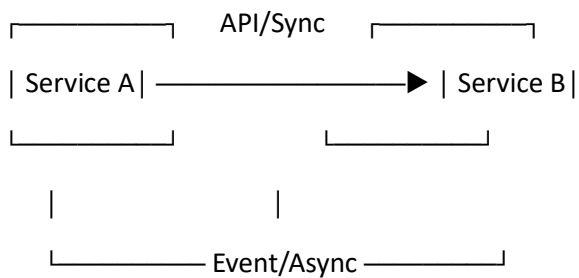
- Creates tight coupling
- Limits scalability
- Prevents independent deployment

Data Ownership Examples

| Service | Data Owned |
|----------|--------------------------------------|
| Order | order_id, items, order_status |
| Customer | customer_id, name, address |
| Payment | payment_id, order_id, payment_status |
| Shipping | shipment_id, tracking_status |

Communication Without Shared DB

text



Key Challenge: Eventual Consistency

Solution Patterns:

- **Saga Pattern** for distributed transactions
- **CQRS** for separating read/write models
- **Event Sourcing** for maintaining consistency

Mental Model

- **Monolith** = one shared brain
- **Microservices** = multiple independent brains

PART 3: RESILIENCE PATTERNS

The Resilience Stack (Order of Protection)

text

Level 1: TIMEOUT —————▶ "Don't wait forever"

|



Level 2: RETRY —————▶ "Try again if temporary"

|



Level 3: CIRCUIT BREAKER —▶ "Stop calling failing service"

|



Level 4: FALLBACK —————▶ "Provide alternative response"

1. Timeout

| Aspect | Detail |
|-------------|---|
| What | Limits max wait time for a call |
| Rule | EVERY remote call needs timeout |
| Why | Prevents thread blocking, resource exhaustion |

2. Retry Pattern

| Aspect | Detail |
|-----------------------|--|
| When | Network glitches, temporary overload, 5xx errors |
| When NOT | 4xx errors, non-idempotent operations |
| Best Practices | Exponential backoff + jitter + limited attempts |

Retry Backoff Diagram

text

Attempt 1: Wait 100ms —▶ Fail

Attempt 2: Wait 200ms —▶ Fail

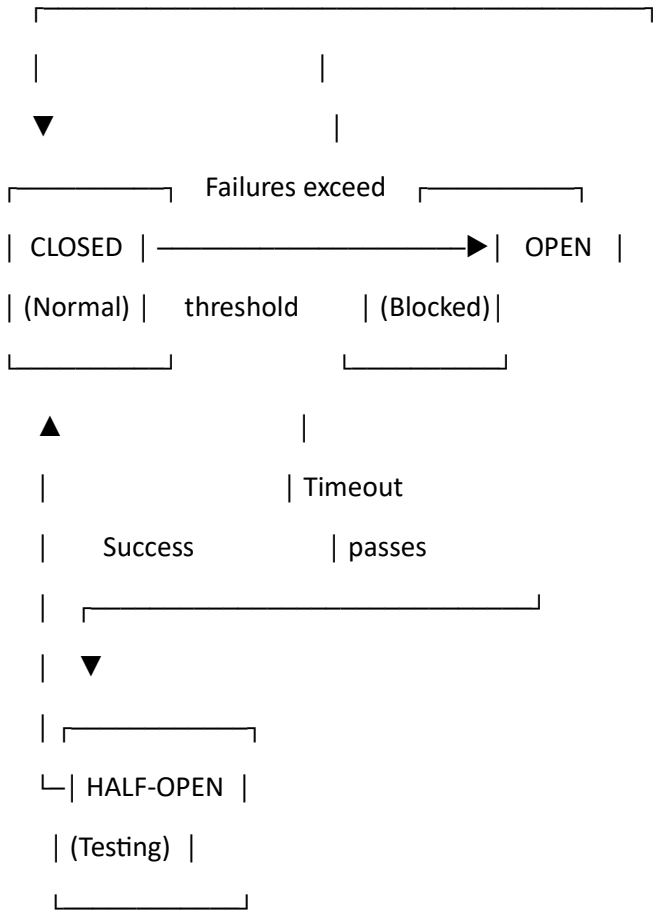
Attempt 3: Wait 400ms —▶ Success ✓

(Exponential backoff with jitter)

3. Circuit Breaker

States Flow

text



4. Fallback Pattern

Type

Example

Cached data

Serve stale cache when live API fails

Default response

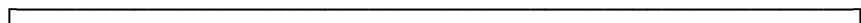
"Service temporarily unavailable"

Secondary provider

Switch to backup service

 Real-World Example (Payment Flow)

text



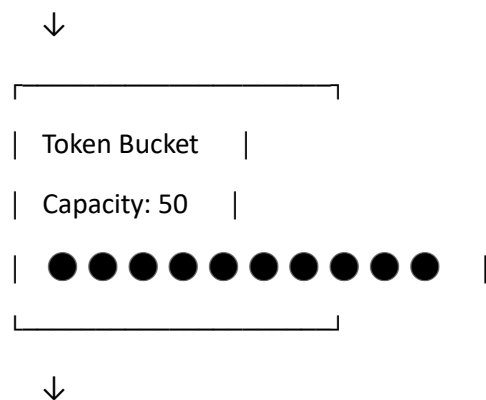
| PAYMENT FLOW | |
|--|--|
| 1. Timeout: 2 seconds max wait | |
| 2. Retry: 2 attempts with exponential backoff | |
| 3. Circuit Breaker: Opens after 5 failures in 10 sec | |
| 4. Fallback: Mark order as "payment pending" | |

What It Does

Algorithms Comparison

Token Bucket Explained

Tokens added at steady rate (10/sec)



Each request consumes 1 token



Burst: 50 requests can come instantly!

HTTP Status Code

429 Too Many Requests → Rate limit exceeded

🧠 Mental Model

Rate limiter = Traffic police (decides who goes, who waits, who stops)

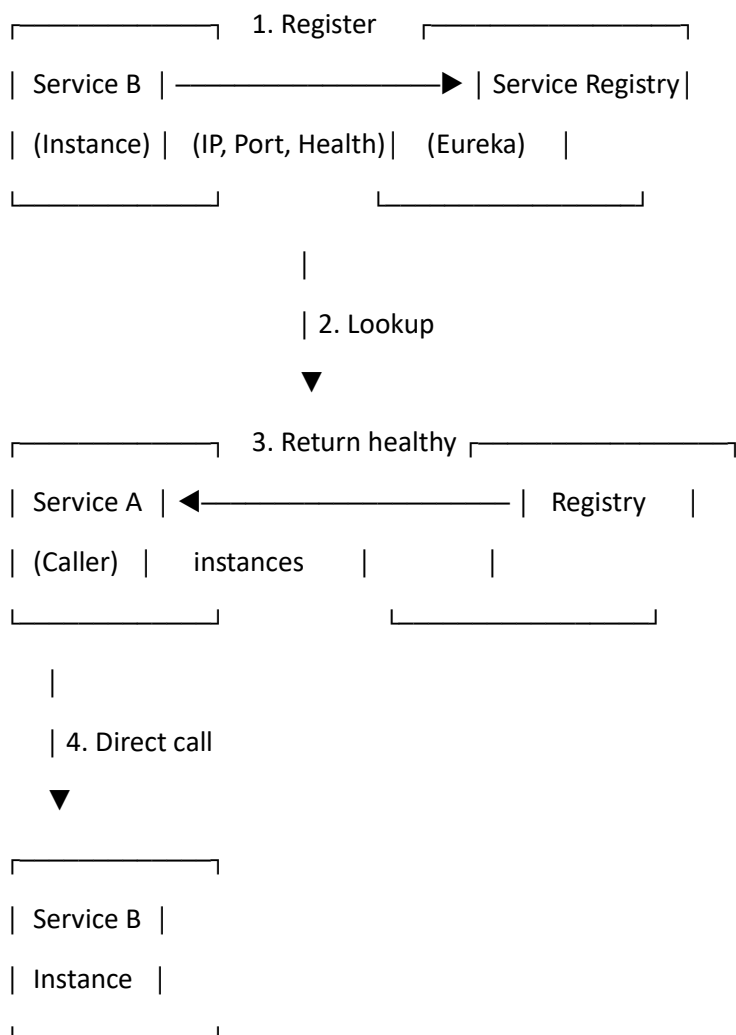
PART 5: SERVICE DISCOVERY

Definition

Mechanism that allows microservices to find and communicate with each other dynamically.

How It Works

text



Types

| Type | Who Decides Routing | Example Tools |
|-------------|---------------------|---------------------|
| Client-Side | Client | Eureka + Ribbon |
| Server-Side | Load Balancer | Kubernetes, AWS ALB |

Mental Model

- Service Discovery = "Google Maps for microservices"
- Registry = Directory of all services
- Load Balancer = Traffic controller

PART 6: SAGA PATTERN

Definition

Manages distributed transactions across multiple microservices without using a single ACID transaction.

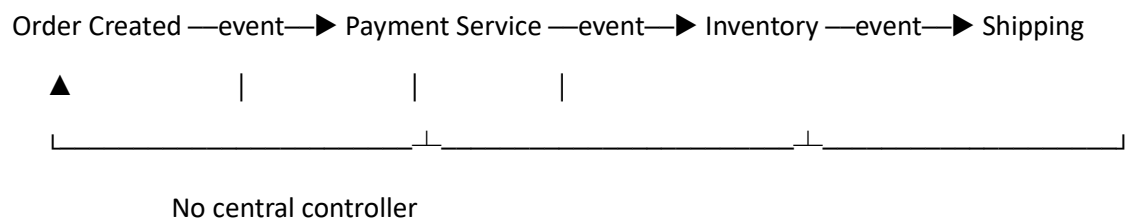
Why Needed

- No shared database
- No global ACID transaction support
- Failures are common

Types of Saga

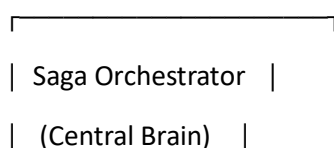
1. Choreography (Event-Driven)

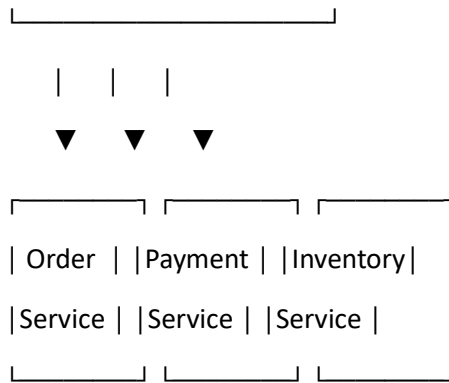
text



2. Orchestration (Central Controller)

text





Saga vs ACID

| Feature | ACID Transaction | Saga Pattern |
|------------------|------------------|----------------------|
| Scope | Single DB | Multiple services |
| Consistency | Strong | Eventual |
| Locking | Yes | No |
| Scalability | Limited | High |
| Failure handling | Rollback | Compensating actions |

Compensating Transactions Example

text

Forward Actions: Compensating Actions:

1. Create Order → Cancel Order
2. Deduct Payment → Refund Payment
3. Reserve Stock → Release Stock
4. Create Shipment → Cancel Shipment



Mental Model

- **ACID** = Single chef in one kitchen
- **Saga** = Multiple chefs in different kitchens coordinating via messages

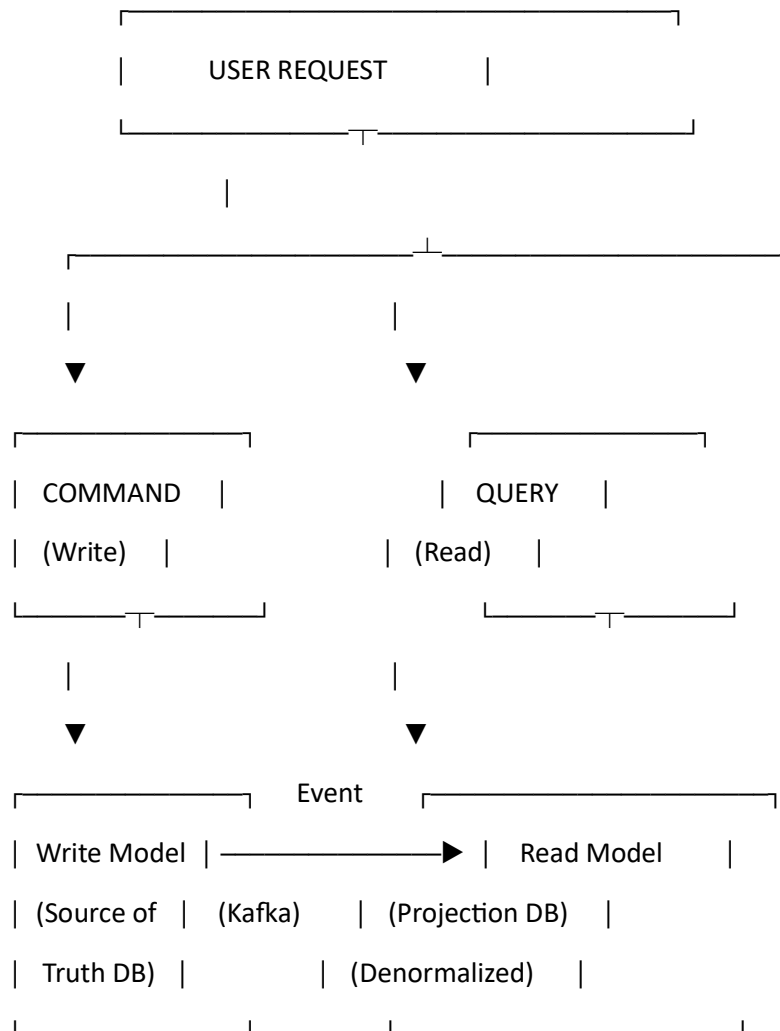
PART 7: CQRS & DATA DUPLICATION

Definition

CQRS = Command Query Responsibility Segregation

Core Architecture

text



Intentional Data Duplication

Write Model

Read Model

Normalized data

Denormalized data

Transactional focus

Query-optimized focus

Single source of truth

Multiple projections possible

Example: Order Data

Write Model (Normalized):

sql

orders: order_id, customer_id, status

order_items: order_id, product_id, quantity

Read Model (Denormalized):

sql

order_view: order_id, customer_name, product_names, order_status, total_amount



Mental Model

- Write Model = Official record ledger
- Read Model = Optimized display copy
- *Users never query the ledger directly*

PART 8: IDEMPOTENCY

Definition

Executing the same operation multiple times produces the same result as executing it once.

Implementation Strategies

text

| IDEMPOTENCY IMPLEMENTATIONS | | |
|--|--|--|
| | | |
| 1. Idempotency Key | | |
| Client: POST /orders + Header: Idempotency-Key: abc123 | | |
| Server: Store processed keys in Redis/DB | | |
| | | |
| 2. Database Constraints | | |
| UNIQUE constraint on order_id, payment_id | | |
| | | |
| 3. State-based Updates | | |
| ✓ SET balance = 1000 (idempotent) | | |
| ✗ ADD 100 to balance (NOT idempotent) | | |
| | | |

| | |
|--|--|
| 4. Message Deduplication | |
| Kafka consumer tracks message IDs in deduplication store | |
| | |

Where Critical

- Payment systems
- Order creation
- Inventory updates
- Event processing systems

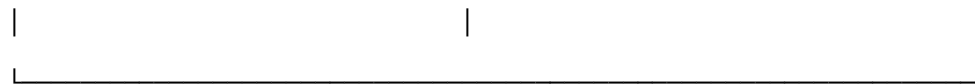
 Mental Model

- **Idempotent** = Light switch (ON → ON → ON = same state)
- **Non-idempotent** = Adding coins (each action changes state)

PART 9: OBSERVABILITY (3 Pillars)

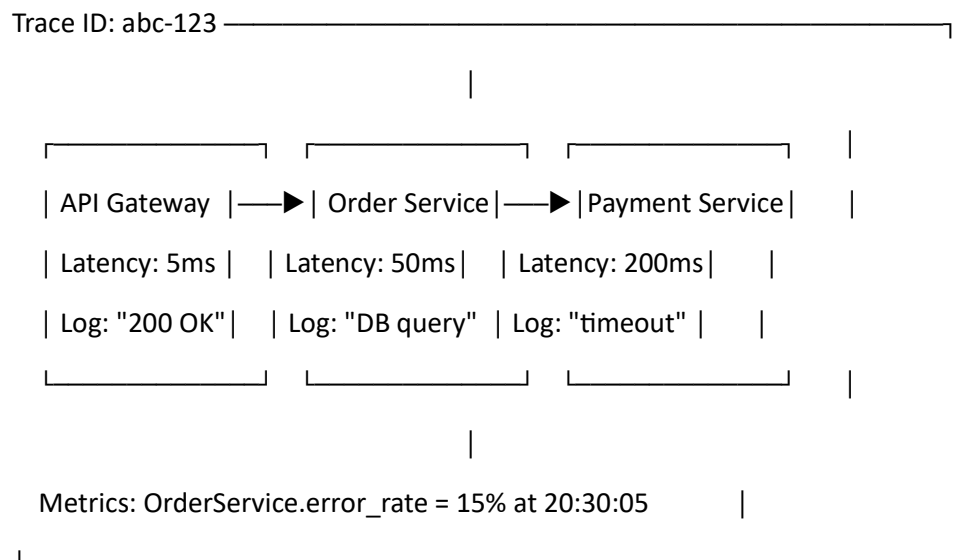
text

| | | | | | | | | | |
|--------------------------------|--|--|-----------------------|--|--|----------------------|--|--|--|
| THREE PILLARS OF OBSERVABILITY | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| LOGS | | | METRICS | | | TRACES | | | |
| "What happened?" | | | "How many/ how fast?" | | | "End-to-end journey" | | | |
| Events | | | Numeric | | | Request ID | | | |
| Timestamps | | | time-series | | | Span context | | | |
| ELK Stack | | | Prometheus | | | OpenTelemetry | | | |
| Grafana | | | Jaeger | | | | | | |



How They Work Together

text



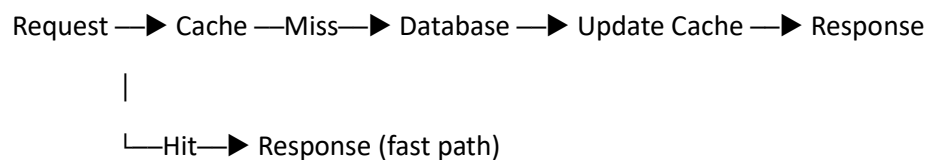
Mental Model

- **Logs** = Diary entries
- **Metrics** = Heartbeat monitor
- **Traces** = GPS route map

PART 10: CACHING STRATEGIES

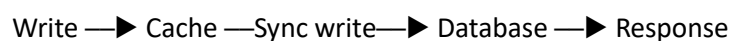
Cache Aside (Lazy Loading) — MOST COMMON

text



Write-Through

text



Write-Back (Write-Behind)

text





Cache Invalidation Strategies

| Strategy | How It Works | Best For |
|-------------|-----------------------------|-------------------------|
| TTL | Data expires after time | Product catalogs |
| Event-based | Update cache on data change | User profiles |
| Manual | Explicit invalidation API | Admin-triggered updates |

Cache Stampede Problem & Solutions

text

Problem: Cache expires → 1000 requests hit DB simultaneously

Solutions:

1. Locking (only 1 request fetches)
2. Request coalescing (deduplicate in-flight requests)
3. Random TTL jitter (stagger expiration times)

Mental Model

- **Cache** = Fast memory desk
- **Database** = Slow library
- *You keep notes on your desk instead of going to the library every time*

PART 11: DEPLOYMENT MODELS

Quick Comparison Matrix

| Model | Downtime | Risk | Cost | Complexity | Use Case |
|------------|----------|--------|--------|------------|----------------------|
| Monolithic | High | High | Low | Low | Legacy systems |
| Rolling | None | Medium | Medium | Medium | Standard deployments |

| Model | Downtime | Risk | Cost | Complexity | Use Case |
|-----------------------|----------|----------|--------|------------|------------------------|
| Blue-Green | None | Low | High | Medium | Critical apps |
| Canary | None | Very Low | Medium | High | Large-scale systems |
| Active-Passive | Minimal | Low | Medium | Low | DR systems |
| Active-Active | None | Low | High | Very High | Global high-scale apps |

Visual Summary

text

Rolling:  →  →  →  → 

(Gradual instance replacement)

Blue-Green:  BLUE  GREEN

\ /

└—Switch—┘

(Instant traffic cutover)

Canary:   5% traffic

Main version New version

(Gradual traffic increase)

Active-Active:  US $\uparrow$

└— All active, load balanced

 EU $\downarrow$

 **ULTRA CHEAT SHEET (1-Page Revision)**

CORE PATTERNS MAP

text

| | | |
|---------------------------|--|------------------------|
| MICROSERVICES PATTERN MAP | | |
| | | |
| Data Ownership | → Data Decomposition | → Database per Service |
| | | |
| Traffic Control | → Rate Limiter | → Token Bucket |
| | | |
| Service Lookup | → Service Discovery | → Eureka/Kubernetes |
| | | |
| Communication | → Sync (REST/gRPC) + Async (Kafka/RabbitMQ) | |
| | | |
| Consistency | → Saga Pattern (Choreography/Orchestration) | |
| | | |
| Scalability | → Async + Event-Driven Architecture | |
| | | |
| Resilience | → Timeout → Retry → Circuit Breaker → Fallback | |
| | | |
| Performance | → CQRS + Caching (Cache Aside) | |
| | | |
| Safety | → Idempotency (Idempotency Key + Unique Constraints) | |
| | | |
| Visibility | → Observability (Logs + Metrics + Traces) | |
| | | |
| Deployment | → Blue-Green / Canary / Active-Active | |
| | | |

QUICK REFERENCE CARD

| Pattern | One-Line Summary |
|------------------------|---|
| Sync | Immediate response, tight coupling, user-facing ops |
| Async | Event-driven, loose coupling, scalable processing |
| Saga | Distributed transactions via compensating actions |
| CQRS | Separate read/write models with intentional duplication |
| Circuit Breaker | Stop calling failing services, prevent cascading failures |
| Retry | Handle transient failures with exponential backoff |
| Timeout | Never wait indefinitely for any remote call |
| Idempotency | Same operation multiple times = same result |
| Rate Limiter | Control request flow, return 429 when exceeded |

MENTAL MODEL CHEAT SHEET

text

API Gateway = Front door of the system

Rate Limiter = Security guard at entrance

Service Registry = Phone directory of services

Kafka = Postal system (async messages)

Circuit Breaker = Emergency electrical switch

Saga = Workflow conductor/orchestrator

Cache = Sticky notes on your desk

Trace = GPS tracker of a request

Idempotency Key = Unique receipt ID for each operation

INTERVIEW POWER PHRASES

"Microservices are independently deployable services, each owning its own data model aligned to business domains."

"We use synchronous communication for real-time user-driven flows, and asynchronous for scalability and decoupling."

"The resilience stack is timeout first, then retry, then circuit breaker, finally fallback."

"Saga replaces ACID with eventual consistency using compensating transactions."

"CQRS intentionally duplicates data — that's the trade-off for scalability and performance."

"Idempotency is essential for safe retries in distributed systems."

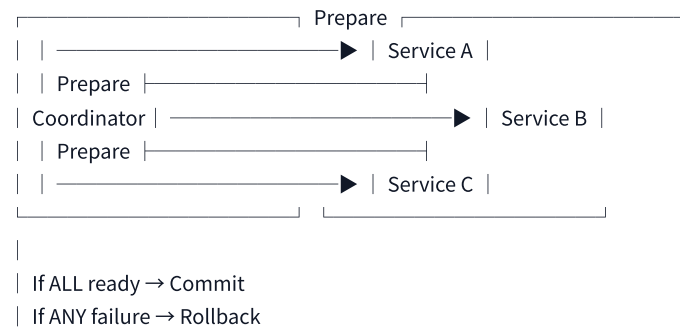
This document now serves as a complete interview preparation guide with:

-  Clean formatting with emojis and visual hierarchy
-  Text-based diagrams for easy recall
-  Comparison tables for quick reference
-  Mental models for intuitive understanding
-  One-line summaries for interview delivery

TOPIC 1: DISTRIBUTED TRANSACTIONS — BEYOND SAGA

Two-Phase Commit (2PC) — Why Microservices Avoid It

text



Aspect 2PC Saga

Locking Yes (blocks resources) No

Consistency Strong Eventual

Availability Lower Higher

Scalability Poor Excellent

Microservices fit ☒ No ☐ Yes

Interview Insight: "2PC doesn't work in microservices because locks can't span services and network partitions kill availability."

XA Transactions vs Saga vs TCC (Try-Confirm-Cancel)

Pattern Locking Compensating Best For

XA / 2PC Long locks Rollback Monoliths, not microservices

Saga No locks Compensating actions Long-running workflows

TCC Short "try" locks Explicit cancel High-performance scenarios

TCC Pattern Explained

text

Phase 1: TRY

├── Reserve resources (soft lock)

├── No actual business execution

└── Return confirmation token

Phase 2: CONFIRM (if all TRY succeed)

├── Execute actual business

└── Release soft locks

Phase 2 (alt): CANCEL (if any TRY fails)

├── Rollback reservations

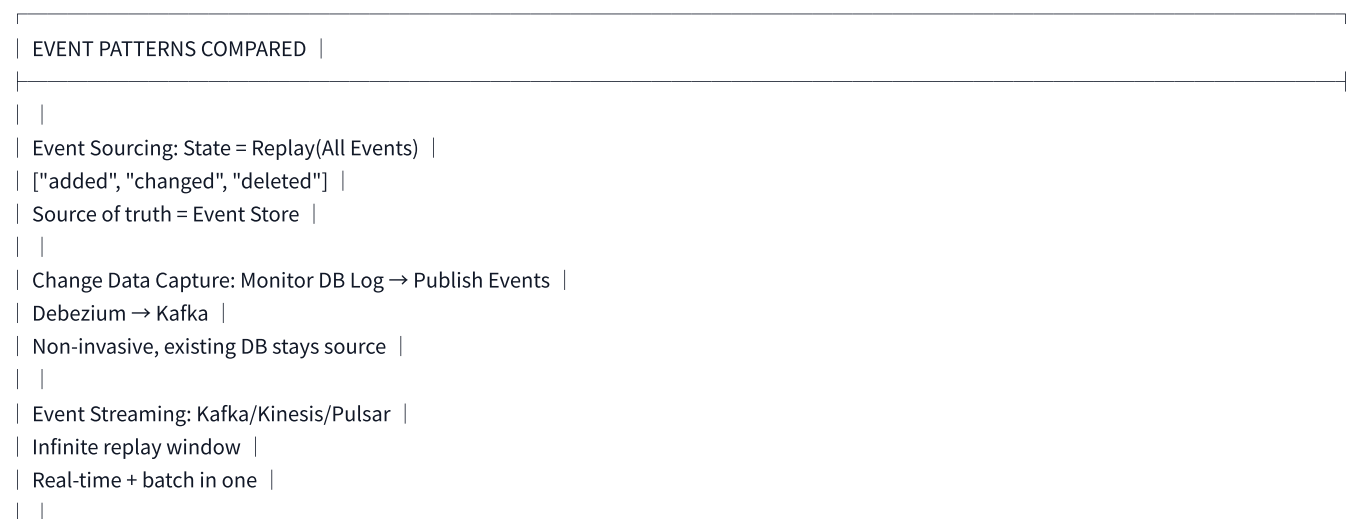
└── Release soft locks

When TCC > Saga: Financial transactions where double-spending must be prevented during the reservation phase.

TOPIC 2: EVENT DRIVEN ARCHITECTURE — DEEP DIVE

Event Sourcing vs Change Data Capture vs Event Streaming

text



When to Use Which

Pattern When to Choose

Event Sourcing Need full audit trail, time travel, debugging past states

CDC Existing monolith, can't change application code

Event Streaming Real-time analytics, data pipelines, Kafka as backbone

Idempotent Event Processing — Deep Dive

text

IDEMPOTENT EVENT PROCESSOR PATTERN |

Consumer: |

1. Receive event { id: "evt-123", type: "order.created" } |

2. Check Redis SETNX "processed:evt-123" |

3. If redis says "already seen" → SKIP |

4. If new → process + store result in DB |

5. Atomic commit: DB update + Redis mark |

Exactly-Once Semantics = Idempotent Consumer + Transactional |

Outbox + Kafka idempotent producer |

TOPIC 3: API GATEWAY — ADVANCED PATTERNS

Gateway Responsibilities — Complete Map

text

API GATEWAY RESPONSIBILITIES |

Layer 1: Security |

├── JWT/OAuth2 validation |

├── API key management |

├── IP whitelisting |

└── Bot detection |

Layer 2: Traffic Control |

├── Rate limiting (per user/per endpoint) |

├── Circuit breaking |

├── Retry with backoff |

└── Request deduplication |

Layer 3: Transformation |

├── Request/response transformation |

├── GraphQL → REST aggregation |

├── Protocol translation (gRPC → HTTP) |

└── Response caching |

Layer 4: Observability |

├── Distributed tracing header injection |

├── Request/response logging |

├── Metrics collection |

└── Alerting |

Gateway Implementation Comparison

Gateway Best For Notable Features

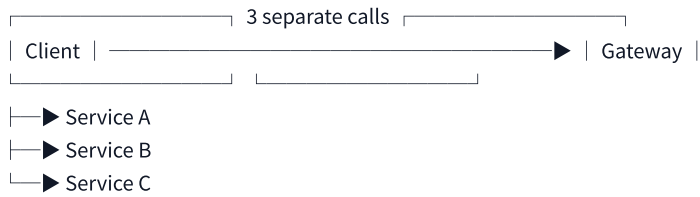
Kong Enterprise Plugin ecosystem, DB-backed

NGINX High performance Low latency, small memory

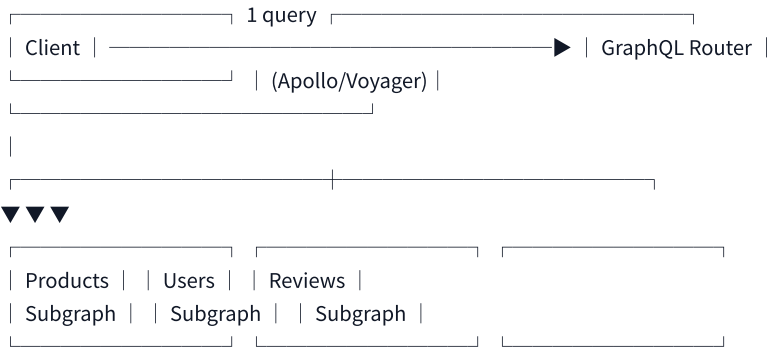
Envoy Service mesh integration xDS protocol, gRPC native

Spring Cloud Gateway Java ecosystem Reactive, Spring integrated
AWS API Gateway Serverless Lambda integration, managed
GraphQL Federation vs API Gateway Aggregation
text

API Gateway Aggregation:



GraphQL Federation:



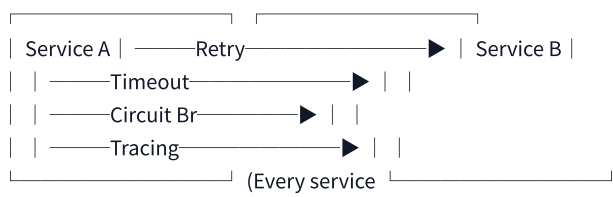
Interview Insight: "API Gateway aggregates at request level; GraphQL federation aggregates at field level with type safety."

TOPIC 4: SERVICE MESH — ENVOY, ISTIO, LINKERD

What Problem Service Mesh Solves

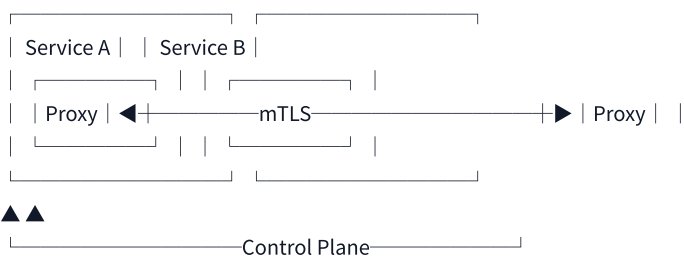
text

Without Service Mesh:



implements its own!)

With Service Mesh:



(Istio/Pilot, Linkerd control)

Control Plane vs Data Plane

Plane Responsibility Components

Data Plane Handles actual traffic Envoy proxies (sidecars)

Control Plane Manages configuration Pilot, Mixer, Citadel (Istio)

Service Mesh Features Matrix

Feature Envoy Istio (on Envoy) Linkerd

mTLS ✓✓✓

Retry/Timeout ✓✓✓

Circuit Breaking ✓✓✓

Traffic splitting ✓✓✓

Fault injection Limited ✓ Limited

Complexity Medium High Low

When to Adopt Service Mesh

text

✓ GOOD FIT:

- Many services (50+)
- Multiple languages (Java, Go, Python, Node)
- Security/compliance requiring mTLS
- Dedicated platform team exists

✗ NOT NEEDED:

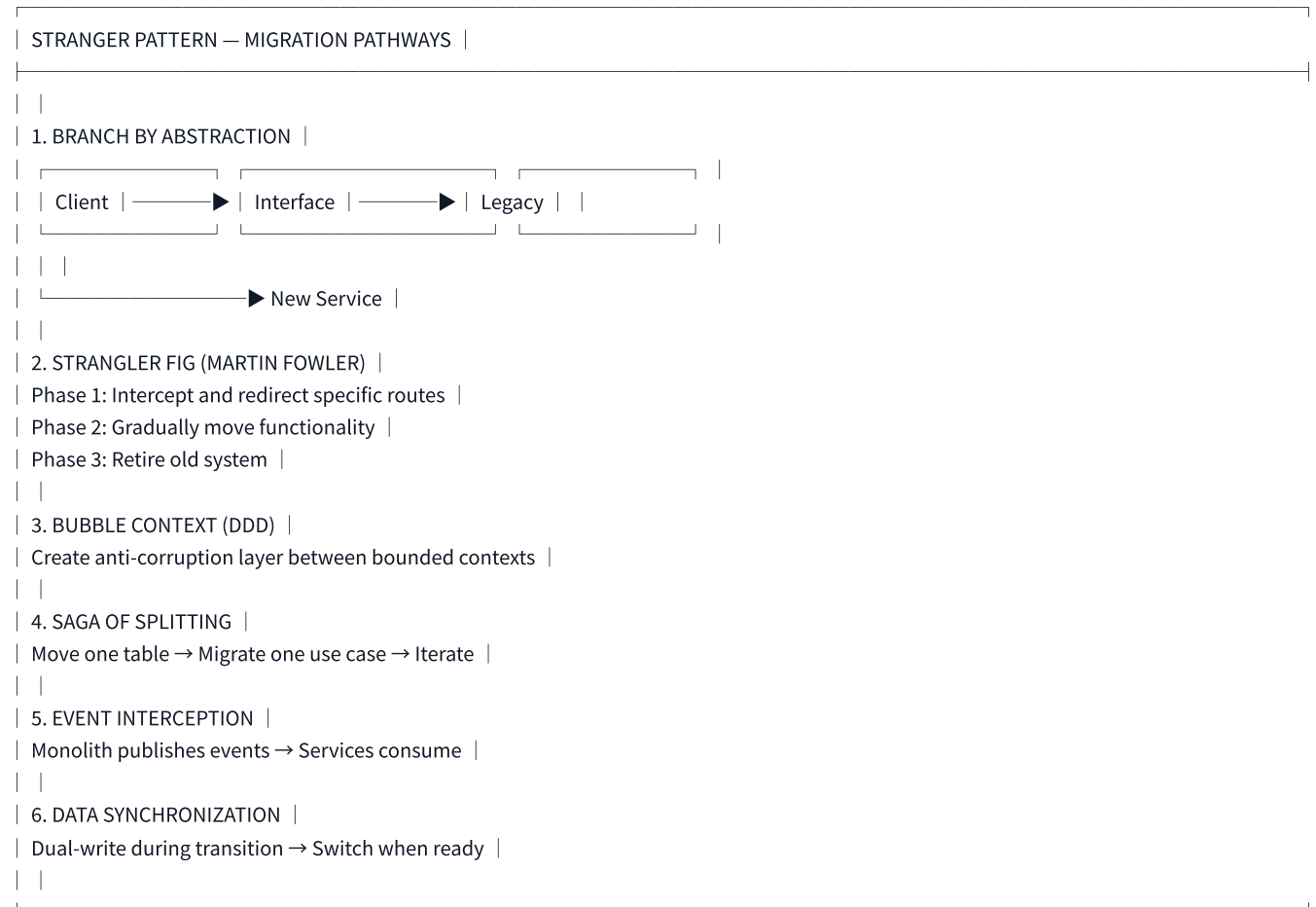
- < 10 services
- Single language (library approach works)
- No security requirements
- Small team, no platform expertise

Interview Insight: "Service mesh is NOT for everyone. For 10-20 services, client libraries (Resilience4j, Hystrix) are simpler. At 50+ services, the standardization of service mesh pays off."

TOPIC 5: STRANGER PATTERN — MONOLITH TO MICROSERVICES

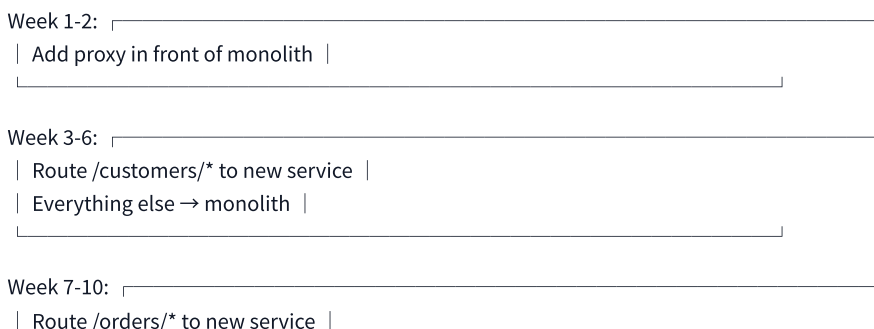
The Six Stranger Pattern Strategies

text



Strangler Fig — Detailed Timeline

text



| Keep /reports/* on monolith |

Week 11-14: |
| Route /reports/* → new service |
| Monolith = empty shell |

Week 15: |
| Decommission monolith |

Interview Insight: "Never do Big Bang rewrites. The Strangler Pattern is the only proven safe path to microservices."

TOPIC 6: DORA METRICS & MICROSERVICES MATURITY

The Four Key DORA Metrics

Metric Definition Target (Elite)

Deployment Frequency How often deploy to production Multiple times/day

Lead Time for Changes Code commit → deployment < 1 hour

Time to Restore Incident → resolution < 1 hour

Change Failure Rate % of deployments causing incidents 0-15%

Microservices Maturity Model

text

Level 1: MONOLITH

- |—— Single deployment unit
- |—— Shared database
- |—— Deploy frequency: Monthly

Level 2: MICROSERVICES NAIVE

- |—— Split by technical layers (not domains)
- |—— Shared database hidden
- |—— Synchronous calls everywhere
- |—— Deploy frequency: Weekly

Level 3: DOMAIN-DRIVEN

- |—— Split by business domains
- |—— Database per service
- |—— Hybrid sync/async
- |—— Saga for transactions
- |—— Deploy frequency: Daily

Level 4: OPERATIONALLY EXCELLENT

- |—— Fully independent deployments
- |—— Blue-green/canary
- |—— Service mesh
- |—— Platform engineering team
- |—— DORA Elite metrics
- |—— Deploy frequency: Multiple times/day

Level 5: CELLULAR ARCHITECTURE

- |—— Self-contained cells
- |—— Regional isolation
- |—— Chaos engineering in production
- |—— Deploy frequency: Hourly

TOPIC 7: CELLULAR ARCHITECTURE — THE NEXT STEP

What is Cellular Architecture

text

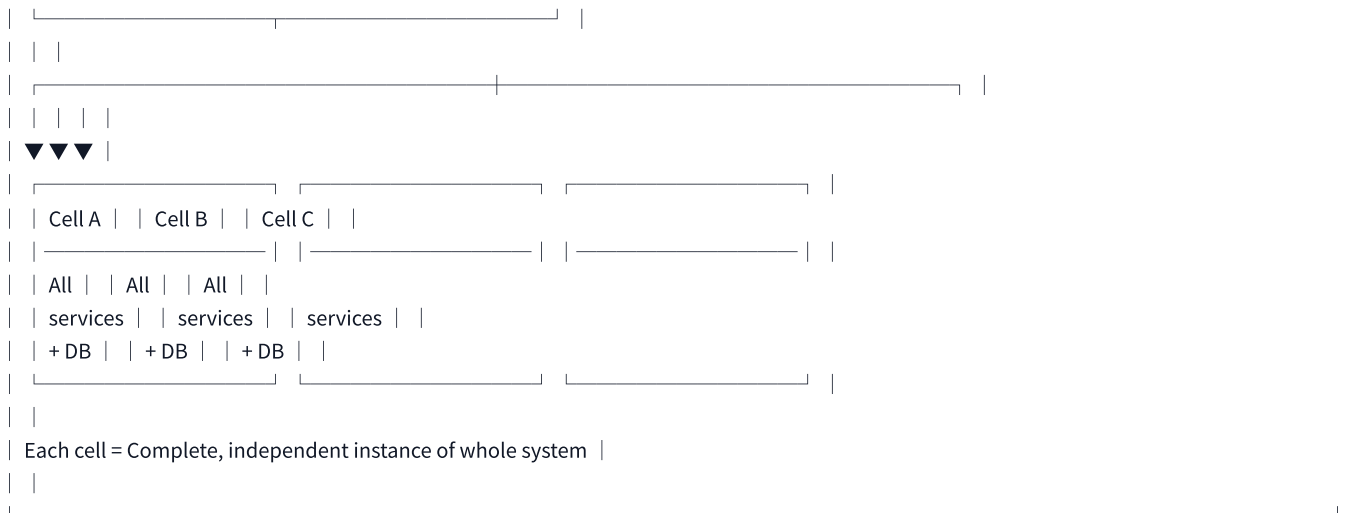
| CELLULAR ARCHITECTURE |

| |

| |

| Global Router | |

| (No business logic) | |



Why Cellular > Traditional Microservices

Aspect Traditional Microservices Cellular Architecture

Blast radius Full system Single cell

Deployment risk Global Per cell (canary cells)

Scaling Per service Whole cells

Regional failover Complex Built-in (cells per region)

Tenancy Hard Easy (tenant → cell mapping)

Complexity High per service High per cell but isolated

Companies Using Cellular Architecture

Company Scale Why Cells

Uber Thousands of services Blast radius containment

Netflix Global streaming Regional isolation

DoorDash 10K+ microservices Developer independence

TOPIC 8: DATABASE PER SERVICE — DEEP TRADE-OFFS

The Hidden Costs

text

Cost 1: Distributed Transactions

Monolith: BEGIN TX → Update A → Update B → COMMIT |
 Microservices: Saga with 5 compensating actions |
 + idempotency |
 + retry logic |
 + dead letter queues |

Cost 2: Reporting

Solutions: |
 — CQRS (separate read models) |
 — Data warehouse (ETL from all services) |
 — Materialized views across services |
 — Kafka + Flink for real-time analytics |

Cost 3: Shared Reference Data

Problem: Product catalog shared across 10 services |
 Solutions: |
 — Product Service as source + cache elsewhere |
 — Replicate product data to each service (eventually) |
 — Library for product validation (breaking independence!) |

When Database Per Service Fails

text

✗ Anti-patterns to recognize:

- 1. Too many join operations across services
→ Suggests wrong service boundaries
- 2. Report queries constantly aggregating
→ Needs CQRS or data warehouse
- 3. Shared reference data changing rapidly
→ Cache invalidation becomes nightmare
- 4. Transactional boundaries span 5+ services
→ Maybe the monolith was better

TOPIC 9: CHAOS ENGINEERING

Principles of Chaos Engineering

text

| |
|---|
| CHAOS ENGINEERING MATURITY MODEL |
| |
| |
| Level 0: No testing |
| |
| Level 1: Test in staging only |
| |
| Level 2: Test in production with small blast radius |
| Example: Kill one pod out of 1000 |
| |
| Level 3: Production experiments with automated rollback |
| Example: 2% latency injection, rollback if error rate >0.1% |
| |
| Level 4: Game days (simulate major outages proactively) |
| Example: Simulate entire region failure |
| |

Common Chaos Experiments

Experiment Target Success Criteria

Pod kill Kubernetes deployment Self-healing, no user impact
Latency injection Service dependency Timeout + circuit breaker works
Database failover Primary → replica Zero downtime failover
Network partition Service A ↔ Service B Graceful degradation
Resource exhaustion CPU/Memory limits Throttling, not crash
Certificate expiry mTLS certs Auto-rotation works

Tools

Tool Use Case Company

Chaos Monkey Kill instances Netflix
Gremlin Commercial platform Gremlin Inc
Litmus Kubernetes native ChaosNative
PowerfulSeal Pod/VMI chaos Bloomberg

Interview Insight: "Chaos engineering isn't about breaking things randomly. It's about building confidence in system resilience by running controlled experiments."

TOPIC 10: PLATFORM ENGINEERING FOR MICROSERVICES

Internal Developer Platform (IDP) — Golden Paths

text

| |
|--------------------------------------|
| PLATFORM AS A PRODUCT — GOLDEN PATHS |
| |
| |
| Developer Experience |
| |
| ▼ |

4. Resilience over correctness

└── Better to be partially available than fully unavailable

5. Simplicity over completeness

└── The best architecture is the simplest one that meets requirements

6. Discovery over configuration

└── Services find each other; don't hardcode anything

7. Idempotency over retry

└── Without idempotency, retry is dangerous

8. Exponential backoff over immediate retry

└── Give failing systems time to recover

9. Circuit breaking over endless retry

└── Know when to stop calling

10. Platform over point solutions

└── Standardize the infrastructure, not the applications

INTERVIEW POWER PHRASES (Advanced Edition)

"Two-phase commit doesn't work in microservices because network partitions are unavoidable. We use Saga with compensating transactions for eventual consistency."

"Service mesh shifts complexity from application to infrastructure — you pay with operational overhead, but gain standardization across polyglot services."

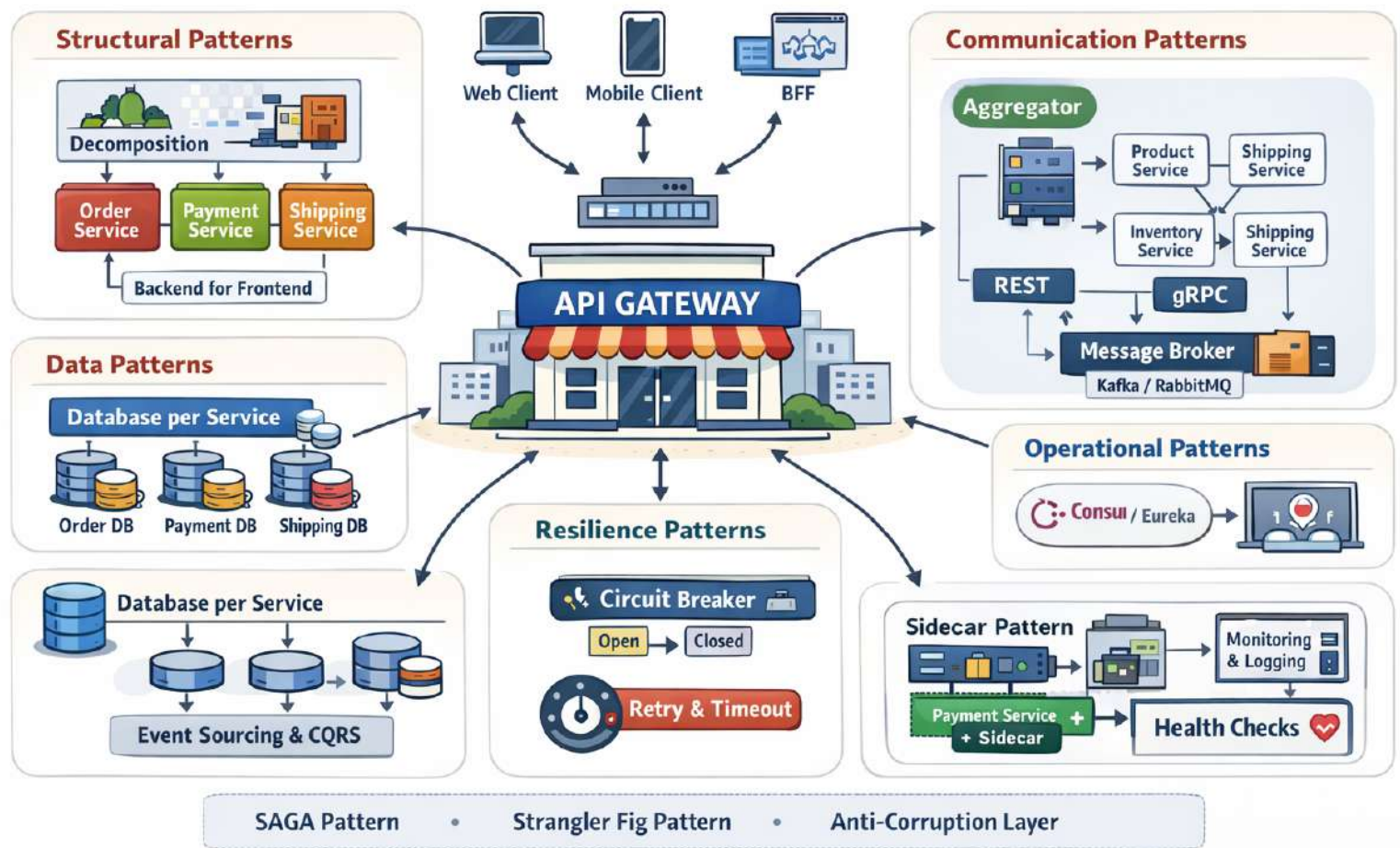
"The Strangler Pattern is the only safe path to microservices. Anyone proposing a big bang rewrite hasn't experienced one."

"Database per service isn't dogma — it's a trade-off. You pay with distributed transactions and reporting complexity in exchange for independent scalability."

"Chaos engineering is about controlled experiments, not random breaking. We start with 1% blast radius and expand confidence over time."

"Cellular architecture is microservices' answer to blast radius: each cell fails independently, the system as a whole doesn't."

"Platform engineering is a product. Our customers are internal developers. We build golden paths, not golden cages."

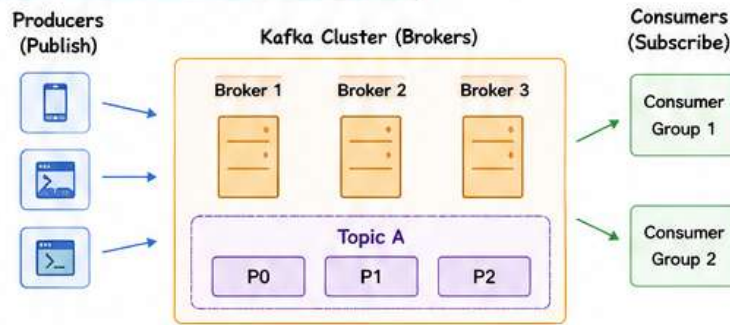


WHAT IS KAFKA?

Kafka is a distributed publish-subscribe messaging system designed for high throughput, fault tolerance and real-time data pipelines.

- High Throughput
- Scalable
- Fault Tolerant

HOW KAFKA WORKS (HIGH LEVEL)



DATA FLOW

- Producers publish records to topics.
- Topics are split into partitions and distributed across brokers.
- Consumers subscribe to topics and read records.
- Kafka stores records durably and delivers them in order within a partition.

CORE CONCEPTS

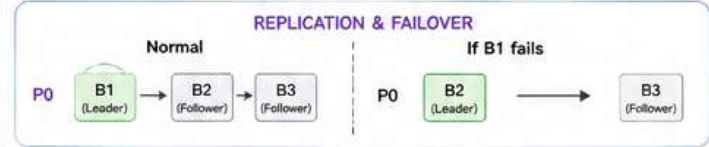
| | | |
|--|---|-----------------------------|
| | Topic
A category or feed name to which events are published. | example: orders
 |
| | Partition
A topic is divided into one or more partitions for scalability. | orders
P0 P1 P2 |
| | Broker
A Kafka server that stores data and serves clients. | |
| | Producer
An application that publishes records to a topic. | |
| | Consumer
An application that subscribes to a topic and processes records. | |
| | Consumer Group
A group of consumers that work together to consume a topic. Each partition is consumed by one consumer in the group. | Group
 |
| | Offset
A unique ID of each record within a partition. Consumers use offsets to track their position. | P0: [0] [1] [2] [3] [4] ... |
| | Replication
Each partition is replicated across brokers for fault tolerance. | P1 → B1 B2 B3 |

TOPIC, PARTITIONS & REPLICATION

Topic: orders (3 partitions, replication factor = 3)

| Partition | Broker 1 | Broker 2 | Broker 3 |
|-------------|----------|----------|----------|
| Partition 0 | Leader | Follower | Follower |
| Partition 1 | Follower | Leader | Follower |
| Partition 2 | Follower | Follower | Leader |

Leader: handles all read/write for the partition Follower: replicates data from leader



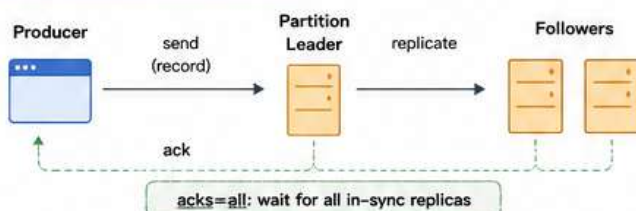
PRODUCER ACKS

- acks = 0
No guarantee
- acks = 1
Leader received
- acks = all (-1)
All in-sync replicas received (strongest)

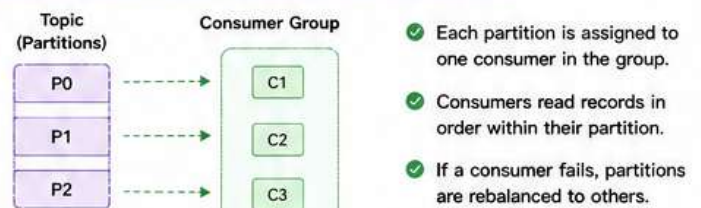
DELIVERY SEMANTICS

- At most once
Message may be lost
- At least once
Message will be delivered one or more times
- Exactly once
Message will be delivered once and only once

PRODUCER WRITE PATH



CONSUMER READ PATH (CONSUMER GROUP)



- Each partition is assigned to one consumer in the group.
- Consumers read records in order within their partition.
- If a consumer fails, partitions are rebalanced to others.

COMMON USE CASES

- Real-time data pipelines (Stream processing, ETL)
- Log aggregation (Centralized logging)
- Event-driven architectures (Microservices communication)
- Monitoring & metrics (Operational monitoring)

KAFKA ECOSYSTEM

- Kafka Connect**
Data integration (sources & sinks)
- Kafka Streams**
Stream processing library
- ksqldb**
SQL interface for Kafka
- Schema Registry**
Manage and enforce schemas

KEY FEATURES

- High Throughput**
Millions of records per second
- Durable & Fault Tolerant**
Data is persisted and replicated
- Scalable**
Horizontal scale with ease
- Ordered per Partition**
Order is guaranteed within a partition
- Retention & Replay**
Data can be retained and reprocessed

USEFUL COMMANDS (kafka-topics.sh)

```
# Create topic
kafka-topics.sh --create \
  --topic orders \
  --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 3

# List topics
kafka-topics.sh --list \
  --bootstrap-server localhost:9092

# Describe topic
kafka-topics.sh --describe --topic orders \
  --bootstrap-server localhost:9092
```

BEST PRACTICES

- Choose the right number of partitions (too few = limited scale, too many = overhead)
- Set appropriate replication factor (≥ 3 for prod)
- Use idempotent producers and transactions for exactly-once semantics
- Use meaningful topic names and configs
- Monitor lag, throughput, and consumer health
- Test failover and disaster recovery regularly

GLOSSARY

Broker - Kafka server
Topic - Feed/category name
Partition - Subdivision of a topic
Offset - Position of a record in a partition
Consumer Group - Group of consumers working together

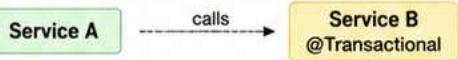
Spring @Transactional Cheat Sheet



@Transactional in Spring manages database transactions. It has 2 important dimensions: **Propagation** and **Isolation**

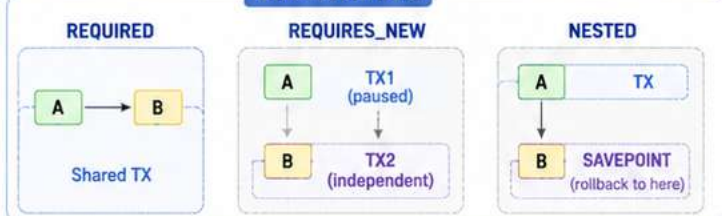
PROPAGATION

What happens if a transaction already exists?



| | | |
|------------------------------|--|---|
| REQUIRED
(Default) | | Join existing transaction if exists, otherwise create a new one. |
| REQUIRES_NEW | | Suspend the existing transaction and create a new one. |
| SUPPORTS | | Join existing transaction if exists, otherwise run non-transactionally. |
| NOT_SUPPORTED | | Suspend the existing transaction and run non-transactionally. |
| MANDATORY | | Must run within an existing transaction, otherwise throw exception. |
| NEVER | | Must run outside a transaction, otherwise throw exception. |
| NESTED | | Run within a nested transaction using a savepoint. Rollback only affects the nested part. |

VISUAL SUMMARY



- TIPS**
- REQUIRED is the most commonly used.
 - REQUIRES_NEW is useful for audit logging, notification, outbox, etc.
 - NESTED works only with JDBC resource transactions (DataSourceTransactionManager).

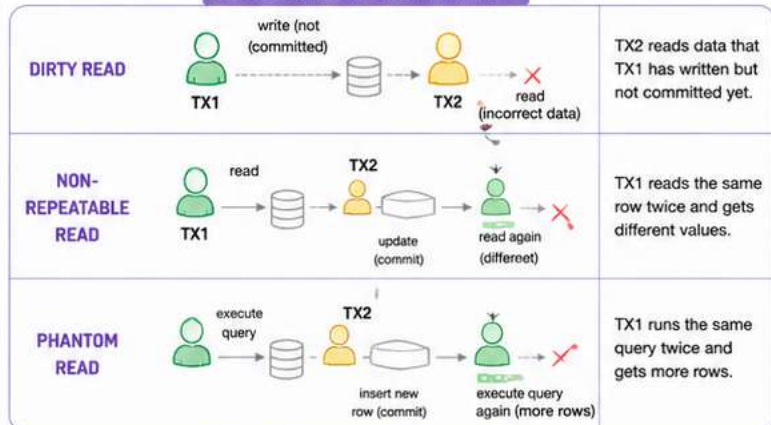
ISOLATION

How "visible" your data is to other transactions?

ISOLATION LEVELS

| Level (from low to high) | Prevents | Description |
|--|--|--|
| READ_UNCOMMITTED | ✗ Nothing | Dirty reads, non-repeatable reads and phantom reads can occur. |
| READ_COMMITTED
(Default in most DBs) | ✓ Dirty reads | Can see only committed data. Non-repeatable and phantom reads can occur. |
| REPEATABLE_READ | ✓ Dirty reads
✓ Non-repeatable reads | Same rows read multiple times won't change. Phantom reads can occur. |
| SERIALIZABLE | ✓ Dirty reads
✓ Non-repeatable reads
✓ Phantom reads | Full isolation. Transactions behave as if executed one by one. |

CONCURRENCY PROBLEM MAP



ISOLATION STRENGTH LADDER



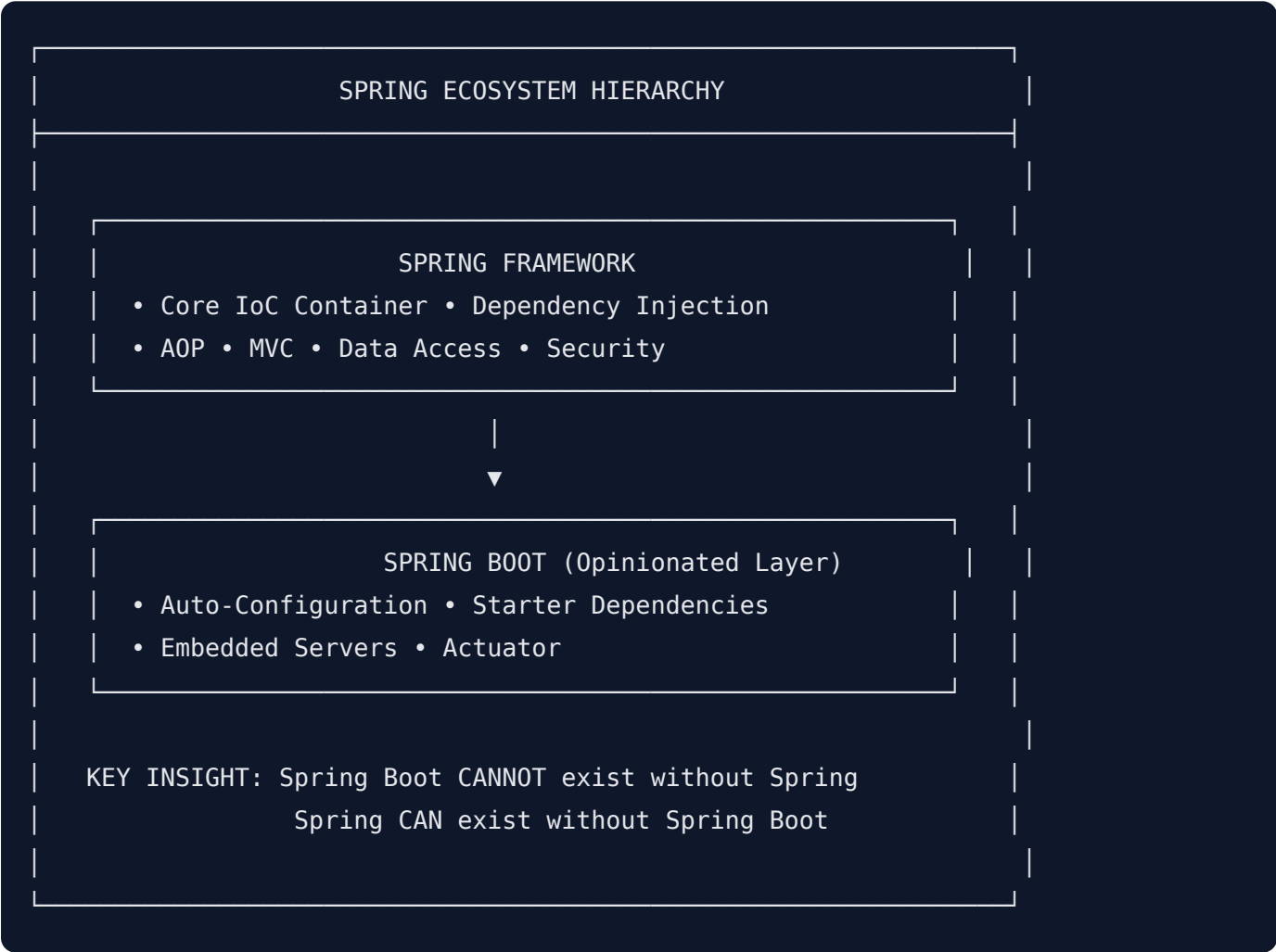
- REMEMBER**
- Propagation:** controls how transactions nest or interact.
 - Isolation:** controls what data is visible between transactions.
 - Choose the weakest isolation level that meets your consistency needs.

SPRING & SPRING BOOT DEEP DIVE — Advanced Interview Guide

For the 16-Year Experience Professional

PART 1: SPRING VS SPRING BOOT — THE BIG PICTURE

Core Relationship



Detailed Comparison Matrix

| Dimension | Spring Framework | Spring Boot |
|---------------|------------------------|---|
| Configuration | Manual XML/Java config | Auto-configuration + minimal properties |

| | | |
|-----------------------|---------------------------------------|--|
| Dependency Management | Manual version resolution | Starter POMs with curated versions |
| Deployment | WAR to external server (Tomcat/JBoss) | Executable JAR with embedded server |
| Development Speed | Slower (configuration overhead) | Very fast (convention over configuration) |
| Control Level | High (everything explicit) | Moderate (opinionated defaults) |
| Use Case | Large enterprise, custom requirements | Microservices, cloud-native, rapid development |
| Learning Curve | Steeper | Gentler |

When to Choose Which

FRAMEWORK SELECTION GUIDE

Choose SPRING FRAMEWORK when:

- You need fine-grained control over every configuration
- Legacy system integration requires specific setup
- Team is already experienced with traditional Spring
- Complex custom requirements not covered by Boot defaults

Choose SPRING BOOT when:

- Starting new microservices project
- Rapid development and prototyping
- Cloud-native or containerized deployment
- Reducing boilerplate configuration is priority
- Standard enterprise patterns suffice

PRO TIP: Most NEW projects (2024-2026) should start with Boot

PART 2: SPRING BOOT INTERNALS — HOW IT REALLY WORKS

Application Startup Flow

SPRING BOOT INTERNAL STARTUP SEQUENCE

@SpringBootApplication (Main Entry Point)



@EnableAutoConfiguration

- Reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
- Evaluates @Conditional annotations
- Registers auto-configuration classes



@ComponentScan

- Scans package for @Component, @Service, @Repository
- Uses index (if spring-context-indexer present)
- Registers found components in IoC container



@Configuration & @Bean Processing

- Parses @Configuration classes
- Executes @Bean methods (CGLIB proxied)
- Ensures singleton scope



IoC Container (ApplicationContext)

- Manages bean lifecycle (init, destroy)
- Handles dependency injection (@Autowired)
- BeanPostProcessors for AOP, etc.



Embedded Web Server (Tomcat/Jetty/Undertow) Startup

@Configuration vs @AutoConfiguration — Critical Distinction

| Aspect | @Configuration | @AutoConfiguration |
|----------------------|--------------------------------|--------------------------------------|
| Introduced | Spring 3.0 (2009) | Spring Boot 2.7 (2022) |
| Primary Use | User-defined application beans | Library/Starter auto-configuration |
| Loading Order | After all auto-configurations | Before user configurations |
| Component Scanning | Participates | Opt-out by default |
| Condition Evaluation | Standard | Optimized + cached |
| Registration | Implicit via @ComponentScan | Via META-INF/spring/... imports file |

Correct Usage Pattern

```
// ✅ CORRECT for user application configuration
@Configuration
@EnableConfigurationProperties(MyAppProperties.class)
public class MyAppConfig {
    @Bean
    public MyService myService() {
        return new MyService();
    }
}

// ✅ CORRECT for library/starter auto-configuration (Boot 2.7+)
@Configuration
@ConditionalOnClass(SomeLibrary.class)
@EnableConfigurationProperties(MyStarterProperties.class)
public class MyAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean // Let users override
    public SomeService someService() {
        return new SomeService();
    }
}
```

Auto-Configuration Loading Order Control

```
// Control execution order between auto-configurations
@Configuration(after = DataSourceAutoConfiguration.class)
@Configuration(before = WebMvcAutoConfiguration.class)
public class MyCustomAutoConfiguration {
    // Ensures this runs AFTER DataSource is set up
    // but BEFORE Web MVC configuration
}
```

PART 3: STARTER DEPENDENCIES DEEP DIVE

Common Starters and What They Include

| Starter | Transitive Dependencies | Use Case |
|---------|-------------------------|----------|
|---------|-------------------------|----------|

| | | |
|----------------------------------|---|------------------------------|
| spring-boot-starter-web | Spring MVC, Jackson, Tomcat, Validation | REST APIs, web apps |
| spring-boot-starter-webflux | Spring WebFlux, Reactor Netty | Reactive applications |
| spring-boot-starter-data-jpa | Hibernate, Spring Data JPA, HikariCP | Database access with ORM |
| spring-boot-starter-data-mongodb | MongoDB driver, Spring Data MongoDB | Document database |
| spring-boot-starter-security | Spring Security Core, OAuth2 client | Authentication/Authorization |
| spring-boot-starter-actuator | Micrometer, health endpoints | Monitoring, metrics |
| spring-boot-starter-cloud | Spring Cloud context, discovery client | Microservices |
| spring-boot-starter-test | JUnit 5, Mockito, AssertJ, Testcontainers | Unit/integration testing |

Custom Starter — When and How

WHEN TO CREATE CUSTOM STARTER

✓ Good candidates:

- Internal company libraries used across microservices
- Common security configuration (JWT validation)
- Standard logging/MDC setup
- Database/queue connection factories
- Shared DTOs and validation logic

✗ Not recommended:

- Single-use configuration
- Highly volatile business logic
- Domain-specific aggregates

PART 4: PRODUCTION-GRADE CONFIGURATION

Essential application.yml for Production

```
# =====
# PRODUCTION SPRING BOOT CONFIGURATION (APPLICATION.YAML)
# =====

server:
  port: 8080
  tomcat:
    max-connections: 10000      # Handle concurrent connections
    threads:
      max: 800                 # Worker threads (CPU cores * 2-4)
      min-spare: 100           # Ready for traffic spikes
    accept-count: 100           # Queue when threads busy
    connection-timeout: 20000   # 20 seconds max wait

spring:
  datasource:
    hikari:
      maximum-pool-size: 50     # NOT infinite—database has limits
      minimum-idle: 10
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000
      leak-detection-threshold: 60000 # Detect connection leaks

  jackson:
    time-zone: UTC              # No server-local time surprises
    date-format: yyyy-MM-dd HH:mm:ss
    serialization:
      write-dates-as-timestamps: false

  servlet:
    multipart:
      max-file-size: 100MB
      max-request-size: 100MB

  task:
    execution:
      pool:
        core-size: 8            # @Async thread pool
        max-size: 16
        queue-capacity: 100
        thread-name-prefix: async-
```

```
logging:
  level:
    com.mycompany: INFO
    org.springframework.web: WARN
  logback:
    rollingpolicy:
      max-file-size: 100MB
      max-history: 30
      total-size-cap: 3GB

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: when-authorized
  metrics:
    export:
      prometheus:
        enabled: true
```

PART 5: PERFORMANCE TUNING — PRODUCTION READY

Dependency Management for Fast Startup

STARTUP TIME OPTIMIZATION STRATEGIES

1. Remove Unused Dependencies

```
$ mvn dependency:analyze
```

Identifies declared but unused dependencies

2. Exclude Unnecessary Auto-Configurations

```
@SpringBootApplication(exclude = {  
    DataSourceAutoConfiguration.class,  
    SecurityAutoConfiguration.class  
})
```

3. Enable Lazy Initialization

```
spring.main.lazy-initialization=true
```

⚠ Warning: First request pays initialization cost

4. Use Spring Context Indexer

```
<dependency>  
    <groupId>org.springframework</groupId>  
    <artifactId>spring-context-indexer</artifactId>  
    <optional>true</optional>  
</dependency>
```

Generates META-INF/spring.components → eliminates scanning

JVM Tuning for Spring Boot

Heap Configuration

```
# CONTAINERIZED ENVIRONMENT (Docker/K8s with 1GB limit)
java -Xms600m -Xmx600m \
    -XX:+UseG1GC \
    -XX:MaxGCPauseMillis=200 \
    -jar app.jar

# DEDICATED SERVER (16GB available)
java -Xms8g -Xmx8g \
    -XX:+UseZGC \
    -XX:+HeapDumpOnOutOfMemoryError \
    -XX:HeapDumpPath=/var/log/app \
    -jar app.jar
```

Garbage Collector Selection Guide

GC	When to Use	Heap Size	Trade-off
G1GC (Default)	General purpose, predictable pauses	4-32GB	Balanced throughput/latency
ZGC	Ultra-low latency (<10ms pauses)	Any, excels at large	Slightly lower throughput
Parallel GC	Batch processing, throughput priority	Any	Higher throughput, longer pauses

Embedded Server Tuning

Tomcat Optimization

```
server:
  tomcat:
    threads:
      max: 200                # Default, adjust based on CPU cores
      min-spare: 10           # Keep ready for spikes
    max-connections: 10000    # Connections before queuing
    accept-count: 100         # Queue size when all threads busy
```

Alternative Embedded Servers

Server	Best For	Key Feature
Tomcat (default)	General purpose	Mature, widely understood

Jetty	Cloud-native, memory-constrained	Lower memory footprint
Undertow	High throughput	Non-blocking I/O, WebSocket support

Interview Insight: *"For most Spring Boot applications, default Tomcat works fine. Switch to Jetty if you're memory-constrained (<512MB heap). Undertow shines for WebSocket-heavy or extreme throughput scenarios."*

PART 6: SPRING BOOT ACTUATOR — PRODUCTION READY

Critical Actuator Endpoints

ACTUATOR ENDPOINTS MATRIX	
Endpoint	Purpose
/health	Liveness + Readiness probes for K8s
/info	Build info, git commit, custom metadata
/metrics	JVM memory, GC, thread, HTTP request metrics
/prometheus	Metrics in Prometheus format
/threaddump	Thread dump for deadlock analysis
/heapdump	Heap dump for memory leak investigation
/env	Environment properties (sensitive—secure it)
/configprops	All @ConfigurationProperties beans
/beans	All Spring beans in context
/mappings	Request mapping routes

Production Security Configuration

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus # NOT /env or /heapdump
        base-path: /internal/actuator           # Obscure from attackers
  endpoint:
    health:
      show-details: when-authorized
      probes:
        enabled: true # For Kubernetes
```

Kubernetes Probes Integration

```
# Deployment.yaml
livenessProbe:
  httpGet:
    path: /internal/actuator/health/liveness
    port: 8080
  initialDelaySeconds: 60
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /internal/actuator/health/readiness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 5
```

PART 7: SPRING BOOT WITH MICROSERVICES — SPRING CLOUD

Spring Cloud Component Map

SPRING CLOUD MICROSERVICES STACK

Spring Cloud Gateway (API Gateway)

- Routing • Rate limiting • Security filtering



Spring Cloud Netflix / Kubernetes Native

- Service Discovery (Eureka / K8s API)
- Client-side Load Balancing (Ribbon / Spring Cloud LoadBalancer)



Spring Cloud Circuit Breaker (Resilience4j)

- Retry • Timeout • Circuit Breaker • Rate Limiter



Spring Cloud Stream / Spring for Apache Kafka

- Event-driven communication
- Binders for Kafka, RabbitMQ



Spring Cloud Sleuth / Micrometer Tracing

- Distributed tracing (OpenTelemetry)
- W3C Trace Context propagation

Resilience4j Configuration (Circuit Breaker)

```
# application.yml
resilience4j:
  circuitbreaker:
    instances:
      payment-service:
        sliding-window-size: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 5
        automatic-transition-from-open-to-half-open-enabled: true

  retry:
    instances:
      payment-service:
        max-attempts: 3
        wait-duration: 1s
        retry-exceptions:
          - org.springframework.dao.TransientDataAccessException
```

```
@CircuitBreaker(name = "payment-service", fallbackMethod = "fallbackPayment")
@Retry(name = "payment-service")
public PaymentResponse processPayment(PaymentRequest request) {
    return paymentClient.process(request);
}

public PaymentResponse fallbackPayment(PaymentRequest request, Exception e) {
    return PaymentResponse.pending("Payment queued for retry");
}
```

PART 8: TESTING DEEP DIVE

Testing Annotations Reference

Annotation	Use Case	Loads
@SpringBootTest	Full integration test	Full application context
@WebMvcTest	Controller layer only	Only web layer, @Controller, @ControllerAdvice
@DataJpaTest	Repository layer	JPA repositories, in-memory DB

@JsonTest	JSON serialization	Jackson/ObjectMapper
@DataRedisTest	Redis operations	Redis repositories
@RestClientTest	REST client testing	RestTemplate, WebClient

Mock vs Spy — Staff Level Understanding

```
// MOCK: Complete fake implementation (default: return null/empty)
@Mock
userRepository repository; // ALL methods return null/empty collections

when(repository.findById(1L)).thenReturn(Optional.of(user));

// SPY: Real object with ability to stub specific methods
@Spy
userService service; // REAL methods execute unless stubbed

doReturn("forced response").when(service).getExternalData(any());
// Other methods execute REAL implementation
```

PART 9: SPRING BOOT 3.x — WHAT'S DIFFERENT

Breaking Changes (Boot 2 → Boot 3)

Area	Boot 2	Boot 3
Java Baseline	Java 8/11	Java 17 minimum
Jakarta EE	javax.*	jakarta.*
Hibernate	5.x	6.x
Security	Spring Security 5	Spring Security 6 (lambda DSL)
Tracing	Spring Cloud Sleuth	Micrometer Tracing
GraalVM	Experimental	Native support

Jakarta EE Migration

```
// Boot 2 (Java EE)
import javax.persistence.Entity;
import javax.validation.constraints.NotNull;

// Boot 3 (Jakarta EE)
import jakarta.persistence.Entity;
import jakarta.validation.constraints.NotNull;
```

PART 10: INTERVIEW POWER PHRASES — STAFF LEVEL

Spring vs Spring Boot

"Spring Boot is NOT a replacement for Spring—it's an opinionated layer on top that provides auto-configuration, starter dependencies, and embedded servers. Spring Boot's biggest innovation is shifting from configuration files to conditional auto-configuration based on classpath."

Auto-Configuration Internals

"At startup, @EnableAutoConfiguration reads all META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports files, then evaluates @Conditional annotations. The optimization here is that conditions are evaluated in phases—class-level first, then method-level—with caching through spring-autoconfigure-metadata.json."

Lazy Initialization Trade-offs

"Setting spring.main.lazy-initialization=true reduces startup time by delaying bean creation until first use. But the first request becomes slower, and runtime failures may appear only after deployment. I prefer selective @Lazy for specific beans rather than blanket lazy initialization."

Connection Pool Sizing

*"The formula for HikariCP pool size is (core_count * 2) + spinning_disk_count. For most applications, 50 is too high—you'll get connection contention. Start with 10-20, monitor, adjust. The database has limits too."*

Embedded Server Selection

"Tomcat is fine for 95% of cases. Switch to Jetty if running memory-constrained (<512MB heap because of cloud costs). Undertow gives the best WebSocket performance. The choice reveals more about your operational constraints than technical preference."

Actuator in Production

"Actuator is not just for humans— /health/liveness and /health/readiness are essential for Kubernetes. We also use /metrics to feed Prometheus, and /heapdump on-demand for memory leak investigations. Never expose /env or /configprops without strict network restrictions."

Custom Starters

"We use custom starters for cross-cutting concerns—JWT validation, MDC logging setup, and standard HTTP client configuration. A proper starter uses @AutoConfiguration (not @Configuration) and exports conditionally with @ConditionalOnMissingBean to allow per-service overrides."

QUICK REFERENCE CARD

Essential Annotations

Annotation	Purpose
@SpringBootApplication	Main entry (composes @EnableAutoConfig + @ComponentScan + @Configuration)
@EnableAutoConfiguration	Enables auto-config mechanism
@ConfigurationProperties	Bind external config to typed objects
@ConditionalOnXxx	Conditional auto-config (Class, Bean, Property, MissingBean)
@RestControllerAdvice	Global exception handling
@Async + @EnableAsync	Asynchronous method execution

Production Checklist

- ❑ Actuator endpoints secured (not exposed publicly)
- ❑ Database connection pool sized appropriately
- ❑ Jackson timezone set to UTC
- ❑ Log rolling policy configured
- ❑ Heap size set (-Xms = -Xmx)
- ❑ Unused dependencies removed
- ❑ Unnecessary auto-configurations excluded
- ❑ Health probes for Kubernetes configured
- ❑ Distributed tracing enabled (if microservices)

Performance Baseline Numbers

Metric	Healthy Range
Startup time (simple service)	2-5 seconds
Startup time (complex service)	15-30 seconds
Memory footprint (minimum)	200-400 MB
Heap usage (steady state)	40-70% of -Xmx
GC pause time	< 200ms (G1GC) / < 10ms (ZGC)

This document covers Spring/Spring Boot at the depth expected for a 16-year experienced professional. Focus on understanding the **trade-offs** and **why** decisions are made, not just the syntax.

SPRING ANNOTATIONS MASTER REFERENCE

Complete Guide with Usage Patterns — For 16-Year Experience Professional

PART 1: STEREOTYPE ANNOTATIONS (Component Model)

Core Stereotypes

Annotation	Purpose	Meta-Annotation	Typical Layer
------------	---------	-----------------	---------------

@Component	Generic Spring-managed bean	None	Utility classes, wiring
@Service	Business logic marker	@Component	Service layer
@Repository	DAO marker with exception translation	@Component	Data access layer
@Controller	Web controller (returns view)	@Component	Web layer
@RestController	REST controller (returns data)	@Controller + @ResponseBody	REST API layer

Usage Patterns

```

// =====
// @Component - Generic Spring Bean
// =====
@Component
public class EmailValidator {
    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,6}$",
Pattern.CASE_INSENSITIVE);

    public boolean isValid(String email) {
        return EMAIL_PATTERN.matcher(email).matches();
    }
}

// =====
// @Service - Business Logic (Explicit intent)
// =====
@Service
@Slf4j
@Transactional(readOnly = true)
public class OrderService {

    private final OrderRepository orderRepository;
    private final PaymentClient paymentClient;

    public OrderService(OrderRepository orderRepository, PaymentClient
paymentClient) {
        this.orderRepository = orderRepository;
        this.paymentClient = paymentClient;
    }

    @Transactional
    public Order createOrder(OrderRequest request) {
        log.info("Creating order for customer: {}", request.getCustomerId());
        Order order = Order.fromRequest(request);
        return orderRepository.save(order);
    }
}

// =====
// @Repository - Data Access with Exception Translation
// =====

```

```

@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {
    // Spring automatically translates SQLException to DataAccessException
    List<Order> findByCustomerIdAndStatus(Long customerId, OrderStatus status);
}

// =====
// @RestController - REST API Controller
// =====
@RestController
@RequestMapping("/api/v1/orders")
@Slf4j
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public OrderResponse createOrder(@Valid @RequestBody OrderRequest request) {
        return orderService.createOrder(request);
    }
}

```

PART 2: DEPENDENCY INJECTION ANNOTATIONS

Injection Annotations Matrix

Annotation	Resolution	Best For	Null Handling
@Autowired	By type, then by name	Constructor injection (recommended)	required=true by default
@Inject (JSR-330)	Same as @Autowired	Portable code (JSR-330 compatibility)	Optional with @Nullable

@Resource (JSR-250)	By name first, then by type	Java EE legacy, named resources	Required by default
@Qualifier	Disambiguates multiple beans	When multiple beans of same type	N/A
@Primary	Preferred bean when ambiguous	Default implementation override	N/A

Constructor Injection (Recommended Pattern)

```

// =====
// RECOMMENDED: Constructor Injection (Immutable, testable)
// =====
@RestController
public class OrderController {

    private final OrderService orderService;
    private final PaymentService paymentService;
    private final NotificationClient notificationClient;

    // @Autowired optional on single constructor (Spring 4.3+)
    public OrderController(
        OrderService orderService,
        PaymentService paymentService,
        NotificationClient notificationClient) {
        this.orderService = orderService;
        this.paymentService = paymentService;
        this.notificationClient = notificationClient;
    }
}

// =====
// Field Injection (NOT recommended - hard to test)
// =====
@RestController
public class OrderController {

    @Autowired // ❌ Reflection-based, not testable easily
    private OrderService orderService;
}

// =====
// Setter Injection (Optional dependencies)
// =====
@RestController
public class OrderController {

    private AuditService auditService; // optional dependency

    @Autowired(required = false)
    public void setAuditService(AuditService auditService) {
        this.auditService = auditService;
    }
}

```

```

    }
}

// =====
// @Qualifier - Multiple beans of same type
// =====
@Service
public class PaymentService {

    @Autowired
    @Qualifier("primaryPaymentGateway")
    private PaymentGateway paymentGateway;

    @Autowired
    @Qualifier("fallbackPaymentGateway")
    private PaymentGateway fallbackGateway;
}

// =====
// @Primary - Define default implementation
// =====
@Component
@Primary
public class PrimaryPaymentGateway implements PaymentGateway {
    // This bean is chosen when no @Qualifier specified
}

@Component
public class SecondaryPaymentGateway implements PaymentGateway {
    // Explicit @Qualifier needed to use this
}

```

PART 3: CONFIGURATION ANNOTATIONS

Configuration Annotations

Annotation	Purpose	Key Feature
@Configuration	Define bean definitions	Full configuration class

@Bean	Declare Spring bean inside @Configuration	Method returns bean instance
@ConfigurationProperties	Bind external properties to typed object	Type-safe configuration
@PropertySource	Load properties file	External configuration source
@Profile	Conditional bean activation per environment	Environment-specific beans
@Conditional	Custom conditional bean registration	Advanced condition logic
@Import	Import additional configuration classes	Modular configuration

Configuration Patterns

```
// =====
// @Configuration + @Bean - Full Configuration Class
// =====
@Configuration
@EnableConfigurationProperties(AppProperties.class)
@Slf4j
public class AppConfig {

    @Bean
    @Profile("development")
    public DataSource devDataSource() {
        log.info("Creating development H2 database");
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .addScript("schema.sql")
            .build();
    }

    @Bean
    @Profile("production")
    @ConditionalOnProperty(name = "database.type", havingValue = "postgres")
    public DataSource prodDataSource(AppProperties properties) {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl(properties.getDatabase().getUrl());
        config.setUsername(properties.getDatabase().getUsername());
        config.setPassword(properties.getDatabase().getPassword());
        config.setMaximumPoolSize(properties.getDatabase().getPoolSize());
        return new HikariDataSource(config);
    }

    @Bean
    @DependsOn({"dataSource", "cacheManager"}) // Ensure order
    public InitializationService initializationService() {
        return new InitializationService();
    }
}

// =====
// @ConfigurationProperties - Type-safe Configuration
// =====
@ConfigurationProperties(prefix = "app.payment")
@Component
```

```

@Data
public class PaymentProperties {

    private Gateway gateway = new Gateway();
    private Retry retry = new Retry();
    private CircuitBreaker circuitBreaker = new CircuitBreaker();

    @Data
    public static class Gateway {
        private String url = "https://default.payment.com";
        private int timeoutSeconds = 30;
        private String apiKey;
    }

    @Data
    public static class Retry {
        private int maxAttempts = 3;
        private Duration initialDelay = Duration.ofSeconds(1);
        private double multiplier = 2.0;
    }

    @Data
    public static class CircuitBreaker {
        private int failureThreshold = 5;
        private Duration timeout = Duration.ofSeconds(30);
    }
}

// application.yml
// app:
//   payment:
//     gateway:
//       url: https://prod.payment.com
//       timeout-seconds: 10
//     retry:
//       max-attempts: 5
//       initial-delay: 500ms

// =====
// @PropertySource - Custom Properties File
// =====
@Configuration
@PropertySource(value = "classpath:secrets.properties", ignoreResourceNotFound =

```

```

true)
@PropertySource(value = "file:/etc/app/secrets.properties", ignoreResourceNotFound
= true)
public class SecretsConfig {

    @Value("${encryption.key}")
    private String encryptionKey;

    @Value("${jwt.secret:defaultSecret}") // with default value
    private String jwtSecret;
}

// =====
// @Import - Modular Configuration
// =====
@Configuration
@Import({DatabaseConfig.class, CacheConfig.class, SecurityConfig.class})
public class RootConfig {
    // Aggregates multiple configuration classes
}

```

Conditional Annotations (Spring Boot)

```
// =====
// CONDITIONAL ANNOTATIONS - Production Patterns
// =====

@Configuration
public class ConditionalConfig {

    // Conditional on class presence
    @Bean
    @ConditionalOnClass(name = "org.apache.kafka.clients.producer.KafkaProducer")
    public KafkaTemplate<String, Object> kafkaTemplate() {
        return new KafkaTemplate<>();
    }

    // Conditional on missing bean
    @Bean
    @ConditionalOnMissingBean(ObjectMapper.class)
    public ObjectMapper defaultObjectMapper() {
        return new ObjectMapper()
            .registerModule(new JavaTimeModule())
            .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    }

    // Conditional on property value
    @Bean
    @ConditionalOnProperty(name = "cache.enabled", havingValue = "true",
matchIfMissing = false)
    public CacheManager cacheManager() {
        return new CaffeineCacheManager();
    }

    // Conditional on web application
    @Bean
    @ConditionalOnWebApplication
    public WebMvcConfigurer webMvcConfigurer() {
        return new CustomWebMvcConfigurer();
    }

    // Conditional on expression
    @Bean
    @ConditionalOnExpression("'${app.environment}' != 'test' &&
${app.monitoring.enabled:true}")

```

```

public MonitoringService monitoringService() {
    return new MonitoringService();
}

// Java 8+ Optional pattern
@Bean
@ConditionalOnJava(JavaVersion.SEVENTEEN)
public Java17FeatureService java17Service() {
    return new Java17FeatureService();
}

// Custom annotation combining conditions
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@ConditionalOnClass(RedisConnectionFactory.class)
@ConditionalOnProperty(name = "redis.enabled", havingValue = "true")
public @interface ConditionalOnRedisEnabled {
}

@Bean
@ConditionalOnRedisEnabled
public RedisTemplate<String, Object> redisTemplate() {
    return new RedisTemplate<>();
}
}

```

PART 4: WEB & REST ANNOTATIONS

Request Mapping Annotations

Annotation	Purpose	Default Method
@RequestMapping	General mapping	GET (if method not specified)
@GetMapping	HTTP GET	GET
@PostMapping	HTTP POST	POST
@PutMapping	HTTP PUT	PUT
@DeleteMapping	HTTP DELETE	DELETE

@PatchMapping	HTTP PATCH	PATCH
---------------	------------	-------

Parameter Binding Annotations

Annotation	Binds To	Example
@PathVariable	URL template variable	/orders/{id} → @PathVariable Long id
@RequestParam	Query parameter	?page=1&size=10 → @RequestParam int page
@RequestBody	HTTP request body	JSON body → @RequestBody Order order
@RequestHeader	HTTP header	Authorization: Bearer xxx → @RequestHeader String auth
@CookieValue	Cookie value	JSESSIONID=abc → @CookieValue String sessionId
@ModelAttribute	Form data or query params	Form POST → @ModelAttribute User user
@MatrixVariable	Matrix variable	/cars;color=red;year=2020

Complete REST Controller Patterns

```

// =====
// COMPLETE REST CONTROLLER – PRODUCTION PATTERN
// =====
@RestController
@RequestMapping("/api/v1/orders")
@Slf4j
@Validated // Enable method-level validation
public class OrderController {

    private final OrderService orderService;
    private final OrderMapper orderMapper;

    public OrderController(OrderService orderService, OrderMapper orderMapper) {
        this.orderService = orderService;
        this.orderMapper = orderMapper;
    }

    // =====
    // GET with Path Variable + Request Params
    // =====
    @GetMapping("/{orderId}")
    public ResponseEntity<OrderResponse> getOrder(
        @PathVariable("orderId") Long id,
        @RequestParam(value = "includeDetails", defaultValue = "false")
boolean includeDetails,
        @RequestHeader(value = "X-Request-ID", required = false) String
requestId) {

        log.info("Fetching order {} (request-id: {})", id, requestId);
        Order order = orderService.findById(id);
        OrderResponse response = orderMapper.toResponse(order, includeDetails);
        return ResponseEntity.ok(response);
    }

    // =====
    // GET with Pagination
    // =====
    @GetMapping
    public Page<OrderSummary> listOrders(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size,
        @RequestParam(required = false) OrderStatus status,

```



```

        @RequestParam(defaultValue = "createdAt") String sortBy,
        @PageableDefault(size = 20, sort = "createdAt", direction =
Sort.Direction.DESC) Pageable pageable) {

    return orderService.findAll(status, pageable);
}

// =====
// POST with Validation
// =====
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public OrderResponse createOrder(
    @Valid @RequestBody OrderRequest request,
    @RequestHeader("Authorization") String authToken) {

    // @Valid triggers validation automatically
    Order order = orderService.createOrder(request, authToken);
    return orderMapper.toResponse(order, false);
}

// =====
// PUT (Full Update)
// =====
@PutMapping("/{orderId}")
public OrderResponse updateOrder(
    @PathVariable Long orderId,
    @Valid @RequestBody OrderUpdateRequest request) {

    Order updated = orderService.updateOrder(orderId, request);
    return orderMapper.toResponse(updated, false);
}

// =====
// PATCH (Partial Update)
// =====
@PatchMapping("/{orderId}/status")
public OrderResponse updateOrderStatus(
    @PathVariable Long orderId,
    @RequestBody StatusUpdateRequest request) {

    Order updated = orderService.updateStatus(orderId, request.getStatus());
    return orderMapper.toResponse(updated, false);
}

```

```

    }

    // =====
    // DELETE
    // =====
    @DeleteMapping("/{orderId}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteOrder(@PathVariable Long orderId) {
        orderService.deleteOrder(orderId);
    }
}

// =====
// @ModelAttribute - Form Binding (Traditional MVC)
// =====
@Controller
@RequestMapping("/users")
public class UserController {

    @ModelAttribute("countries")
    public List<String> populateCountries() {
        // This attribute available to ALL handlers in this controller
        return List.of("USA", "Canada", "Mexico");
    }

    @PostMapping("/register")
    public String registerUser(@ModelAttribute UserRegistrationForm form, Model
model) {
        // form populated from form parameters
        userService.register(form);
        model.addAttribute("message", "Registration successful");
        return "success";
    }
}

```

PART 5: VALIDATION ANNOTATIONS (Bean Validation)

Jakarta Validation Annotations (javax.validation → jakarta.validation)

Annotation	Purpose	Example
<code>@NotNull</code>	Value cannot be null	<code>@NotNull String name</code>
<code>@NotEmpty</code>	CharSequence/Collection/Map not null and not empty	<code>@NotEmpty List<Item> items</code>
<code>@NotBlank</code>	String not null and trimmed length > 0	<code>@NotBlank String username</code>
<code>@Size</code>	Size between min and max	<code>@Size(min=2, max=50) String name</code>
<code>@Min</code> / <code>@Max</code>	Numeric minimum/maximum	<code>@Min(0) @Max(100) int percent</code>
<code>@Pattern</code>	Regex pattern match	<code>@Pattern(regex="^[A-Z]+\$") String code</code>
<code>@Email</code>	Valid email format	<code>@Email String email</code>
<code>@Past</code> / <code>@Future</code>	Date in past/future	<code>@Past LocalDate birthDate</code>
<code>@Positive</code> / <code>@Negative</code>	Positive or negative number	<code>@Positive int quantity</code>
<code>@Digits</code>	Integer/fraction digit limit	<code>@Digits(integer=10, fraction=2) BigDecimal amount</code>

Custom Validation Example

```
// =====
// DTO with Validation Annotations
// =====
@Data
public class OrderRequest {

    @NotBlank(message = "Customer ID is required")
    private String customerId;

    @NotNull(message = "Order items cannot be null")
    @Size(min = 1, message = "Order must have at least one item")
    private List<OrderItem> items;

    @Positive(message = "Total amount must be positive")
    private BigDecimal totalAmount;

    @Email(message = "Invalid email format")
    @NotBlank(message = "Email is required")
    private String email;

    @Pattern(regex = "^[A-Z]{2}[0-9]{6}$", message = "Invalid promotion code
format")
    private String promotionCode;

    @Future(message = "Delivery date must be in the future")
    private LocalDate deliveryDate;
}

// =====
// Group Validation – Different rules for different operations
// =====
public class ValidationGroups {
    public interface Create {}
    public interface Update {}
    public interface Patch {}
}

@Data
public class ProductRequest {

    @NotNull(groups = {Create.class, Update.class})
    private Long id;
}
```

```

@NotBlank(groups = {Create.class, Update.class, Patch.class})
private String name;

@Positive(groups = {Create.class, Update.class})
private BigDecimal price;

@Null(groups = Create.class) // Can't set on create
@Positive(groups = Update.class)
private BigDecimal discountedPrice;
}

@RestController
public class ProductController {

    @PostMapping("/products")
    public Product create(@Validated(ValidationGroups.Create.class) @RequestBody
ProductRequest request) {
        return productService.create(request);
    }

    @PutMapping("/products/{id}")
    public Product update(@Validated(ValidationGroups.Update.class) @RequestBody
ProductRequest request) {
        return productService.update(request);
    }
}

// =====
// Custom Validation Annotation
// =====
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = UniqueUsernameValidator.class)
@Documented
public @interface UniqueUsername {
    String message() default "Username already exists";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

@Component
public class UniqueUsernameValidator implements

```

```
ConstraintValidator<UniqueUsername, String> {

    @Autowired
    private UserRepository userRepository;

    @Override
    public boolean isValid(String username, ConstraintValidatorContext context) {
        if (username == null) return true;
        return !userRepository.existsByUsername(username);
    }
}

// Usage
@Data
public class UserRegistration {
    @UniqueUsername
    @NotBlank
    private String username;
}
```

PART 6: TRANSACTION MANAGEMENT ANNOTATIONS

@Transactional Deep Dive

```

// =====
// @TRANSACTIONAL – COMPLETE REFERENCE
// =====

@Service
public class OrderService {

    // =====
    // Basic Usage – Class Level
    // =====
    @Transactional(readOnly = true) // All read-only methods inherit
    public class ReadOnlyService {

        @Transactional // Overrides to read-write
        public Order createOrder(OrderRequest request) {
            // Transaction starts here
            Order order = orderRepository.save(new Order());
            paymentService.process(order); // Propagates transaction
            return order;
            // Commit on successful return
        }

        @Transactional(readOnly = true)
        public Order findById(Long id) {
            return orderRepository.findById(id).orElseThrow();
        }
    }

    // =====
    // Propagation Levels
    // =====
    @Transactional(propagation = Propagation.REQUIRED) // DEFAULT – Join or
create
    public void requiredExample() {}

    @Transactional(propagation = Propagation.REQUIRES_NEW) // Always suspend
current, create new
    public void requiresNewExample() {
        // Suspends existing transaction, starts new one
        auditService.log("operation"); // Logs even if outer transaction rolls
back
    }
}

```

```

    @Transactional(propagation = Propagation.NESTED) // Savepoint within existing
transaction
    public void nestedExample() throws Exception {
        // Creates savepoint, can roll back partial work
        try {
            riskyOperation();
        } catch (Exception e) {
            TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
            // Only nested part rolls back
        }
    }

    @Transactional(propagation = Propagation.SUPPORTS) // Execute within
transaction if exists, else non-transactional
    public void supportsExample() {
        // Fine for read queries
    }

    @Transactional(propagation = Propagation.NOT_SUPPORTED) // Suspend
transaction, execute non-transactional
    public void notSupportedExample() {
        // Long-running non-transactional operation
        generateLargeReport();
    }

    @Transactional(propagation = Propagation.MANDATORY) // Must have existing
transaction
    public void mandatoryExample() {
        // Fails if called without transaction
    }

    @Transactional(propagation = Propagation.NEVER) // Must NOT have existing
transaction
    public void neverExample() {
        // Fails if called within transaction
    }

    // =====
    // Isolation Levels
    // =====

    // READ UNCOMMITTED – Dirty reads possible (lowest isolation)

```



```

@Transactional(isolation = Isolation.READ_UNCOMMITTED)
public void readUncommittedExample() {
    // Sees uncommitted changes from other transactions
    // ❌ Don't use – data consistency issues
}

// READ COMMITTED – Default in PostgreSQL, SQL Server
@Transactional(isolation = Isolation.READ_COMMITTED)
public void readCommittedExample() {
    // Only sees committed data
    // ✅ Good default for most OLTP systems
}

// REPEATABLE READ – Default in MySQL
@Transactional(isolation = Isolation.REPEATABLE_READ)
public void repeatableReadExample() {
    // Same query returns same data within transaction
    // Prevents non-repeatable reads
}

// SERIALIZABLE – Highest isolation, lowest concurrency
@Transactional(isolation = Isolation.SERIALIZABLE)
public void serializableExample() {
    // Complete isolation, no phantom reads
    // Use: Financial reconciliation, inventory allocation
}

// =====
// Rollback Rules
// =====

@Transactional(rollbackFor = {BusinessException.class,
DataAccessException.class})
public void specificRollbackExample() throws BusinessException {
    // Rolls back only on these exceptions
    if (invalidState) {
        throw new BusinessException("Invalid");
    }
    // RuntimeException triggers rollback by default
}

@Transactional(noRollbackFor = {OptimisticLockException.class})
public void noRollbackExample() {

```

```

        // OptimisticLockException won't cause rollback
        // Useful for retry scenarios
    }

    @Transactional(rollbackForClassName = {"BusinessException",
"ValidationException"})
    public void rollbackByClassNameExample() throws Exception {
        // Same as rollbackFor but with class names (for AOP proxies)
    }

    // =====
    // Timeout
    // =====

    @Transactional(timeout = 30) // Seconds
    public void timeoutExample() {
        // Transaction will be rolled back after 30 seconds
        // Prevents long-running transactions locking resources
    }

    // =====
    // Advanced – Transaction Template (Programmatic Control)
    // =====
    @Service
    public class AdvancedOrderService {

        private final TransactionTemplate transactionTemplate;

        public AdvancedOrderService(TransactionTemplate transactionTemplate) {
            this.transactionTemplate = transactionTemplate;
        }

        public Order createOrderWithRetry(OrderRequest request) {
            return transactionTemplate.execute(status -> {
                try {
                    Order order = createOrderInternal(request);
                    return order;
                } catch (OptimisticLockException e) {
                    status.setRollbackOnly();
                    throw new RetryableException(e);
                }
            });
        }
    }

```

```
}  
}
```

PART 7: CACHE ANNOTATIONS

Spring Cache Annotations

Annotation	Purpose	Example
@EnableCaching	Enable cache abstraction	@SpringBootApplication @EnableCaching
@Cacheable	Store result in cache	@Cacheable("products")
@CacheEvict	Remove from cache	@CacheEvict(value="products", key="#id")
@CachePut	Update cache without method execution	@CachePut(value="products", key="#product.id")
@Caching	Group multiple cache operations	Multiple evicts + caches
@CacheConfig	Class-level cache configuration	Shared cache names, key generator

Complete Caching Patterns

```
// =====
// CACHE CONFIGURATION
// =====
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager cacheManager = new CaffeineCacheManager();
        cacheManager.setCaffeine(Caffeine.newBuilder()
            .expireAfterWrite(10, TimeUnit.MINUTES)
            .expireAfterAccess(5, TimeUnit.MINUTES)
            .maximumSize(1000)
            .recordStats());
        return cacheManager;
    }

    @Bean
    public KeyGenerator customKeyGenerator() {
        return (target, method, params) -> {
            return target.getClass().getSimpleName() + "_" +
                method.getName() + "_" +
                Arrays.deepHashCode(params);
        };
    }
}

// =====
// @Cacheable – Store and Retrieve
// =====
@Service
@CacheConfig(cacheNames = "products", keyGenerator = "customKeyGenerator")
public class ProductService {

    // Simple caching
    @Cacheable(value = "products", key = "#id")
    public Product findById(Long id) {
        simulateSlowDatabase();
        return productRepository.findById(id).orElseThrow();
    }
}
```

```

// Conditional caching
@Cacheable(value = "products", condition = "#id > 100", unless =
"#result.price > 1000")
public Product findWithCondition(Long id) {
    return productRepository.findById(id).orElseThrow();
    // Caches only if id > 100 AND result price ≤ 1000
}

// Cache with list of keys
@Cacheable(value = "products", key = "{#category, #status}")
public List<Product> findByCategoryAndStatus(String category, ProductStatus
status) {
    return productRepository.findByCategoryAndStatus(category, status);
}

// Sync – Prevents cache stampede
@Cacheable(value = "products", key = "#id", sync = true)
public Product findByIdSync(Long id) {
    // Multiple concurrent calls wait for one to populate cache
    return productRepository.findById(id).orElseThrow();
}
}

// =====
// @CachePut – Force Cache Update
// =====
@Service
public class ProductUpdateService {

    // Always executes method, always updates cache
    @CachePut(value = "products", key = "#product.id")
    public Product updateProduct(Product product) {
        return productRepository.save(product);
        // Cache updated with new value
    }
}

// =====
// @CacheEvict – Remove from Cache
// =====
@Service
public class ProductDeleteService {

```

```

// Remove single entry
@CacheEvict(value = "products", key = "#id")
public void deleteProduct(Long id) {
    productRepository.deleteById(id);
}

// Remove all entries from cache
@CacheEvict(value = "products", allEntries = true)
public void clearAllProducts() {
    // Useful after bulk operations
}

// Evict before method execution
@CacheEvict(value = "products", key = "#id", beforeInvocation = true)
public Product updateWithFreshCache(Long id, ProductUpdateRequest request) {
    // Removes old cache BEFORE method executes
    return productRepository.save(updatedProduct);
}
}

// =====
// @Caching – Multiple Operations
// =====
@Service
public class BulkCacheService {

    @Caching(
        put = {
            @CachePut(value = "products", key = "#result.id"),
            @CachePut(value = "productsByName", key = "#result.name")
        },
        evict = {
            @CacheEvict(value = "productList", allEntries = true)
        }
    )
    public Product createProduct(Product product) {
        // Updates multiple caches, evicts list cache
        return productRepository.save(product);
    }

    @Caching(evict = {
        @CacheEvict(value = "products", key = "#id"),
        @CacheEvict(value = "productsByName", allEntries = true)
    })

```

```
    })  
    public void deleteWithMultipleEvicts(Long id) {  
        productRepository.deleteById(id);  
    }  
}
```

PART 8: ASYNCHRONOUS & SCHEDULING ANNOTATIONS

@Async & @EnableAsync

```

// =====
// ASYNC CONFIGURATION
// =====
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    @Bean(name = "taskExecutor")
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(25);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
        executor.setRejectedExecutionHandler(new
ThreadPoolExecutor.CallerRunsPolicy());
        executor.setWaitForTasksToCompleteOnShutdown(true);
        executor.setAwaitTerminationSeconds(60);
        executor.initialize();
        return executor;
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) ->
            log.error("Async method {} failed with exception: {}",
method.getName(), ex.getMessage());
    }
}

// =====
// @Async USAGE PATTERNS
// =====
@Service
@Slf4j
public class NotificationService {

    // Fire and forget
    @Async
    public void sendEmail(String to, String subject, String body) {
        // Executes in separate thread
    }
}

```



```

        // Caller doesn't wait
        try {
            emailClient.send(to, subject, body);
            log.info("Email sent to {}", to);
        } catch (Exception e) {
            log.error("Email failed to {}", to, e);
        }
    }

    // Async with return value (Future)
    @Async
    public CompletableFuture<String> processFileAsync(String filePath) {
        String result = fileProcessor.process(filePath);
        return CompletableFuture.completedFuture(result);
    }

    // Async with specific executor
    @Async("mailExecutor")
    public void sendBulkEmails(List<String> recipients) {
        // Uses dedicated email executor
        recipients.forEach(this::sendIndividualEmail);
    }

    // ListenableFuture (Spring 4.3+)
    @Async
    public ListenableFuture<Report> generateReport(ReportRequest request) {
        Report report = reportGenerator.generate(request);
        return new AsyncResult<>(report);
    }
}

// =====
// CALLER PATTERN – Using Async Methods
// =====
@RestController
public class ReportController {

    private final NotificationService notificationService;

    public ReportController(NotificationService notificationService) {
        this.notificationService = notificationService;
    }
}

```

```
@PostMapping("/reports/generate")
public ResponseEntity<String> generateReport(@RequestBody ReportRequest
request) {
    // Fire async
    notificationService.sendEmail(request.getEmail(), "Report Ready", "Your
report is processing");

    // Wait for async result
    CompletableFuture<String> future =
notificationService.processFileAsync(request.getFilePath());
    String result = future.get(30, TimeUnit.SECONDS); // Block with timeout

    return ResponseEntity.ok(result);
}
}
```

@Scheduled & @EnableScheduling

```
// =====
// SCHEDULING CONFIGURATION
// =====
@Configuration
@EnableScheduling
public class SchedulingConfig {

    @Bean
    public TaskScheduler taskScheduler() {
        ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
        scheduler.setPoolSize(5);
        scheduler.setThreadNamePrefix("scheduled-");
        scheduler.setWaitForTasksToCompleteOnShutdown(true);
        scheduler.setAwaitTerminationSeconds(60);
        return scheduler;
    }
}

// =====
// @SCHEDULED USAGE PATTERNS
// =====
@Component
@Slf4j
public class ScheduledTasks {

    // Fixed delay – between completion of last execution and start of next
    @Scheduled(fixedDelay = 5000) // 5 seconds after previous completion
    public void fixedDelayTask() {
        log.info("Running with fixed delay");
        // Execution time doesn't affect schedule start
    }

    // Fixed rate – between start times (may overlap)
    @Scheduled(fixedRate = 60000) // Every 60 seconds regardless of duration
    public void fixedRateTask() {
        log.info("Running with fixed rate");
        // ⚠ Risk of overlapping if task takes > interval
    }

    // Initial delay before first execution
    @Scheduled(fixedRate = 60000, initialDelay = 30000) // Start after 30 seconds
    public void delayedStartTask() {

```

```
        log.info("First run after 30 seconds, then every minute");
    }

    // Cron expression (most flexible)
    @Scheduled(cron = "0 0 2 * * MON-FRI") // 2 AM every weekday
    public void weeklyReportTask() {
        // Generate nightly reports
        reportService.generateDailyReport();
    }

    // Cron with timezone
    @Scheduled(cron = "0 0 9 * * *", zone = "America/New_York")
    public void usMarketOpenTask() {
        // Runs at 9 AM US Eastern time
    }

    // Using ZoneDateTime for dynamic scheduling
    @Scheduled(cron = "0 #{new java.util.Date().getHours()} * * * ?")
    public void dynamicCronTask() {
        // Cron expression can use placeholders
        log.info("Dynamic cron task");
    }
}
```

SPRING SECURITY & OAUTH2 / JWT DEEP DIVE

Complete Interview Guide — For 16-Year Experience Professional

PART 1: SPRING SECURITY ARCHITECTURE — THE BIG PICTURE

Core Components — What Talks to What

SPRING SECURITY ARCHITECTURE

Client
(Browser/
Mobile)

HTTP Request + Credentials



FILTER CHAIN (DelegatingFilterProxy)

Username Password Filter	▶ Basic Auth Filter	▶ Security Context Holder	▶ Exception Translate Filter
--------------------------------	---------------------------	---------------------------------	------------------------------------



AuthenticationManager

ProviderManager

DaoProvider	LdapProvider	...
-------------	--------------	-----



UserDetailsService
loadUserByUsername()



Response



Key Components — Quick Reference

Component	Responsibility	When Used
SecurityContextHolder	Stores current authentication	Every request
SecurityContext	Holds Authentication object	Per thread
Authentication	Principal + Credentials + Authorities	After authentication
AuthenticationManager	Main authentication interface	Entry point for auth
ProviderManager	Delegates to auth providers	Implementation of AuthManager
AuthenticationProvider	Specific auth logic (DB, LDAP, OAuth)	Actual authentication
UserDetailsService	Loads user by username	DB authentication
PasswordEncoder	Encodes/verifies passwords	Password storage
FilterChainProxy	Manages security filters	Every request
AccessDecisionManager	Authorization voting	URL/method access

PART 2: JWT DEEP DIVE

JWT Structure — What's Inside

JWT STRUCTURE

JWT = Header . Payload . Signature

HEADER (Base64Url encoded)

```
{
  "alg": "RS256",           // Algorithm (RS256, HS256)
  "typ": "JWT",
  "kid": "key-id-123"      // Key ID (for rotation)
}
```

+

PAYLOAD (Claims - Base64Url encoded)

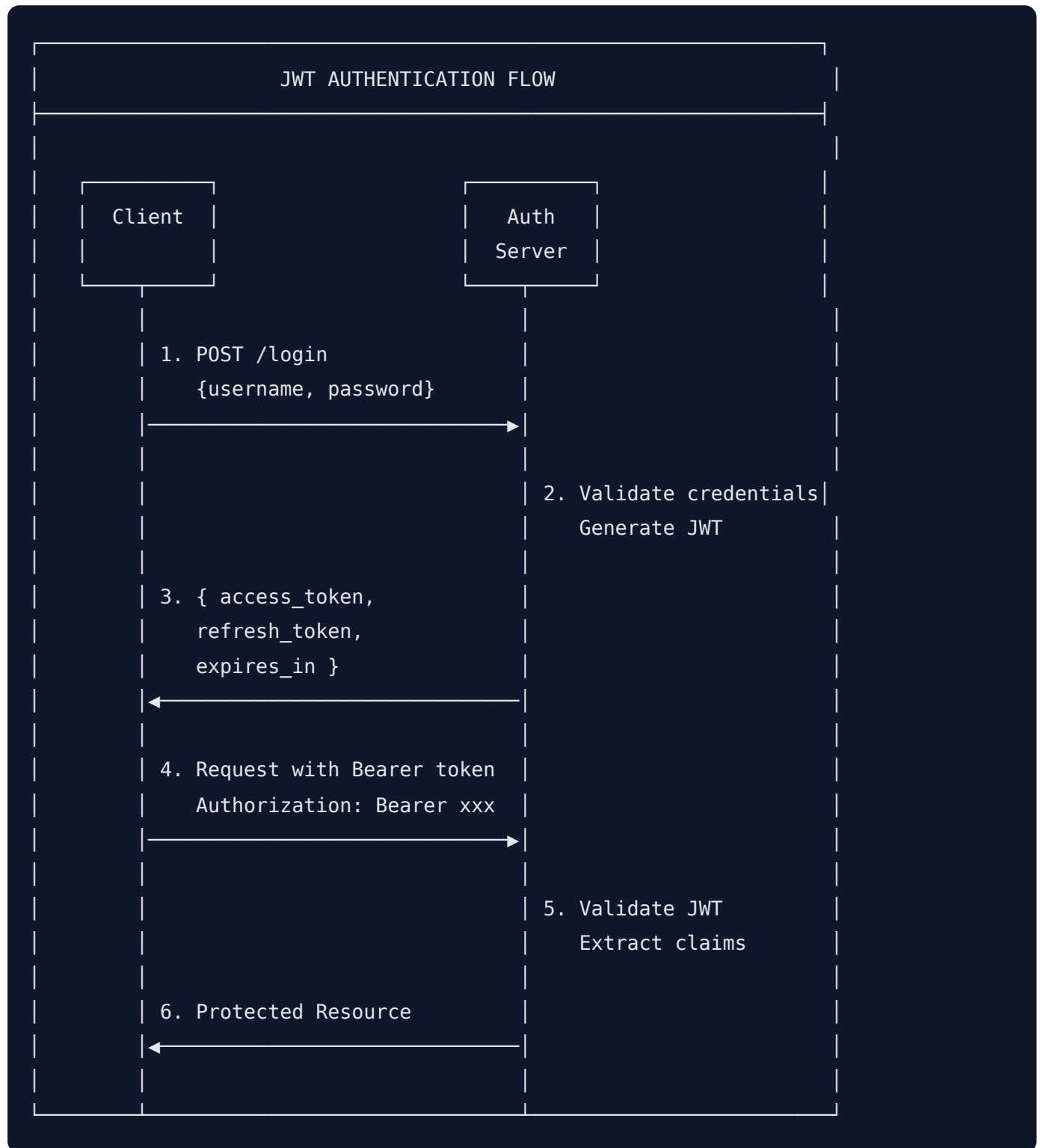
```
{
  // Registered Claims
  "iss": "https://auth.myapp.com", // Issuer
  "sub": "user123",                // Subject
  "aud": "api.myapp.com",          // Audience
  "exp": 1735689600,               // Expiration
  "nbf": 1735686000,               // Not Before
  "iat": 1735686000,               // Issued At
  "jti": "unique-token-id",        // JWT ID
  //
  // Custom Claims
  "roles": ["ADMIN", "USER"],
  "email": "user@example.com",
  "tenantId": "tenant_001"
}
```

+

SIGNATURE (Verifies integrity)

```
HMAC-SHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret
)
```


JWTs in Authentication Flow



JWT vs Session — Staff Level Comparison

Aspect	Session	JWT
Storage	Server-side memory/Redis	Client-side (no server storage)
Scalability	Shared session store needed	Stateless, scales infinitely
Revocation	Instant (delete session)	Hard (needs blacklist)
Size	Session ID only (~50 bytes)	Large token (500+ bytes)
Security	Not in client control	XSS/CSRF risks
Performance	DB/Redis lookup per request	Validates locally (fast)
Logout	Simple (clear session)	Complex (token still valid until expiry)

Interview Insight: "JWTs are NOT automatically more secure than sessions. They trade server-side storage for client-side storage. The main advantage is scalability — no session affinity or shared cache needed."

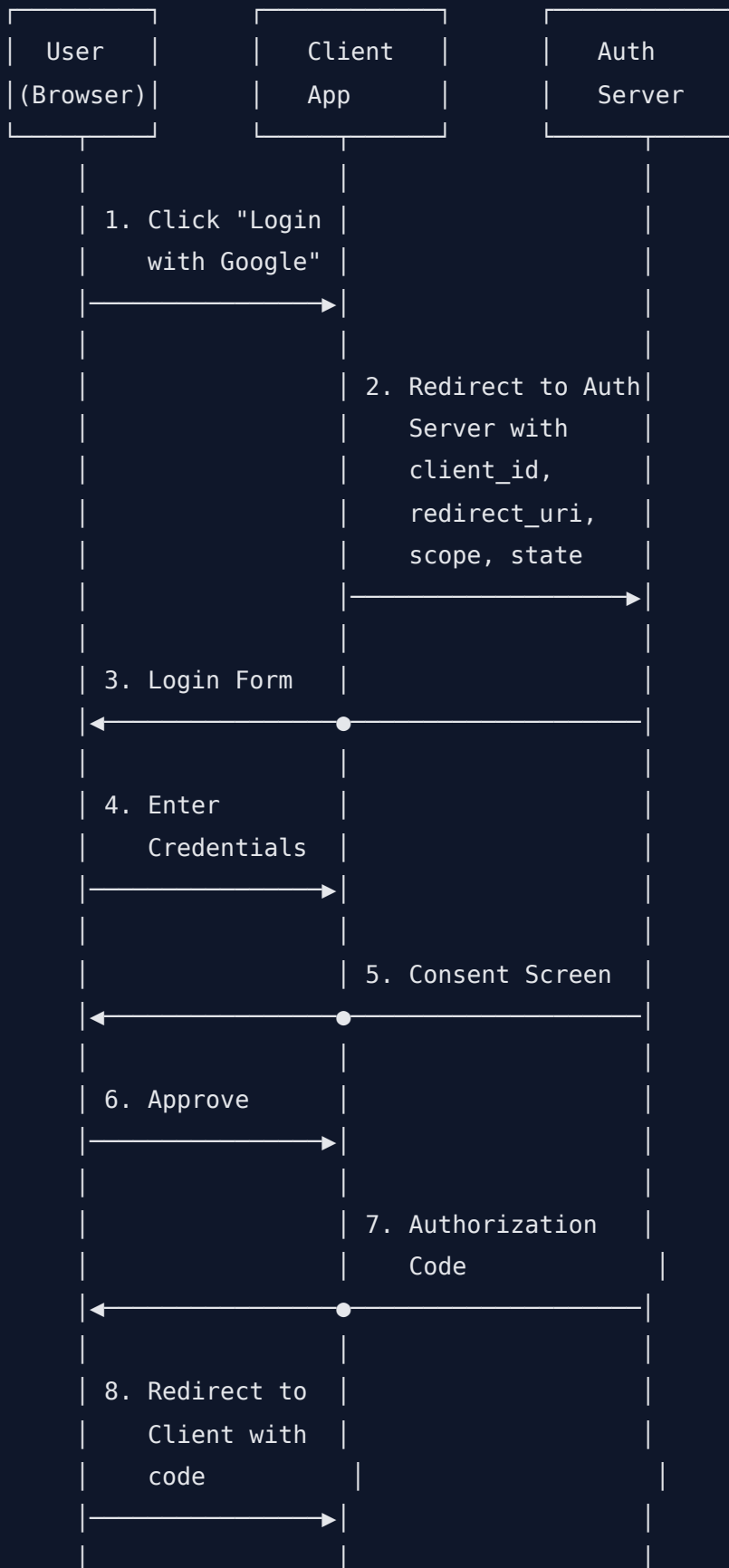
PART 3: OAUTH2 — COMPLETE FLOW

OAuth2 Roles — Who Does What

Role	Responsibility	Example
Resource Owner	Owns the data	End user
Client	App requesting access	Mobile app, Web app
Authorization Server	Issues tokens	Auth0, Keycloak, custom
Resource Server	Hosts protected data	API Gateway, Backend API

Authorization Code Flow (Most Common)

OAUTH2 AUTHORIZATION CODE FLOW





OAuth2 Grant Types — When to Use Which

Grant Type	When to Use	Security Level
Authorization Code	Web apps, mobile apps (with PKCE)	Highest
Client Credentials	Machine-to-machine (no user)	High
Password	Legacy/trusted apps (deprecated)	Low (avoid)
Implicit	SPA (deprecated, use PKCE)	Low (avoid)
Refresh Token	Get new access token without user	High

OAuth2 vs JWT vs OIDC — Clarification

OAUTH2 vs JWT vs OPENID CONNECT

OAUTH2 (Authorization Framework)

- Delegated access to resources
- Gives ACCESS to APIs (what can I do?)
- Defines flows (Authorization Code, Client Cred)
- NOT for authentication (doesn't verify identity)



OPENID CONNECT (OIDC) – Identity layer on OAuth2

- Authenticates USER (who am I?)
- Returns ID Token (JWT format)
- Adds id_token, userinfo endpoint
- Standard for "Login with Google/Facebook"



JWT (Token Format – Not a protocol)

- How tokens are FORMATTED
- Can be Access Token OR ID Token
- Self-contained, signed, optionally encrypted
- Alternative: Opaque tokens (database lookup)

PART 4: SPRING SECURITY + JWT — COMPLETE IMPLEMENTATION

Dependencies

```
<dependencies>
  <!-- Spring Security -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- JWT Support -->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
  </dependency>

  <!-- Web -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

JWT Service — Generate, Validate, Extract

```

@Component
@Slf4j
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.access-token-expiration}")
    private long accessTokenExpiration;

    @Value("${jwt.refresh-token-expiration}")
    private long refreshTokenExpiration;

    private final String ISSUER = "my-auth-server";

    // =====
    // Generate Access Token
    // =====
    public String generateAccessToken(UserDetails userDetails) {
        return generateToken(
            userDetails.getUsername(),
            userDetails.getAuthorities(),
            accessTokenExpiration
        );
    }

    // =====
    // Generate Token with Custom Claims
    // =====
    public String generateToken(String username,
                                Collection<? extends GrantedAuthority>
authorities,
                                long expiration) {

        Map<String, Object> claims = new HashMap<>();
        claims.put("roles", authorities.stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.toList()));
        claims.put("type", "access");

        return Jwts.builder()
            .setClaims(claims)

```

```

        .setSubject(username)
        .setIssuer(ISSUER)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + expiration))
        .signWith(getSignKey(), SignatureAlgorithm.HS256)
        .compact();
    }

    // =====
    // Generate Refresh Token (Longer expiry, fewer claims)
    // =====
    public String generateRefreshToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuer(ISSUER)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() +
refreshTokenExpiration))
            .claim("type", "refresh")
            .signWith(getSignKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    // =====
    // Validate Token
    // =====
    public boolean validateToken(String token) {
        try {
            Jwts.parserBuilder()
                .setSigningKey(getSignKey())
                .build()
                .parseClaimsJws(token);
            return true;
        } catch (ExpiredJwtException e) {
            log.warn("JWT expired: {}", e.getMessage());
        } catch (UnsupportedJwtException e) {
            log.warn("Unsupported JWT: {}", e.getMessage());
        } catch (MalformedJwtException e) {
            log.warn("Malformed JWT: {}", e.getMessage());
        } catch (SignatureException e) {
            log.warn("Invalid signature: {}", e.getMessage());
        } catch (IllegalArgumentException e) {
            log.warn("Illegal argument: {}", e.getMessage());
        }
    }

```



```

    }
    return false;
}

// =====
// Extract Username from Token
// =====
public String extractUsername(String token) {
    return extractAllClaims(token).getSubject();
}

// =====
// Extract Roles
// =====
@SuppressWarnings("unchecked")
public List<String> extractRoles(String token) {
    return extractAllClaims(token).get("roles", List.class);
}

// =====
// Check if Token is Refresh Token
// =====
public boolean isRefreshToken(String token) {
    return "refresh".equals(extractAllClaims(token).get("type"));
}

// =====
// Extract All Claims
// =====
private Claims extractAllClaims(String token) {
    return Jwts.parserBuilder()
        .setSigningKey(getSignKey())
        .build()
        .parseClaimsJws(token)
        .getBody();
}

// =====
// Get Signing Key
// =====
private Key getSignKey() {
    byte[] keyBytes = Decoders.BASE64.decode(secretKey);
    return Keys.hmacShaKeyFor(keyBytes);
}

```

```
}  
}
```

JWT Authentication Filter

```
@Component
@Slf4j
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtService jwtService;

    @Autowired
    private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");

        // Skip if no token
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        try {
            final String jwt = authHeader.substring(7);
            final String username = jwtService.extractUsername(jwt);

            // If username exists and not already authenticated
            if (username != null &&
                SecurityContextHolder.getContext().getAuthentication() == null) {

                UserDetails userDetails =
                    userDetailsService.loadUserByUsername(username);

                if (jwtService.validateToken(jwt)) {
                    // Create authentication object
                    UsernamePasswordAuthenticationToken authToken =
                        new UsernamePasswordAuthenticationToken(
                            userDetails,
                            null,
                            userDetails.getAuthorities())
                }
            }
        }
    }
}
```

```

        );

        authToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));

SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}
} catch (Exception e) {
    log.error("JWT authentication error: {}", e.getMessage());
    response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Invalid
token");
    return;
}

filterChain.doFilter(request, response);
}
}

```

Security Configuration

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationFilter jwtAuthFilter;

    @Autowired
    private UserDetailsService userDetailsService;

    // =====
    // Password Encoder
    // =====
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12); // Strength 12
    }

    // =====
    // Authentication Manager
    // =====
    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    // =====
    // Security Filter Chain (Spring Security 6+)
    // =====
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

        http
            .csrf(csrf -> csrf.disable()) // JWT doesn't need CSRF
            .cors(cors -> cors.configure(http))
            .authorizeHttpRequests(auth -> auth
                // Public endpoints
                .requestMatchers("/api/auth/**", "/api/public/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
            )
    }
}

```

```

        // Role-based access
        .requestMatchers("/api/admin/**").hasRole("ADMIN")
        .requestMatchers("/api/users/**").hasAnyRole("ADMIN", "USER")
        // Any other request needs authentication
        .anyRequest().authenticated()
    )
    .sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS) // No
sessions
    )
    .authenticationProvider(authenticationProvider())
    .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

// =====
// Custom Authentication Provider
// =====
@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    authProvider.setUserDetailsService(userDetailsService);
    authProvider.setPasswordEncoder(passwordEncoder());
    return authProvider;
}
}

```

Custom UserDetailsService

```

@Service
@Slf4j
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {

        User user = userRepository.findByEmail(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " +
username));

        return org.springframework.security.core.userdetails.User.builder()
            .username(user.getEmail())
            .password(user.getPassword())
            .authorities(user.getRoles().stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .toArray(GrantedAuthority[]::new))
            .accountExpired(!user.isAccountNonExpired())
            .accountLocked(!user.isAccountNonLocked())
            .credentialsExpired(!user.isCredentialsNonExpired())
            .disabled(!user.isEnabled())
            .build();
    }
}

```

Authentication Controller

```

@RestController
@RequestMapping("/api/auth")
@Slf4j
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtService jwtService;

    @Autowired
    private UserDetailsService userDetailsService;

    // =====
    // Login – Returns Access + Refresh Tokens
    // =====
    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest request) {

        // Authenticate
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.getEmail(),
                request.getPassword()
            )
        );

        // Generate tokens
        UserDetails userDetails = (UserDetails) authentication.getPrincipal();
        String accessToken = jwtService.generateAccessToken(userDetails);
        String refreshToken =
jwtService.generateRefreshToken(userDetails.getUsername());

        return ResponseEntity.ok(AuthResponse.builder()
            .accessToken(accessToken)
            .refreshToken(refreshToken)
            .tokenType("Bearer")
            .expiresIn(3600) // 1 hour
            .build());
    }
}

```



```

// =====
// Refresh Token – Get New Access Token
// =====
@PostMapping("/refresh")
public ResponseEntity<AuthResponse> refreshToken(@RequestBody RefreshRequest
request) {

    String refreshToken = request.getRefreshToken();

    // Validate refresh token
    if (!jwtService.validateToken(refreshToken) ||
!jwtService.isRefreshToken(refreshToken)) {
        throw new IllegalArgumentException("Invalid refresh token");
    }

    String username = jwtService.extractUsername(refreshToken);
    UserDetails userDetails = userDetailsService.loadUserByUsername(username);

    String newAccessToken = jwtService.generateAccessToken(userDetails);

    return ResponseEntity.ok(AuthResponse.builder()
        .accessToken(newAccessToken)
        .refreshToken(refreshToken) // Same refresh token (or rotate)
        .tokenType("Bearer")
        .expiresIn(3600)
        .build());
}

// =====
// Logout (Client-side only for JWT – need blacklist)
// =====
@PostMapping("/logout")
public ResponseEntity<Void> logout(@RequestHeader("Authorization") String
authHeader) {
    // For JWT, logout is client-side
    // For production, add token to blacklist (Redis)
    return ResponseEntity.ok().build();
}
}

```

Role-Based Method Security

```

@RestController
@RequestMapping("/api/orders")
@PreAuthorize("isAuthenticated()")
public class OrderController {

    // Any authenticated user
    @GetMapping
    @PreAuthorize("hasAuthority('ROLE_USER') or hasAuthority('ROLE_ADMIN')")
    public List<Order> getOrders() {
        return orderService.findAll();
    }

    // Only ADMIN
    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public void deleteOrder(@PathVariable Long id) {
        orderService.delete(id);
    }

    // Specific user check (method parameter)
    @GetMapping("/user/{userId}")
    @PreAuthorize("#userId == authentication.principal.username or
hasRole('ADMIN')")
    public List<Order> getUserOrders(@PathVariable String userId) {
        return orderService.findByUserId(userId);
    }

    // Permission evaluator for complex logic
    @GetMapping("/{id}/sensitive")
    @PreAuthorize("@orderPermissionEvaluator.canAccess(authentication, #id)")
    public Order getSensitiveOrder(@PathVariable Long id) {
        return orderService.findById(id);
    }
}

```

Permission Evaluator (Complex Authorization)

```

@Component
public class OrderPermissionEvaluator implements PermissionEvaluator {

    @Autowired
    private OrderRepository orderRepository;

    @Override
    public boolean hasPermission(Authentication authentication,
                                Object targetDomainObject,
                                Object permission) {

        if (authentication == null || targetDomainObject == null) {
            return false;
        }

        Order order = (Order) targetDomainObject;
        String username = authentication.getName();

        return order.getCustomerEmail().equals(username)
            || authentication.getAuthorities().stream()
                .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
    }

    @Override
    public boolean hasPermission(Authentication authentication,
                                Serializable targetId,
                                String targetType,
                                Object permission) {

        if (authentication == null || targetId == null) {
            return false;
        }

        if ("Order".equals(targetType)) {
            Order order = orderRepository.findById((Long) targetId).orElse(null);
            return hasPermission(authentication, order, permission);
        }

        return false;
    }
}

```

```
// Enable in SecurityConfig
@EnableGlobalMethodSecurity(prePostEnabled = true)
```

PART 5: TOKEN STORAGE — WHERE TO KEEP JWT

Storage Location Comparison

Storage	XSS Risk	CSRF Risk	Persistence	Size Limit	Best For
LocalStorage	High	None	Session or persistent	~10MB	SPA (with caution)
SessionStorage	High	None	Tab session	~10MB	Temporary data
Cookie (HttpOnly)	None	High	Configurable	4KB	Most secure (with CSRF)
Memory (React state)	Low	None	Page refresh loses	Unlimited	Most secure but loses on refresh

Recommended Pattern: HttpOnly Cookie

```

@PostMapping("/login")
public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest request,
                                          HttpServletResponse response) {

    // Authenticate and generate tokens
    Authentication auth = authenticationManager.authenticate(...);
    String accessToken = jwtService.generateAccessToken(...);

    // Set HttpOnly cookie
    Cookie cookie = new Cookie("access_token", accessToken);
    cookie.setHttpOnly(true);      // Not accessible via JavaScript
    cookie.setSecure(true);        // HTTPS only
    cookie.setPath("/");
    cookie.setMaxAge(3600);        // 1 hour
    cookie.setAttribute("SameSite", "Strict"); // CSRF protection

    response.addCookie(cookie);

    return ResponseEntity.ok().build();
}

// Read from cookie in filter
@Override
protected void doFilterInternal(HttpServletRequest request, ...) {
    Cookie[] cookies = request.getCookies();
    if (cookies != null) {
        for (Cookie cookie : cookies) {
            if ("access_token".equals(cookie.getName())) {
                jwt = cookie.getValue();
                break;
            }
        }
    }
}
}

```

PART 6: OAUTH2 RESOURCE SERVER (Spring Boot)

```
# application.yml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com
          jwk-set-uri: https://auth.example.com/.well-known/jwks.json
```

```

@Configuration
@EnableWebSecurity
public class OAuth2ResourceServerConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt
                    .jwtAuthenticationConverter(jwtAuthenticationConverter())
                )
            )
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .anyRequest().authenticated()
            );

        return http.build();
    }

    @Bean
    public JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            // Extract roles from custom claim
            List<String> roles = jwt.getClaimAsStringList("roles");
            return roles.stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .collect(Collectors.toList());
        });
        return converter;
    }
}

```

PART 7: TOKEN REVOCATION STRATEGIES

Problem with JWT — No Built-in Revocation

TOKEN REVOCATION STRATEGIES

Strategy 1: Short Expiry + Refresh Token Rotation

Access Token: 15 minutes

Refresh Token: 7 days, rotates on each use

Revocation: Just don't issue new refresh token

Strategy 2: Token Blacklist (Redis)

On logout/revoke:

```
redis.setex("blacklist:" + jwt, ttl, "revoked")
```

In filter:

```
if (redis.exists("blacklist:" + jwt)) reject();
```

Strategy 3: Versioned User Key

User has "tokenVersion" field (incremented on logout)

JWT includes version claim

Reject if version mismatch

Blacklist Implementation


```

@Component
public class TokenBlacklistService {

    @Autowired
    private RedisTemplate<String, String> redisTemplate;

    private static final String BLACKLIST_PREFIX = "blacklist:";

    public void revokeToken(String jwt, long expirationMillis) {
        String key = BLACKLIST_PREFIX + jwt;
        redisTemplate.opsForValue().set(key, "revoked", expirationMillis,
TimeUnit.MILLISECONDS);
    }

    public boolean isRevoked(String jwt) {
        return Boolean.TRUE.equals(redisTemplate.hasKey(BLACKLIST_PREFIX + jwt));
    }
}

// In filter
if (jwtService.validateToken(jwt) && !blacklistService.isRevoked(jwt)) {
    // authenticate
}

```

PART 8: INTERVIEW POWER PHRASES

Staff-Level Insights

"JWTs are NOT more secure than sessions — they just scale better. The security trade-off is XSS/CSRF risk vs server-side storage. For financial apps, I prefer HttpOnly cookies with CSRF tokens."

"The biggest JWT mistake is putting too much in the payload. Keep minimal claims — user ID, roles, expiry. Every byte adds to every request overhead."

"OAuth2 Authorization Code with PKCE is the only flow I recommend for mobile/SPA. Password grant is deprecated for good reason — it gives the client too much power."

"Spring Security's filter chain order matters. Put JWT filter BEFORE UsernamePasswordAuthenticationFilter but AFTER SecurityContextPersistenceFilter."

"Refresh token rotation is essential for security. Each refresh should issue a new token and invalidate the old one. This prevents token replay attacks."

"Stateless JWT is a lie — you always need state for revocation. Either use short expiry (15 min) or a blacklist. There's no free lunch."

QUICK REFERENCE CARD

Security Annotations

Annotation	Purpose
@EnableWebSecurity	Enable Spring Security
@EnableMethodSecurity	Enable method-level security (prePostEnabled)
@PreAuthorize	Check before method execution
@PostAuthorize	Check after method execution (with result)
@PreFilter	Filter collection arguments
@PostFilter	Filter returned collection
@Secured	Simple role check (legacy)
@RolesAllowed	JSR-250 role check

JWT Claims (Registered)

Claim	Name	Meaning
iss	Issuer	Who created the token
sub	Subject	Who the token is about (user)
aud	Audience	Intended recipient

exp	Expiration	When token expires
nbf	Not Before	Token valid after this time
iat	Issued At	When token was created
jti	JWT ID	Unique token identifier

OAuth2 Endpoints

Endpoint	Purpose
/authorize	Get authorization code (user-facing)
/token	Exchange code for tokens
/token/refresh	Get new access token
/revoke	Revoke a token
/introspect	Validate token (resource server)
/userinfo	Get user info (OIDC)

Environment Variables (Production)

```
jwt.secret=${JWT_SECRET:defaultSecretKeyForDevOnly}
jwt.access-token-expiration=${JWT_ACCESS_EXPIRATION:900000}
jwt.refresh-token-expiration=${JWT_REFRESH_EXPIRATION:604800000}
spring.security.oauth2.resourceserver.jwt.issuer-uri=${OAUTH_ISSUER_URI}
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=${JWK_SET_URI}
```

One-Line Summary

"Spring Security provides the filter chain, OAuth2 defines the authorization flows, and JWT is the stateless token format — together they enable scalable, delegated authentication and authorization for microservices."

Ready for tomorrow's topics! 🚀

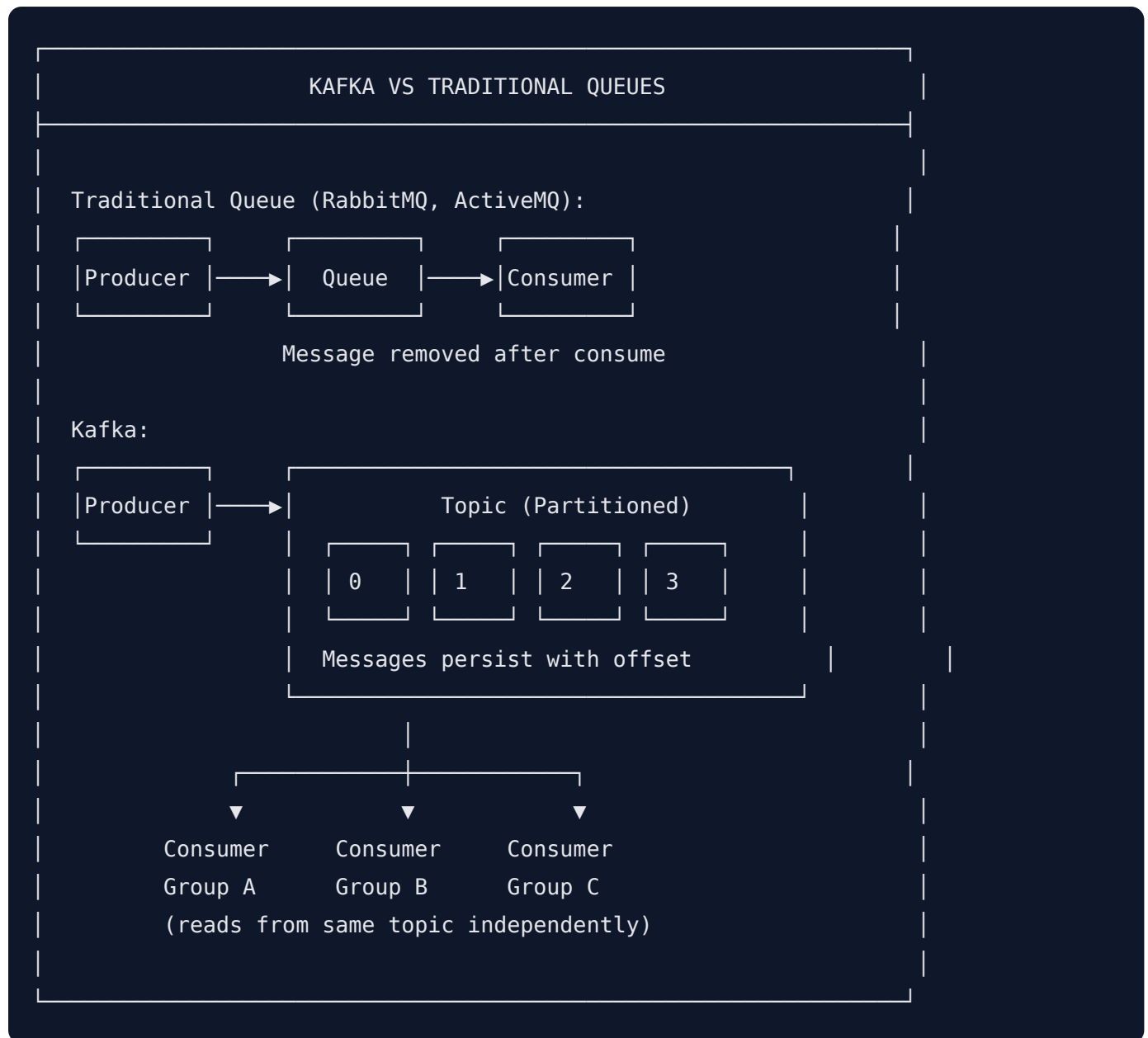
Here is your **Kafka Deep Dive** — comprehensive, interview-focused, and formatted in clean markdown for PDF export.

KAFKA DEEP DIVE — Advanced Microservices Interview Guide

For the 16-Year Experience Professional

PART 1: KAFKA CORE ARCHITECTURE

What Makes Kafka Different



Kafka Terminology — Must Know

Term	Definition	Interview Importance
Topic	Logical stream of messages	★★★★
Partition	Ordered, immutable sequence of messages	★★★★
Offset	Unique ID per message within partition	★★★★
Broker	Kafka server (one node in cluster)	★★★
Producer	Publishes messages to topics	★★★★
Consumer	Reads messages from topics	★★★★
Consumer Group	Multiple consumers reading together	★★★★

Leader/Follower	Partition replication for HA	★★
ISR	In-Sync Replicas	★★★
Log Segment	Physical file storing messages	★

PART 2: PARTITIONING DEEP DIVE

How Partitioning Works



Partitioning Strategies Comparison

Strategy	How It Works	Use Case	Ordering Guarantee
Default (Round Robin)	Cycle through partitions	High throughput, no key requirement	No ordering across partitions

Keyed	hash(key) % num_partitions	Need ordering per entity (e.g., per customer)	Per-key ordering preserved
Custom Partitioner	Custom logic	Complex routing, sticky partitioning	Depends on implementation
Sticky Partitioner	Batch to same partition, switch when full	Optimal batching + reduced latency	No ordering guarantee

Partition Count Decisions

PARTITION COUNT FORMULA (Rough Rule)

$\text{Max Partitions} = \text{Target Throughput} / \text{Max Throughput per Partition}$

Example:

- Target: 100 MB/s total throughput
- One partition: ~10 MB/s max
- Partitions needed = $100 / 10 = 10$

WARNINGS:

- More partitions → more file handles
- More partitions → slower leader election
- More partitions → higher latency for small messages
- More partitions → more consumer rebalancing time

Rule of thumb: 1 partition = 10 MB/s throughput

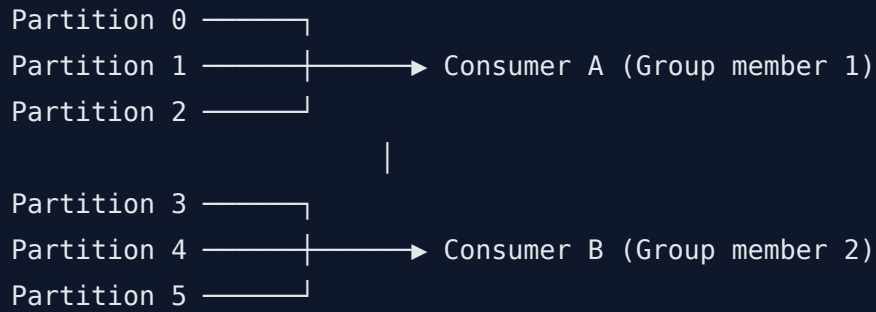
Sweet spot: 100-200 partitions per broker max

PART 3: CONSUMER GROUPS & REBALANCING

Consumer Group Architecture

CONSUMER GROUP – DETAILED

Topic: "orders" with 6 partitions



RULES:

1. Each partition assigned to exactly ONE consumer in group
2. One consumer can handle multiple partitions
3. Max consumers in group = number of partitions
(Extra consumers sit idle)

Rebalancing Trigger:

- Consumer joins or leaves group
- Consumer commits too slowly (session timeout)
- Partition count changes
- Topic reassignment happens

Rebalancing Protocols

Protocol	How It Works	Pros	Cons
Eager Rebalancing (Classic)	All consumers revoke all partitions → Redistribute	Simple, predictable	Stop-the-world during rebalance
Cooperative (Incremental)	Only revoke some partitions, reassign gradually	Minimal pause, faster	Complex, newer

Sticky Assignor	Keep previous assignments when possible	Less movement	Implementation complexity
-----------------	---	---------------	---------------------------

Consumer Heartbeat & Session Timeouts

HEARTBEAT CONFIGURATION DEEP DIVE

Config:

```
session.timeout.ms = 45000 (45 sec default)
heartbeat.interval.ms = 3000 (1/3 of session)
max.poll.interval.ms = 300000 (5 min default)
```

TIMELINE OF FAILURE:

```
t=0s: Consumer receives message, starts processing
t=45s: No heartbeat sent (processing still ongoing)
t=46s: Broker marks consumer dead → Rebalance
t=47s: Original consumer finishes processing
      → Now processing same message again!
```

Interview Insight:

```
"Long processing needs increased max.poll.interval.ms,
not increased session.timeout.ms"
```

PART 4: MESSAGE DELIVERY SEMANTICS

The Three Delivery Guarantees

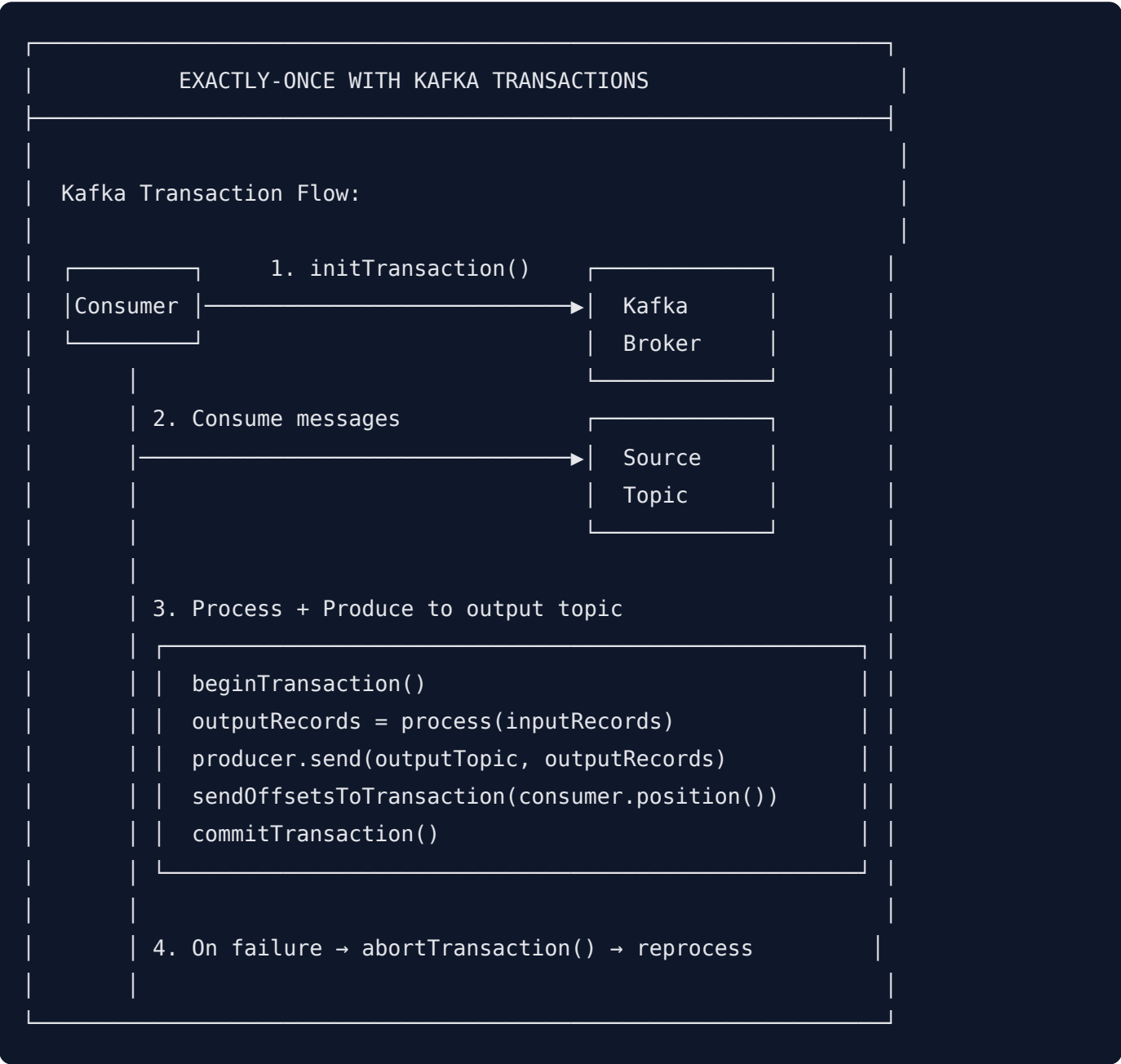


Producer Delivery Configurations

Config	Value	What It Does	Delivery Guarantee
acks=0	Producer doesn't wait	Fastest, no guarantee	At-most-once
acks=1	Leader only	Low latency, risk on leader fail	At-least/At-most

<code>acks=all</code>	All ISRs	Strongest durability	At-least-once
<code>enable.idempotence=true</code>	Exactly-once within partition	Prevents duplicate sends	Exactly-once per partition
<code>max.in.flight.requests.per.connection=1</code>	Single request at a time	Required for exactly-once	Ordering guarantee

Exactly-Once Processing — The Real Way



Exactly-Once Trade-offs

Aspect	At-least-once + Idempotent	Exactly-Once with Transactions
--------	----------------------------	--------------------------------

Throughput	High	30-50% lower
Latency	Low	Higher (coordinator overhead)
Complexity	Low	High
Recovery	Manual dedup needed	Automatic
When to use	Most cases	Financial, critical counting

Interview Insight: *"True exactly-once is expensive. Most production systems use at-least-once with idempotent consumers — same effect, better performance."*

PART 5: IDEMPOTENT PRODUCER DEEP DIVE

How Idempotent Producer Works

IDEMPOTENT PRODUCER INTERNALS

Producer sends message with:

```
Producer ID (PID) = Unique per producer instance
Sequence Number = Increments per message per partition
Producer Epoch = Increments when PID reused
```

Broker maintains:

```
PID → Last Sequence Number (per partition)

When new message arrives:
if (sequence == last_sequence + 1) → Accept
if (sequence <= last_sequence) → Reject (duplicate)
if (sequence > last_sequence + 1) → Reject (gap)
```

Example:

```
Send(seq=1) → Accepted (last_seq=0, new_seq=1) ✓
Send(seq=2) → Accepted (last_seq=1, new_seq=2) ✓
Send(seq=2) → Rejected (duplicate) ✗
Send(seq=4) → Rejected (gap, seq=3 missing) ✗
```

PART 6: COMPRESSION & SERIALIZATION

Compression Comparison

Compression	Speed	Ratio	CPU Usage	Network Saving	Best For
None	Fastest	0%	Minimal	0%	Low throughput

gzip	Slow	Best (70-80%)	High	High	Text, batches, archives
snappy	Fast	Good (30-40%)	Low	Medium	Balanced, Google style
lz4	Fastest	Good (30-40%)	Very Low	Medium	Java ecosystem
zstd	Medium	Better than gzip (80%+)	Medium	Highest	Newer, best combo

Serialization Formats Deep Dive

SERIALIZATION FORMAT COMPARISON

JSON:

Pros: Human readable, ubiquitous

Cons: Large (20-50+ bytes overhead per message)
No schema evolution

Avro:

Schema Registry Required!

Message = schema_id (4 bytes) + binary data

Schema stored separately, only ID in message

Evolution: Add optional field → old readers ignore
Remove field → old readers use default

Best for: Long-term storage, streaming, evolved schemas

Protobuf:

Pros: Smaller than Avro? Actually similar
Schema in code (no registry required)
Multiple language support

Cons: Schema changes require code changes

Sizing Comparison (100,000 records):

JSON raw: 50 MB

JSON + gzip: 15 MB

Avro: 12 MB

Avro + snappy: 8 MB

Protobuf + lz4: 7 MB (plus codegen)

PART 7: KAFKA CONNECT & KAFKA STREAMS

Kafka Connect — Plug and Play Integration



Kafka Streams vs ksqlDB vs Flink

STREAM PROCESSING COMPARISON

Kafka Streams (Java/Scala Library)

- No additional cluster
- Exactly-once semantics
- State stores (RocksDB in consumer)
- Use when: You're in JVM, want tight integration

ksqlDB (SQL on Kafka)

- Declarative SQL
- Interactive queries (pull queries)
- Materialized views
- Use when: Analysts, simpler pipelines, SQL preferred

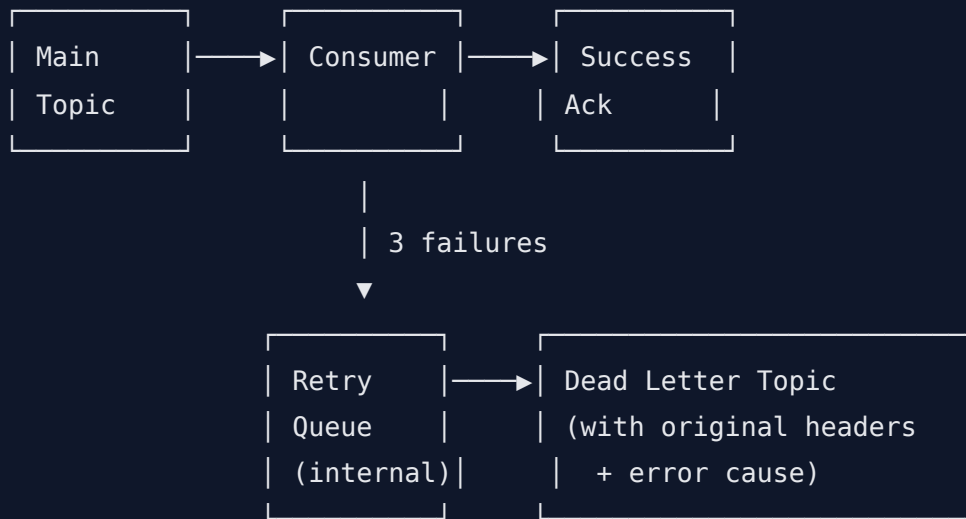
Flink (Separate processing cluster)

- True streaming (not micro-batch)
- Event time processing
- Complex event processing (CEP)
- Use when: Low-latency (sub-second), Python, complex

PART 8: DEAD LETTER QUEUES (DLQ) & ERROR HANDLING

DLQ Architecture

DEAD LETTER QUEUE PATTERN



Retry Strategy:

Attempt 1: Immediate
Attempt 2: Delay 100ms
Attempt 3: Delay 500ms
Attempt 4: Delay 2s
Attempt 5: Delay 10s → DLQ after this

DLQ Consumer (Separate team/app):

- Alerts on DLQ messages
- Manual inspection
- Repair and republish
- Dead letter → reprocessed or archived

Spring Kafka DLQ Configuration

```
// Interview-ready code understanding
@RetryableTopic(
    attempts = "4",
    backoff = @Backoff(delay = 1000, multiplier = 2),
    topicSuffixingStrategy = TopicSuffixingStrategy.SUFFIX_WITH_INDEX_VALUE,
    dltTopicSuffix = ".dlq"
)
@KafkaListener(topics = "orders")
public void consume(Order order) {
    // processing logic
}
```

PART 9: MONITORING & PRODUCTION TUNING

Critical Kafka Metrics

MUST-MONITOR KAFKA METRICS

Broker Metrics:

- Under-replicated partitions (should be 0)
- Offline partitions (should be 0)
- ISR shrink/expand rate (stable)
- Request handler avg idle (>30% is healthy)
- Network handler avg idle
- Log flush rate

Producer Metrics:

- request-latency-avg (target: <100ms)
- record-queue-time-avg (check batching)
- record-error-rate (spikes = issues)
- compression-rate (target: >0.3)

Consumer Metrics:

- records-lag-max (most important!)
- records-lag-avg
- fetch-latency-avg
- commit-latency-avg
- rebalance rate

Lag Monitoring Alert Thresholds

Metric	Warning	Critical	Action
Consumer lag	>10,000	>100,000	Scale consumers, increase partitions
Under-replicated partitions	>0 for 5 min	>0 for 30 min	Network or broker issue
ISR count	<2 for partition	=1 for partition	Broker failure, urgent

Producer error rate	>1%	>5%	Check serialization, timeout config
Rebalance frequency	>1/hour	>5/hour	Check session timeout config

PART 10: KAFKA TROUBLESHOOTING — REAL SCENARIOS

Scenario 1: Consumer Lag Growing Forever

Symptoms:

- Lag increasing monotonically
- Consumer processing not keeping up

Root Causes:

1. Slow processing (DB bottleneck, external API)
2. Single consumer with max.poll.records too high
3. Not enough partitions (parallelism limit)
4. Head-of-line blocking with large messages

Solutions:

- Solution A: More consumers (if partitions available)
- Solution B: Increase max.poll.records (batch size)
- Solution C: Increase partitions (with careful planning)
- Solution D: Async processing with manual offset commit
- Solution E: Co-partition join optimization

Scenario 2: Rebalancing Too Often

Symptoms:

- "Rebalancing" logs every few minutes
- Processing pauses frequently

Root Causes:

1. Consumer processing > max.poll.interval.ms
2. Network issues causing heartbeat loss
3. Garbage collection pauses > session.timeout.ms
4. Consumer group metadata updates too frequent

Diagnostic:

Check consumer logs for:

"Member xxx has left group" → session timeout

"Revoking partitions" → triggered rebalance

Solutions:

- Increase max.poll.interval.ms (if long processing)
- Increase session.timeout.ms (if GC/network issues)
- Use cooperative rebalancing (rebalance protocol)
- Move to async processing + manual commit

Scenario 3: Message Ordering Violated

Symptoms:

- Processed events out of sequence for same key
- Downstream system sees inconsistent state

Root Causes:

1. `max.in.flight.requests.per.connection > 1` without idempotence
2. Multiple partitions for same key (impossible)
3. Consumer multi-threaded without key-based partitioning
4. Retries on producer without idempotence

Solutions:

- ✓ `enable.idempotence=true` (implies `max.in.flight=1/5`)
- ✓ Ensure consistent key hashing across partitions
- ✓ Use partition-level processing in consumer (single thread)
- ✓ Enable idempotent producer to prevent duplicate reorders

PART 11: KAFKA INTERVIEW POWER PHRASES

Key Differentiators (For Senior/Staff Level)

Topic	Power Phrase
Kafka vs Queue	"Kafka isn't a traditional queue — it's a distributed commit log with consumer groups that maintain independent offsets, enabling replayability."
Partitioning	"Partitions are the unit of parallelism and ordering. Messages with the same key go to the same partition, preserving order per entity."
Exactly-Once	"True exactly-once requires both idempotent producers and transactional consumers, but at-least-once with idempotent processing is often the pragmatic choice."
Consumer Groups	"A consumer group provides load balancing AND fault tolerance — but maximum parallelism equals partition count."
Rebalancing	"Cooperative rebalancing is the modern approach — incremental reassignment rather than stop-the-world."

Lag Management	"Lag is the most critical metric. Monitor records-lag-max per consumer group. Zero lag doesn't always mean healthy — could mean not processing."
Compression	"lz4 or zstd for speed, gzip for space. Never use JSON without compression — Avro + zstd is the modern sweet spot."
Retries	"Retry without idempotence is dangerous. Always enable idempotent producer when retries are needed."
Kafka vs Flink	"Kafka Streams is for stateful processing within Kafka's ecosystem. Flink is for complex event processing with sub-second latency across multiple systems."

QUICK REFERENCE CARD — KAFKA

Configurations Cheat Sheet

```
# Producer – Production Settings
producer:
  acks: all
  enable.idempotence: true
  max.in.flight.requests.per.connection: 5 # if idempotence=true
  compression.type: lz4|zstd
  retries: 2147483647 # effectively unlimited
  linger.ms: 10
  batch.size: 16384

# Consumer – Production Settings
consumer:
  group.id: ${service}.${environment}
  enable.auto.commit: false
  auto.offset.reset: earliest # or latest per use case
  fetch.min.bytes: 50000
  max.poll.records: 500
  session.timeout.ms: 45000
  max.poll.interval.ms: 300000 # adjust for processing time

# Broker – Cluster Settings
broker:
  min.insync.replicas: 2
  default.replication.factor: 3
  unclean.leader.election.enable: false
  log.retention.hours: 168
  log.segment.bytes: 1073741824 # 1GB
```

Topic Design Decisions

Decision	Consideration
Partitions	Throughput per partition (~10 MB/s), rebalance time, file handles
Replication factor	3 for production, 2 for dev, 1 only for test
**min.insync.replic	

JAVA STREAMS & ALGORITHMS — Complete Program Library

STREAM METHOD NAMES — Read Like English Examples That Read as Natural Sentences

Pattern 1: Filter → Transform → Collect

```
// "From list, keep adults, get their names, collect to list"
list.stream().filter(Person::isAdult).map(Person::getName).collect(toList());

// "From products, keep in stock, get prices, collect to set"
products.stream().filter(Product::isInStock).map(Product::getPrice).collect(toSet());

// "From employees, keep IT department, get salaries, collect to array"
employees.stream().filter(e ->
    "IT".equals(e.getDept()))
    .map(Employee::getSalary).toArray();

// "From orders, keep completed, get amounts, collect to map"
orders.stream().filter(Order::isCompleted).collect(toMap(Order::getId,
    Order::getAmount));
```

Pattern 2: Find First/Any Matching

```
// "From users, find first active user"
users.stream().filter(User::isActive).findFirst();

// "From numbers, find any even number"
numbers.stream().filter(n -> n % 2 == 0).findAny();

// "From strings, find first longer than 5 characters"
strings.stream().filter(s -> s.length() > 5).findFirst();

// "From products, find any with price less than 100"
products.stream().filter(p -> p.getPrice() < 100).findAny();
```

Pattern 3: Check Conditions (Any/All/None)

```
// "From list, check if any price is over 1000"
prices.stream().anyMatch(p -> p > 1000);

// "From students, check if all passed"
students.stream().allMatch(Student::hasPassed);

// "From strings, check if none are empty"
strings.stream().noneMatch(String::isEmpty);

// "From employees, check if all have valid email"
employees.stream().allMatch(e -> e.getEmail().contains("@"));
```

Pattern 4: Count Elements

```
// "From list, count active users"
users.stream().filter(User::isActive).count();

// "From orders, count completed orders"
orders.stream().filter(Order::isCompleted).count();

// "From products, count out of stock items"
products.stream().filter(p -> p.getStock() == 0).count();

// "From sentences, count words longer than 7 letters"
words.stream().filter(w -> w.length() > 7).count();
```

Pattern 5: Sum and Average

```
// "From salaries, sum all values"
salaries.stream().mapToDouble(Double::doubleValue).sum();

// "From ages, calculate average"
ages.stream().mapToInt(Integer::intValue).average();

// "From transactions, sum amounts"
transactions.stream().mapToDouble(Transaction::getAmount).sum();

// "From scores, get average score"
scores.stream().mapToInt(Integer::intValue).average().orElse(0);
```

Pattern 6: Find Min and Max

```
// "From products, find cheapest"
products.stream().min(Comparator.comparing(Product::getPrice));

// "From employees, find highest paid"
employees.stream().max(Comparator.comparing(Employee::getSalary));

// "From dates, find earliest"
dates.stream().min(Comparator.naturalOrder());

// "From numbers, find maximum"
numbers.stream().max(Integer::compareTo);
```

Pattern 7: Group By Something

```
// "From employees, group by department"
employees.stream().collect(groupingBy(Employee::getDepartment));

// "From orders, group by status"
orders.stream().collect(groupingBy(Order::getStatus));

// "From products, group by category"
products.stream().collect(groupingBy(Product::getCategory));

// "From students, group by grade"
students.stream().collect(groupingBy(Student::getGrade));
```

Pattern 8: Group and Count

```
// "From employees, group by department and count each"
employees.stream().collect(groupingBy(Employee::getDepartment, counting()));

// "From orders, group by status and count"
orders.stream().collect(groupingBy(Order::getStatus, counting()));

// "From words, group by first letter and count"
words.stream().collect(groupingBy(w -> w.charAt(0), counting()));

// "From logs, group by level and count"
logs.stream().collect(groupingBy(Log::getLevel, counting()));
```

Pattern 9: Convert List to Map

```
// "From employees, convert to map by id"
employees.stream().collect(toMap(Employee::getId, Function.identity()));

// "From products, convert to map by name"
products.stream().collect(toMap(Product::getName, Function.identity()));

// "From users, convert to map by email"
users.stream().collect(toMap(User::getEmail, Function.identity()));

// "From accounts, convert to map by account number"
accounts.stream().collect(toMap(Account::getNumber, Function.identity()));
```

Pattern 10: Join Strings

```
// "From names, join with comma"
names.stream().collect(joining(", "));

// "From words, join with space"
words.stream().collect(joining(" "));

// "From ids, join with hyphen"
ids.stream().collect(joining("-"));

// "From lines, join with newline"
lines.stream().collect(joining("\n"));
```

Pattern 11: Sort Elements

```
// "From employees, sort by salary"
employees.stream().sorted(comparing(Employee::getSalary));

// "From products, sort by price descending"
products.stream().sorted(comparing(Product::getPrice).reversed());

// "From names, sort alphabetically"
names.stream().sorted();

// "From dates, sort oldest first"
dates.stream().sorted();
```

Pattern 12: Limit and Skip


```
// "From list, take top 10"
items.stream().limit(10);

// "From results, skip first 5"
results.stream().skip(5);

// "From sales, take best 3"
sales.stream().sorted(comparing(Sale::getAmount).reversed()).limit(3);

// "From logs, skip first 100 lines"
logs.stream().skip(100);
```

Pattern 13: Remove Duplicates

```
// "From list, get unique names"
names.stream().distinct();

// "From numbers, get unique values"
numbers.stream().distinct();

// "From emails, remove duplicates"
emails.stream().distinct();

// "From ids, get unique ids"
ids.stream().distinct();
```

Pattern 14: Flatten Nested Structures

```
// "From list of lists, flatten to single list"
listOfLists.stream().flatMap(List::stream);

// "From sentences, get all words"
sentences.stream().flatMap(s -> Arrays.stream(s.split(" ")));

// "From orders, get all items"
orders.stream().flatMap(o -> o.getItems().stream());

// "From customers, get all phone numbers"
customers.stream().flatMap(c -> c.getPhones().stream());
```

Pattern 15: Peek (Debug)

```
// "From stream, log each element then collect"
stream.peek(System.out::println).collect(toList());

// "From orders, log processing then filter"
orders.stream().peek(o -> log.info("Processing: {}",
o)).filter(Order::isValid).collect(toList());

// "From users, log before and after transformation"
users.stream().peek(u -> log.debug("Before: {}", u)).map(User::normalize).peek(u -
> log.debug("After: {}", u)).collect(toList());
```

Pattern 16: Reduce to Single Value

```
// "From numbers, sum all using reduce"
numbers.stream().reduce(0, Integer::sum);

// "From strings, concatenate all"
strings.stream().reduce("", String::concat);

// "From products, find most expensive"
products.stream().reduce((p1, p2) -> p1.getPrice() > p2.getPrice() ? p1 : p2);

// "From transactions, calculate total with initial value"
transactions.stream().reduce(BigDecimal.ZERO, BigDecimal::add, BigDecimal::add);
```

Pattern 17: Partition Into Two Groups

```
// "From employees, partition by salary > 50000"
employees.stream().collect(partitioningBy(e -> e.getSalary() > 50000));

// "From orders, partition by completed"
orders.stream().collect(partitioningBy(Order::isCompleted));

// "From numbers, partition by even or odd"
numbers.stream().collect(partitioningBy(n -> n % 2 == 0));

// "From students, partition by pass or fail"
students.stream().collect(partitioningBy(Student::hasPassed));
```

Pattern 18: Get Summary Statistics

```
// "From ages, get statistics"
ages.stream().mapToInt(Integer::intValue).summaryStatistics();

// "From salaries, get stats"
salaries.stream().mapToDouble(Double::doubleValue).summaryStatistics();

// "From scores, get all stats at once"
scores.stream().mapToInt(Integer::intValue).summaryStatistics();

// "From transaction amounts, get summary"
transactions.stream().mapToDouble(Transaction::getAmount).summaryStatistics();
```

Pattern 19: Handle Nulls

```
// "From list, filter out nulls then process"
list.stream().filter(Objects::nonNull).map(String::toUpperCase).collect(toList());

// "From values, use orElse for default"
stream.findFirst().orElse(defaultValue);

// "From optional, get or throw"
optional.orElseThrow(() -> new NotFoundException());

// "From results, provide fallback value"
results.stream().findAny().orElseGet(() -> createDefault());
```

Pattern 20: Parallel Processing

```
// "From large list, process in parallel"
largeList.parallelStream().filter(Item::isValid).collect(toList());

// "From employees, calculate sum in parallel"
employees.parallelStream().mapToDouble(Employee::getSalary).sum();

// "From orders, group in parallel"
orders.parallelStream().collect(groupingByConcurrent(Order::getStatus));

// "From transactions, process fastest way"
transactions.parallelStream().map(Transaction::process).collect(toList());
```

Pattern 21: Combine Multiple Conditions

```
// "From products, keep in stock AND price < 100"
products.stream().filter(p -> p.isInStock() && p.getPrice() <
100).collect(toList());

// "From employees, keep IT department AND salary > 80000"
employees.stream().filter(e -> "IT".equals(e.getDept()) && e.getSalary() >
80000).collect(toList());

// "From orders, keep completed AND amount > 1000"
orders.stream().filter(o -> o.isCompleted() && o.getAmount() >
1000).collect(toList());

// "From students, keep age > 18 AND grade A"
students.stream().filter(s -> s.getAge() > 18 &&
"A".equals(s.getGrade())).collect(toList());
```

Pattern 22: Chain Transformations

```
// "From list, filter, then transform, then collect"
list.stream().filter(Item::isActive).map(Item::getName).map(String::toUpperCase).c
ollect(toList());

// "From orders, filter completed, get amounts, sum them"
orders.stream().filter(Order::isCompleted).map(Order::getAmount).reduce(BigDecimal
.ZERO, BigDecimal::add);

// "From employees, keep seniors, get emails, collect to set"
employees.stream().filter(e -> e.getYears() >
10).map(Employee::getEmail).collect(toSet());

// "From products, apply discount, filter by price, get names"
products.stream().map(p -> p.withDiscount(0.1)).filter(p -> p.getPrice() <
50).map(Product::getName).collect(toList());
```

Pattern 23: Custom Comparisons

```
// "From employees, sort by salary, then by name"
employees.stream().sorted(comparing(Employee::getSalary).thenComparing(Employee::get
etName)).collect(toList());

// "From products, sort by category, then by price"
products.stream().sorted(comparing(Product::getCategory).thenComparing(Product::ge
tPrice)).collect(toList());

// "From students, sort by grade descending, then by name"
students.stream().sorted(comparing(Student::getGrade).reversed().thenComparing(Stu
dent::getName)).collect(toList());

// "From tasks, sort by priority, then by due date"
tasks.stream().sorted(comparing(Task::getPriority).thenComparing(Task::getDueDate)
).collect(toList());
```

Pattern 24: Collect to Specific Collection Type

```
// "From items, collect to linked list"
items.stream().collect(toCollection(LinkedList::new));

// "From keys, collect to tree set"
keys.stream().collect(toCollection(TreeSet::new));

// "From values, collect to array list with capacity"
values.stream().collect(toCollection(() -> new ArrayList<>(100)));

// "From unique items, collect to concurrent set"
items.stream().collect(toCollection(ConcurrentSkipListSet::new));
```

Pattern 25: Handle Empty Results Gracefully

```
// "From list, find first or return null"
list.stream().filter(Item::isValid).findFirst().orElse(null);

// "From results, get value or compute default"
stream.findAny().orElseGet(() -> createFallbackValue());

// "From search, get or throw meaningful exception"
searchResults.stream().findFirst().orElseThrow(() -> new
NoSuchElementException("Not found"));

// "From empty stream, return empty list safely"
emptyStream.collect(toList()); // Just works, returns empty list
```

Memory Trick: Read as English Sentence

Source	+ Operation Chain	+ Terminal
"From list"	"keep adults"	"get names"
"From users"	"find first"	"active"
"From orders"	"group by"	"status"
"From numbers"	"check if"	"all positive"
"From products"	"find"	"cheapest"
"From words"	"join with"	"comma"
"From employees"	"sort by"	"salary"
		"take top 5"

Pro Tip: Method Names Are Verbs

Method	Think of it as
<code>filter()</code>	"keep only..."
<code>map()</code>	"convert each..."
<code>flatMap()</code>	"explode each into..."
<code>sorted()</code>	"arrange in order..."
<code>limit()</code>	"take first..."
<code>skip()</code>	"ignore first..."

<code>distinct()</code>	"remove duplicates..."
<code>peek()</code>	"look at each..."
<code>collect()</code>	"gather into..."
<code>groupingBy()</code>	"put into buckets by..."
<code>partitioningBy()</code>	"split into two groups..."
<code>joining()</code>	"glue together with..."
<code>findFirst()</code>	"get the first one..."
<code>anyMatch()</code>	"is there any..."
<code>allMatch()</code>	"do all..."
<code>noneMatch()</code>	"is there no..."

JAVA 8 STREAMS — Pure Methods Reference Table

No Code — Only Method Names & When to Use

1. STREAM CREATION METHODS

Method	When to Use
<code>stream()</code>	Convert collection to stream
<code>of()</code>	Create stream from individual elements
<code>iterate()</code>	Generate infinite sequential stream
<code>generate()</code>	Create infinite stream from supplier
<code>range()</code>	Create numeric range (exclusive end)
<code>rangeClosed()</code>	Create numeric range (inclusive end)
<code>concat()</code>	Merge two streams together
<code>empty()</code>	Create empty stream

<code>builder()</code>	Build stream dynamically
<code>chars()</code>	Convert string to IntStream

2. INTERMEDIATE OPERATIONS (Transform)

Filtering Methods

Method	When to Use
<code>filter()</code>	Keep elements matching condition
<code>distinct()</code>	Remove duplicates
<code>limit()</code>	Take first N elements
<code>skip()</code>	Discard first N elements
<code>takeWhile()</code>	Keep until condition fails (Java 9+)
<code>dropWhile()</code>	Discard until condition fails (Java 9+)

Transformation Methods

Method	When to Use
<code>map()</code>	Convert each element 1→1
<code>flatMap()</code>	Convert each element 1→many
<code>mapToInt()</code>	Convert to IntStream
<code>mapToLong()</code>	Convert to LongStream
<code>mapToDouble()</code>	Convert to DoubleStream
<code>flatMapToInt()</code>	Flatten to IntStream
<code>boxed()</code>	Convert primitive stream to object stream

Sorting Methods

Method	When to Use
<code>sorted()</code>	Sort naturally

<code>sorted(Comparator)</code>	Sort with custom rule
---------------------------------	-----------------------

Debugging Method

Method	When to Use
<code>peek()</code>	Debug - see intermediate values

3. TERMINAL OPERATIONS (Execute)

Iteration Method

Method	When to Use
<code>forEach()</code>	Perform action on each element
<code>forEachOrdered()</code>	Preserve order in parallel streams

Aggregation Methods

Method	When to Use
<code>count()</code>	Count total elements
<code>min()</code>	Find smallest element
<code>max()</code>	Find largest element
<code>sum()</code>	Add all numbers (primitive streams)
<code>average()</code>	Calculate mean (primitive streams)
<code>reduce()</code>	Combine all to one value

Matching Methods

Method	When to Use
<code>anyMatch()</code>	Check if ANY element matches
<code>allMatch()</code>	Check if ALL elements match
<code>noneMatch()</code>	Check if NO elements match

Finding Methods

Method	When to Use
<code>findFirst()</code>	Get first element (ordered)
<code>findAny()</code>	Get any element (parallel friendly)

Collection Methods

Method	When to Use
<code>collect()</code>	Gather results into container
<code>toList()</code>	Simplified collection (Java 16+)
<code>toArray()</code>	Convert to array

4. COLLECTORS METHODS (Inside collect)

Grouping Methods

Method	When to Use
<code>groupingBy()</code>	Group elements by key
<code>groupingByConcurrent()</code>	Thread-safe grouping
<code>partitioningBy()</code>	Split into true/false groups

Joining Methods

Method	When to Use
<code>joining()</code>	Combine strings
<code>joining(delimiter)</code>	Join with separator
<code>joining(delimiter, prefix, suffix)</code>	Join with wrapper

Summarizing Methods

Method	When to Use
<code>summarizingInt()</code>	Get all stats (count, sum, min, max, avg)

<code>summarizingLong()</code>	Same for long values
<code>summarizingDouble()</code>	Same for double values

Mapping & Reducing

Method	When to Use
<code>mapping()</code>	Transform before collecting
<code>flatMap()</code>	Flatten then collect
<code>filtering()</code>	Filter before collecting
<code>reducing()</code>	Custom reduction
<code>collectingAndThen()</code>	Post-process result

To Collection Methods

Method	When to Use
<code>toList()</code>	Collect to ArrayList
<code>toSet()</code>	Collect to HashSet
<code>toCollection()</code>	Collect to specific collection type
<code>toMap()</code>	Convert to Map
<code>toConcurrentMap()</code>	Thread-safe Map
<code>toUnmodifiableList()</code>	Create immutable list (Java 10+)

Counting & Averaging

Method	When to Use
<code>counting()</code>	Count elements in group
<code>averagingInt()</code>	Average of ints
<code>averagingDouble()</code>	Average of doubles
<code>summingInt()</code>	Sum of ints

5. PRIMITIVE STREAM METHODS

IntStream Methods

Method	When to Use
<code>range()</code>	Create sequence start to end-1
<code>rangeClosed()</code>	Create sequence start to end
<code>sum()</code>	Add all values
<code>average()</code>	Calculate mean
<code>summaryStatistics()</code>	Get all stats at once
<code>boxed()</code>	Convert to Stream

DoubleStream Methods

Method	When to Use
<code>of()</code>	Create from double values
<code>sum()</code>	Add all values
<code>average()</code>	Calculate mean

LongStream Methods

Method	When to Use
<code>range()</code>	Sequence of longs
<code>sum()</code>	Add all values

6. PARALLEL STREAM METHODS

Method	When to Use
<code>parallelStream()</code>	Convert collection to parallel stream
<code>parallel()</code>	Convert sequential to parallel
<code>sequential()</code>	Convert parallel to sequential

<code>isParallel()</code>	Check if stream is parallel
<code>unordered()</code>	Remove order constraint (improves performance)

7. OPTIONAL METHODS (Stream Results)

Method	When to Use
<code>get()</code>	Get value (unsafe - only if present)
<code>orElse()</code>	Provide default value
<code>orElseGet()</code>	Provide default from supplier
<code>orElseThrow()</code>	Throw exception if empty
<code>ifPresent()</code>	Execute action if present
<code>isPresent()</code>	Check if value exists
<code>isEmpty()</code>	Check if empty (Java 11+)
<code>filter()</code>	Apply condition to Optional
<code>map()</code>	Transform Optional value
<code>flatMap()</code>	Flatten nested Optional
<code>stream()</code>	Convert Optional to Stream

8. COMPARATOR METHODS (For Sorting)

Method	When to Use
<code>comparing()</code>	Sort by field
<code>comparingInt()</code>	Sort by int field
<code>comparingDouble()</code>	Sort by double field
<code>thenComparing()</code>	Secondary sort
<code>reversed()</code>	Reverse order

<code>naturalOrder()</code>	Default natural order
<code>reverseOrder()</code>	Reverse natural order
<code>nullsFirst()</code>	Put nulls at beginning
<code>nullsLast()</code>	Put nulls at end

9. QUICK DECISION MATRIX

I Need To...	Use Method
Start streaming	<code>stream()</code> , <code>of()</code> , <code>range()</code>
Remove elements	<code>filter()</code>
Remove duplicates	<code>distinct()</code>
Take first few	<code>limit()</code>
Skip first few	<code>skip()</code>
Convert each element	<code>map()</code>
Merge nested lists	<code>flatMap()</code>
Sort elements	<code>sorted()</code>
Count elements	<code>count()</code>
Find smallest	<code>min()</code>
Find largest	<code>max()</code>
Add all numbers	<code>sum()</code>
Get average	<code>average()</code>
Find any match	<code>anyMatch()</code>
Get first element	<code>findFirst()</code>
Save as list	<code>collect(toList())</code>
Save as map	<code>collect(toMap())</code>

Group by key	<code>collect(groupingBy())</code>
Join strings	<code>collect(joining())</code>
Loop through	<code>forEach()</code>
Debug pipeline	<code>peek()</code>
Speed up processing	<code>parallelStream()</code>

10. METHOD CATEGORY MEMORY TRICK

CREATE → FILTER → TRANSFORM → COLLECT

<code>of()</code>	<code>filter()</code>	<code>map()</code>	<code>collect()</code>
<code>stream()</code>	<code>distinct()</code>	<code>flatMap()</code>	<code>toList()</code>
<code>range()</code>	<code>limit()</code>	<code>sorted()</code>	<code>toMap()</code>
<code>generate()</code>	<code>skip()</code>	<code>peek()</code>	<code>groupingBy()</code>
<code>iterate()</code>	<code>takeWhile()</code>	<code>boxed()</code>	<code>joining()</code>
<code>concat()</code>	<code>dropWhile()</code>	<code>mapToInt()</code>	<code>summingInt()</code>
<code>empty()</code>			→ <code>summarizingInt()</code>

Pro Tip: Read Method Names Like English

```
// Reads like: "From list, keep active items, get their prices, collect to list"
list.stream().filter(Item::isActive).map(Item::getPrice).collect(toList());
```

```
// Reads like: "From employees, group by department, count each group"
employees.stream().collect(groupingBy(Employee::getDepartment, counting()));
```

```
// Reads like: "From numbers, find first even number or return -1"
numbers.stream().filter(n -> n % 2 == 0).findFirst().orElse(-1);
```

JAVA 8 STREAMS CHEAT SHEET — Quick Reference

When to Use What — Decision Guide

1. STREAM CREATION

```
// When: You have data and need to start streaming
Stream.of("a", "b", "c")           // Individual elements
Arrays.stream(new int[]{1, 2, 3})  // From array
list.stream()                     // From Collection
Stream.iterate(0, n -> n + 2)      // Infinite sequence
Stream.generate(Math::random)     // Infinite supplier
IntStream.range(1, 100)           // Range of numbers
"string".chars()                  // String to IntStream
Stream.empty()                    // Empty stream
Stream.concat(stream1, stream2)   // Combine two streams
```

2. INTERMEDIATE OPERATIONS (Lazy)

Filtering

```
// When: Remove elements that don't match condition
.filter(s -> s.startsWith("a"))    // Keep only 'a' strings
.distinct()                         // Remove duplicates
.limit(10)                          // Take first 10
.skip(5)                            // Skip first 5
.takeWhile(s -> s.length() > 2)     // Java 9+: Take while condition true
.dropWhile(s -> s.length() > 2)     // Java 9+: Drop while condition true
```

Transforming

```
// When: Convert each element to something else
.map(String::toUpperCase)          // Transform each element
.mapToInt(String::length)           // To IntStream
.mapToDouble(Employee::getSalary)  // To DoubleStream
.flatMap(line -> Arrays.stream(line.split(" "))) // One-to-many transformation
.boxed()                            // IntStream -> Stream<Integer>
```

Sorting

```
// When: Need ordered results
.sorted() // Natural order
.sorted(Comparator.reverseOrder()) // Reverse natural order
.sorted(Comparator.comparing(Employee::getSalary)) // By field
.sorted(Comparator.comparing(Employee::getSalary).reversed()) // Descending
.sorted(Comparator.comparing(Employee::getSalary).thenComparing(Employee::getName))
)
```

Peeking (Debugging)

```
// When: Debug intermediate values (side effect - be careful!)
.peek(System.out::println) // Print each element (debug only)
.peek(e -> log.info("Processing: {}", e)) // Logging
```

3. TERMINAL OPERATIONS (Execute Stream)

Aggregation

```
// When: Need single value from stream
.count() // Number of elements
.min(Comparator.naturalOrder()) // Minimum element
.max(Comparator.comparing(Employee::getSalary)) // Maximum element
.sum() // IntStream/DoubleStream sum
.average() // IntStream/DoubleStream average
.reduce(0, Integer::sum) // Combine to single value
.reduce((a, b) -> a + b) // Without identity
.findFirst() // First element (Optional)
.findAny() // Any element (parallel friendly)
```

Collection

```
// When: Need to collect results into container
.collect(Collectors.toList()) // To ArrayList
.collect(Collectors.toSet()) // To HashSet
.collect(Collectors.toCollection(LinkedList::new)) // Specific collection
.collect(Collectors.toUnmodifiableList()) // Java 10+: Immutable list
.toList() // Java 16+: Simplified
```

Iteration

```
// When: Perform action for each element
.forEach(System.out::println)           // Print each (terminal)
.forEachOrdered(System.out::println)    // Preserve order in parallel
```

Matching

```
// When: Check conditions on stream
.anyMatch(s -> s.length() > 5)         // Any element matches
.allMatch(s -> s.length() > 0)         // All elements match
.noneMatch(s -> s.isEmpty())           // No element matches
```

4. COLLECTORS — The Most Important

Grouping

```
// When: Group elements by some key
.collect(Collectors.groupingBy(Employee::getDepartment))
.collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()))
.collect(Collectors.groupingBy(Employee::getDepartment,
    Collectors.mapping(Employee::getName, Collectors.toList())))

// When: Partition into true/false
.collect(Collectors.partitioningBy(e -> e.getSalary() > 50000))
```

Joining

```
// When: Combine strings with delimiter
.collect(Collectors.joining(", "))      // "a, b, c"
.collect(Collectors.joining(", ", "[", "]")) // "[a, b, c]"
```

Summarizing

```
// When: Need multiple stats at once
.collect(Collectors.summarizingInt(Employee::getAge))
// Returns: IntSummaryStatistics{count=5, sum=150, min=25, max=40, average=30.0}
```

Mapping & Reducing

```
// When: Transform before collecting
.collect(Collectors.mapping(Employee::getName, Collectors.toList()))

// When: Custom reduction
.collect(Collectors.reducing(0, Employee::getSalary, Integer::sum))

// When: FlatMap during collect
.collect(Collectors.flatMapping(e -> e.getPhones().stream(), Collectors.toList()))
```

To Map

```
// When: Convert list to map
.collect(Collectors.toMap(Employee::getId, Function.identity()))
.collect(Collectors.toMap(Employee::getId, Employee::getName))
.collect(Collectors.toMap(Employee::getId, Function.identity(), (e1, e2) -> e1))
.collect(Collectors.toMap(Employee::getId, Function.identity(), (e1, e2) -> e1,
LinkedHashMap::new))
```

5. PRIMITIVE STREAMS (Performance)

IntStream

```
// When: Working with primitives (faster, saves memory)
IntStream.range(1, 100)                // 1 to 99
IntStream.rangeClosed(1, 100)           // 1 to 100
IntStream.of(1, 2, 3)                   // From values
Arrays.stream(intArray)                  // From int[]

// Operations
.sum() .average() .min() .max() .count()
.sorted() .distinct() .filter(n -> n > 5)
.boxed()                                // To Stream<Integer>
.mapToObj(i -> "Number: " + i)           // To object stream
```

DoubleStream & LongStream

```
DoubleStream.of(1.0, 2.0, 3.0)
LongStream.range(1, 100)
```

6. COMMON PATTERNS (Copy-Paste Ready)

Pattern 1: Filter Nulls

```
list.stream()
    .filter(Objects::nonNull)
    .collect(Collectors.toList());
```

Pattern 2: Find Duplicates

```
Set<String> seen = new HashSet<>();
list.stream()
    .filter(e -> !seen.add(e))
    .collect(Collectors.toSet());
```

Pattern 3: Most Frequent Element

```
list.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse(null);
```

Pattern 4: Chunk/Batch Processing

```
IntStream.range(0, list.size())
    .boxed()
    .collect(Collectors.groupingBy(i -> i / batchSize))
    .values()
    .stream()
    .map(indices -> indices.stream().map(list::get).collect(Collectors.toList()))
    .forEach(batch -> process(batch));
```

Pattern 5: Cartesian Product

```
list1.stream()
    .flatMap(a -> list2.stream().map(b -> new Pair<>(a, b)))
    .collect(Collectors.toList());
```

Pattern 6: Nested Collection Flatten

```
List<List<String>> nested = ...;
nested.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

Pattern 7: Custom Collector (Top N by Value)

```
list.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        l -> l.stream().limit(5).collect(Collectors.toList())
    ));
```

Pattern 8: Pagination Simulation

```
IntStream.range(0, (list.size() + pageSize - 1) / pageSize)
    .mapToObj(page -> list.stream()
        .skip(page * pageSize)
        .limit(pageSize)
        .collect(Collectors.toList()))
    .collect(Collectors.toList());
```

7. PERFORMANCE TIPS (Quick Memory)

When	Use	Avoid
Large dataset, multi-core	<code>.parallelStream()</code>	<code>.stream()</code>
Small dataset	<code>.stream()</code>	<code>.parallelStream()</code> (overhead)
Simple operations	<code>.map().filter()</code>	Custom Splitterator
Need order	<code>.forEachOrdered()</code>	<code>.forEach()</code>
Infinite streams	<code>.limit()</code> or <code>.findFirst()</code>	No limit (memory crash)
Primitive operations	<code>IntStream</code> , <code>LongStream</code>	<code>Stream<Integer></code> (boxing)

8. COMMON MISTAKES TO AVOID

```
// ❌ WRONG: Reusing stream
Stream<String> stream = list.stream();
stream.forEach(System.out::println);
stream.count(); // IllegalStateException!

// ✅ RIGHT: Create new stream each time

// ❌ WRONG: Modifying source while streaming
list.stream().filter(e -> list.remove(e)); // ConcurrentModificationException

// ✅ RIGHT: Collect and modify separately

// ❌ WRONG: Using parallel on non-thread-safe collections
ArrayList<String> list = new ArrayList<>();
list.parallelStream().forEach(s -> list.add(s)); // Race condition!

// ✅ RIGHT: Use Concurrent collections or collect results

// ❌ WRONG: Calling terminal operation multiple times
Stream<Integer> stream = Stream.of(1,2,3);
stream.count();
stream.collect(Collectors.toList()); // Already consumed!

// ✅ RIGHT: Chain everything before terminal operation
```

9. QUICK VISUAL REFERENCE

Source → Intermediate → Intermediate → Terminal → Result

list.stream()	.filter()	.map()	.collect()
array.stream()	.distinct()	.flatMap()	.forEach()
Stream.of()	.sorted()	.limit()	.reduce()
	.peek()	.skip()	.findFirst()

10. ONE-LINE SOLUTIONS (Interview Gems)

```
// Remove duplicates
list.stream().distinct().collect(Collectors.toList());

// Convert list to map
list.stream().collect(Collectors.toMap(Item::getId, Function.identity()));

// Sum of salaries
employees.stream().mapToDouble(Employee::getSalary).sum();

// Group by department
employees.stream().collect(Collectors.groupingBy(Employee::getDepartment));

// Filter and transform
list.stream().filter(x -> x > 5).map(String::valueOf).collect(Collectors.toList());

// Check if any matches condition
list.stream().anyMatch(x -> x > 100);

// First element or default
list.stream().findFirst().orElse(defaultValue);

// Join strings with delimiter
list.stream().collect(Collectors.joining(","));

// Average of integers
ints.stream().mapToInt(i -> i).average().orElse(0);

// Sort descending
list.stream().sorted(Comparator.reverseOrder()).collect(Collectors.toList());

// Max by field
employees.stream().max(Comparator.comparing(Employee::getSalary)).get();

// Square and sum
numbers.stream().mapToInt(n -> n * n).sum();

// Partition by condition
list.stream().collect(Collectors.partitioningBy(x -> x > 10));
```

Memory Aid: COMMAND Pattern


```
Create → Filter → Transform → Collect/Save
.of()      .filter() .map()      .collect()
.range()   .distinct() .flatMap() .toList()
.stream()  .limit()   .sorted()  .toSet()
```

Real Example:

```
List<Employee> seniorDevelopers = employees.stream()
    .filter(e -> e.getAge() > 30)
    .filter(e -> "IT".equals(e.getDepartment()))
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .limit(5)
    .collect(Collectors.toList());
```

Pro Tip: Method References vs Lambdas

```
// Lambda (verbose)
.map(s -> s.toUpperCase())

// Method Reference (clean)
.map(String::toUpperCase)

// Common method references:
String::toUpperCase    // instance method
String::length         // instance method
Employee::getName      // getter
Math::max              // static method
System.out::println    // specific instance
this::processMethod    // this instance
```

Easy, Medium, Hard with Stream Solutions

EASY LEVEL

Problem 1: Find Duplicate Characters in String

Stream-Based Solution

```

import java.util.*;
import java.util.stream.*;
import java.util.function.Function;

public class DuplicateCharacters {

    // Method 1: Using Streams with Collections.frequency
    public static Set<Character> findDuplicatesStreams(String str) {
        if (str == null || str.isEmpty()) return Collections.emptySet();

        return str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() > 1)
            .map(Map.Entry::getKey)
            .collect(Collectors.toSet());
    }

    // Method 2: Without streams (traditional)
    public static Set<Character> findDuplicatesTraditional(String str) {
        Set<Character> duplicates = new HashSet<>();
        Set<Character> seen = new HashSet<>();

        for (char c : str.toCharArray()) {
            if (!seen.add(c)) {
                duplicates.add(c);
            }
        }
        return duplicates;
    }

    // Method 3: Find duplicates with their counts
    public static Map<Character, Long> findDuplicateCounts(String str) {
        return str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() > 1)
            .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
    }
}

```

```

    }

    // Method 4: First non-repeating character using streams
    public static Character firstNonRepeatingChar(String str) {
        return str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(),
LinkedHashMap::new, Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() == 1)
            .map(Map.Entry::getKey)
            .findFirst()
            .orElse(null);
    }

    public static void main(String[] args) {
        String input = "programming";
        System.out.println("Duplicate characters: " +
findDuplicatesStreams(input));
        System.out.println("Duplicate counts: " + findDuplicateCounts(input));
        System.out.println("First non-repeating: " +
firstNonRepeatingChar(input));
        // Output: Duplicate characters: [r, g, m]
    }
}

```

Problem 2: Fibonacci Series

Multiple Solutions

```

import java.util.stream.*;
import java.util.*;

public class FibonacciGenerator {

    // Method 1: Using Stream.iterate (Infinite stream)
    public static List<Long> fibonacciStreams(int n) {
        return Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] +
f[1]})
            .limit(n)
            .map(f -> f[0])
            .collect(Collectors.toList());
    }

    // Method 2: Using IntStream.generate
    public static List<Integer> fibonacciGenerate(int n) {
        return IntStream.generate(new Supplier<Integer>() {
            private int prev = 0;
            private int current = 1;

            @Override
            public Integer get() {
                int next = prev;
                int temp = prev + current;
                prev = current;
                current = temp;
                return next;
            }
        }).limit(n)
            .boxed()
            .collect(Collectors.toList());
    }

    // Method 3: Recursive (classic, but inefficient for large n)
    public static long fibonacciRecursive(int n) {
        if (n <= 1) return n;
        return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
    }

    // Method 4: Memoized recursion (efficient)
    private static Map<Integer, Long> memo = new HashMap<>();

```

```

public static long fibonacciMemoized(int n) {
    if (n <= 1) return n;
    return memo.computeIfAbsent(n,
        k -> fibonacciMemoized(n - 1) + fibonacciMemoized(n - 2));
}

// Method 5: Iterative (most efficient)
public static long fibonacciIterative(int n) {
    if (n <= 1) return n;
    long prev = 0, current = 1;
    for (int i = 2; i <= n; i++) {
        long next = prev + current;
        prev = current;
        current = next;
    }
    return current;
}

// Method 6: Find first fibonacci number greater than threshold
public static long firstFibonacciAbove(long threshold) {
    return Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] +
f[1]})
        .filter(f -> f[0] > threshold)
        .findFirst()
        .map(f -> f[0])
        .orElse(-1L);
}

// Method 7: Check if number is fibonacci
public static boolean isFibonacci(long num) {
    // A number is Fibonacci if  $(5*n^2 + 4)$  or  $(5*n^2 - 4)$  is perfect square
    long square1 = 5 * num * num + 4;
    long square2 = 5 * num * num - 4;
    return isPerfectSquare(square1) || isPerfectSquare(square2);
}

private static boolean isPerfectSquare(long x) {
    long s = (long) Math.sqrt(x);
    return s * s == x;
}

public static void main(String[] args) {
    System.out.println("First 10 Fibonacci: " + fibonacciStreams(10));
}

```

```

        // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

        System.out.println("First > 100: " + firstFibonacciAbove(100));
        // 144
    }
}

```

Problem 3: Palindrome Check

```

public class PalindromeChecker {

    // Stream-based solution
    public static boolean isPalindromeStream(String str) {
        String cleaned = str.toLowerCase().replaceAll("[^a-zA-Z0-9]", "");
        return IntStream.range(0, cleaned.length() / 2)
            .allMatch(i -> cleaned.charAt(i) == cleaned.charAt(cleaned.length() -
1 - i));
    }

    // Using StringBuilder (simplest)
    public static boolean isPalindromeSimple(String str) {
        String cleaned = str.toLowerCase().replaceAll("[^a-zA-Z0-9]", "");
        return new StringBuilder(cleaned).reverse().toString().equals(cleaned);
    }

    // Two-pointer technique
    public static boolean isPalindromeTwoPointer(String str) {
        String cleaned = str.toLowerCase().replaceAll("[^a-zA-Z0-9]", "");
        int left = 0, right = cleaned.length() - 1;
        while (left < right) {
            if (cleaned.charAt(left++) != cleaned.charAt(right--)) {
                return false;
            }
        }
        return true;
    }
}

```

MEDIUM LEVEL

Problem 4: Second Highest Salary (Classic Stream Problem)

Employee Class and Solutions

```
import java.util.*;
import java.util.stream.*;

class Employee {
    private String name;
    private double salary;
    private String department;
    private int age;

    // Constructor, getters, toString
    public Employee(String name, double salary, String department, int age) {
        this.name = name;
        this.salary = salary;
        this.department = department;
        this.age = age;
    }

    public double getSalary() { return salary; }
    public String getName() { return name; }
    public String getDepartment() { return department; }
    public int getAge() { return age; }

    @Override
    public String toString() {
        return String.format("%s - $%.2f - %s - %d", name, salary, department,
age);
    }
}

public class SalaryAnalyzer {

    // Method 1: Second highest salary overall
    public static Optional<Double> secondHighestSalary(List<Employee> employees) {
        return employees.stream()
            .map(Employee::getSalary)
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(1)
            .findFirst();
    }

    // Method 2: Second highest employee (full object)
```



```

    public static Optional<Employee> secondHighestEmployee(List<Employee>
employees) {
        return employees.stream()
            .sorted(Comparator.comparing(Employee::getSalary).reversed())
            .distinct()
            .skip(1)
            .findFirst();
    }

    // Method 3: Second highest salary per department
    public static Map<String, Optional<Double>>
secondHighestPerDepartment(List<Employee> employees) {
        return employees.stream()
            .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.collectingAndThen(
                    Collectors.mapping(Employee::getSalary, Collectors.toList()),
                    salaries -> salaries.stream()
                        .distinct()
                        .sorted(Comparator.reverseOrder())
                        .skip(1)
                        .findFirst()
                )
            ));
    }

    // Method 4: Nth highest salary (generic)
    public static Optional<Double> nthHighestSalary(List<Employee> employees, int
n) {
        if (n <= 0) return Optional.empty();

        return employees.stream()
            .map(Employee::getSalary)
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(n - 1)
            .findFirst();
    }

    // Method 5: Second highest with duplicates handled
    public static Optional<Employee> secondHighestByRank(List<Employee> employees)
{
        return employees.stream()

```

```

        .collect(Collectors.groupingBy(Employee::getSalary))
        .entrySet().stream()
        .sorted(Map.Entry.<Double, List<Employee>>comparingByKey().reversed())
        .skip(1)
        .findFirst()
        .map(Map.Entry::getValue)
        .flatMap(list -> list.stream().findFirst());
    }

    // Method 6: Top 3 salaries across all departments
    public static List<Employee> topThreeSalaries(List<Employee> employees) {
        return employees.stream()
            .sorted(Comparator.comparing(Employee::getSalary).reversed())
            .limit(3)
            .collect(Collectors.toList());
    }

    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("John", 100000, "IT", 30),
            new Employee("Jane", 95000, "HR", 28),
            new Employee("Bob", 100000, "IT", 35), // Duplicate salary
            new Employee("Alice", 90000, "HR", 32),
            new Employee("Charlie", 110000, "IT", 40),
            new Employee("Diana", 85000, "Finance", 29)
        );

        secondHighestSalary(employees).ifPresent(s ->
            System.out.println("Second highest salary: $" + s));
        // Second highest salary: $100000 (not 95000 because duplicate handling)

        secondHighestPerDepartment(employees).forEach((dept, salary) ->
            System.out.println(dept + " second highest: $" + salary.orElse(0.0)));

        // IT second highest: $100000.0
        // HR second highest: $90000.0
        // Finance second highest: $0.0 (only one employee)
    }
}

```

Problem 5: Find All Permutations of String (Medium/Hard)

```

public class StringPermutations {

    // Method 1: Generate all distinct permutations using Streams approach
    public static Set<String> permutationsStream(String str) {
        if (str == null || str.isEmpty()) {
            return new HashSet<>();
        }

        return permutationsHelper(str, "")
            .collect(Collectors.toSet());
    }

    private static Stream<String> permutationsHelper(String remaining, String
prefix) {
        if (remaining.isEmpty()) {
            return Stream.of(prefix);
        }

        return IntStream.range(0, remaining.length())
            .boxed()
            .flatMap(i -> {
                char ch = remaining.charAt(i);
                String newRemaining = remaining.substring(0, i) +
remaining.substring(i + 1);
                return permutationsHelper(newRemaining, prefix + ch);
            });
    }

    // Method 2: Backtracking approach (more efficient)
    public static List<String> permutationsBacktrack(String str) {
        List<String> result = new ArrayList<>();
        char[] chars = str.toCharArray();
        backtrack(chars, 0, result);
        return result;
    }

    private static void backtrack(char[] chars, int index, List<String> result) {
        if (index == chars.length - 1) {
            result.add(new String(chars));
            return;
        }
    }
}

```

```

        for (int i = index; i < chars.length; i++) {
            swap(chars, index, i);
            backtrack(chars, index + 1, result);
            swap(chars, index, i); // backtrack
        }
    }
}

```

```

private static void swap(char[] chars, int i, int j) {
    char temp = chars[i];
    chars[i] = chars[j];
    chars[j] = temp;
}

```

```

// Method 3: Unique permutations only (handle duplicates)
public static List<String> uniquePermutations(String str) {
    List<String> result = new ArrayList<>();
    char[] chars = str.toCharArray();
    Arrays.sort(chars); // Sort to handle duplicates
    boolean[] used = new boolean[chars.length];
    backtrackUnique(chars, used, new StringBuilder(), result);
    return result;
}

```

```

private static void backtrackUnique(char[] chars, boolean[] used,
                                    StringBuilder current, List<String>
result) {
    if (current.length() == chars.length) {
        result.add(current.toString());
        return;
    }

    for (int i = 0; i < chars.length; i++) {
        if (used[i]) continue;
        // Skip duplicates
        if (i > 0 && chars[i] == chars[i - 1] && !used[i - 1]) continue;

        used[i] = true;
        current.append(chars[i]);
        backtrackUnique(chars, used, current, result);
        current.deleteCharAt(current.length() - 1);
        used[i] = false;
    }
}

```

```
public static void main(String[] args) {  
    System.out.println("Permutations of 'abc': " +  
permutationsBacktrack("abc"));  
    // [abc, acb, bac, bca, cab, cba]  
  
    System.out.println("Unique permutations of 'aab': " +  
uniquePermutations("aab"));  
    // [aab, aba, baa]  
}  
}
```

Problem 6: Group Anagrams

```

public class AnagramGrouper {

    // Stream-based solution - group anagrams
    public static Map<String, List<String>> groupAnagrams(String[] words) {
        return Arrays.stream(words)
            .collect(Collectors.groupingBy(
                word -> word.chars()
                    .sorted()
                    .collect(StringBuilder::new,
                        StringBuilder::appendCodePoint,
                        StringBuilder::append)
                    .toString()
            ));
    }

    // More efficient with char array sorting
    public static Map<String, List<String>> groupAnagramsEfficient(String[] words)
    {
        return Arrays.stream(words)
            .collect(Collectors.groupingBy(word -> {
                char[] chars = word.toCharArray();
                Arrays.sort(chars);
                return new String(chars);
            }));
    }

    // Find all anagram groups and return only those with size > 1
    public static List<List<String>> findAnagramGroups(String[] words) {
        return new ArrayList<>(groupAnagramsEfficient(words).values().stream()
            .filter(group -> group.size() > 1)
            .collect(Collectors.toList()));
    }

    public static void main(String[] args) {
        String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
        System.out.println(groupAnagrams(words));
        // {aet=[eat, tea, ate], ant=[tan, nat], abt=[bat]}
    }
}

```

HARD LEVEL

Problem 7: Tree Problem — Lowest Common Ancestor in Binary Tree

```

import java.util.*;
import java.util.stream.*;

// Binary Tree Node
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
    }

    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }

    @Override
    public String toString() {
        return String.valueOf(val);
    }
}

public class BinaryTreeLCA {

    // Problem 1: Lowest Common Ancestor (LCA) in Binary Tree
    public static TreeNode findLCA(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) {
            return root;
        }

        TreeNode left = findLCA(root.left, p, q);
        TreeNode right = findLCA(root.right, p, q);

        if (left != null && right != null) {
            return root; // Found LCA
        }

        return left != null ? left : right;
    }
}

```



```

// Problem 2: LCA in Binary Search Tree (more efficient)
public static TreeNode findLCAInBST(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null) return null;

    // If both values are smaller, go left
    if (p.val < root.val && q.val < root.val) {
        return findLCAInBST(root.left, p, q);
    }

    // If both values are larger, go right
    if (p.val > root.val && q.val > root.val) {
        return findLCAInBST(root.right, p, q);
    }

    // One on each side, current node is LCA
    return root;
}

// Problem 3: Find path from root to node (using streams)
public static List<TreeNode> findPath(TreeNode root, TreeNode target) {
    List<TreeNode> path = new ArrayList<>();
    findPathHelper(root, target, path);
    return path;
}

private static boolean findPathHelper(TreeNode root, TreeNode target,
List<TreeNode> path) {
    if (root == null) return false;

    path.add(root);

    if (root == target) return true;

    if (findPathHelper(root.left, target, path) || findPathHelper(root.right,
target, path)) {
        return true;
    }

    path.remove(path.size() - 1);
    return false;
}

```

```
// Problem 4: Find distance between two nodes
```

```
public static int findDistance(TreeNode root, TreeNode p, TreeNode q) {  
    TreeNode lca = findLCA(root, p, q);  
    int distP = findLevel(lca, p, 0);  
    int distQ = findLevel(lca, q, 0);  
    return distP + distQ;  
}
```

```
private static int findLevel(TreeNode root, TreeNode target, int level) {  
    if (root == null) return -1;  
    if (root == target) return level;  
  
    int left = findLevel(root.left, target, level + 1);  
    return left != -1 ? left : findLevel(root.right, target, level + 1);  
}
```

```
// Problem 5: Get all nodes at distance K from target
```

```
public static List<TreeNode> nodesAtDistanceK(TreeNode root, TreeNode target,  
int k) {  
    List<TreeNode> result = new ArrayList<>();  
    Map<TreeNode, TreeNode> parentMap = new HashMap<>();  
    buildParentMap(root, null, parentMap);  
  
    Queue<TreeNode> queue = new LinkedList<>();  
    Set<TreeNode> visited = new HashSet<>();  
  
    queue.offer(target);  
    visited.add(target);  
    int distance = 0;  
  
    while (!queue.isEmpty() && distance < k) {  
        int size = queue.size();  
        for (int i = 0; i < size; i++) {  
            TreeNode node = queue.poll();  
  
            // Check left child  
            if (node.left != null && !visited.contains(node.left)) {  
                visited.add(node.left);  
                queue.offer(node.left);  
            }  
  
            // Check right child  
            if (node.right != null && !visited.contains(node.right)) {
```

```

        visited.add(node.right);
        queue.offer(node.right);
    }

    // Check parent
    TreeNode parent = parentMap.get(node);
    if (parent != null && !visited.contains(parent)) {
        visited.add(parent);
        queue.offer(parent);
    }
}
distance++;
}

result.addAll(queue);
return result;
}

```

```

private static void buildParentMap(TreeNode root, TreeNode parent,
Map<TreeNode, TreeNode> map) {
    if (root == null) return;
    map.put(root, parent);
    buildParentMap(root.left, root, map);
    buildParentMap(root.right, root, map);
}

```

```

// Problem 6: Binary Tree Level Order Traversal (using streams)
public static List<List<Integer>> levelOrderTraversal(TreeNode root) {
    if (root == null) return Collections.emptyList();

    List<List<Integer>> result = new ArrayList<>();
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }
}

```

```

    }
    result.add(level);
}
return result;
}

```

```

public static void main(String[] args) {

```

```

    // Build binary tree:

```

```

    //      3
    //     / \
    //    5   1
    //   / \ / \
    //  6  2 0 8
    //   / \
    //  7   4

```

```

    TreeNode root = new TreeNode(3);
    TreeNode node5 = new TreeNode(5);
    TreeNode node1 = new TreeNode(1);
    TreeNode node6 = new TreeNode(6);
    TreeNode node2 = new TreeNode(2);
    TreeNode node0 = new TreeNode(0);
    TreeNode node8 = new TreeNode(8);
    TreeNode node7 = new TreeNode(7);
    TreeNode node4 = new TreeNode(4);

```

```

    root.left = node5;
    root.right = node1;
    node5.left = node6;
    node5.right = node2;
    node1.left = node0;
    node1.right = node8;
    node2.left = node7;
    node2.right = node4;

```

```

    System.out.println("LCA of 5 and 1: " + findLCA(root, node5, node1).val);
    // Output: 3

```

```

    System.out.println("Distance between 7 and 4: " + findDistance(root,
node7, node4));
    // Output: 4 (7→2→5→3→1? Actually 7→2→5→3→1→0? Let's compute properly)

```

```

    System.out.println("Level order: " + levelOrderTraversal(root));

```

```
        // [[3], [5,1], [6,2,0,8], [7,4]]  
    }  
}
```

Problem 8: Graph Problem — Clone Graph & Detect Cycle

```

import java.util.*;

// Graph Node
class GraphNode {
    int val;
    List<GraphNode> neighbors;

    GraphNode(int val) {
        this.val = val;
        this.neighbors = new ArrayList<>();
    }

    GraphNode(int val, List<GraphNode> neighbors) {
        this.val = val;
        this.neighbors = neighbors;
    }

    @Override
    public String toString() {
        return String.valueOf(val);
    }
}

public class GraphProblems {

    // Problem 1: Clone Graph (Deep Copy using BFS)
    public static GraphNode cloneGraph(GraphNode node) {
        if (node == null) return null;

        Map<GraphNode, GraphNode> visited = new HashMap<>();
        Queue<GraphNode> queue = new LinkedList<>();

        visited.put(node, new GraphNode(node.val));
        queue.offer(node);

        while (!queue.isEmpty()) {
            GraphNode current = queue.poll();

            for (GraphNode neighbor : current.neighbors) {
                if (!visited.containsKey(neighbor)) {
                    visited.put(neighbor, new GraphNode(neighbor.val));
                    queue.offer(neighbor);
                }
            }
        }
    }
}

```

```

        }
        visited.get(current).neighbors.add(visited.get(neighbor));
    }
}

return visited.get(node);
}

```

// Problem 2: Detect Cycle in Directed Graph (DFS)

```

public static boolean hasCycleDirected(List<GraphNode> graph) {
    Set<GraphNode> visited = new HashSet<>();
    Set<GraphNode> recursionStack = new HashSet<>();

    for (GraphNode node : graph) {
        if (detectCycleDFSUtil(node, visited, recursionStack)) {
            return true;
        }
    }
    return false;
}

```

```

private static boolean detectCycleDFSUtil(GraphNode node, Set<GraphNode>
visited,
                                Set<GraphNode> recursionStack) {
    if (recursionStack.contains(node)) return true;
    if (visited.contains(node)) return false;

    visited.add(node);
    recursionStack.add(node);

    for (GraphNode neighbor : node.neighbors) {
        if (detectCycleDFSUtil(neighbor, visited, recursionStack)) {
            return true;
        }
    }

    recursionStack.remove(node);
    return false;
}

```

// Problem 3: Detect Cycle in Undirected Graph (Union-Find)

```

public static boolean hasCycleUndirected(List<Edge> edges, int vertices) {
    int[] parent = new int[vertices];
}

```

```

    for (int i = 0; i < vertices; i++) {
        parent[i] = i;
    }

    for (Edge edge : edges) {
        int root1 = find(parent, edge.src);
        int root2 = find(parent, edge.dest);

        if (root1 == root2) {
            return true; // Cycle detected
        }

        union(parent, root1, root2);
    }
    return false;
}

private static int find(int[] parent, int x) {
    if (parent[x] != x) {
        parent[x] = find(parent, parent[x]); // Path compression
    }
    return parent[x];
}

private static void union(int[] parent, int x, int y) {
    int rootX = find(parent, x);
    int rootY = find(parent, y);
    if (rootX != rootY) {
        parent[rootY] = rootX;
    }
}

static class Edge {
    int src, dest;
    Edge(int src, int dest) {
        this.src = src;
        this.dest = dest;
    }
}

// Problem 4: Find if path exists between two nodes (BFS)
public static boolean hasPath(GraphNode start, GraphNode end) {
    if (start == end) return true;

```



```

Set<GraphNode> visited = new HashSet<>();
Queue<GraphNode> queue = new LinkedList<>();

visited.add(start);
queue.offer(start);

while (!queue.isEmpty()) {
    GraphNode current = queue.poll();

    for (GraphNode neighbor : current.neighbors) {
        if (neighbor == end) return true;
        if (!visited.contains(neighbor)) {
            visited.add(neighbor);
            queue.offer(neighbor);
        }
    }
}
return false;
}

```

```

// Problem 5: Topological Sort (Kahn's Algorithm)
public static List<Integer> topologicalSort(List<GraphNode> graph, int
vertices) {
    int[] inDegree = new int[vertices];

    // Calculate in-degree for each node
    for (GraphNode node : graph) {
        for (GraphNode neighbor : node.neighbors) {
            inDegree[neighbor.val]++;
        }
    }

    // Add all nodes with 0 in-degree to queue
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < vertices; i++) {
        if (inDegree[i] == 0) {
            queue.offer(i);
        }
    }

    List<Integer> result = new ArrayList<>();
    while (!queue.isEmpty()) {

```

```

        int current = queue.poll();
        result.add(current);

        // Decrease in-degree of neighbors
        for (GraphNode neighbor : graph.get(current).neighbors) {
            inDegree[neighbor.val]--;
            if (inDegree[neighbor.val] == 0) {
                queue.offer(neighbor.val);
            }
        }
    }

    return result.size() == vertices ? result : new ArrayList<>(); // Cycle
detected
}

// Problem 6: Find shortest path in unweighted graph (BFS)
public static List<GraphNode> shortestPath(GraphNode start, GraphNode end) {
    if (start == end) return List.of(start);

    Map<GraphNode, GraphNode> parent = new HashMap<>();
    Queue<GraphNode> queue = new LinkedList<>();

    parent.put(start, null);
    queue.offer(start);

    while (!queue.isEmpty()) {
        GraphNode current = queue.poll();

        for (GraphNode neighbor : current.neighbors) {
            if (!parent.containsKey(neighbor)) {
                parent.put(neighbor, current);
                if (neighbor == end) {
                    return reconstructPath(parent, end);
                }
                queue.offer(neighbor);
            }
        }
    }

    return Collections.emptyList(); // No path found
}

```

```

    private static List<GraphNode> reconstructPath(Map<GraphNode, GraphNode>
parent, GraphNode end) {
        List<GraphNode> path = new ArrayList<>();
        GraphNode current = end;

        while (current != null) {
            path.add(0, current);
            current = parent.get(current);
        }

        return path;
    }

    public static void main(String[] args) {
        // Create graph: 0->1, 0->2, 1->2, 2->0, 2->3, 3->3
        GraphNode node0 = new GraphNode(0);
        GraphNode node1 = new GraphNode(1);
        GraphNode node2 = new GraphNode(2);
        GraphNode node3 = new GraphNode(3);

        node0.neighbors = Arrays.asList(node1, node2);
        node1.neighbors = Arrays.asList(node2);
        node2.neighbors = Arrays.asList(node0, node3);
        node3.neighbors = Arrays.asList(node3);

        List<GraphNode> graph = Arrays.asList(node0, node1, node2, node3);

        System.out.println("Has cycle: " + hasCycleDirected(graph)); // true

        // Shortest path
        List<GraphNode> path = shortestPath(node0, node3);
        System.out.println("Shortest path 0→3: " + path);
        // [0, 2, 3]
    }
}

```

Problem 9: Backtracking — N-Queens Problem

```

import java.util.*;
import java.util.stream.IntStream;

public class NQueensProblem {

    // Problem: Place N queens on NxN chessboard so no two attack each other

    // Solution 1: Standard backtracking
    public static List<List<String>> solveNQueens(int n) {
        List<List<String>> solutions = new ArrayList<>();
        char[][] board = new char[n][n];

        for (int i = 0; i < n; i++) {
            Arrays.fill(board[i], '.');
        }

        backtrack(board, 0, solutions);
        return solutions;
    }

    private static void backtrack(char[][] board, int row, List<List<String>>
solutions) {
        if (row == board.length) {
            solutions.add(constructSolution(board));
            return;
        }

        for (int col = 0; col < board.length; col++) {
            if (isSafe(board, row, col)) {
                board[row][col] = 'Q';
                backtrack(board, row + 1, solutions);
                board[row][col] = '.'; // Undo
            }
        }
    }

    private static boolean isSafe(char[][] board, int row, int col) {
        // Check column
        for (int i = 0; i < row; i++) {
            if (board[i][col] == 'Q') return false;
        }
    }

```

```

        // Check diagonal (top-left to bottom-right)
        for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
            if (board[i][j] == 'Q') return false;
        }

        // Check diagonal (top-right to bottom-left)
        for (int i = row - 1, j = col + 1; i >= 0 && j < board.length; i--, j++) {
            if (board[i][j] == 'Q') return false;
        }

        return true;
    }

    private static List<String> constructSolution(char[][] board) {
        return Arrays.stream(board)
            .map(String::new)
            .collect(Collectors.toList());
    }

    // Solution 2: Optimized with column and diagonal tracking
    public static List<List<String>> solveNQueensOptimized(int n) {
        List<List<String>> solutions = new ArrayList<>();
        int[] queens = new int[n]; // queens[row] = column
        boolean[] cols = new boolean[n];
        boolean[] diag1 = new boolean[2 * n - 1]; // row - col + n - 1
        boolean[] diag2 = new boolean[2 * n - 1]; // row + col

        backtrackOptimized(queens, cols, diag1, diag2, 0, n, solutions);
        return solutions;
    }

    private static void backtrackOptimized(int[] queens, boolean[] cols, boolean[]
diag1,
                                           boolean[] diag2, int row, int n,
                                           List<List<String>> solutions) {
        if (row == n) {
            solutions.add(constructSolutionOptimized(queens, n));
            return;
        }

        for (int col = 0; col < n; col++) {
            int d1 = row - col + n - 1;
            int d2 = row + col;

```

```

        if (!cols[col] && !diag1[d1] && !diag2[d2]) {
            queens[row] = col;
            cols[col] = true;
            diag1[d1] = true;
            diag2[d2] = true;

            backtrackOptimized(queens, cols, diag1, diag2, row + 1, n,
solutions);

            cols[col] = false;
            diag1[d1] = false;
            diag2[d2] = false;
        }
    }
}

private static List<String> constructSolutionOptimized(int[] queens, int n) {
    return IntStream.range(0, n)
        .mapToObj(row -> {
            char[] line = new char[n];
            Arrays.fill(line, '.');
            line[queens[row]] = 'Q';
            return new String(line);
        })
        .collect(Collectors.toList());
}

// Solution 3: Count total solutions (no board construction)
public static int totalNQueens(int n) {
    int[] count = new int[]{0};
    boolean[] cols = new boolean[n];
    boolean[] diag1 = new boolean[2 * n - 1];
    boolean[] diag2 = new boolean[2 * n - 1];

    backtrackCount(cols, diag1, diag2, 0, n, count);
    return count[0];
}

private static void backtrackCount(boolean[] cols, boolean[] diag1, boolean[]
diag2,
                                int row, int n, int[] count) {
    if (row == n) {

```

```

        count[0]++;
        return;
    }

    for (int col = 0; col < n; col++) {
        int d1 = row - col + n - 1;
        int d2 = row + col;

        if (!cols[col] && !diag1[d1] && !diag2[d2]) {
            cols[col] = true;
            diag1[d1] = true;
            diag2[d2] = true;

            backtrackCount(cols, diag1, diag2, row + 1, n, count);

            cols[col] = false;
            diag1[d1] = false;
            diag2[d2] = false;
        }
    }
}

// Solution 4: Stream-based solution verification (check if board is valid)
public static boolean isValidNQueensSolution(List<String> board) {
    int n = board.size();
    List<int[]> queens = IntStream.range(0, n)
        .boxed()
        .flatMap(row -> IntStream.range(0, n)
            .filter(col -> board.get(row).charAt(col) == 'Q')
            .mapToObj(col -> new int[]{row, col}))
        .collect(Collectors.toList());

    if (queens.size() != n) return false;

    return queens.stream().noneMatch(q1 ->
        queens.stream().anyMatch(q2 ->
            q1 != q2 && (q1[0] == q2[0] || q1[1] == q2[1] ||
                Math.abs(q1[0] - q2[0]) == Math.abs(q1[1] - q2[1]))
        )
    );
}

public static void main(String[] args) {

```

```

        System.out.println("N-Queens Solutions for N=4:");
        List<List<String>> solutions = solveNQueens(4);
        solutions.forEach(solution -> {
            System.out.println("Solution:");
            solution.forEach(System.out::println);
            System.out.println();
        });

        System.out.println("Total solutions for N=8: " + totalNQueens(8));
        // Output: 92
    }
}

```

QUICK REFERENCE SUMMARY

Difficulty	Problem	Key Technique	Stream Usage
Easy	Duplicate Characters	GroupingBy, Filter	✓ Heavy
Easy	Fibonacci	Stream.iterate, Generate	✓ Heavy
Easy	Palindrome	IntStream.range, allMatch	✓ Heavy
Medium	Second Highest Salary	Sorting, Skip, Distinct	✓ Heavy
Medium	Permutations	Backtracking	✗ Minimal
Medium	Group Anagrams	GroupingBy, Sorting	✓ Heavy
Hard	LCA in Tree	DFS, Recursion	✗ None
Hard	Clone Graph	BFS, HashMap	✗ None
Hard	N-Queens	Backtracking	✗ None

One-Line Stream Gems (Bonus)


```
// Reverse a string
String reversed = new StringBuilder(str).reverse().toString();

// Check if string contains only digits
boolean isNumeric = str.chars().allMatch(Character::isDigit);

// Find most frequent character
char mostFrequent = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .get().getKey();

// Check if two strings are anagrams
boolean areAnagrams = str1.length() == str2.length() &&
    str1.chars().sorted().boxed().collect(Collectors.toList())
    .equals(str2.chars().sorted().boxed().collect(Collectors.toList()));

// Remove duplicates from list preserving order
List<Integer> distinct = list.stream()
    .distinct()
    .collect(Collectors.toList());

// Find common elements between two lists
List<Integer> common = list1.stream()
    .filter(list2::contains)
    .collect(Collectors.toList());
```

1. Filter & Map

```
List<String> result = people.stream()
    .filter(p -> p.getAge() > 25)
    .map(p -> p.getName())
    .toList();
```

✓ filter → condition

✓ map → transform

2. Find First

```
Optional<Integer> num = list.stream()
    .filter(n -> n % 2 == 0)
    .findFirst();
```

✓ findFirst → ordered

✓ findAny → faster

3. Remove Duplicates

```
List<Integer> unique = list.stream()
    .distinct()
    .toList();
```

4. Sorting

```
List<Integer> sorted = list.stream()
    .sorted()
    .toList();
```

Reverse:

```
.sorted(Comparator.reverseOrder())
```

5. Count

```
long count = list.stream()
    .filter(n -> n > 10)
    .count();
```

6. Group By

```
Map<String, List<Employee>> map =  
    employees.stream()  
        .collect(Collectors.groupingBy(Employee::getDept));
```

7. Count by Group

```
Map<String, Long> count =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDept,  
            Collectors.counting()  
        ));
```

8. Max / Min

```
Optional<Integer> max = list.stream()  
    .max(Integer::compareTo);
```

9. Reduce (Sum)

```
int sum = list.stream()  
    .reduce(0, (a, b) -> a + b);
```

Better:

```
int sum = list.stream()  
    .mapToInt(Integer::intValue)  
    .sum();
```

10. FlatMap

```
List<Integer> flat = nested.stream()  
    .flatMap(List::stream)  
    .toList();
```

11. List → Map

```
Map<Integer, String> map =  
    list.stream()  
        .collect(Collectors.toMap(  
            e -> e.getId(),  
            e -> e.getName()  
        ));
```

12. Partitioning

```
Map<Boolean, List<Integer>> result =  
    list.stream()  
        .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

13. Join Strings

```
String result = list.stream()  
    .map(String::valueOf)  
    .collect(Collectors.joining(", "));
```

14. Skip & Limit

```
list.stream()  
    .skip(2)  
    .limit(5)  
    .forEach(System.out::println);
```

15. Match Methods

```
list.stream().allMatch(n -> n > 0);  
list.stream().anyMatch(n -> n > 10);  
list.stream().noneMatch(n -> n < 0);
```

16. Peek (Debug)

```
list.stream()  
    .peek(System.out::println)  
    .map(n -> n * 2)  
    .toList();
```

17. Parallel Stream

```
list.parallelStream()  
    .forEach(System.out::println);
```

⚠ Order not guaranteed

18. Max Salary

```
Optional<Employee> emp =  
    employees.stream()  
        .max(Comparator.comparing(Employee::getSalary));
```

19. Second Highest

```
int secondHighest = list.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElseThrow();
```

20. Frequency Count

```
Map<Integer, Long> freq =
    list.stream()
        .collect(Collectors.groupingBy(
            n -> n,
            Collectors.counting()
        ));
```

Stream Flow Diagram

```
Collection
↓
stream()
↓
filter → map → sorted
↓
collect / reduce / count
```

Cheat Sheet

```
filter()    → condition
map()       → transform
flatMap()   → flatten
sorted()    → sort
distinct()  → remove duplicates
limit()     → restrict
skip()      → skip elements
collect()   → convert
reduce()    → aggregate
findFirst() → first match
groupingBy() → grouping
partitioningBy() → split
```

Interview Tip

 Always follow:

stream() → filter → map → collect

Here are  **Advanced Java Stream Problems (FAANG Level)** with **clean solutions + thinking approach**. These are the kinds of questions asked in **Amazon, Google, Microsoft** interviews.

1. Top K Frequent Elements

Problem

Find top k frequent numbers.

```
List<Integer> list = List.of(1,1,1,2,2,3);  
int k = 2;
```

Solution

```
Map<Integer, Long> freq = list.stream()  
    .collect(Collectors.groupingBy(n -> n, Collectors.counting()));  
  
List<Integer> result = freq.entrySet().stream()  
    .sorted(Map.Entry.<Integer, Long>comparingByValue().reversed())  
    .limit(k)  
    .map(Map.Entry::getKey)  
    .toList();
```

Key Idea

 groupingBy + sorting + limit

2. Longest String in List

```
String longest = list.stream()  
    .max(Comparator.comparingInt(String::length))  
    .orElse("");
```

3. Group Anagrams

Input

```
["eat", "tea", "tan", "ate", "nat", "bat"]
```

Solution

```
Map<String, List<String>> map =  
    list.stream()  
    .collect(Collectors.groupingBy(s -> {
```

```
char[] arr = s.toCharArray();
Arrays.sort(arr);
return new String(arr);
});
```

 Idea

 Same sorted string → anagram group

 4. Find First Non-Repeating Character

```
Character ch = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(
        c -> c,
        LinkedHashMap::new,
        Collectors.counting()
    ))
    .entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null);
```

 5. Merge Two Lists Without Duplicates

```
List<Integer> result = Stream.concat(list1.stream(), list2.stream())
    .distinct()
    .toList();
```

 6. Find Duplicate Elements

```
Set<Integer> duplicates = list.stream()
    .collect(Collectors.groupingBy(n -> n, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toSet());
```

 7. Employee with Highest Salary per Department

```
Map<String, Optional<Employee>> result =
    employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
```

```
Collectors.maxBy(  
    Comparator.comparing(Employee::getSalary)  
)  
));
```

8. Flatten Nested Objects

```
List<Order> orders = users.stream()  
    .flatMap(user -> user.getOrders().stream())  
    .toList();
```

9. Partition Employees by Salary

```
Map<Boolean, List<Employee>> result =  
    employees.stream()  
        .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));
```

10. Convert List to Map with Duplicate Keys

```
Map<String, Integer> map =  
    list.stream()  
        .collect(Collectors.toMap(  
            e -> e.getName(),  
            e -> e.getAge(),  
            (oldVal, newVal) -> oldVal // resolve conflict  
        ));
```

11. Running Total (Cumulative Sum) ⚡

👉 Streams don't directly support this (tricky!)

```
AtomicInteger sum = new AtomicInteger(0);
```

```
List<Integer> result = list.stream()  
    .map(sum::addAndGet)  
    .toList();
```

12. Find Second Highest Salary 🔥

```
Optional<Integer> second =  
    list.stream()  
        .distinct()  
        .sorted(Comparator.reverseOrder())
```



```
.skip(1)
.findFirst();
```

13. Check Palindrome Strings

```
list.stream()
    .filter(s -> s.equals(new StringBuilder(s).reverse().toString()))
    .toList();
```

14. Find Missing Number

```
int n = 5;

int missing = IntStream.rangeClosed(1, n)
    .sum() - list.stream().mapToInt(Integer::intValue).sum();
```

15. Sort Map by Value 🔥

```
LinkedHashMap<String, Integer> sorted =
    map.entrySet().stream()
        .sorted(Map.Entry.comparingByValue())
        .collect(Collectors.toMap(
            Map.Entry::getKey,
            Map.Entry::getValue,
            (a,b) -> a,
            LinkedHashMap::new
        ));
```

16. Find All Pairs with Given Sum

```
int target = 10;

list.stream()
    .flatMap(i -> list.stream()
        .filter(j -> i + j == target)
        .map(j -> List.of(i, j)))
    .toList();
```

17. Group Words by Length

```
Map<Integer, List<String>> map =
    list.stream()
        .collect(Collectors.groupingBy(String::length));
```

18. Most Frequent Character in String

```
Character result = str.chars()
    .mapToObj(c -> (char)c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .get()
    .getKey();
```

19. Custom Sorting (Multiple Fields)

```
employees.stream()
    .sorted(Comparator
        .comparing(Employee::getDept)
        .thenComparing(Employee::getSalary))
    .toList();
```

20. Detect Cycle-like Duplicate Pattern (Advanced Thinking)

👉 Example: detect repeated pattern

```
boolean hasDuplicates = list.size() != list.stream().distinct().count();
```

FAANG-Level Thinking Pattern

1. Convert → stream()
 2. Transform → map / flatMap
 3. Filter → filter()
 4. Aggregate → collect / reduce
 5. Optimize → parallelStream (if needed)
-

Advanced Cheat Sheet

```
groupingBy + counting → frequency
flatMap → flatten nested data
sorted + limit → top K
distinct → remove duplicates
partitioningBy → split
reduce → aggregation
mapToInt → numeric operations
```

Interview Strategy (VERY IMPORTANT)

When stuck:

👉 Break problem:

- Can I **group** it?
- Can I **sort + limit**?
- Can I **flatten**?
- Can I **reduce**?

Problem Solving Patterns Cheatsheet (15 Core Patterns)

1. Sliding Window

Subarray/substring optimization using expanding and shrinking window.

2. Two Pointers

Left-right or fast-slow pointer technique for arrays/lists.

3. Fast & Slow Pointer

Cycle detection and middle element problems.

4. Binary Search

Efficient searching and optimization in sorted/monotonic space.

5. BFS

Level-order traversal for shortest path and spreading problems.

6. DFS

Deep exploration for trees, graphs, and paths.

7. Backtracking

Try choices, undo if invalid (permutations, Sudoku).

8. Dynamic Programming

Break into overlapping subproblems with memoization/tabulation.

9. Greedy

Local optimal choices leading to global solution.

10. Prefix Sum

Precompute cumulative sums for fast range queries.

11. Hashing

Use maps/sets for fast lookup and frequency tracking.

12. Heap / Priority Queue

Efficient access to min/max elements.

13. Interval Merging

Sort and merge overlapping intervals.

14. Monotonic Stack

Next greater/smaller element problems.

15. Trie

Prefix tree for word-based search and autocomplete.

Array

Two Sum

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to target*.

You may assume that each input would have ***exactly one solution***, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Output: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`

Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`

Output: `[0,1]`

Constraints:

- $2 \leq \text{nums.length} \leq 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$
- **Only one valid answer exists.**

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

Best Time to Buy and Sell Stock

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: `prices = [7,6,4,3,1]`

Output: 0

Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 10^5$
- $0 \leq \text{prices}[i] \leq 10^4$

Contains Duplicate

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`
Output: `true`

Example 2:

Input: `nums = [1,2,3,4]`
Output: `false`

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`
Output: `true`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

Product of Array Except Self

Given an integer array `nums`, return *an array answer such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`*.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]`

Example 2:

Input: `nums = [-1,1,0,-3,3]`

Output: `[0,0,9,0,0]`

Constraints:

- $2 \leq \text{nums.length} \leq 10^5$
- $-30 \leq \text{nums}[i] \leq 30$
- The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array **does not** count as extra space for space complexity analysis.)

Maximum Subarray

Given an integer array `nums`, find the contiguous subarray (containing at least one number) which has the largest sum and return *its sum*.

A **subarray** is a **contiguous** part of an array.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: `[4,-1,2,1]` has the largest sum = 6.

Example 2:

Input: `nums = [1]`

Output: 1

Example 3:

Input: `nums = [5,4,-1,7,8]`

Output: 23

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-10^5 \leq \text{nums}[i] \leq 10^5$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

Maximum Product Subarray

Given an integer array `nums`, find a contiguous non-empty subarray within the array that has the largest product, and return *the product*.

It is **guaranteed** that the answer will fit in a **32-bit** integer.

A **subarray** is a contiguous subsequence of the array.

Example 1:

Input: `nums = [2,3,-2,4]`

Output: 6

Explanation: `[2,3]` has the largest product 6.

Example 2:

Input: `nums = [-2,0,-1]`

Output: 0

Explanation: The result cannot be 2, because `[-2,-1]` is not a subarray.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-10 \leq \text{nums}[i] \leq 10$
- The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

Find Minimum in Rotated Sorted Array

Suppose an array of length n sorted in ascending order is **rotated** between 1 and n times. For example, the array `nums = [0, 1, 2, 4, 5, 6, 7]` might become:

- `[4, 5, 6, 7, 0, 1, 2]` if it was rotated 4 times.
- `[0, 1, 2, 4, 5, 6, 7]` if it was rotated 7 times.

Notice that **rotating** an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of **unique** elements, return *the minimum element of this array*.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [3,4,5,1,2]`

Output: 1

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`

Output: 0

Explanation: The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

Example 3:

Input: `nums = [11,13,15,17]`

Output: 11

Explanation: The original array was `[11,13,15,17]` and it was rotated 4 times.

Constraints:

- `n == nums.length`
- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- All the integers of `nums` are **unique**.
- `nums` is sorted and rotated between 1 and `n` times.

Search in Rotated Sorted Array

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **rotated** at an unknown pivot index `k` ($0 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be rotated at pivot index 3 and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the rotation and an integer `target`, return *the index of target if it is in nums, or -1 if it is not in nums*.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`

Output: 4

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`

Output: -1

Example 3:

Input: `nums = [1]`, `target = 0`

Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- All values of `nums` are **unique**.

- `nums` is guaranteed to be rotated at some pivot.
- $-10^4 \leq \text{target} \leq 10^4$

3Sum

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Example 2:

Input: `nums = []`

Output: `[]`

Example 3:

Input: `nums = [0]`

Output: `[]`

Constraints:

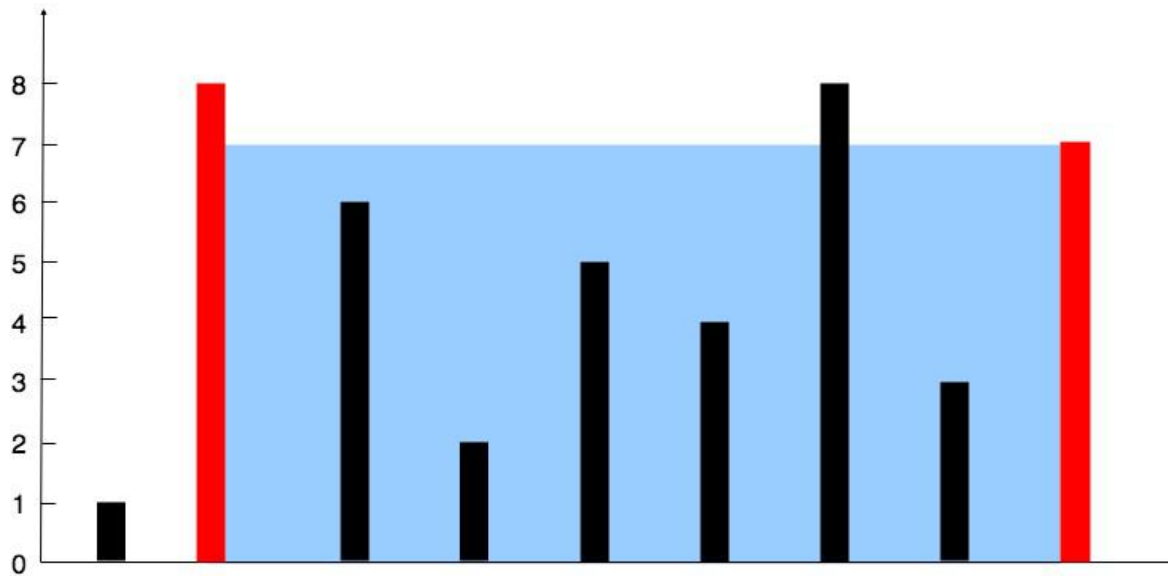
- $0 \leq \text{nums.length} \leq 3000$
- $-10^5 \leq \text{nums}[i] \leq 10^5$

Container With Most Water

Given n non-negative integers $a_1, a_2, \dots, a_n$, where each represents a point at coordinate (i, a_i) . n vertical lines are drawn such that the two endpoints of the line i is at (i, a_i) and $(i, 0)$. Find two lines, which, together with the x-axis forms a container, such that the container contains the most water.

Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]
Output: 1

Example 3:

Input: height = [4,3,2,1,4]
Output: 16

Example 4:

Input: height = [1,2,1]
Output: 2

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 10^5$
- $0 \leq \text{height}[i] \leq 10^4$

Binary

Sum of Two Integers

Given two integers a and b, return *the sum of the two integers without using the operators + and -*.

Example 1:

Input: a = 1, b = 2

Output: 3

Example 2:

Input: a = 2, b = 3

Output: 5

Constraints:

- $-1000 \leq a, b \leq 1000$

Number of 1 Bits

Write a function that takes an unsigned integer and returns the number of '1' bits it has (also known as the [Hamming weight](#)).

Note:

- Note that in some languages, such as Java, there is no unsigned integer type. In this case, the input will be given as a signed integer type. It should not affect your implementation, as the integer's internal binary representation is the same, whether it is signed or unsigned.
- In Java, the compiler represents the signed integers using [2's complement notation](#). Therefore, in **Example 3**, the input represents the signed integer. - 3.

Example 1:

Input: n = 000000000000000000000000000000001011

Output: 3

[illegible]

Example 2:

[illegible]

Output: 1

Explanation: The input binary string 000000000000000000000000010000000 has a total of one '1' bit.

Example 3:

Input: n = 111111111111111111111111111111101

Output: 31

[illegible]

Constraints:

- The input must be a **binary string** of length 32.

Follow up: If this function is called many times, how would you optimize it?

Counting Bits

Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the **number of 1's** in the binary representation of i .

Example 1:

Input: $n = 2$

Output: `[0,1,1]`

Explanation:

0 --> 0

1 --> 1

2 --> 10

Example 2:

Input: $n = 5$

Output: `[0,1,1,2,1,2]`

Explanation:

0 --> 0

1 --> 1

2 --> 10

3 --> 11

4 --> 100

5 --> 101

Constraints:

- $0 \leq n \leq 10^5$

Follow up:

- It is very easy to come up with a solution with a runtime of $O(n \log n)$. Can you do it in linear time $O(n)$ and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

Missing Number

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return *the only number in the range that is missing from the array*.

Follow up: Could you implement a solution using only $O(1)$ extra space complexity and $O(n)$ runtime complexity?

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation: $n = 3$ since there are 3 numbers, so all numbers are in the range $[0,3]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Explanation: $n = 2$ since there are 2 numbers, so all numbers are in the range $[0,2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation: $n = 9$ since there are 9 numbers, so all numbers are in the range $[0,9]$. 8 is the missing number in the range since it does not appear in `nums`.

Example 4:

Input: `nums = [0]`

Output: 1

Explanation: $n = 1$ since there is 1 number, so all numbers are in the range $[0,1]$. 1 is the missing number in the range since it does not appear in `nums`.

Constraints:

- `n == nums.length`
- `1 <= n <= 104`
- `0 <= nums[i] <= n`
- All the numbers of `nums` are **unique**.

Reverse Bits

Reverse bits of a given 32 bits unsigned integer.

Note:

- Note that in some languages such as Java, there is no unsigned integer type. In this case, both input and output will be given as a signed integer type. They should not affect your implementation, as the integer's internal binary representation is the same, whether it is signed or unsigned.
- In Java, the compiler represents the signed integers using [2's complement notation](#). Therefore, in **Example 2** above, the input represents the signed integer -3 and the output represents the signed integer -1073741825.

Follow up:

If this function is called many times, how would you optimize it?

Example 1:

Input: n = 00000010100101000001111010011100

Output: 964176192 (00111001011110000010100101000000)

Explanation: The input binary string 00000010100101000001111010011100 represents the unsigned integer 43261596, so return 964176192 which its binary representation is 00111001011110000010100101000000.

Example 2:

Input: n = 11111111111111111111111111111101

Output: 3221225471 (10111111111111111111111111111111)

Explanation: The input binary string 11111111111111111111111111111101 represents the unsigned integer 4294967293, so return 3221225471 which its binary representation is 10111111111111111111111111111111.

Constraints:

- The input must be a **binary string** of length 32

Dynamic Programming

Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$

Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$

Output: 3

Explanation: There are three ways to climb to the top.

1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

Coin Change

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

Input: `coins = [1,2,5]`, `amount = 11`

Output: `3`

Explanation: $11 = 5 + 5 + 1$

Example 2:

Input: `coins = [2]`, `amount = 3`

Output: `-1`

Example 3:

Input: `coins = [1]`, `amount = 0`

Output: `0`

Example 4:

Input: `coins = [1]`, `amount = 1`

Output: `1`

Example 5:

Input: coins = [1], amount = 2
Output: 2

Constraints:

- $1 \leq \text{coins.length} \leq 12$
- $1 \leq \text{coins}[i] \leq 2^{31} - 1$
- $0 \leq \text{amount} \leq 10^4$

Longest Increasing Subsequence

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

A **subsequence** is a sequence that can be derived from an array by deleting some or no elements without changing the order of the remaining elements. For example, `[3, 6, 2, 7]` is a subsequence of the array `[0, 3, 1, 6, 2, 2, 7]`.

Example 1:

Input: `nums = [10,9,2,5,3,7,101,18]`

Output: 4

Explanation: The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`

Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7,7,7]`

Output: 1

Constraints:

- `1 <= nums.length <= 2500`
- `-104 <= nums[i] <= 104`

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

Longest Common Subsequence

Given two strings `text1` and `text2`, return *the length of their longest **common subsequence***. If there is no **common subsequence**, return `0`.

A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A **common subsequence** of two strings is a subsequence that is common to both strings.

Example 1:

Input: `text1 = "abcde"`, `text2 = "ace"`

Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: `text1 = "abc"`, `text2 = "abc"`

Output: 3

Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: `text1 = "abc"`, `text2 = "def"`

Output: 0

Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- `1 <= text1.length, text2.length <= 1000`
- `text1` and `text2` consist of only lowercase English characters.

Word Break

Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: `s = "leetcode", wordDict = ["leet","code"]`

Output: `true`

Explanation: Return `true` because "leetcode" can be segmented as "leet code".

Example 2:

Input: `s = "applepenapple", wordDict = ["apple","pen"]`

Output: `true`

Explanation: Return `true` because "applepenapple" can be segmented as "apple pen apple".

Note that you are allowed to reuse a dictionary word.

Example 3:

Input: `s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]`

Output: `false`

Constraints:

- `1 <= s.length <= 300`
- `1 <= wordDict.length <= 1000`
- `1 <= wordDict[i].length <= 20`
- `s` and `wordDict[i]` consist of only lowercase English letters.
- All the strings of `wordDict` are **unique**.

Combination Sum IV

Given an array of **distinct** integers `nums` and a target integer `target`, return *the number of possible combinations that add up to target*.

The answer is **guaranteed** to fit in a **32-bit** integer.

Example 1:

Input: `nums = [1,2,3]`, `target = 4`

Output: 7

Explanation:

The possible combination ways are:

(1, 1, 1, 1)

(1, 1, 2)

(1, 2, 1)

(1, 3)

(2, 1, 1)

(2, 2)

(3, 1)

Note that different sequences are counted as different combinations.

Example 2:

Input: `nums = [9]`, `target = 3`

Output: 0

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 1000`
- All the elements of `nums` are **unique**.

- $1 \leq \text{target} \leq 1000$

Follow up: What if negative numbers are allowed in the given array? How does it change the problem? What limitation we need to add to the question to allow negative numbers?

House Robber

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night.**

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight without alerting the police.*

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = $1 + 3 = 4$.

Example 2:

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob = $2 + 9 + 1 = 12$.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

House Robber II

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are **arranged in a circle**. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have a security system connected, and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight without alerting the police*.

Example 1:

Input: `nums = [2,3,2]`

Output: 3

Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are adjacent houses.

Example 2:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 3:

Input: `nums = [1,2,3]`

Output: 3

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 1000`

Decode Ways

A message containing letters from A-Z can be **encoded** into numbers using the following mapping:

'A' -> "1"
'B' -> "2"
⋮
'Z' -> "26"

To **decode** an encoded message, all the digits must be grouped then mapped back into letters using the reverse of the mapping above (there may be multiple ways). For example, "11106" can be mapped into:

- "AAJF" with the grouping (1 1 10 6)
- "KJF" with the grouping (11 10 6)

Note that the grouping (1 11 06) is invalid because "06" cannot be mapped into 'F' since "6" is different from "06".

Given a string *s* containing only digits, return *the number of ways to decode it*.

The answer is guaranteed to fit in a **32-bit** integer.

Example 1:

Input: *s* = "12"

Output: 2

Explanation: "12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: *s* = "226"

Output: 3

Explanation: "226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: s = "0"

Output: 0

Explanation: There is no character that is mapped to a number starting with 0.

The only valid mappings with 0 are 'J' -> "10" and 'T' -> "20", neither of which start with 0.

Hence, there are no valid ways to decode this since all digits need to be mapped.

Example 4:

Input: s = "06"

Output: 0

Explanation: "06" cannot be mapped to "F" because of the leading zero ("6" is different from "06").

Constraints:

- $1 \leq s.length \leq 100$
- s contains only digits and may contain leading zero(s).

Unique Paths

A robot is located at the top-left corner of a $m \times n$ grid (marked 'Start' in the diagram below).

The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid (marked 'Finish' in the diagram below).

How many possible unique paths are there?

Example 1:



Input: $m = 3$, $n = 7$

Output: 28

Example 2:

Input: $m = 3$, $n = 2$

Output: 3

Explanation:

From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:

1. Right -> Down -> Down

2. Down -> Down -> Right
3. Down -> Right -> Down

Example 3:

Input: $m = 7, n = 3$
Output: 28

Example 4:

Input: $m = 3, n = 3$
Output: 6

Constraints:

- $1 \leq m, n \leq 100$
- It's guaranteed that the answer will be less than or equal to $2 * 10^9$.

Jump Game

You are given an integer array `nums`. You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position.

Return `true` *if you can reach the last index, or false otherwise.*

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: `true`

Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: `nums = [3,2,1,0,4]`

Output: `false`

Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $0 \leq \text{nums}[i] \leq 10^5$

Graph

Clone Graph

Given a reference of a node in a [connected](#) undirected graph.

Return a [deep copy](#) (clone) of the graph.

Each node in the graph contains a value (`int`) and a list (`List[Node]`) of its neighbors.

```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

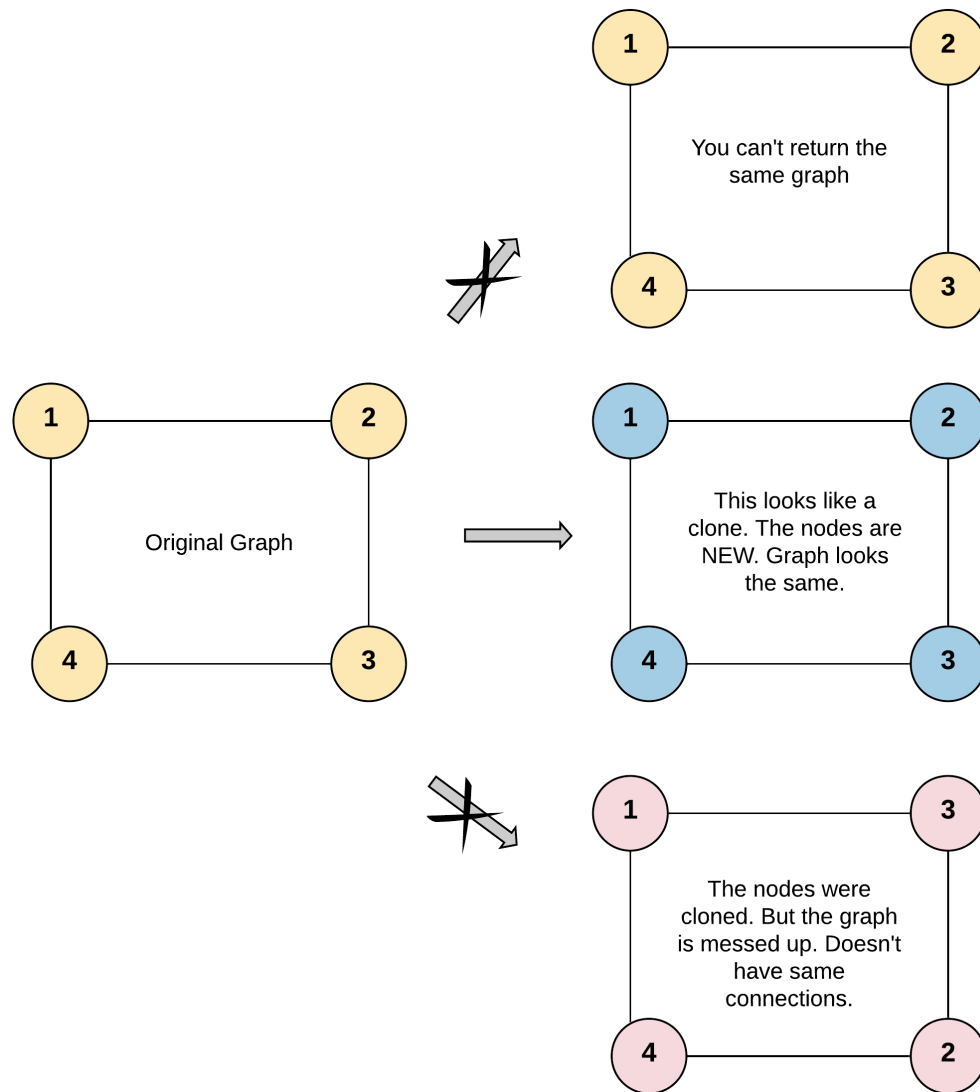
Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered **lists** used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with `val = 1`. You must return the **copy of the given node** as a reference to the cloned graph.

Example 1:



Input: adjList = [[2,4],[1,3],[2,4],[1,3]]

Output: [[2,4],[1,3],[2,4],[1,3]]

Explanation: There are 4 nodes in the graph.

1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).
4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

Example 2:



Input: adjList = [[]]

Output: [[]]

Explanation: Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.

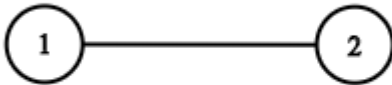
Example 3:

Input: adjList = []

Output: []

Explanation: This an empty graph, it does not have any nodes.

Example 4:



Input: `adjList = [[2],[1]]`

Output: `[[2],[1]]`

Constraints:

- The number of nodes in the graph is in the range $[0, 100]$.
- $1 \leq \text{Node.val} \leq 100$
- `Node.val` is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

Course Schedule

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `true`

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`

Output: `false`

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- `1 <= numCourses <= 105`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- All the pairs `prerequisites[i]` are **unique**.

Pacific Atlantic Water Flow

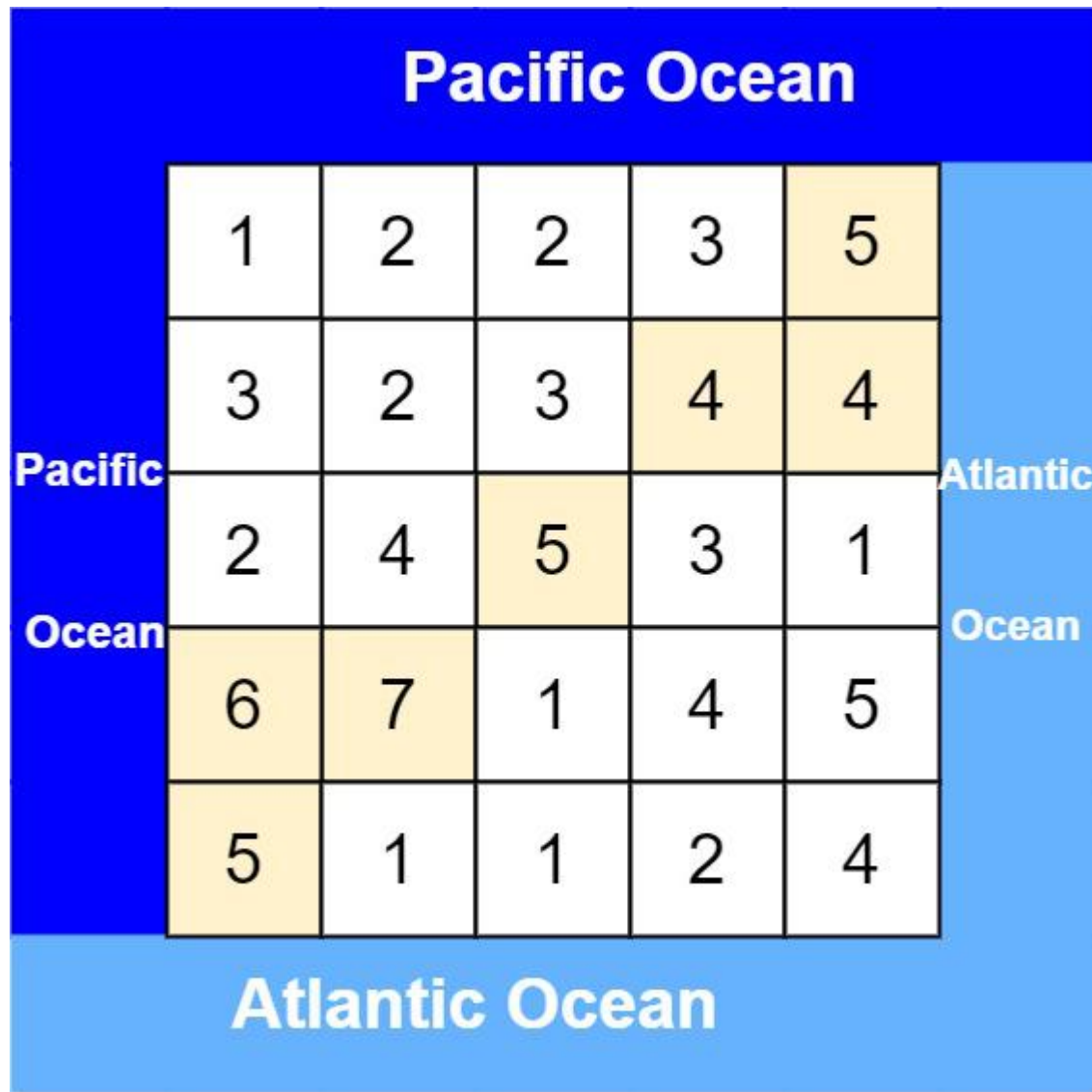
There is an $m \times n$ rectangular island that borders both the **Pacific Ocean** and **Atlantic Ocean**. The **Pacific Ocean** touches the island's left and top edges, and the **Atlantic Ocean** touches the island's right and bottom edges.

The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix `heights` where `heights[r][c]` represents the **height above sea level** of the cell at coordinate (r, c) .

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is **less than or equal to** the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a **2D list** of grid coordinates `result` where `result[i] = [ri, ci]` denotes that rain water can flow from cell (r_i, c_i) to **both** the Pacific and Atlantic oceans.

Example 1:



Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]

Example 2:

Input: heights = [[2,1],[1,2]]
Output: [[0,0],[0,1],[1,0],[1,1]]

Constraints:

- $m == \text{heights.length}$
- $n == \text{heights}[r].\text{length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{heights}[r][c] \leq 10^5$

Number of Islands

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
```

Output: 1

Example 2:

```
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
```

Output: 3

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`

- `grid[i][j]` is '0' or '1'.

Longest Consecutive Sequence

Given an unsorted array of integers `nums`, return *the length of the longest consecutive elements sequence*.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: `nums = [100,4,200,1,3,2]`

Output: 4

Explanation: The longest consecutive elements sequence is `[1, 2, 3, 4]`. Therefore its length is 4.

Example 2:

Input: `nums = [0,3,7,2,5,8,4,6,0,1]`

Output: 9

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

Alien Dictionary

Graph Valid Tree

Number of Connected Components in an Undirected Graph

Interval

Insert Interval

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [starti, endi]` represent the start and the end of the i^{th} interval and `intervals` is sorted in ascending order by `starti`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `starti` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return `intervals` *after the insertion*.

Example 1:

Input: `intervals = [[1,3],[6,9]]`, `newInterval = [2,5]`

Output: `[[1,5],[6,9]]`

Example 2:

Input: `intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]]`, `newInterval = [4,8]`

Output: `[[1,2],[3,10],[12,16]]`

Explanation: Because the new interval `[4,8]` overlaps with `[3,5]`, `[6,7]`, `[8,10]`.

Example 3:

Input: `intervals = []`, `newInterval = [5,7]`

Output: `[[5,7]]`

Example 4:

Input: `intervals = [[1,5]]`, `newInterval = [2,3]`

Output: `[[1,5]]`

Example 5:

Input: intervals = [[1,5]], newInterval = [2,7]
Output: [[1,7]]

Constraints:

- $0 \leq \text{intervals.length} \leq 10^4$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 10^5$
- intervals is sorted by start_i in **ascending** order.
- $\text{newInterval.length} == 2$
- $0 \leq \text{start} \leq \text{end} \leq 10^5$

Merge Intervals

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

Input: `intervals = [[1,3],[2,6],[8,10],[15,18]]`

Output: `[[1,6],[8,10],[15,18]]`

Explanation: Since intervals `[1,3]` and `[2,6]` overlaps, merge them into `[1,6]`.

Example 2:

Input: `intervals = [[1,4],[4,5]]`

Output: `[[1,5]]`

Explanation: Intervals `[1,4]` and `[4,5]` are considered overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 10^4$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 10^4$

Non-overlapping Intervals

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return *the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.*

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: `intervals = [[1,2],[1,2],[1,2]]`

Output: 2

Explanation: You need to remove two `[1,2]` to make the rest of the intervals non-overlapping.

Example 3:

Input: `intervals = [[1,2],[2,3]]`

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 10^5$
- $\text{intervals}[i].\text{length} == 2$
- $-5 * 10^4 \leq \text{start}_i < \text{end}_i \leq 5 * 10^4$

Meeting Rooms

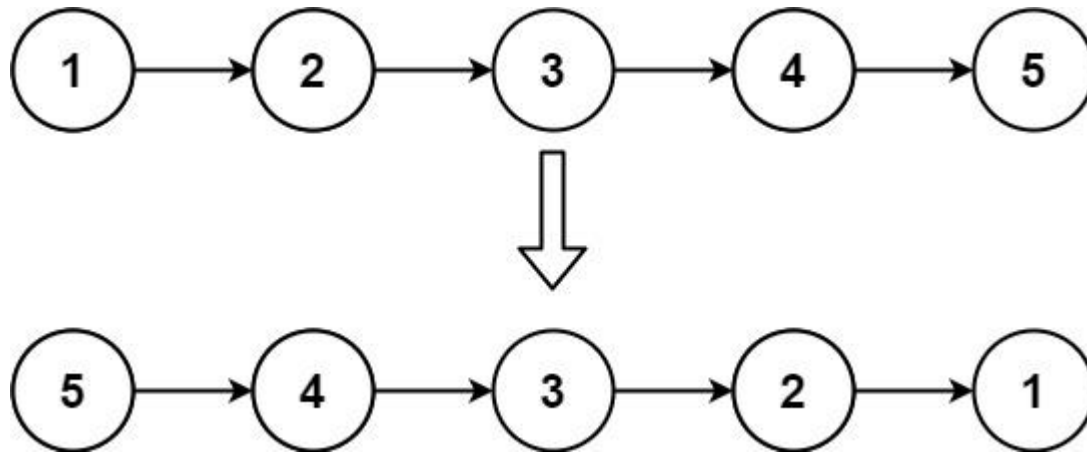
Meeting Rooms II

Linked List

Reverse Linked List

Given the head of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:



Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]

Example 2:

Input: head = [1,2]

Output: [2,1]

Example 3:

Input: head = []

Output: []

Constraints:

- The number of nodes in the list is the range $[0, 5000]$.
- $-5000 \leq \text{Node.val} \leq 5000$

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

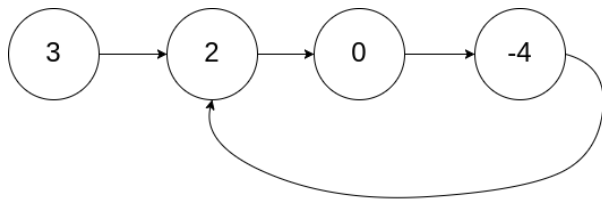
Linked List Cycle

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter.**

Return `true` if there is a cycle in the linked list. Otherwise, return `false`.

Example 1:

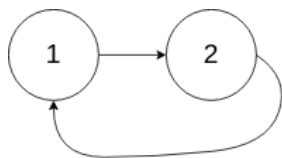


Input: `head = [3,2,0,-4]`, `pos = 1`

Output: `true`

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:



Input: head = [1,2], pos = 0

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:



Input: head = [1], pos = -1

Output: false

Explanation: There is no cycle in the linked list.

Constraints:

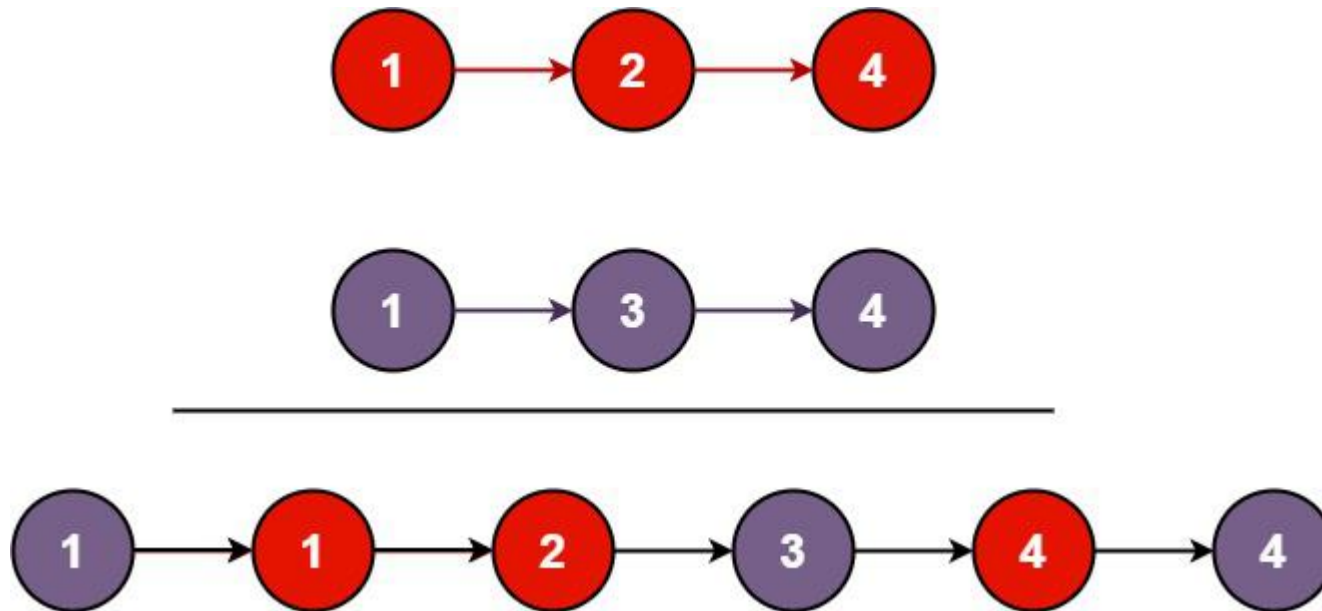
- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- pos is -1 or a **valid index** in the linked-list.

Follow up: Can you solve it using $O(1)$ (i.e. constant) memory?

Merge Two Sorted Lists

Merge two sorted linked lists and return it as a **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Example 1:



Input: l1 = [1,2,4], l2 = [1,3,4]

Output: [1,1,2,3,4,4]

Example 2:

Input: l1 = [], l2 = []

Output: []

Example 3:

Input: l1 = [], l2 = [0]
Output: [0]

Constraints:

- The number of nodes in both lists is in the range $[0, 50]$.
- $-100 \leq \text{Node.val} \leq 100$
- Both l1 and l2 are sorted in **non-decreasing** order.

Merge k Sorted Lists

You are given an array of k linked-lists `lists`, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

Input: `lists = [[1,4,5],[1,3,4],[2,6]]`

Output: `[1,1,2,3,4,4,5,6]`

Explanation: The linked-lists are:

```
[
  1->4->5,
  1->3->4,
  2->6
]
```

merging them into one sorted list:

`1->1->2->3->4->4->5->6`

Example 2:

Input: `lists = []`

Output: `[]`

Example 3:

Input: `lists = [[]]`

Output: `[]`

Constraints:

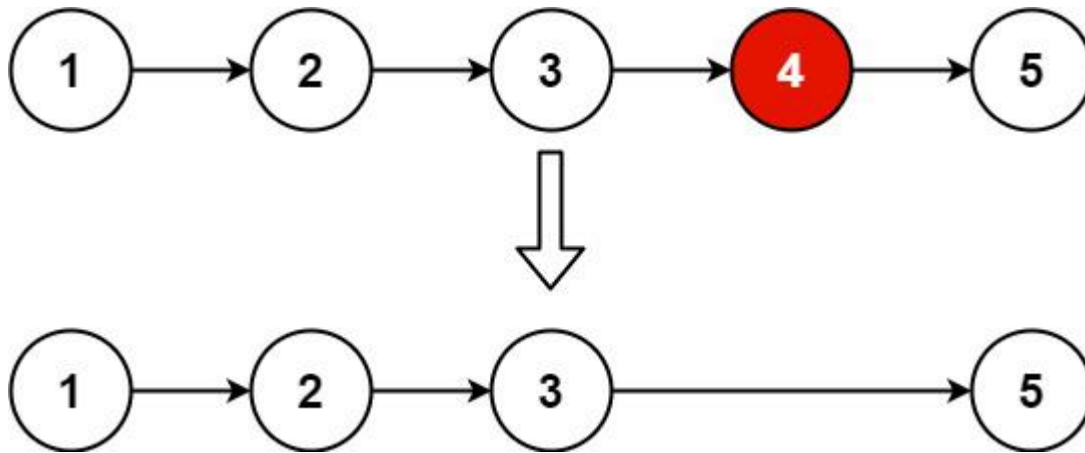
- `k == lists.length`

- $0 \leq k \leq 10^4$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-10^4 \leq \text{lists}[i][j] \leq 10^4$
- $\text{lists}[i]$ is sorted in **ascending order**.
- The sum of $\text{lists}[i].\text{length}$ won't exceed 10^4 .

Remove Nth Node From End of List

Given the head of a linked list, remove the n^{th} node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2

Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1

Output: []

Example 3:

Input: head = [1,2], n = 1

Output: [1]

Constraints:

- The number of nodes in the list is `sz`.
- `1 <= sz <= 30`
- `0 <= Node.val <= 100`
- `1 <= n <= sz`

Follow up: Could you do this in one pass?

Reorder List

You are given the head of a singly linked-list. The list can be represented as:

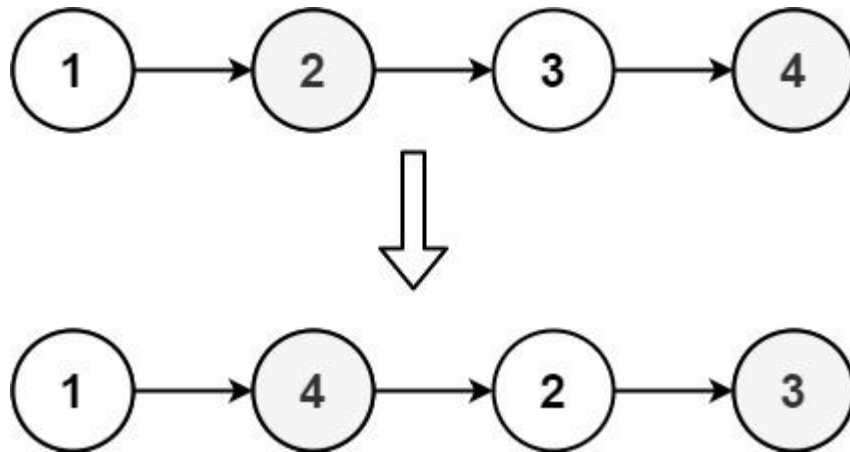
$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$$

Reorder the list to be on the following form:

$$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$$

You may not modify the values in the list's nodes. Only nodes themselves may be changed.

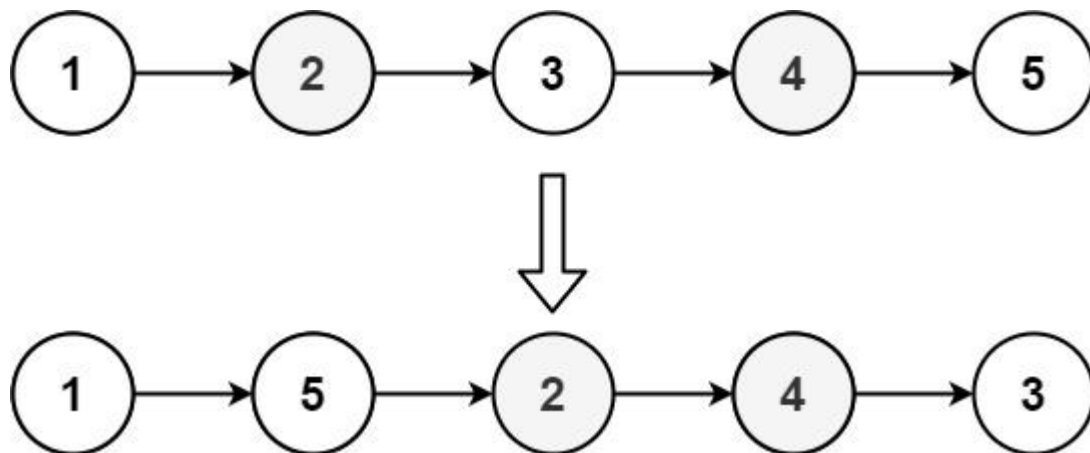
Example 1:



Input: head = [1,2,3,4]

Output: [1,4,2,3]

Example 2:



Input: head = [1,2,3,4,5]

Output: [1,5,2,4,3]

Constraints:

- The number of nodes in the list is in the range $[1, 5 * 10^4]$.
- $1 \leq \text{Node.val} \leq 1000$

Matrix

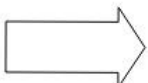
Set Matrix Zeroes

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's, and return *the matrix*.

You must do it [in place](#).

Example 1:

1	1	1
1	0	1
1	1	1



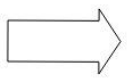
1	0	1
0	0	0
1	0	1

Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`

Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

0	1	2	0
3	4	5	2
1	3	1	5



0	0	0	0
0	4	5	0
0	3	1	0

Input: `matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]`

Output: `[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`

Constraints:

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- `-231 <= matrix[i][j] <= 231 - 1`

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.
- Could you devise a constant space solution?

Spiral Matrix

Given an $m \times n$ matrix, return *all elements of the matrix in spiral order*.

Example 1:

1	→	2	→	3
4	→	5		↓ 6
↑ 7	←	8	←	↓ 9

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

Example 2:

1 →	2 →	3 →	4
5 →	6 →	7	8
9 ←	10 ←	11 ←	12

Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Constraints:

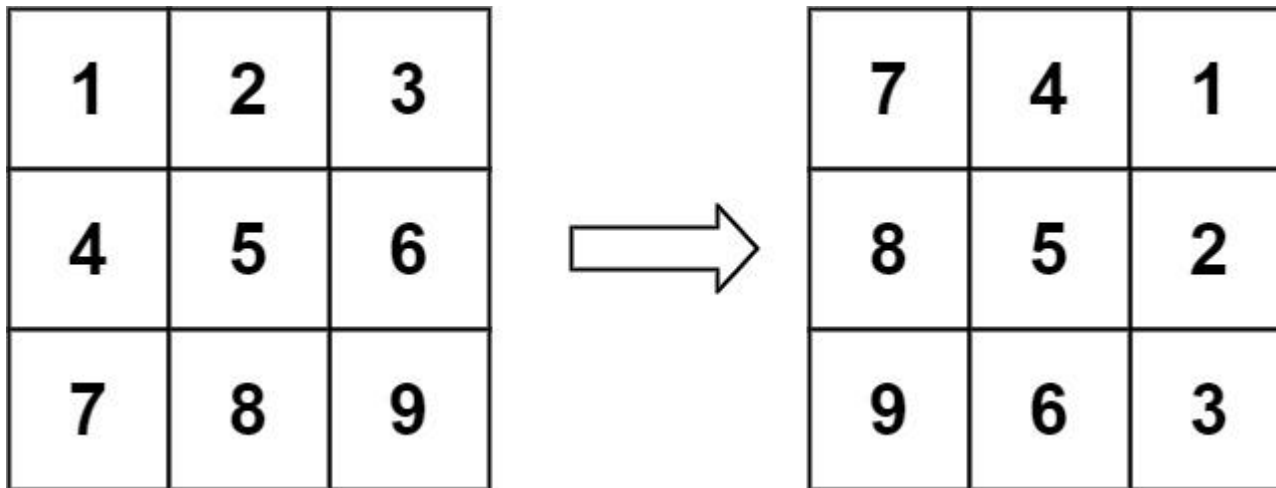
- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 10
- -100 <= matrix[i][j] <= 100

Rotate Image

You are given an $n \times n$ 2D matrix representing an image, rotate the image by **90** degrees (clockwise).

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:

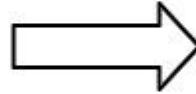


Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16



15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Example 3:

Input: matrix = [[1]]

Output: [[1]]

Example 4:

Input: matrix = [[1,2],[3,4]]

Output: [[3,1],[4,2]]

Constraints:

- matrix.length == n

- `matrix[i].length == n`
- `1 <= n <= 20`
- `-1000 <= matrix[i][j] <= 1000`

Word Search

Given an $m \times n$ grid of characters `board` and a string `word`, return `true` *if word exists in the grid*.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: `board = [ ["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"] ]`, `word = "ABCCED"`

Output: `true`

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, word = "SEE"
Output: true

Example 3:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, word = "ABCB"
Output: false

Constraints:

- `m == board.length`
- `n = board[i].length`
- `1 <= m, n <= 6`
- `1 <= word.length <= 15`
- board and word consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger board?

String

Longest Substring Without Repeating Characters

Given a string s , find the length of the **longest substring** without repeating characters.

Example 1:

Input: $s = \text{"abcabcbb"}$

Output: 3

Explanation: The answer is "abc", with the length of 3.

Example 2:

Input: $s = \text{"bbbbbb"}$

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: $s = \text{"pwwkew"}$

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Example 4:

Input: $s = \text{" "}$

Output: 0

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- s consists of English letters, digits, symbols and spaces.

Longest Repeating Character Replacement

You are given a string s and an integer k . You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return *the length of the longest substring containing the same letter you can get after performing the above operations*.

Example 1:

Input: $s = \text{"ABAB"}, k = 2$

Output: 4

Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:

Input: $s = \text{"AABABBA"}, k = 1$

Output: 4

Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".
The substring "BBBB" has the longest repeating letters, which is 4.

Constraints:

- $1 \leq s.length \leq 10^5$
- s consists of only uppercase English letters.
- $0 \leq k \leq s.length$

Minimum Window Substring

Given two strings s and t of lengths m and n respectively, return *the **minimum window substring** of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string ""*.

The testcases will be generated such that the answer is **unique**.

A **substring** is a contiguous sequence of characters within the string.

Example 1:

Input: $s = \text{"ADOBECODEBANC"}, t = \text{"ABC"}$

Output: `"BANC"`

Explanation: The minimum window substring `"BANC"` includes 'A', 'B', and 'C' from string t .

Example 2:

Input: $s = \text{"a"}, t = \text{"a"}$

Output: `"a"`

Explanation: The entire string s is the minimum window.

Example 3:

Input: $s = \text{"a"}, t = \text{"aa"}$

Output: `""`

Explanation: Both 'a's from t must be included in the window.

Since the largest window of s only has one 'a', return empty string.

Constraints:

- $m == s.length$
- $n == t.length$

- $1 \leq m, n \leq 10^5$
- s and t consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in $O(m + n)$ time?

Valid Anagram

Given two strings `s` and `t`, return `true` *if t is an anagram of s*, and `false` otherwise.

Example 1:

Input: `s = "anagram", t = "nagaram"`

Output: `true`

Example 2:

Input: `s = "rat", t = "car"`

Output: `false`

Constraints:

- `1 <= s.length, t.length <= 5 * 104`
- `s` and `t` consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

Group Anagrams

Given an array of strings `strs`, group **the anagrams** together. You can return the answer in **any order**.

An **Anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`
Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Example 2:

Input: `strs = [""]`
Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`
Output: `[["a"]]`

Constraints:

- `1 <= strs.length <= 104`
- `0 <= strs[i].length <= 100`
- `strs[i]` consists of lowercase English letters.

Valid Parentheses

Given a string *s* containing just the characters '(', ')', '{', '}', '[', and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: *s* = "("

Output: false

Example 4:

Input: *s* = "([)]"

Output: false

Example 5:

Input: *s* = "{[]}"

Output: true

Constraints:

- $1 \leq s.length \leq 10^4$
- s consists of parentheses only '()[]{}'.

Valid Palindrome

Given a string S , determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Example 1:

Input: $s = \text{"A man, a plan, a canal: Panama"}$

Output: true

Explanation: "amanaplanacanalpanama" is a palindrome.

Example 2:

Input: $s = \text{"race a car"}$

Output: false

Explanation: "raceacar" is not a palindrome.

Constraints:

- $1 \leq s.length \leq 2 * 10^5$
- s consists only of printable ASCII characters.

Longest Palindromic Substring

Given a string S , return *the longest palindromic substring* in S .

Example 1:

Input: $s = \text{"babad"}$

Output: "bab"

Note: "aba" is also a valid answer.

Example 2:

Input: $s = \text{"cbbdd"}$

Output: "bb"

Example 3:

Input: $s = \text{"a"}$

Output: "a"

Example 4:

Input: $s = \text{"ac"}$

Output: "a"

Constraints:

- $1 \leq s.length \leq 1000$
- s consist of only digits and English letters.

Palindromic Substrings

Given a string S , return *the number of palindromic substrings in it*.

A string is a **palindrome** when it reads the same backward as forward.

A **substring** is a contiguous sequence of characters within the string.

Example 1:

Input: $s = \text{"abc"}$

Output: 3

Explanation: Three palindromic strings: "a", "b", "c".

Example 2:

Input: $s = \text{"aaa"}$

Output: 6

Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of lowercase English letters.

Encode and Decode Strings

Tree

Maximum Depth of Binary Tree

Given the root of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:

Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

Input: root = [1,null,2]
Output: 2

Example 3:

Input: root = []
Output: 0

Example 4:

Input: root = [0]
Output: 1

Constraints:

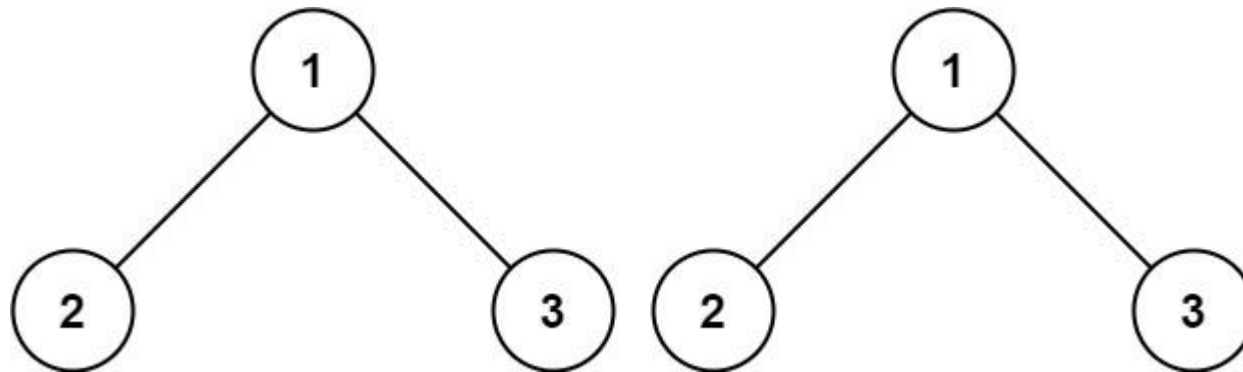
- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$

Same Tree

Given the roots of two binary trees p and q, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

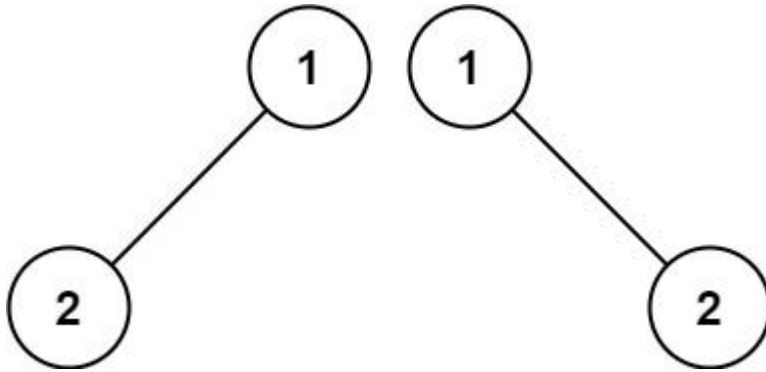
Example 1:



Input: p = [1,2,3], q = [1,2,3]

Output: true

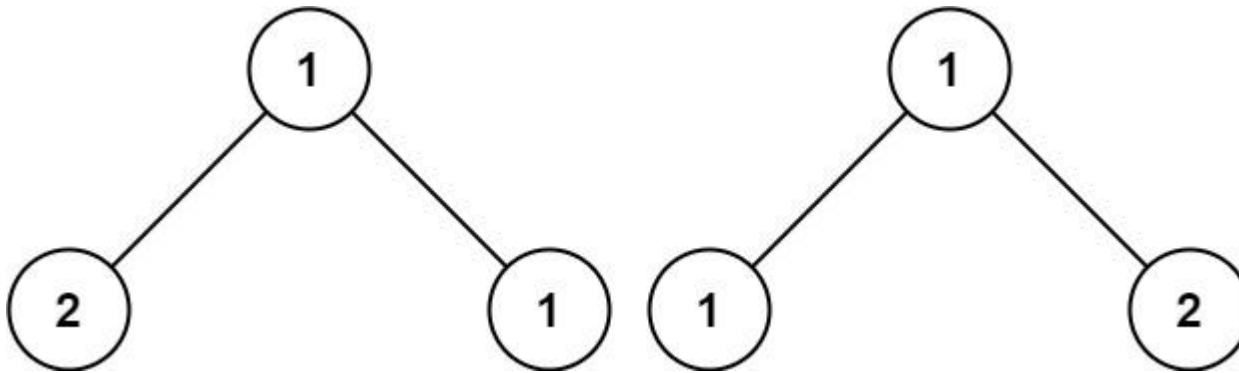
Example 2:



Input: `p = [1,2]`, `q = [1,null,2]`

Output: `false`

Example 3:



Input: `p = [1,2,1]`, `q = [1,1,2]`

Output: `false`

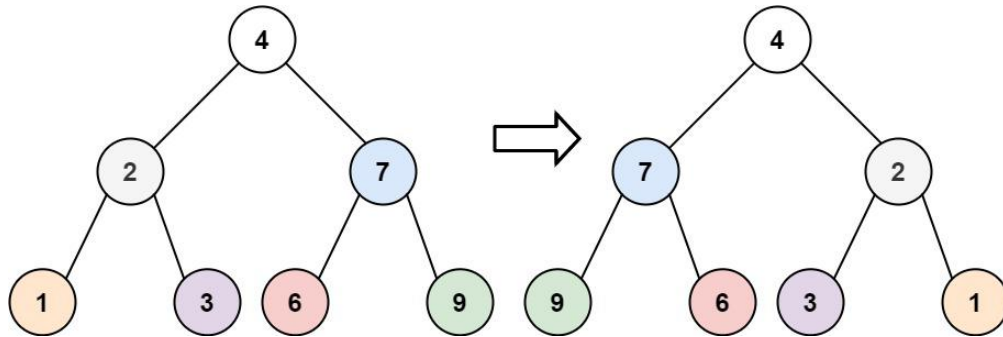
Constraints:

- The number of nodes in both trees is in the range `[0, 100]`.
- $-10^4 \leq \text{Node.val} \leq 10^4$

Invert Binary Tree

Given the root of a binary tree, invert the tree, and return *its root*.

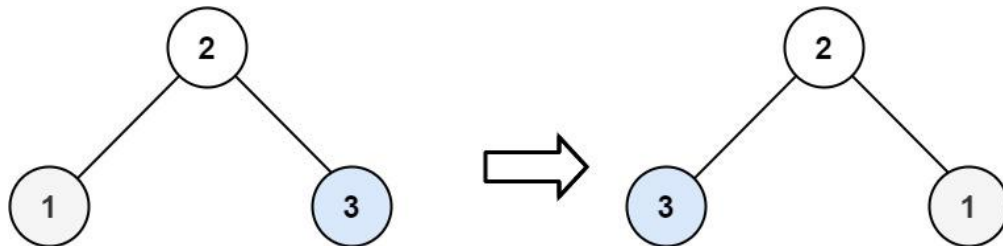
Example 1:



Input: root = [4,2,7,1,3,6,9]

Output: [4,7,2,9,6,3,1]

Example 2:



Input: root = [2,1,3]

Output: [2,3,1]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 100]$.
- $-100 \leq \text{Node.val} \leq 100$

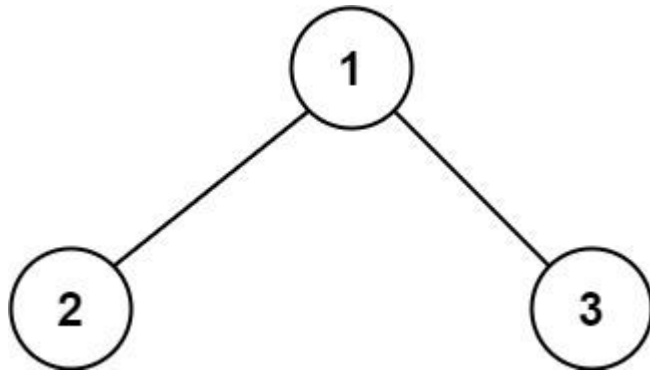
Binary Tree Maximum Path Sum

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence **at most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

Given the root of a binary tree, return *the maximum path sum of any path*.

Example 1:

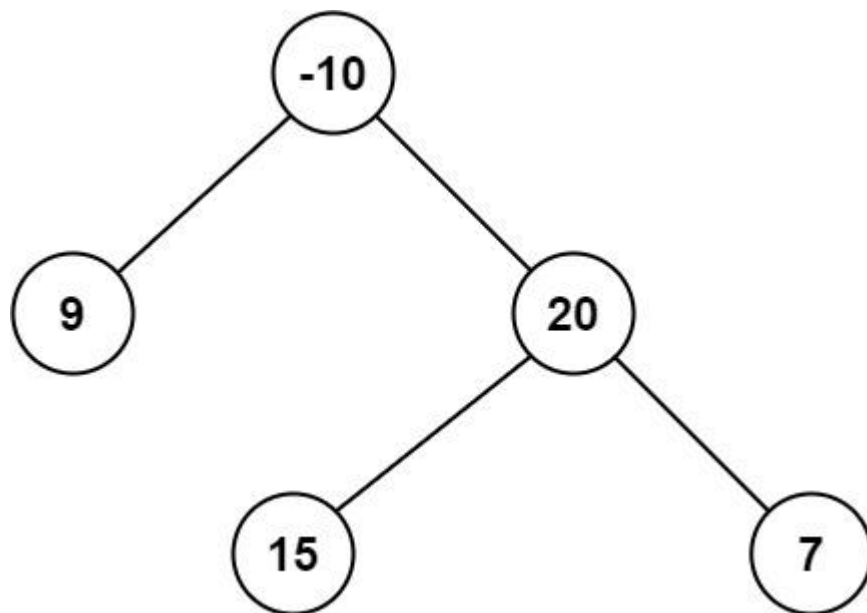


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of $2 + 1 + 3 = 6$.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of $15 + 20 + 7 = 42$.

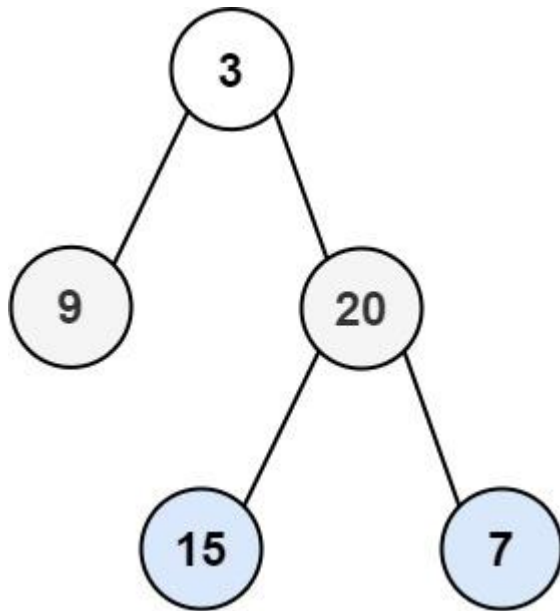
Constraints:

- The number of nodes in the tree is in the range $[1, 3 * 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

Binary Tree Level Order Traversal

Given the root of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]

Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- $-1000 \leq \text{Node.val} \leq 1000$

Serialize and Deserialize Binary Tree

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as [how LeetCode serializes a binary tree](#). You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:

Input: root = [1,2,3,null,null,4,5]

Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []

Output: []

Example 3:

Input: root = [1]

Output: [1]

Example 4:

Input: root = [1,2]

Output: [1,2]

Constraints:

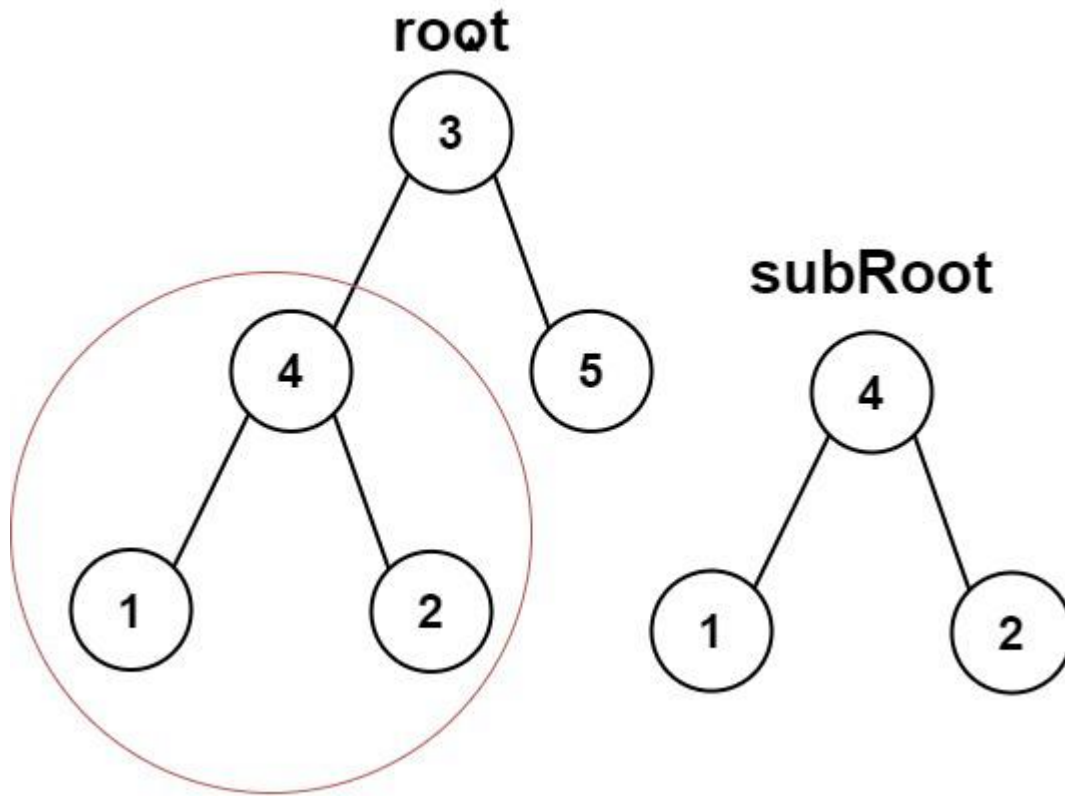
- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

Subtree of Another Tree

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

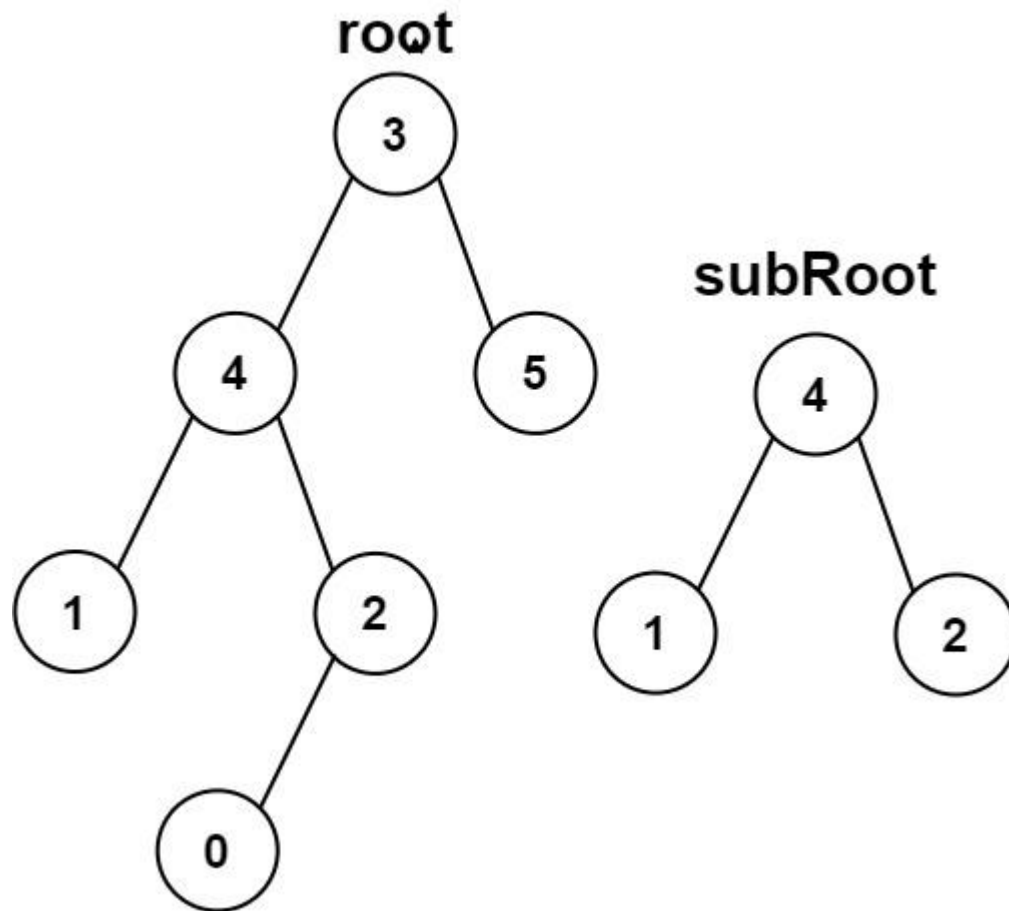
A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

Example 1:



Input: root = [3,4,5,1,2], subRoot = [4,1,2]
Output: true

Example 2:



Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
Output: false

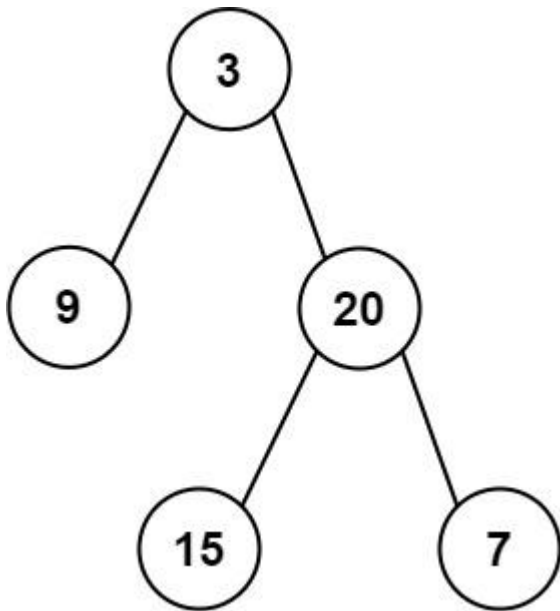
Constraints:

- The number of nodes in the `root` tree is in the range $[1, 2000]$.
- The number of nodes in the `subRoot` tree is in the range $[1, 1000]$.
- $-10^4 \leq \text{root.val} \leq 10^4$
- $-10^4 \leq \text{subRoot.val} \leq 10^4$

Construct Binary Tree from Preorder and Inorder Traversal

Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return *the binary tree*.

Example 1:



Input: `preorder = [3,9,20,15,7]`, `inorder = [9,3,15,20,7]`

Output: `[3,9,20,null,null,15,7]`

Example 2:

Input: `preorder = [-1]`, `inorder = [-1]`

Output: `[-1]`

Constraints:

- `1 <= preorder.length <= 3000`
- `inorder.length == preorder.length`
- `-3000 <= preorder[i], inorder[i] <= 3000`
- preorder and inorder consist of **unique** values.
- Each value of inorder also appears in preorder.
- preorder is **guaranteed** to be the preorder traversal of the tree.
- inorder is **guaranteed** to be the inorder traversal of the tree.

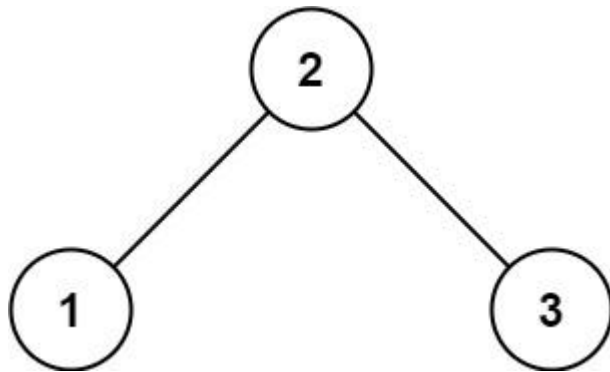
Validate Binary Search Tree

Given the root of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

- The left subtree of a node contains only nodes with keys **less than** the node's key.
- The right subtree of a node contains only nodes with keys **greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

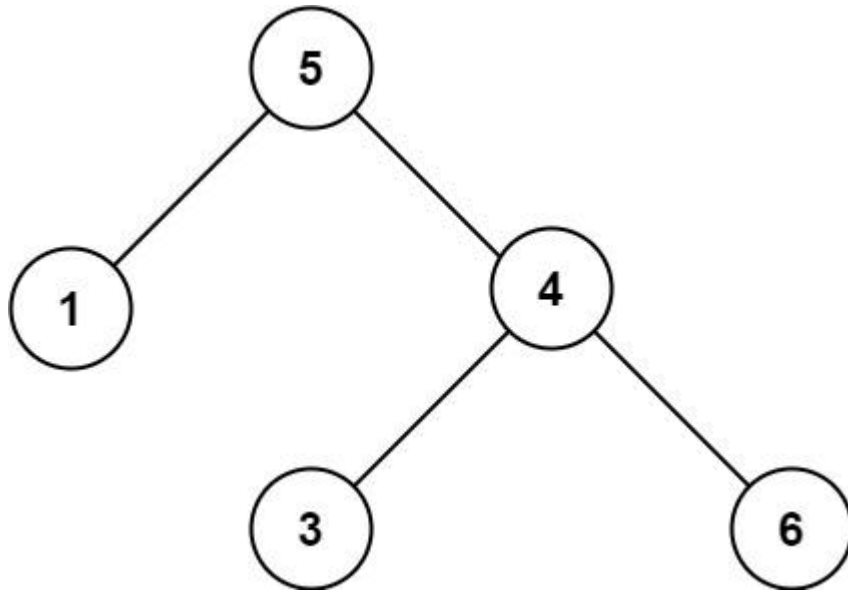
Example 1:



Input: root = [2,1,3]

Output: true

Example 2:



Input: root = [5,1,4,null,null,3,6]

Output: false

Explanation: The root node's value is 5 but its right child's value is 4.

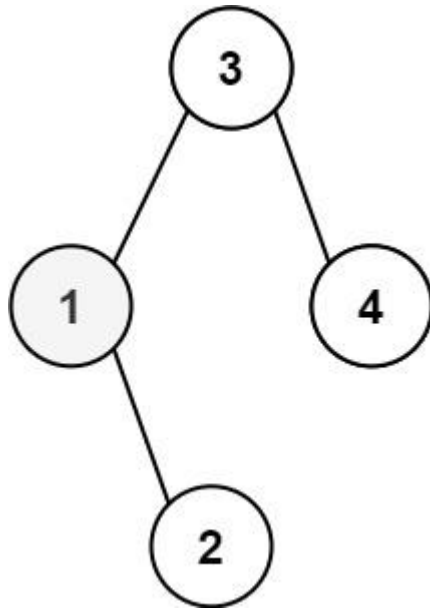
Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

Kth Smallest Element in a BST

Given the root of a binary search tree, and an integer k, return *the* k^{th} (**1-indexed**) *smallest element in the tree*.

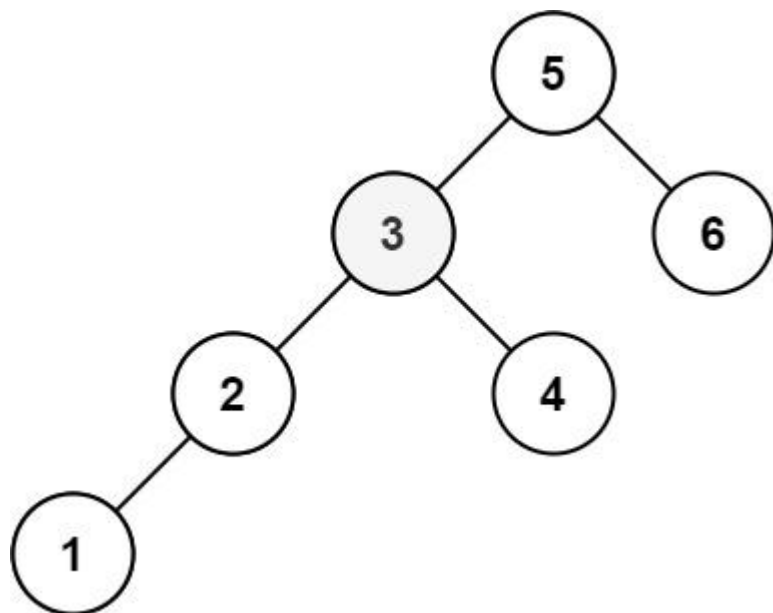
Example 1:



Input: root = [3,1,4,null,2], k = 1

Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3

Output: 3

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

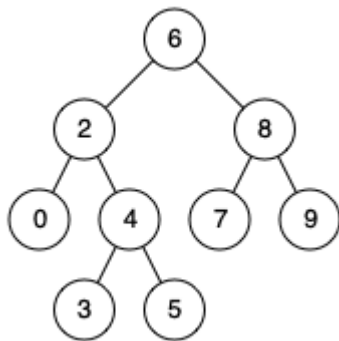
Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

Lowest Common Ancestor of a Binary Search Tree

Given a binary search tree (BST), find the lowest common ancestor (LCA) of two given nodes in the BST.

According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:

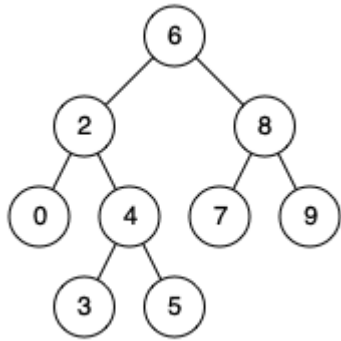


Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

Output: 2

Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [2,1], p = 2, q = 1

Output: 2

Constraints:

- The number of nodes in the tree is in the range $[2, 10^5]$.
- $-10^9 \leq \text{Node.val} \leq 10^9$
- All `Node.val` are **unique**.
- $p \neq q$
- p and q will exist in the BST.

Implement Trie (Prefix Tree)

A [trie](#) (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string word into the trie.
- `boolean search(String word)` Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- `boolean startsWith(String prefix)` Returns true if there is a previously inserted string word that has the prefix prefix, and false otherwise.

Example 1:

Input

```
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
```

```
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
```

Output

```
[null, null, true, false, true, null, true]
```

Explanation

```
Trie trie = new Trie();
trie.insert("apple");
trie.search("apple");    // return True
trie.search("app");      // return False
trie.startsWith("app");  // return True
trie.insert("app");
trie.search("app");      // return True
```

Constraints:

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- word and prefix consist only of lowercase English letters.
- At most $3 \cdot 10^4$ calls **in total** will be made to insert, search, and startsWith.

Word Search II

Given an $m \times n$ board of characters and a list of strings `words`, return *all words on the board*.

Each word must be constructed from letters of sequentially adjacent cells, where **adjacent cells** are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:

Input: `board = [ ["o","a","a","n"], ["e","t","a","e"], ["i","h","k","r"], ["i","f","l","v"] ],`
`words = ["oath","pea","eat","rain"]`
Output: `["eat","oath"]`

Example 2:

a	b
c	d

Input: `board = [ ["a","b"], ["c","d"] ], words = ["abcb"]`
Output: `[]`

Constraints:

- `m == board.length`
- `n == board[i].length`

- $1 \leq m, n \leq 12$
- `board[i][j]` is a lowercase English letter.
- $1 \leq \text{words.length} \leq 3 * 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$
- `words[i]` consists of lowercase English letters.
- All the strings of `words` are unique.

Heap

Merge k Sorted Lists

You are given an array of k linked-lists `lists`, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

Input: `lists = [[1,4,5],[1,3,4],[2,6]]`

Output: `[1,1,2,3,4,4,5,6]`

Explanation: The linked-lists are:

```
[
  1->4->5,
  1->3->4,
  2->6
]
```

merging them into one sorted list:

`1->1->2->3->4->4->5->6`

Example 2:

Input: `lists = []`

Output: `[]`

Example 3:

Input: `lists = [[]]`

Output: `[]`

Constraints:

- `k == lists.length`

- $0 \leq k \leq 10^4$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-10^4 \leq \text{lists}[i][j] \leq 10^4$
- $\text{lists}[i]$ is sorted in **ascending order**.
- The sum of $\text{lists}[i].\text{length}$ won't exceed 10^4 .

Top K Frequent Elements

Given an integer array `nums` and an integer `k`, return *the k most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Constraints:

- `1 <= nums.length <= 105`
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is **guaranteed** that the answer is **unique**.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where `n` is the array's size.

Find Median from Data Stream

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value and the median is the mean of the two middle values.

- For example, for `arr = [2, 3, 4]`, the median is 3.
- For example, for `arr = [2, 3]`, the median is $(2 + 3) / 2 = 2.5$.

Implement the `MedianFinder` class:

- `MedianFinder()` initializes the `MedianFinder` object.
- `void addNum(int num)` adds the integer `num` from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
```

```
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1);    // arr = [1]
medianFinder.addNum(2);    // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3);    // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0
```

Constraints:

- $-10^5 \leq \text{num} \leq 10^5$
- There will be at least one element in the data structure before calling `findMedian`.
- At most $5 * 10^4$ calls will be made to `addNum` and `findMedian`.

Follow up:

- If all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?

